

Tri-Mode Ethernet MAC v9.0

LogiCORE IP Product Guide

PG051 (v9.0) December 11, 2024

AMD Adaptive Computing is creating an environment where employees, customers, and partners feel welcome and included. To that end, we're removing non-inclusive language from our products and related collateral. We've launched an internal initiative to remove language that could exclude people or reinforce historical biases, including terms embedded in our software and IPs. You may still find examples of non-inclusive language in our older products as we work to make these changes and align with evolving industry standards. Follow this [link](#) for more information.





Table of Contents

- Chapter 1: Introduction..... 5**
 - Features..... 5
 - IP Facts.....6
- Chapter 2: Overview.....7**
 - Core Overview.....7
 - Recommended Design Experience..... 7
 - Ethernet Overview.....7
 - Core Overview.....9
 - Feature Summary..... 14
 - Applications.....15
 - Licensing and Ordering..... 17
- Chapter 3: Product Specification..... 18**
 - Standards..... 18
 - Performance..... 18
 - Resource Utilization..... 19
 - Port Descriptions.....19
 - Register Space..... 39
 - System Requirements.....69
- Chapter 4: Designing with the Core..... 70**
 - General Design Guidelines.....70
 - Shared Logic..... 73
 - Clocking..... 84
 - Resets.....85
 - Protocol Description..... 86
 - AXI4-Stream User Interface..... 92
 - Flow Control Using IEEE 802.3..... 104
 - Using Priority Flow Control..... 111
 - Statistics Counters.....120
 - Frame Filter..... 122

Ethernet AVB Endpoint.....	127
Configuration and Status.....	146
TEMAC Configuration Settings.....	155
Physical Interface for 7 Series and Zynq 7000 Devices.....	156
Physical Interface for UltraScale Devices.....	192
Interfacing to Other AMD Ethernet Cores.....	216
Chapter 5: Design Flow Steps.....	217
Customizing and Generating the Core.....	217
Constraining the Core.....	226
Simulation.....	230
Synthesis and Implementation.....	230
Chapter 6: Example Design.....	231
10, 100, 1000 Mbps Ethernet FIFO.....	233
2.5 Gbps Ethernet FIFO.....	235
Basic Pattern Generator Module for 10, 100, 1000 Mbps Data Rates.....	236
Basic Pattern Generator Module for 2.5 Gbps Data Rate.....	239
AXI4-Lite Control State Machine.....	240
Targeting the Example Design to a Board.....	242
Chapter 7: Test Bench.....	247
Test Bench Functionality.....	247
Changing the Test Bench.....	251
Appendix A: Upgrading.....	253
Migrating to the Vivado Design Suite.....	253
Upgrading in the Vivado Design Suite.....	253
Appendix B: Calculating the MMCM Phase Shift or IODelay Tap	
Setting.....	259
MMCM Usage.....	259
IODelay Usage.....	260
Appendix C: Verification, Compliance, and Interoperability.....	262
Simulation.....	262
Hardware Testing.....	262
Appendix D: Debugging.....	263



Finding Help with AMD Adaptive Computing Solutions.....263

Debug Tools..... 264

Simulation Debug.....265

Implementation and Timing Errors..... 267

Hardware Debug..... 270

Appendix E: Additional Resources and Legal Notices.....273

Finding Additional Documentation.....273

Support Resources..... 274

References.....274

Revision History..... 275

Please Read: Important Legal Notices..... 279

Introduction

The AMD LogiCORE™ IP Tri-Mode Ethernet Media Access Controller (TEMAC) solution comprises the 10/100/1000 Mbps Ethernet MAC, 1 Gbps Ethernet MAC, 2.5 Gbps Ethernet MAC, and 10/100 Mbps Ethernet MAC IP core. All cores support half-duplex and full-duplex operation.

Features

- Designed to IEEE 802.3-2008 specification
 - Configurable half-duplex and full-duplex operation
 - Supports 10/100 Mbps, 1 Gbps, 2.5 Gbps, or 10/100/1000 Mbps IP cores
 - Supports RGMII, GMII, and MII as well as providing connectivity to
 - LogiCORE IP Ethernet 1G/2.5G PCS/PMA or SGMII using transceiver, SelectIO™ or Ten-Bit Interface (TBI)
 - Optional MDIO interface to manage objects in PHY layers (MII Management)
 - Optional frame filter with selectable number of table entries and optional statistics counters
 - Supports Flow Control frames, Virtual LAN (VLAN) frames, jumbo frames, and allows a configurable interframe gap.
 - Optional support for Priority-based Flow Control in both directions as defined in IEEE specification 802.1Qbb
 - Optional fee-based Ethernet Audio Video Bridging (AVB) Endpoint designed to the following IEEE specifications
 - IEEE 802.1AS-2011 and IEEE 1588-2008
- Supports clock master functionality, clock slave functionality, and the Best Master Clock Algorithm (BMCA)
- IEEE 802.1Qav-2009 - Supports arbitration between different priority traffic and implements bandwidth policing

IP Facts

AMD LogiCORE™ IP Facts Table	
Core Specifics	
Supported Device Family ¹	AMD Versal™ Adaptive SoC, AMD UltraScale+™ families, AMD UltraScale™ families, AMD Zynq™ 7000 SoC, 7 series FPGAs
Supported User Interfaces	AXI4-Lite, AXI4-Stream
Resources	Performance and Resource Use web page
Provided with Core	
Design Files	Encrypted RTL
Example Design	VHDL and Verilog
Test Bench	Demonstration Test Bench
Constraints File	XDC
Simulation Model	Verilog and VHDL
Supported S/W Driver ²	N/A
Tested Design Flows ²	
Design Entry	AMD Vivado™ Design Suite
Simulation	For supported simulators, see the <i>Vivado Design Suite User Guide: Release Notes, Installation, and Licensing</i> (UG973).
Synthesis	Vivado Synthesis
Support	
Release Notes and Known Issues	Master Answer Record: 54251
All Vivado IP Change Logs	Master Vivado IP Change Logs: 72775
Support web page	

Notes:

1. For a complete list of supported devices, see the AMD Vivado™ IP catalog.
2. For the supported versions of third-party tools, see the *Vivado Design Suite User Guide: Release Notes, Installation, and Licensing* ([UG973](#)).

Overview

Core Overview

The Tri-Mode Ethernet Media Access Controller (TEMAC) solution comprises the 10/100/1000 Mbps, 1 Gbps, 2.5 Gbps, and 10/100 Mbps Intellectual Property (IP) cores along with the optional Ethernet AVB Endpoint, which are fully-verified designs. In addition, the example design provided with the core is in both Verilog-HDL and VHDL. This chapter introduces the TEMAC solution and provides related information, including recommended design experience, additional resources, technical support, and submitting feedback to AMD.

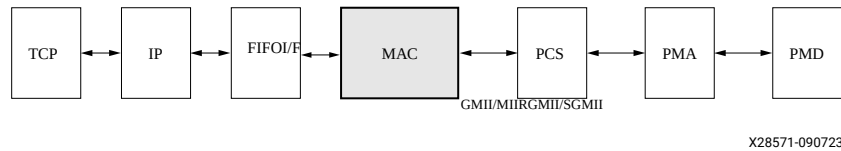
Recommended Design Experience

Although the TEMAC core is fully-verified, the challenge associated with implementing a complete design varies depending on the configuration and functionality of the application. For best results, previous experience building high-performance, pipelined FPGA using AMD implementation tools, and Constraint Files is recommended. Contact your local AMD representative for a closer review and estimation of your specific requirements.

Ethernet Overview

The MAC sublayer provided by this core is part of the Ethernet architecture displayed in [Figure 1: Typical Ethernet Architecture](#). The portion of the architecture, from the MAC to the right, is defined in IEEE 802.3-2008 specification for non-2.5 Gbps data rates. The architecture is similar when the MAC data rate is set to 2.5 Gbps. The only notable difference for 2.5 Gbps is the interface between MAC and PHY is Internal only (GMII with no I/Os). This figure also illustrates where the supported interfaces fit into the architecture.

Figure 1: Typical Ethernet Architecture



MAC

For 10/100 Mbps, 1 Gbps, and 10/100/1000 Mbps, the Ethernet Medium Access Controller (MAC) is defined in IEEE 802.3-2008 specification clauses 2, 3, and 4. The MAC also supports a non-standard based 2.5 Gbps data rate. At 2.5 Gbps data rate, MAC operation is not standard-based and can be interpreted as a MAC with an internal interface, which is clocked at 312.5 MHz clock. A MAC is responsible for the Ethernet framing protocols and error detection of these frames. The MAC is independent of and can be connected to any type of physical layer.



IMPORTANT! The 2.5 Gbps data rate is available for internal mode only. AMD Artix™ 7 support is only available in -2 and -3. All other devices and speed grades are supported at 2.5 Gbps. Half-duplex is not available for the internal mode.

GMII/MII

The Gigabit Media Independent Interface (GMII) is defined in IEEE 802.3-2008 specification, clause 35. At 10 Mbps and 100 Mbps, the Media Independent Interface (MII) is used as defined in IEEE 802.3-2008 specification, clause 22. These are parallel interfaces connecting a MAC to the physical sublayers (PCS, PMA, and PMD).

RGMII

The Reduced Gigabit Media Independent Interface (RGMII) is an alternative to the GMII. RGMII achieves a 50-percent reduction in the pin count, compared with GMII. Therefore, it is preferred over GMII by PCB designers. This is achieved with the use of double-data-rate (DDR) flip-flops. No change in the operation of the core is required to select between GMII and RGMII. However, the clock management logic and Input/Output Block (IOB) logic around the core change. HDL example designs are provided with the core that implements either the GMII or RGMII protocols.

SGMII

The Serial-GMII (SGMII) is an alternative interface to the GMII, which converts the parallel interface of the GMII into a serial format, radically reducing the I/O count (and for this reason often favored by PCB designers).

The TEMAC solution can be extended to include SGMII functionality by internally connecting its PHY side GMII to the [Ethernet 1G/2.5G PCS/PMA or SGMII](#) core from AMD. See [Interfacing to Other AMD Ethernet Cores](#).

PCS, PMA, and PMD

The combination of the Physical Coding Sublayer (PCS), the Physical Medium Attachment (PMA), and the Physical Medium Dependent (PMD) sublayer comprises the physical layers of the Ethernet protocol.

Two main physical standards are specified for Ethernet:

- BASE-T, a copper standard using twisted pair cabling systems
- BASE-X, usually a fiber optical physical standard using short and long wavelength laser

BASE-T devices, supporting 10 Mbps, 100 Mbps, and 1 Gbps Ethernet speeds are readily available as off-the-shelf parts. As illustrated in [Figure 4: Typical Application: Ethernet Switch or Router](#), these can be connected using GMII/MII, RGMII, or SGMII to provide a tri-speed Ethernet port.

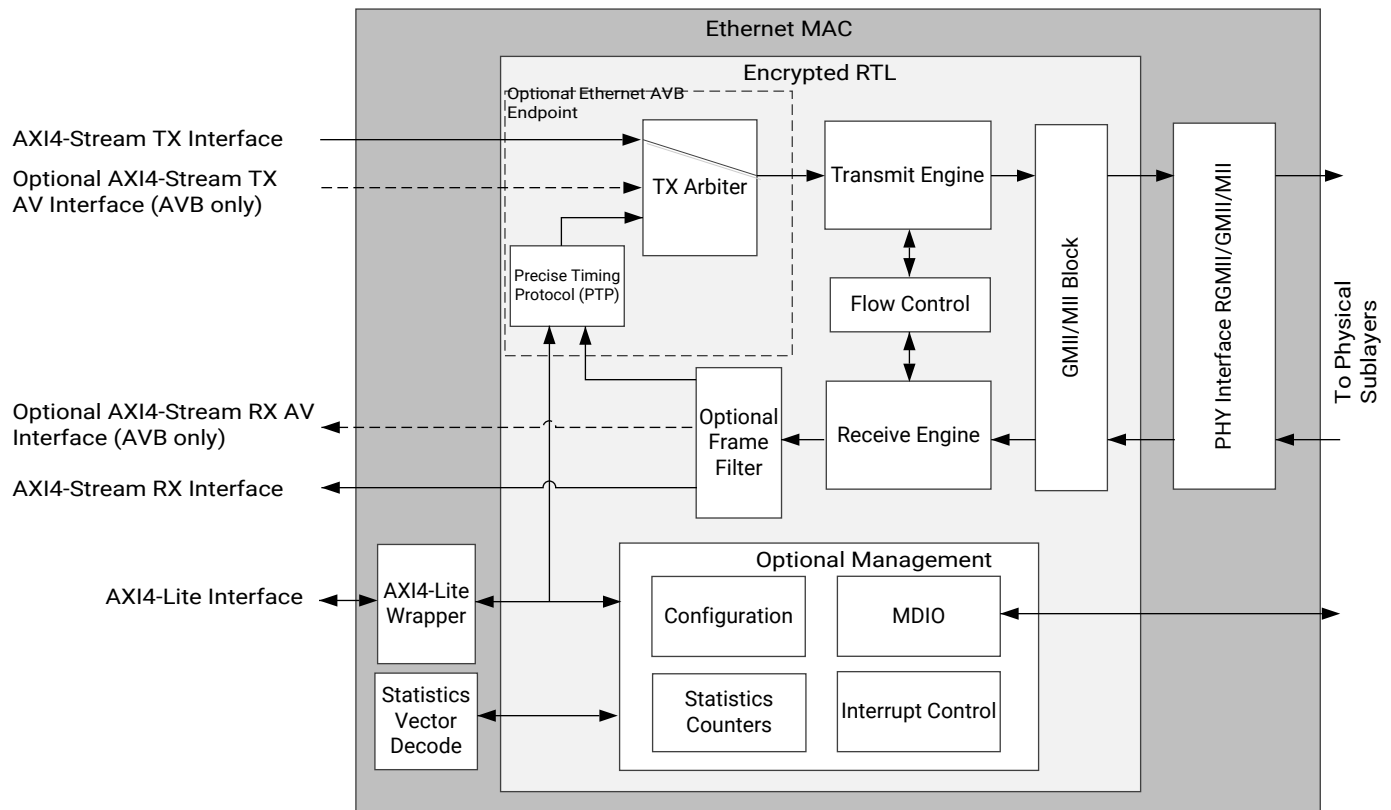
The 1000BASE-X architecture can be provided by connecting the TEMAC core to the Ethernet 1G/2.5G PCS/PMA or SGMII core.

A detailed Ethernet protocol overview is provided in [Protocol Description](#).

Core Overview

The following figure identifies the major functional blocks of the TEMAC solution and optional Ethernet AVB Endpoint cores. Descriptions of the functional blocks and interfaces are provided in the subsequent sections.

Figure 2: TEMAC Functional Block Diagram



X25618-080321

Ethernet MAC Core

The Ethernet MAC core includes the basic blocks required to use the Ethernet MAC.

AXI4-Lite Wrapper

The AXI4-Lite Wrapper allows the Ethernet MAC to be connected to an AXI4-Lite Interface and drives the Ethernet MAC through a processor independent Intellectual Property Interface (IPIF).

Statistics Vector Decode

The Statistics Vector Decode interprets the RX and TX statistics vectors supplied by the Ethernet MAC on a per-frame basis and generates the Statistics counter increment controls. This code is provided as editable HDL to meet specific Statistics counter requirements.

PHY Interface

The PHY Interface provides the required logic to interface with the PHY using either RGMII or GMII/MII. The core can be generated without the PHY Interface to allow direct connection to the AMD LogiCORE™ IP Ethernet 1G/2.5G PCS/PMA or SGMII.

Ethernet AVB Endpoint

The TEMAC can be implemented with an optional Ethernet AVB endpoint that is made up of two key functional blocks. When this functionality is not included, the AXI4-Stream TX Data is passed directly to the transmit engine. The AXI4-Stream RX Data is always passed directly to you, with the relative `t_user` signals being used to validate the data on the required interface.



IMPORTANT! AVB is only available for GMII/RGMII modes at 100/1000 Mbps speeds. It is not available for MII or 2.5 Gbps data rate.

Precise Timing Protocol (PTP)

The Precise Timing Protocol (PTP) block within the core provides the dedicated hardware to implement the IEEE 802.1AS specification. However, full functionality is only achieved using a combination of this hardware block coupled with functions provided by the relevant software drivers (run on an embedded processor). For more information, [Precise Timing Protocol Packet Buffers](#).

TX Arbiter

Data for transmission over an AVB network can be obtained from three source types:

- **AV Traffic:** For transmission from the AV Traffic interface of the core.
- **For transmission from the AV Traffic interface of the core:** Initiated by the software drivers using the dedicated hardware.
- **Legacy Traffic:** For transmission from the Legacy Traffic interface of the core.

The transmitter (TX) arbiter selects from these three sources in the following manner. If there is an AV packet available and the programmed AV bandwidth limitation is not exceeded, then the AV packet is transmitted. Otherwise, the TX arbiter checks to see if there are any PTP packets to be transmitted. If not, it checks to see if there is an available legacy packet to be transmitted. To comply with the specifications, the AV Traffic Interface should not be configured to exceed 75% of the overall Ethernet bandwidth. The arbiter then polices this bandwidth restriction for the AV traffic and ensures that it never exceeds on an average. Consequently, despite the AV traffic having a higher priority than the legacy traffic, there is always a remaining bandwidth available to schedule legacy traffic.

Transmit Engine

The transmit engine takes data from the AXI4-Stream TX interface and converts it into GMII format. Preamble and frame check sequence fields are added and the data is padded if necessary. The transmit engine also provides the transmit statistics vector for each packet and transmits the pause frames generated by the flow control module.

Receive Engine

The receive engine takes the data from the GMII/MII interface and checks it for compliance with IEEE 802.3-2008 specifications. Padding fields are removed and the AXI4-Stream RX interface is presented with the frame data along with a good/bad indication. The receive engine also provides the receive statistics vector for each received packet.

Flow Control

The flow control block is designed to IEEE 802.3-2008 specification, clause 31. The MAC can be configured to send pause frames with a programmable pause value and to act on their reception. These two behaviors can be configured asymmetrically.

As an option, an enhanced flow control block designed to IEEE 802.1Qbb Priority Flow Control (PFC) is also available. The MAC can be configured to send PFC frames with programmable enables, pause values, and also act on their reception. The legacy flow control and the priority flow control are mutually exclusive.

GMII/MII Block

The GMII/MII interface, which only operates at speeds below 1 Gbps, converts between the 4-bit data required by MII and the 8-bit data expected by the Receiver/Transmitter interfaces.

Management Interface

The optional Management Interface is a processor-independent interface with standard address, data, and control signals. It is used for the configuration and monitoring of the MAC and access to the Management Data Input/Output (MDIO) Interface. It is supplied with a wrapper to interface to the industry standard AXI4-Lite. This interface is optional. If it is not present, the device can be configured using configuration vectors.

MDIO Interface

The optional MDIO interface can be written to and read by using the Management Interface. The MDIO is used to monitor and configure PHY devices. The MDIO Interface is defined in IEEE 802.3-2008 specification, clause 22.

Frame Filter

The TEMAC solution can be implemented with an optional frame filter. If the frame filter is enabled, the device does not pass frames that do not contain one of a set of known addresses or match against one of the configurable frame filters. By default, all configurable frame filters are initialized to match against the IEEE 802.3-2008 specification defined Broadcast Address being observed in the destination address field of the MAC frame.

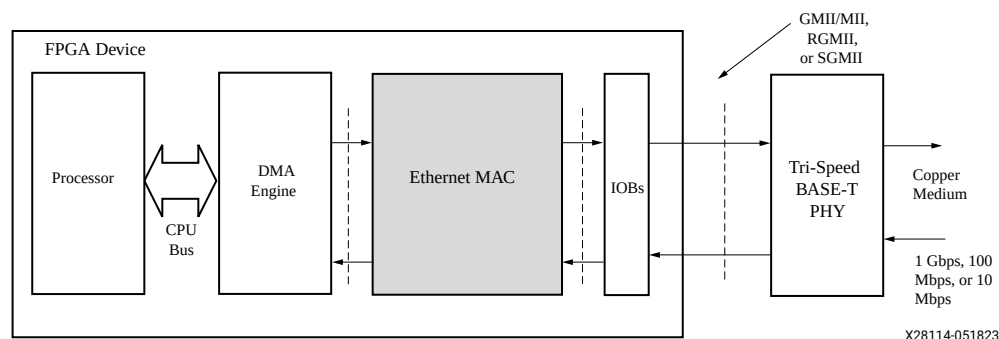
When the AVB Endpoint is included, the frame filter is always present with three filters being dedicated to identifying AV or PTP data. In this case, these filters are initialized to identify the default values for the various frame fields. The number of filters selected by you is in addition to these three filters.

Ethernet Communications Port for an Embedded Processor

The following figure illustrates a typical application for a single Ethernet MAC. The PHY side of the core is connected to an off-the-shelf Ethernet PHY device, which performs the BASE-T standard at 1 Gbps, 100 Mbps, and 10 Mbps speeds. The PHY device can be connected using any of the following supported interfaces: GMII/MII, RGMII, or by additionally using the [Ethernet 1G/2.5G BASE-X PCS/PMA](#) or [SGMII LogiCORE](#), SGMII.

The user side of the MAC is connected to a processor system through a processor DMA engine. This processor could be running a communications stack, such as the Transmission Control Protocol/Internet Protocol (TCP/IP). For applications such as this, see the *AMD Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* ([UG994](#)). The IP integrator contains an additional IP to connect the user interface of the MAC to the DMA port of a processor. The *AXI 1G/2.5G Ethernet Subsystem Product Guide* ([PG138](#)) describes the AXI Ethernet, which can be instantiated for an intended processor application.

Figure 3: Typical Application: Ethernet Communications Port for Embedded Processor



Statistics Counters

The TEMAC solution can be implemented with optional Statistics Counters. See [Statistics Counters](#) for more details.

Feature Summary

The key features of the TEMAC solution are:

- Designed to the IEEE Std 802.3-2008 specification
- Supports five separate IP cores
- 10/100/1000 Mbps Ethernet MAC
- 1 Gbps Ethernet MAC
- 2.5 Gbps Ethernet MAC
- 10/100 Mbps Ethernet MAC
- Optional Ethernet AVB
- Configurable duplex operation
- Support for MII, GMII, RGMII, and connection to the Ethernet 1G/2.5G PCS/PMA or SGMII LogiCORE.
- Optional Management Data Input/Output (MDIO) interface to manage objects in the physical layer
- User-accessible raw statistic vector outputs
- Optional built-in statistics counters
- Optional built-in Ethernet AVB Endpoint designed to the following IEEE specifications
- IEEE802.1AS – Supports clock master functionality, clock slave functionality, and the Best Master Clock Algorithm (BMCA)
- IEEE802.1Qav – Supports arbitration between different priority traffic and implements bandwidth policing
- Support for VLAN frames
- Configurable interframe gap (IFG) adjustment in full-duplex operation
- Configurable in-band Frame Check Sequence (FCS) field passing on both transmit and receive paths
- Auto padding on transmit and stripping on receive paths
- Optional full memory mapped AXI4-Lite interface for configuration and monitoring
- Configurable flow control through Ethernet MAC Control PAUSE frames; symmetrically or asymmetrically enabled. Optional support for Priority-based Flow Control, in both directions, as defined in IEEE specification 802.1Qbb.
- Configurable support for jumbo frames of any length
- Configurable maximum frame length check

- Configurable receive frame filter
- AXI4-Stream user interface for Transmit and Receive frame datapath.

Applications

Typical applications for the Ethernet MAC include:

- [Ethernet Switch or Router](#)
- [Ethernet Communications Port for an Embedded Processor](#)
- [Ethernet AVB Endpoint System](#)

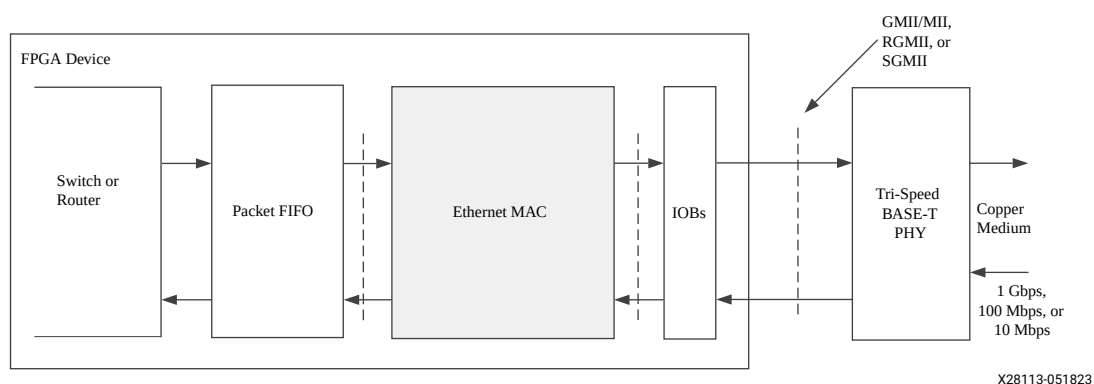
Ethernet Switch or Router

The following figure illustrates a typical application for a single Ethernet MAC. The Physical-side interface (PHY) side of the core is connected to an off-the-shelf Ethernet PHY device, which performs the BASE-T standard at 1 Gbps, 100 Mbps, and 10 Mbps speeds. The PHY device can be connected using any of the following supported interfaces: GMII/MII, RGMII, or by additionally using the [Ethernet 1G/2.5G BASE-X PCS/PMA or SGMII LogiCORE](#).

The user side of the Ethernet MAC is connected to a FIFO to complete a single Ethernet port. This port is connected to a Switch or Routing matrix that might contain several ports.

The TEMAC solution is provided with an example design for any of the supported physical interfaces. A FIFO example is also generated that can be used as the FIFO in the illustration for a typical application.

Figure 4: Typical Application: Ethernet Switch or Router



Ethernet AVB Endpoint System

The following figure illustrates a typical implementation for the TEMAC (100/1000 Mbps) core when the optional Ethernet AVB endpoint is included. Endpoint refers to a talker (for example, DVD player) or listener (for example, TV set) device as opposed to an intermediate bridge function that is not supported. In the implementation, the Tri-Mode Ethernet MAC core, with the AVB front end, is connected to an AVB-capable network.

Figure 5: Ethernet AVB Endpoint System

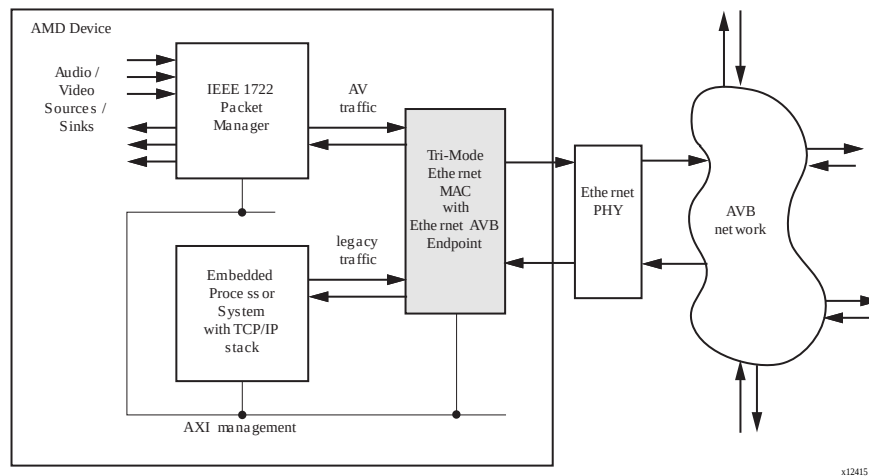


Figure 5: Ethernet AVB Endpoint System illustrates that the Tri-Mode Ethernet MAC core with the Ethernet AVB Endpoint logic supports two main data interfaces at the user side:

1. The *AV traffic* interface is intended for the Quality of Service audio/video data. Illustrated are several audio/video sources (for example, a DVD player) and several audio/video sinks (for example, a TV set). The Ethernet AVB Endpoint gives priority to the *AV traffic* interface over the *legacy traffic* interface, as dictated by IEEE 802.1Q 75% bandwidth restrictions.
2. The *legacy traffic* interface is maintained for best effort Ethernet data: Ethernet as it is known today (for example, a PC surfing the internet). Wherever possible, the priority is given to the *AV traffic* interface (as dictated by IEEE 802.1Q bandwidth restrictions), but a minimum of 25% of the total Ethernet bandwidth is always available for legacy Ethernet applications.

The *AV traffic* interface in the previous figure is shown as interfacing to a 1722 Packet Manager block. The IEEE1722 is another standard that specifies the embedding of audio/video data streams into Ethernet Packets. The 1722 headers within these packets include presentation timestamp information. Contact AMD for an engineering solution and for more system-level information.

Licensing and Ordering

This AMD LogiCORE™ IP module is provided under the terms of the [Core License Agreement](#). The module is shipped as part of the AMD Vivado™ Design Suite. For full access to all core functionalities in simulation and in hardware, you must purchase a license for the core. To generate a full license, visit the [product licensing web page](#). Evaluation licenses and hardware timeout licenses might be available for this core. Contact your [local sales representative](#) for information about pricing and availability.

Note: To verify that you need a license, check the License column of the IP catalog. Included means that a license is included with the AMD Vivado™ Design Suite; Purchase means that you have to purchase a license to use the core.

For more information about this core, visit the <Tri-Mode Ethernet MAC> [product web page](#) and optional fee-based Ethernet AVB Endpoint [product page](#).

Information about other AMD LogiCORE™ IP modules is available at the [Intellectual Property](#) page. For information about pricing and availability of other AMD LogiCORE IP modules and tools, contact your [local sales representative](#).

Following table shows the bundle offerings.

Table 1: TEMAC Bundle Offerings

Part Number	License	IP Cores
EF-DI-TEMAC-SITE	AMD LogiCORE IP Site License	10/100/1000 Mbps, 1 Gbps, 10/100 Mbps
EF-DI-TEMAC-PROJ	AMD LogiCORE IP Project License	10/100/1000 Mbps, 1 Gbps, 10/100 Mbps
EF-DI-EAVB-EPT-SITE	AMD LogiCORE IP Site License	100/1000 Mbps Ethernet AVB Endpoint

License Checkers

If the IP requires a license key, the key must be verified. The AMD Vivado™ design tools have several license checkpoints for gating licensed IP through the flow. If the license check succeeds, the IP can continue generation. Otherwise, generation halts with an error. License checkpoints are enforced by the following tools:

- Vivado Synthesis
- Vivado Implementation
- write_bitstream (Tcl command)



IMPORTANT! IP license level is ignored at checkpoints. The test confirms a valid license exists. It does not check IP license level.

Product Specification

The TEMAC solution is generated through the AMD Vivado™ Design Suite included in the latest IP Update on the AMD IP center. For detailed information about the core, see the TEMAC [product page](#) and the Ethernet AVB Endpoint [product page](#) for that optional feature.

Standards

The System Core adheres to the [AMBA AXI4-Stream Protocol v1.0 Specification \(Arm IHI 0051A\)](#). Designed to IEEE 802.3-2008 specification.

Performance

Latency

The latency figures given in the following sections apply to all permutations of the core.

Receive Path Latency

The receive path latency is measured as the number of valid cycles between a byte being driven into the GMII/MII receive interface (`gmii_rxd`), and it is appearing at the user interface (`rx_axis_mac_tdata`) of the Ethernet MAC core level. So latency values do not include any GMII/MII or RGMII logic within the example design. Receiver path latency has been measured as:

- 15 clock-enabled cycles at 1 Gbps or 2.5 Gbps Ethernet speed.
- 15 or 15.5 clock-enabled cycles at 10 Mbps and 100 Mbps Ethernet speeds. This extra half cycle of uncertainty is due to the conversion of 4-bit MII data width to 8-bit user data conversion.

Transmit Path Latency

The transmit path latency is measured by counting the number of valid cycles between a data byte being placed on the user interface (`tx_axis_mac_tdata`), and it is appearing at the GMII/MII output (`gmii_txd`) of the Ethernet MAC core level. So latency values do not include any GMII/MII or RGMII logic within the example design. Transmitter path latency has been measured as:

- 8 clock-enabled cycles at 1 Gbps or 2.5 Gbps Ethernet speed.
- 7 or 7.5 clock-enabled cycles at 10 Mbps and 100 Mbps Ethernet speeds. This extra half cycle of uncertainty is due to the conversion of 8-bit user data to 4-bit MII width conversion. Data is presented to the MII at the earliest possible opportunity.

Resource Utilization

For full details about performance and resource utilization, visit the [Performance and Resource Utilization web page](#).

Port Descriptions

All ports of the Ethernet MAC core level are internal connections in the FPGA logic. An example HDL design provided in both VHDL and Verilog is delivered with each core. The example design connects the core to a FIFO-based loopback example design and adds Input/Output Block (IOB) flip-flops to the external signals of the GMII/MII (or RGMII).

All clock management logic translated from a single onboard clock to the required system clocks are placed in this example design that provides more you more flexibility in implementation (for example, in designs using multiple cores). For information about the example design, See [Chapter 6: Example Design](#).



IMPORTANT! *The bus width of the write and read addresses depends on whether AVB endpoint is enabled or disabled.*

User Interfaces

Transmitter Interface

[Transmit Interface AXI4-Stream Signal Pins](#) defines the AXI4-Stream transmit signals of the core, which are used to transmit data from the user to the core. [Transmit Interface Sideband Signal Pins](#) defines transmit sideband signals. A detailed description of operation is provided in [Transmitting Outbound Frames](#).

Table 2: Transmit Interface AXI4-Stream Signal Pins

Signal	Direction	Clock Domain	Description
tx_axis_mac_tdata[7:0]	In	tx_mac_aclk	Frame data to be transmitted.
tx_axis_mac_tvalid	In	tx_mac_aclk	Control signal for tx_axis_mac_tdata port. Indicates the data is valid.
tx_axis_mac_tlast	In	tx_mac_aclk	Control signal for tx_axis_mac_tdata port. Indicates the final transfer in a frame.
tx_axis_mac_tuser	In	tx_mac_aclk	Control signal for tx_axis_mac_tdata port. Indicates an error condition, such as FIFO underrun, in the frame allowing the MAC to send an error to the PHY.
tx_axis_mac_tready	Out	tx_mac_aclk	Handshaking signal. Asserted when the current data on tx_axis_mac_tdata has been accepted and tx_axis_mac_tvalid is High. At 10/100 Mbps, this is used to meter the data into the core at the correct rate.

Notes:

1. All signals are active-High.

Table 3: Transmit Interface Sideband Signal Pins

Signal	Direction	Clock Domain	Description
tx_ifg_delay[7:0]	In	tx_mac_aclk	Control signal for configurable interframe gap
tx_collision	Out	tx_mac_aclk	Asserted by the Ethernet MAC core level to signal a collision on the medium and that any transmission in progress should be aborted. Always 0 when the Ethernet MAC core level is in full-duplex mode.
tx_retransmit	Out	tx_mac_aclk	When asserted at the same time as the tx_collision signal, this signals the client to resupply the aborted frame to the Ethernet MAC core level for retransmission. Always 0 when the Ethernet MAC core level is in full-duplex mode.
tx_statistics_vector[31:0] ²	Out	tx_mac_aclk	A statistics vector that gives information on the last frame transmitted.

Table 3: Transmit Interface Sideband Signal Pins (cont'd)

Signal	Direction	Clock Domain	Description
tx_statistics_valid	Out	tx_mac_aclk	Asserted at end of frame transmission, indicating that the tx_statistics_vector is valid.

Notes:

1. All signals are active-High.
2. When PFC is enabled tx_statistics_vector has a bus width of [32:0].

The following table defines the optional AXI4-Stream AV transmit signals included when the AVB functionality is selected.

Table 4: Transmit Interface AXI4-Stream AV Signal Pins

Signal	Direction	Clock Domain	Description
tx_axis_av_tdata[7:0]	In	tx_mac_aclk	Frame data to be transmitted.
tx_axis_av_tvalid	In	tx_mac_aclk	Control signal for tx_axis_av_tdata port. Indicates the data is valid.
tx_axis_av_tlast	In	tx_mac_aclk	Control signal for tx_axis_av_tdata port. Indicates the final transfer in a frame.
tx_axis_av_tuser	In	tx_mac_aclk	Control signal for tx_axis_av_tdata port. Indicates an error condition, such as FIFO underrun, in the frame allowing the MAC to send an error to the PHY.
tx_axis_av_tready	Out	tx_mac_aclk	Handshaking signal. Asserted when the current data on tx_axis_av_tdata has been accepted and tx_axis_av_tvalid is High. At 100 Mbps, this is used to meter the data into the core at the correct rate.

Notes:

1. All signals are active-High.

The statistics for the frame transmitted are contained within the tx_statistics_vector output. The bit field definition for the Vector is defined in the following table.

Table 5: Bit Definition for the Transmitter Statistics Vector

emacclient txstats	Name	Description
32	PFC_FRAME_TRANSMITTED	Extra vector bit included when the PFC functionality is included. This indicates that the Ethernet MAC has generated and transmitted a PFC frame.
31	PAUSE_FRAME_TRANSMITTED	Asserted if the previous frame was a pause frame that the MAC itself initiated in response to a pause_req assertion.

Table 5: Bit Definition for the Transmitter Statistics Vector (cont'd)

emacclient txstats	Name	Description
30	BYTE_VALID	Asserted if a MAC frame byte (DA to FCS inclusive) is in the process of being transmitted. This is valid on every clock cycle. Do not use this as an enable signal to indicate that data is present on (R)(G)MII_TXD.
29	Reserved	Returns logic 0.
28:25	TX_ATTEMPTS[3:0]	The number of attempts that have been made to transmit the previous frame. This is a 4-bit number: 0 should be interpreted as 1 attempt; 1 as 2 attempts, up until 15 as 16 attempts.
24	Reserved	Returns logic 0.
23	EXCESSIVE_COLLISION	Asserted if a collision has been detected on each of the last 16 attempts to transmit the previous frame.
22	LATE_COLLISION	Asserted if a late collision occurred during frame transmission.
21	EXCESSIVE_DEFERRAL	Asserted if the previous frame was deferred for an excessive amount of time as defined by the constant "maxDeferTime" in IEEE 802.3-2008.
20	TX_DEFERRED	Asserted if transmission of the frame was deferred.
19	VLAN_FRAME	Asserted if the previous frame contained a VLAN identifier in the length/type field when transmitter VLAN operation is enabled.
18:5	FRAME_LENGTH_COUNT	The length of the previous frame in number of bytes. The count stays at 16368 for any jumbo frames larger than this value.
4	CONTROL_FRAME	Asserted if the previous frame had the special MAC Control Type code 88-08 in the length/type field.
3	UNDERRUN_FRAME	Asserted if the previous frame contained an underrun error.
2	MULTICAST_FRAME	Asserted if the previous frame contained a multicast address in the destination address field.
1	BROADCAST_FRAME	Asserted if the previous frame contained a broadcast address in the destination address field.
0	SUCCESSFUL_FRAME	Asserted if the previous frame was transmitted without error.

Receiver Interface

[Receive Interface AXI4-Stream Signal Pins](#) describes the receive AXI4-Stream signals used by the core to transfer data to the user. [Receive Interface Sideband Signal Pins](#) describes the related sideband interface signals. A detailed description of operation is provided in [Receiving Inbound Frames](#).

Table 6: Receive Interface AXI4-Stream Signal Pins

Signal	Direction	Clock Domain	Description
rx_axis_mac_tdata[7:0]	Out	rx_mac_aclk	Frame data received is supplied on this port.
rx_axis_mac_tvalid	Out	rx_mac_aclk	Control signal for the rx_axis_mac_tdata port. Indicates that the data is valid.
rx_axis_mac_tlast	Out	rx_mac_aclk	Control signal for the rx_axis_mac_tdata port. Indicates the final byte in the frame.
rx_axis_mac_tuser	Out	rx_mac_aclk	Control signal for rx_axis_mac_tdata. Asserted at the end of frame reception to indicate that the frame had an error.
rx_axis_filter_tuser[x:0]	Out	rx_mac_aclk	Per frame filter tuser output. Can be used to send only data passed by a specific frame filter. See Frame Filter for more information.

Notes:

1. All signals are active-High.

Table 7: Receive Interface Sideband Signal Pins

Signal	Direction	Clock Domain	Description
rx_statistics_vector[27:0] ²	Out	rx_mac_aclk	Provides information about the last frame received.
rx_statistics_valid	Out	rx_mac_aclk	Asserted at the end of frame reception, indicating that the rx_statistics_vector is valid.

Notes:

1. All signals are active-High.
2. When PFC is enabled, rx_statistics_vector has a bus width of Bits[28:0].

The following table defines the optional AXI4-Stream AV receive signals included when the AVB functionality is selected.

Table 8: Receive Interface AXI4-Stream AV Signal Pins

Signal	Direction	Clock Domain	Description
rx_axis_av_tdata[7:0]	Out	rx_mac_aclk	Frame data received is supplied on this port.
rx_axis_av_tvalid	Out	rx_mac_aclk	Control signal for the rx_axis_av_tdata port. Indicates that the data is valid.
rx_axis_av_tlast	Out	rx_mac_aclk	Control signal for the rx_axis_av_tdata port. Indicates the final byte in the frame.
rx_axis_av_tuser	Out	rx_mac_aclk	Control signal for rx_axis_av_tdata. Asserted at end of frame reception to indicate that the frame had an error.

Notes:

1. All signals are active-High.

The statistics for the frame received are contained within the `rx_statistics_vector` output. The following table defines the bit field for the vector.

Table 9: Bit Definition for the Receiver Statistics Vector

emacclient rxstats	Name	Description
28	PFC_FRAME	Extra vector bit included when the PFC functionality is included. This indicates that the Ethernet MAC has received a valid PFC frame.
27	ADDRESS_MATCH	If the optional address filter is included in the core, this bit is asserted in case the address of the incoming frame matches one of the stored or pre-set addresses in the address filter. If the address filter is omitted from the core or is configured in promiscuous mode, this line is held High.
26	ALIGNMENT_ERROR	Asserted at speeds less than 1 Gbps if the frame contains an odd number of nibbles and the FCS for the frame is invalid.
25	LENGTH/TYPE Out of Range	If the length/type field contained a length value that did not match the number of MAC client data bytes received and the length/type field checks are enabled, this bit is asserted. This bit is also asserted if the length/type field is less than 46 and the frame is not padded to exactly 64 bytes. This is independent of whether or not the length/type field checks are enabled.
24	BAD_OPCODE	Asserted if the previous frame was error-free and contained the special control frame identifier in the length/type field, but contained an opcode that is unsupported by the MAC (any opcode other than PAUSE).
23	FLOW_CONTROL_FRAME	Asserted if the previous frame was error-free, contained the special control frame identifier in the length/type field, a destination address that matched either the MAC Control multicast address or the configured source address of the MAC, the supported PAUSE opcode, and was acted upon by the MAC.
22	BYTE_VALID	Asserted if a MAC frame byte (destination address to FCS inclusive) is in the process of being received. This is valid on every clock cycle. Do not use this as an enable signal to indicate that data is present on <code>emacclientrx[7:0]</code> .
21	VLAN_FRAME	Asserted if the previous frame contained a VLAN identifier in the length/type field when receiver VLAN operation is enabled.
20	OUT_OF_BOUNDS	Asserted if the previous frame exceeded the maximum frame size as defined in Maximum Permitted Frame Length . This is only asserted if jumbo frames are disabled.
19	CONTROL_FRAME	Asserted if the previous frame contained the special control frame identifier in the length/type field.
18:5	FRAME_LENGTH_COUNT	The length of the previous frame in number of bytes. The count stays at 16368 for any jumbo frames larger than this value.
4	MULTICAST_FRAME	Asserted if the previous frame contained a multicast address in the destination address field.
3	BROADCAST_FRAME	Asserted if the previous frame contained the broadcast address in the destination address field.
2	FCS_ERROR	Asserted if the previous frame received was correctly aligned but had an incorrect FCS value or the MAC detected error codes during frame reception.
1	BAD_FRAME ¹	Asserted if the previous frame received contained errors.

Table 9: Bit Definition for the Receiver Statistics Vector (cont'd)

emacclient rxstats	Name	Description
0	GOOD_FRAME ¹	Asserted if the previous frame received was error-free.

Notes:

1. If the length/type field error checks are disabled, a frame that has an actual data length that does not match the length/type field value is marked as a GOOD_FRAME providing no additional errors were detected. See [Length/Type Field Error Checks](#).

Flow Control Interface (IEEE 802.3)

See [Flow Control Interface Signal Pinout](#) describes the signals used to request a flow-control action from the transmit engine. Valid flow control frames received by the MAC are automatically handled (if the MAC is configured to do so). The pause value in the received frame is used to inhibit the transmitter operation for the time defined in IEEE 802.3-2008 specification. The frame is then passed to the client with `rx_axis_mac_tuser` asserted to indicate to the client that it should be dropped. See [Flow Control Using IEEE 802.3](#).

Table 10: Flow Control Interface Signal Pinout

Signal	Direction	Clock Domain	Description
pause_req	In	tx_mac_aclk	Pause request: Upon request the MAC transmits a pause frame after the completion of the current data packet. See Transmitting a Pause Control Frame .
pause_val[15:0]	In	tx_mac_aclk	Pause value: Inserted into the parameter field of the transmitted pause frame.

Notes:

1. All signals are active-High.
2. Continuous pause requests are not recommended to provide to the MAC. Assert a pause request pulse and wait for the respective pause valid assertion before applying another pause request pulse.

Priority Flow Control Interface (802.1Qbb)

The Priority Flow Control (PFC) interface is used to initiate the transmission of PFC frames from the core. The ports associated with this interface are shown in the following table. This interface is only present when priority-based flow control is enabled at the core customization stage.

When the optional PFC is enabled, there are eight AXI4-Stream interfaces defined for each Class of Service. The following table describes the AXI4-Stream PFC TX and RX signals.



IMPORTANT! The legacy pause and the priority flow control are mutually exclusive.

Table 11: Priority Flow Control Ports

Signal	Direction	Clock Domain	Description
tx_pfc_p0_tvalid	In	tx_mac_aclk	Pause request from priority 0 FIFO. This results in a PFC frame at the next available point.
tx_pfc_p1_tvalid	In	tx_mac_aclk	Pause request from priority 1 FIFO. This results in a PFC frame at the next available point.
tx_pfc_p2_tvalid	In	tx_mac_aclk	Pause request from priority 2 FIFO. This results in a PFC frame at the next available point.
tx_pfc_p3_tvalid	In	tx_mac_aclk	Pause request from priority 3 FIFO. This results in a PFC frame at the next available point.
tx_pfc_p4_tvalid	In	tx_mac_aclk	Pause request from priority 4 FIFO. This results in a PFC frame at the next available point.
tx_pfc_p5_tvalid	In	tx_mac_aclk	Pause request from priority 5 FIFO. This results in a PFC frame at the next available point.
tx_pfc_p6_tvalid	In	tx_mac_aclk	Pause request from priority 6 FIFO. This results in a PFC frame at the next available point.
tx_pfc_p7_tvalid	In	tx_mac_aclk	Pause request from priority 7 FIFO. This results in a PFC frame at the next available point.
rx_pfc_p0_tvalid	Out	rx_mac_aclk	Pause request to priority 0 RX FIFO.
rx_pfc_p0_tready	In	rx_mac_aclk	Pause acknowledge from priority 0 RX FIFO. The captured quanta only start to expire when this is asserted. If unused this should be tied High.
rx_pfc_p1_tvalid	Out	rx_mac_aclk	Pause request to priority 1 RX FIFO.
rx_pfc_p1_tready	In	rx_mac_aclk	Pause acknowledge from priority 1 RX FIFO. The captured quanta only start to expire when this is asserted. If unused this should be tied High.
rx_pfc_p2_tvalid	Out	rx_mac_aclk	Pause request to priority 2 RX FIFO.
rx_pfc_p2_tready	In	rx_mac_aclk	Pause acknowledge from priority 2 RX FIFO. The captured quanta only start to expire when this is asserted. If unused this should be tied High.
rx_pfc_p3_tvalid	Out	rx_mac_aclk	Pause request to priority 3 FIFO.
rx_pfc_p3_tready	In	rx_mac_aclk	Pause acknowledge from priority 3 RX FIFO. The captured quanta only start to expire when this is asserted. If unused this should be tied High.
rx_pfc_p4_tvalid	Out	rx_mac_aclk	Pause request to priority 4 FIFO.
rx_pfc_p4_tready	In	rx_mac_aclk	Pause acknowledge from priority 4 RX FIFO. The captured quanta only start to expire when this is asserted. If unused this should be tied High.
rx_pfc_p5_tvalid	Out	rx_mac_aclk	Pause request to priority 5 FIFO.

Table 11: Priority Flow Control Ports (cont'd)

Signal	Direction	Clock Domain	Description
rx_pfc_p5_tready	In	rx_mac_aclk	Pause acknowledge from priority 5 RX FIFO. The captured quanta only start to expire when this is asserted. If unused this should be tied High.
rx_pfc_p6_tvalid	Out	rx_mac_aclk	Pause request to priority 6 FIFO.
rx_pfc_p6_tready	In	rx_mac_aclk	Pause acknowledge from priority 6 RX FIFO. The captured quanta only start to expire when this is asserted. If unused this should be tied High.
rx_pfc_p7_tvalid	Out	rx_mac_aclk	Pause request to priority 7 FIFO.
rx_pfc_p7_tready	In	rx_mac_aclk	Pause acknowledge from priority 7 RX FIFO. The captured quanta only start to expire when this is asserted. If unused this should be tied High.

AXI4-Lite Signal Definition

The following table describes the optional signals that you used to access the Ethernet MAC core level, including configuration, status, and MDIO access. See [Management Interface](#).

Note: The bus width of the write and read addresses depends on whether AVB endpoint is enabled or disabled.

Table 12: Optional AXI4-Lite Signal Pinout

Signal	Direction	Clock Domain	Description
s_axi_aclk	In	N/A	Clock for AXI4-Lite
s_axi_resetn	In	s_axi_aclk	Local reset for the clock domain
s_axi_awaddr[16:0]	In	s_axi_aclk	Write Address When AVB endpoint is enabled.
s_axi_awaddr[11:0]	In	s_axi_aclk	Write Address When AVB endpoint is disabled.
s_axi_awvalid	In	s_axi_aclk	Write Address Valid
s_axi_awready	Out	s_axi_aclk	Write Address Ready
s_axi_wdata[31:0]	In	s_axi_aclk	Write Data
s_axi_wvalid	In	s_axi_aclk	Write Data Valid
s_axi_wready	Out	s_axi_aclk	Write Data Ready
s_axi_bresp[1:0]	Out	s_axi_aclk	Write Response
s_axi_bvalid	Out	s_axi_aclk	Write Response Valid
s_axi_bready	In	s_axi_aclk	Write Response Ready
s_axi_araddr[16:0]	In	s_axi_aclk	Read Address When AVB endpoint is enabled.
s_axi_araddr[11:0]	In	s_axi_aclk	Read Address When AVB endpoint is disabled.
s_axi_arvalid	In	s_axi_aclk	Read Address Valid

Table 12: Optional AXI4-Lite Signal Pinout (cont'd)

Signal	Direction	Clock Domain	Description
s_axi_arready	Out	s_axi_aclk	Read Address Ready
s_axi_rdata[31:0]	Out	s_axi_aclk	Read Data
s_axi_rresp[1:0]	Out	s_axi_aclk	Read Response
s_axi_rvalid	Out	s_axi_aclk	Read Data/Response Valid
s_axi_rready	In	s_axi_aclk	Read Data/Response Ready

Configuration Vector Signal Definition

The following table describes the configuration vectors that use direct inputs to the core to replace the functionality of the MAC configuration bits when the Management Interface is not used. The configuration settings described in [Table 28: Receiver Configuration Word 0 \(0x400\)](#) to [Table 34: TX Max Frame Configuration Word \(0x418\)](#) are included in the vector.

Table 13: Alternative to Optional Management Interface – Configuration Vector Signal Pinout

Signal	Direction	Clock Domain	Description
rx_configuration_vector[79:0] ²	In	rx_mac_aclk	The RX Configuration Vector is used to replace the functionality of the MAC RX Configuration registers when the Management interface is not used.
tx_configuration_vector[79:0] ³	In	tx_mac_aclk	The TX Configuration Vector is used to replace the functionality of the MAC TX Configuration registers when the Management interface is not used.

Notes:

1. All bits of the config vectors are registered on input but can be treated as asynchronous inputs.
2. When PFC is enabled rx_configuration_vector has a bus width of Bits[95:0].
3. When PFC is enabled tx_configuration_vector has a bus width of Bits[367:0].

Clock, Speed Indication, and Reset Signal Definition

The following table describes the reset signals, the clock signals that are input to the core, and the outputs that can be used to select between the three operating speeds. The clock signals are generated in the top-level wrapper provided with the core.

Table 14: Clock and Speed Indication Signals

Signal	Direction	Description
gbl_rstn	In	Active-Low asynchronous reset for entire core.
rx_axi_rstn	In	Active-Low RX domain reset
tx_axi_rstn	In	Active-Low TX domain reset
rx_reset	Out	Active-High RX software reset from Ethernet MAC core level

Table 14: Clock and Speed Indication Signals (cont'd)

Signal	Direction	Description
tx_reset	Out	Active-High TX software reset from Ethernet MAC core level
gtx_clk	In	Global 125 MHz clock. For 2.5 Gbps Ethernet speed is 312.5 MHz.
refclk	In	Required for idelayctrl, 200–300 MHz. For UltraScale architecture devices the range is 300–1333 MHz.
rx_usr_clk2	In	Only available when the Physical Interface is set to Internal and Internal Mode Clock Source set to RX User Clk2. 125 MHz Clock for the MAC RX datapath.
tx_mac_aclk	Out	Clock for the transmission of data on the physical interface. 312.5 MHz at 2.5 Gbps, 125 MHz at 1 Gbps, 25 MHz at 100 Mbps, and 2.5 MHz at 10 Mbps. This clock should be used to clock the physical interface transmit circuitry and the TX AXI4-Stream transmit circuitry. See the appropriate section: Physical Interface for 7 Series and Zynq 7000 Devices 1 Gbps Ethernet MAC Core Interfaces Tri-Speed Ethernet MAC Core Interfaces
rx_mac_aclk	Out	Clock for the reception of data on the physical interface. 312.5 MHz at 2.5 Gbps, 125 MHz at 1 Gbps, 25 MHz at 100 Mbps, and 2.5 MHz at 10 Mbps. This clock should be used to clock the physical interface receive circuitry and the RX AXI4-Stream receive circuitry. See the appropriate section: Physical Interface for 7 Series and Zynq 7000 Devices 1 Gbps Ethernet MAC Core Interfaces Tri-Speed Ethernet MAC Core Interfaces
clk_enable	In	Only available when the core is generated in Internal with tri-speed operation. This signal is input from the Ethernet 1G/2.5G PCS/PMA or SGMII core and used by the TEMAC as clock enable signal. For 1000 Mbps speeds, this signal is always asserted and for 10/100 Mbps speeds this signal pulsates once every 100/10 clock cycles. For more details, see <i>1G/2.5G Ethernet PCS/PMA or SGMII LogiCORE IP Product Guide (PG047)</i>.
clk_enable_rx	In	Only available when the Physical Interface is set to Internal and Internal Mode Clock Source set to RX User Clk2. This signal is input from the Ethernet 1G/2.5G PCS/PMA or SGMII core and used by the TEMAC as clock enable signal for the RX datapath. For 1,000 Mbps speeds, this signal is always asserted and for 10/100 Mbps speeds, this signal pulsates once every 100/10 clock cycles. For more details, see <i>1G/2.5G Ethernet PCS/PMA or SGMII LogiCORE IP Product Guide (PG047)</i>.
speedis100	Out	This output is asserted when the core is operating at 100 Mbps. It is derived from either Bits[13:12] of the MAC Speed Configuration register. If the optional Management Interface is not present, this is derived from configuration vector Bits[13:12].

Table 14: Clock and Speed Indication Signals (cont'd)

Signal	Direction	Description
speedis10100	Out	This output is asserted when the core is operating at either 10 Mbps or 100 Mbps. It is derived from either Bits[13:12] of the MAC Speed Configuration register. If the Management Interface is not present, this is derived from configuration vector Bits[13:12].
gtx_clk_out	Out	Only available when using RGMII in AMD Artix™ 7 or AMD Kintex™ 7 devices, and only then when the Shared Logic option (see Shared Logic) is selected with the “Include shared logic in core” option. This output clock can be used by other TEMAC core instances when sharing clocking resources. For AMD Virtex™ 7, MMCM is not a sharable resource and thus gtx_clk_out, and gtx_clk90_out are not listed. See the Physical Interface for 7 Series and Zynq 7000 Devices for details. See Figure 75: Clock Resource Sharing for 1 Gbps RGMII in 7 Series Using HP I/O or Figure 76: Clock Resource Sharing for 1 Gbps RGMII in 7 Series Using HR I/O for a connection illustration. This clock has a 0o phase shift with respect to the gtx_clk input and is used for RGMII data transmission.
gtx_clk90_out	Out	Only available when using RGMII in Artix 7 or Kintex 7 devices, and when the Shared Logic option (see Shared Logic) is selected with the “Include shared logic in core” option. This output clock can be used by other TEMAC core instances when sharing clocking resources. See Figure 75: Clock Resource Sharing for 1 Gbps RGMII in 7 Series Using HP I/O or Figure 76: Clock Resource Sharing for 1 Gbps RGMII in 7 Series Using HR I/O for a connection illustration. This clock has a 90o phase shift with respect to the gtx_clk input and is used for the RGMII transmitter clock forwarding.

Note: Apply same refclk frequency to IDELAYCTRL as that of REFCLK_FREQUENCY attribute of IDELAYE3 when the core is generated with IDELAYCTRL is included in core logic.

Interrupt Signals

The following table describes the interrupt signals provided by the TEMAC core.

Table 15: Interrupt Signals

Signal	Direction	Clock Domain	Description
mac_irq	Out	s_axi_aclk	Only available when the core is generated with AXI4-Lite Management Interface. This is the interrupt output from the interrupt controller. It is asserted when any of the interrupts, namely mdio_ready, interrupt_ptp_rx, interrupt_ptp_tx, or interrupt_ptp_timer has been enabled and set. See Interrupt Controller for more information.
interrupt_ptp_rx	Out	s_axi_aclk	Only available when the core is generated with AVB. This is asserted following the reception of any PTP packet by the RX PTP Packet Buffers. See RX PTP Packet Buffer for more information.
interrupt_ptp_tx	Out	s_axi_aclk	Only available when the core is generated with AVB. This is asserted following the transmission of any PTP packet from the TX PTP Packet Buffers. See TX PTP Packet Buffer for more information.
interrupt_ptp_timer	Out	s_axi_aclk	Only available when the core is generated with AVB. This interrupt asserts every 1/128 seconds as measured by the RTC. This acts as a timer for the PTP software algorithms. See Real-Time Clock for more information.

Ethernet AVB Endpoint PTP Signals

The following table defines the signals output from the core by the [Precise Timing Protocol \(PTP\)](#) block in the [Figure 2: TEMAC Functional Block Diagram](#). These signals, present only when the AVB Endpoint is included in the TEMAC, are provided for reference only and can be used by an application.

Table 16: AVB Specific Signals

Signal	Direction	Clock Domain	Description
rtc_nanosec_field	Out	gtx_clk	This is the synchronized nanoseconds field from the RTC. Updates to RTC Nanoseconds Field Offset register are reflected here.
rtc_sec_field	Out	gtx_clk	This is the synchronized seconds fields from the RTC. Updates to RTC Seconds Field Offset register are not reflected here.
clk8k	Out	gtx_clk	This is an 8 kHz clock, which is derived from and synchronized in frequency to the Real-Time Clock . The period of this clock, that is, 125 μ s is useful in timing the SR class measurement intervals.

Table 16: AVB Specific Signals (cont'd)

Signal	Direction	Clock Domain	Description
rtc_nanosec_field_1722	Out	gtx_clk	The IEEE1722 specification contains a different format for the Real-Time Clock provided here as an extra port. This is derived and is in sync with the IEEE802.1 AS Real-Time Clock.

Physical Interface Signals

MDIO Signal Definition

The following table describes the MDIO (MII Management) interface signals of the core when the *Add IO Buffers for MDIO Interface Ports* option is not selected. These signals are typically connected to the MDIO port of an on-chip PHY device, such as the multi-gigabit transceivers. These signals are present whenever the optional Management Interface is used. The MDIO format is defined in the *IEEE 802.3-2008 specification*, clause 2.

Table 17: MDIO Interface Signal Pinout

Signal	Direction	Description
mdc	Out	MDIO Management Clock: derived from s_axi_aclk on the basis of supplied configuration data when the optional Management Interface is used.
mdio_i	In	In data signal for communication with PHY configuration and status. Tie High if unused.
mdio_o	Out	Output data signal for communication with PHY configuration and status.
mdio_t	Out	3-state control for MDIO signals; 0 signals that the value on MDIO_OUT should be asserted onto the MDIO bus.

The following table shows the MDIO signals when the *Add IO Buffers for MDIO Interface Ports* option is selected. These signals are typically connected to the MDIO bus connects to an off-chip PHY device.

Table 18: MDIO Interface Signal Pinout with I/O Buffers

Signal	Direction	Description
mdc	In/Out	MDIO Management Clock: Derived from s_axi_aclk on the basis of supplied configuration data when the optional Management Interface is used.
mdio	In/Out	MDIO bus from the bidirectional I/O buffer inserted by the core. The core internally maps the mdio_i, mdio_o, and mdio_t signals to this bidirectional I/O buffer.

PHY Interface Signal Definition

The following tables describe the three possible interface standards supported, RGMII, GMII and MII, which are typically attached to a PHY module, either off-chip or internally integrated. The RGMII is defined in *Reduced Gigabit Media Independent Interface (RGMII)*, version 2.0, the GMII is defined in IEEE 802.3-2008 specification, clause 35, and MII is defined in IEEE 802.3-2008 specification, clause 22.

Table 19: Optional GMII Interface Signal Pinout

Signal	Direction	Clock Domain	Description
gmii_txd[7:0]	Out	tx_mac_aclk	Transmit data to PHY
gmii_tx_en	Out	tx_mac_aclk	Data Enable control signal to PHY
gmii_tx_er	Out	tx_mac_aclk	Error control signal to PHY
mii_tx_clk	In	-	Clock from PHY (used for 10/100)
gmii_col	In	N/A	Control signal from PHY
gmii_crs	In	N/A	Control signal from PHY
gmii_rxd[7:0]	In	gmii_rx_clk	Received data from PHY
gmii_rx_dv	In	gmii_rx_clk	Data Valid control signal from PHY
gmii_rx_er	In	gmii_rx_clk	Error control signal from PHY
gmii_rx_clk	In	-	Clock from PHY

Table 20: Optional MII Interface Signal Pinout

Signal	Direction	Clock Domain	Description
mii_tx_clk	In	-	Clock from PHY
mii_txd[3:0]	Out	mii_tx_clk	Transmit data to PHY
mii_tx_en	Out	mii_tx_clk	Data Enable control signal to PHY
mii_tx_er	Out	mii_tx_clk	Error control signal to PHY
mii_col	In	N/A	Control signal from PHY
mii_crs	In	N/A	Control signal from PHY
mii_rxd[3:0]	In	mii_rx_clk	Received data from PHY
mii_rx_dv	In	mii_rx_clk	Data Valid control signal from PHY
mii_rx_er	In	mii_rx_clk	Error control signal from PHY
mii_rx_clk	In	-	Clock from PHY

Table 21: Optional RGMII Interface Signal Pinout

Signal	Direction	Clock Domain	Description
rgmii_txd[3:0]	Out	tx_mac_aclk	Transmit data to PHY
rgmii_tx_ctl	Out	tx_mac_aclk	control signal to PHY
rgmii_txc	Out	-	Clock to PHY
rgmii_rxd[3:0]	In	rgmii_rxc	Received data from PHY

Table 21: Optional RGMII Interface Signal Pinout (cont'd)

Signal	Direction	Clock Domain	Description
rgmii_rx_ctl	In	rgmii_rxc	Control signal from PHY
rgmii_rxc	In	–	Clock from PHY
inband_link_status	Out	rgmii_rxc	Link Status from the PHY
inband_clock_speed	Out	rgmii_rxc	Link Speed from the PHY
inband_duplex_status	Out	rgmii_rxc	Duplex Status from the PHY

I/O Delay Calibration Ports for TEMAC RGMII Interface

The following ports are used to calibrate the IO Delays that the RGMII interface uses for Versal devices. Refer to [AR 000033647](#) to calibrate the values of these ports.

Table 22: IO Delay Calibration Ports for TEMAC RGMII Interface

Signal Name	Direction	Description
idelay_cnt_val[4:0]	In	CNTVALUEIN input of all the IDELAYS that are driven by rgmii_rxd[3:0] and rgmii_rx_ctl
idelay_load	In	LOAD input of all the IDELAYS that are driven by rgmii_rxd[3:0] and rgmii_rxc
odelay_cnt_val[4:0]	In	CNTVALUEIN input of the ODELAY that drives rgmii_tx_clk
odelay_load	In	LOAD input of the ODELAY that drives rgmii_tx_clk

Configuration Vector

If the optional management interface is omitted from the core, all of the relevant configuration signals are brought out of the core. These signals are bundled into the `rx_configuration_vector` and the `tx_configuration_vector` signals. The bit mapping of these signals is defined in the following tables.

You can permanently set the vector bits to logic 0 or 1 or change the configuration vector signals at any time. However, with the exception of the reset signals, they do not take effect until the current frame has completed transmission or reception.

Bits 367:80 of `tx_configuration_vector` [Bit Definitions](#) are only present if PFC has been enabled.

Bits 95:80 of `rx_configuration_vector` [Bit Definitions](#) are only present if PFC has been enabled.

Table 23: tx_configuration_vector Bit Definitions

Bits	Description
367:352	<i>Legacy Pause refresh value</i> . When the PFC feature is included, the 802.3 flow control logic also has the capability of being used as a XON/XOFF interface. If the pause_request input is asserted and held High, a pause frame is transmitted as normal and then refreshed when the internal quanta count reaches this value. When the pause request is de-asserted, an XON frame can be automatically sent if the TX Auto XON feature is enabled.
351:336	<i>TX Priority 7 Pause Quanta Refresh value</i> . This provides the quanta count value at which a new PFC frame is automatically generated if this priority is active and held High.
335:320	<i>TX Priority 7 Pause Quanta</i> . This provides the quanta value that is included in a transmitted PFC frame if this priority is enabled and asserted.
319:304	<i>TX Priority 6 Pause Quanta Refresh value</i> . This provides the quanta count value at which a new PFC frame is automatically generated if this priority is active and held High.
303:288	<i>TX Priority 6 Pause Quanta</i> . This provides the quanta value that is included in a transmitted PFC frame if this priority is enabled and asserted.
287:272	<i>TX Priority 5 Pause Quanta Refresh value</i> . This provides the quanta count value at which a new PFC frame is automatically generated if this priority is active and held High.
271:256	<i>TX Priority 5 Pause Quanta</i> . This provides the quanta value that is included in a transmitted PFC frame if this priority is enabled and asserted.
255:240	<i>TX Priority 4 Pause Quanta Refresh value</i> . This provides the quanta count value at which a new PFC frame is automatically generated if this priority is active and held High.
239:224	<i>TX Priority 4 Pause Quanta</i> . This provides the quanta value which is included in a transmitted PFC frame if this priority is enabled and asserted.
223:208	<i>TX Priority 3 Pause Quanta Refresh value</i> . This provides the quanta count value at which a new PFC frame is automatically generated if this priority is active and held High.
207:192	<i>TX Priority 3 Pause Quanta</i> . This provides the quanta value that is included in a transmitted PFC frame if this priority is enabled and asserted.
191:176	<i>TX Priority 2 Pause Quanta Refresh value</i> . This provides the quanta count value at which a new PFC frame is automatically generated if this priority is active and held High.
175:160	<i>TX Priority 2 Pause Quanta</i> . This provides the quanta value that is included in a transmitted PFC frame if this priority is enabled and asserted.
159:144	<i>TX Priority 1 Pause Quanta Refresh value</i> . This provides the quanta count value at which a new PFC frame is automatically generated if this priority is active and held High.
143:128	<i>TX Priority 1 Pause Quanta</i> . This provides the quanta value that is included in a transmitted PFC frame if this priority is enabled and asserted.
127:112	<i>TX Priority 0 Pause Quanta Refresh value</i> . This provides the quanta count value at which a new PFC frame is automatically generated if this priority is active and held High.
111:96	<i>TX Priority 0 Pause Quanta</i> . This provides the quanta value that is included in a transmitted PFC frame if this priority is enabled and asserted.
95	<i>TX Priority 7 Flow control enable</i> . If set, this enables the use of tx_pfc_p7_tvalid to generate PFC frames.
94	<i>TX Priority 6 Flow control enable</i> . If set, this enables the use of tx_pfc_p6_tvalid to generate PFC frames.

Table 23: tx_configuration_vector Bit Definitions (cont'd)

Bits	Description
93	<i>TX Priority 5 Flow control enable</i> . If set, this enables the use of tx_pfc_p5_tvalid to generate PFC frames.
92	<i>TX Priority 4 Flow control enable</i> . If set, this enables the use of tx_pfc_p4_tvalid to generate PFC frames.
91	<i>TX Priority 3 Flow control enable</i> . If set, this enables the use of tx_pfc_p3_tvalid to generate PFC frames.
90	<i>TX Priority 2 Flow control enable</i> . If set, this enables the use of tx_pfc_p2_tvalid to generate PFC frames.
89	<i>TX Priority 1 Flow control enable</i> . If set, this enables the use of tx_pfc_p1_tvalid to generate PFC frames.
88	<i>TX Priority 0 Flow control enable</i> . If set, this enables the use of tx_pfc_p0_tvalid to generate PFC frames.
87:82	Reserved
81	<i>Auto XON enable</i> . If set, the Ethernet MAC automatically generates a flow control frame with the relevant quanta set to zero when the associated tvalid or pause request is de-asserted (provided it has been asserted for more than one cycle.)
80	<i>Priority Flow Control Enable</i> . If set, this enables the TX PFC feature. This should not be set at the same time as the Transmit Flow control Enable defined in bit 5.
79:32	<i>Transmitter Pause Frame Source Address[47:0]</i> . This MAC Address is used by the MAC core as the source address for any outbound flow control frames only when the core is generated without management interface. The bits in this vector field are ordered so that the least significant bit of the MAC Address (IEEE802.3 definition) is stored in the least significant bit of this vector field. Consequently, Bit[0] of this field differentiates between an individual or group (multicast) address. The transmission order within a MAC frame is to send the least significant bit of the MAC Address first. Consequently, Bits[7:0] of this vector field represent the first byte to appear in frame transmission
31:16	<i>Transmitter Max Frame Size[15:0]</i> . This specifies the maximum frame size supported when Transmitter Max Frame Enable is set to 1 and Transmitter Jumbo Frame Enable is set to 0. This should always be set to 1518 or more.
15	Reserved
14	<i>Transmitter Max Frame Enable</i> . When this bit is set to 1 and Transmitter Jumbo Frame Enable is set to 0, the MAC transmitter allows frames larger than the maximum legal frame length specified in IEEE 802.3-2008 to be sent, provided they are smaller than the size specified in Transmitter Max Frame Length. This is described in See Maximum Permitted Frame Length . When set to 0, the MAC transmitter only allows frames up to the legal maximum to be sent.
13:12	<i>Transmitter Speed Configuration</i> 00 - 10 Mbps 01 - 100 Mbps 10 - 1 Gbps When the TEMAC solution is generated for 1 Gbps or 2.5 Gbps speed support, these inputs are unused. When the TEMAC solution is generated for 10 Mbps or 100 Mbps speed support, only Bit[12] is used to differentiate the speed: Bit[13] is unused. CAUTION! Issue the core with a system-wide reset following a speed change.
11:9	Reserved

Table 23: tx_configuration_vector Bit Definitions (cont'd)

Bits	Description
8	<i>Transmitter Interframe Gap Adjust Enable</i> . If 1, and the MAC is set to operate in full-duplex mode the transmitter reads the value of the tx_ifg_delay port and set the Interframe Gap accordingly. If 0, the transmitter always inserts at least the legal minimum interframe gap.
7	Reserved
6	<i>Transmitter Half-Duplex</i> . If 1, the transmitter operates in half-duplex mode. If 0, the transmitter operates in full-duplex mode. If the TEMAC solution has been generated without half-duplex support, this input to the core is unused.
5	<i>Transmitter Flow Control Enable</i> . When this bit is 1, asserting the pause_req signal causes the MAC core to send a flow control frame out of the transmitter as described in Transmitting a Pause Control Frame . when this bit is 0, asserting the pause_req signal has no effect.
4	<i>Transmitter Jumbo Frame Enable</i> . When this bit is 1, the MAC transmitter allows frames larger than the maximum legal frame length specified in IEEE 802.3-2008 to be sent. When set to 0, the maximum frame size is dependent upon the setting of Transmitter Max Frame Enable and Transmitter Max Frame Length.
3	<i>Transmitter In-Band FCS Enable</i> . When this bit is 1, the MAC transmitter expects the FCS field and any padding to take the frame up to 64 bytes to be passed by the user as described in User-Supplied FCS Passing . When it is 0, the MAC transmitter appends padding as required, compute the FCS and append it to the frame.
2	<i>Transmitter VLAN Enable</i> . When this bit is set to 1, the transmitter allows the transmission of VLAN tagged frames up to 1522 bytes in size.
1	<i>Transmitter Enable</i> . When this bit is set to 1, the transmitter is operational. When set to 0, the transmitter is disabled.
0	<i>Transmitter Reset</i> . When this bit is 1, the MAC transmitter is held in reset. This signal is an input to the reset circuit for the transmitter block.

Table 24: rx_configuration_vector Bit Definitions

Bits	Description
95	<i>RX Priority 7 Flow control enable</i> . If set, this allows received error free PFC frames with priority 7 enabled to assert the rx_pfc_p7_tvalid output for the requested duration. A new RX PFC frame can always refresh this or cancel.
94	<i>RX Priority 6 Flow control enable</i> . If set, this allows received error free PFC frames with priority 6 enabled to assert the rx_pfc_p6_tvalid output for the requested duration. A new RX PFC frame can always refresh this or cancel.
93	<i>RX Priority 5 Flow control enable</i> . If set, this allows received error free PFC frames with priority 5 enabled to assert the rx_pfc_p5_tvalid output for the requested duration. A new RX PFC frame can always refresh this or cancel.
92	<i>RX Priority 4 Flow control enable</i> . If set, this allows received error free PFC frames with priority 4 enabled to assert the rx_pfc_p4_tvalid output for the requested duration. A new RX PFC frame can always refresh this or cancel.
91	<i>RX Priority 3 Flow control enable</i> . If set, this allows received error free PFC frames with priority 3 enabled to assert the rx_pfc_p3_tvalid output for the requested duration. A new RX PFC frame can always refresh this or cancel.
90	<i>RX Priority 2 Flow control enable</i> . If set, this allows received error free PFC frames with priority 2 enabled to assert the rx_pfc_p2_tvalid output for the requested duration. A new RX PFC frame can always refresh this or cancel.

Table 24: rx_configuration_vector Bit Definitions (cont'd)

Bits	Description
89	<i>RX Priority 1 Flow control enable</i> . If set, this allows received error free PFC frames with priority 1 enabled to assert the rx_pfc_p1_tvalid output for the requested duration. A new RX PFC frame can always refresh this or cancel.
88	<i>RX Priority 0 Flow control enable</i> . If set, this allows received error free PFC frames with priority 0 enabled to assert the rx_pfc_p0_tvalid output for the requested duration. A new RX PFC frame can always refresh this or cancel.
87:81	Reserved
80	<i>Priority Flow Control Enable</i> . If set, this enables the RX PFC feature and any received PFCframes is marked as bad at the client interface. If set to 0 then PFC frames are ignored and marked as good at the client interface. This should not be set at the same time as the Receive Flow control Enable defined in bit 5.
79:32	<i>Receiver Pause Frame Source Address[47:0]</i> . This MAC Address is used by the MAC core to match against the destination address of any incoming flow control only when the core is generated without management interface. The bits in this vector field are ordered so that the least significant bit of the MAC Address (IEEE802.3 definition) is stored in the least significant bit of this vector field. Consequently, Bit[0] of this field differentiates between an individual or group (multicast) address. The reception order within a MAC frame is to receive the least significant bit of the MAC Address first. Consequently, Bits[7:0] of this vector field represent the first byte to appear in frame reception.
31:16	<i>Receiver Max Frame Size[15:0]</i> . This specifies the maximum frame size supported when Receiver Max Frame Enable is set to 1 and Receiver Jumbo Frame Enable is set to 0. This should always be set to 1518 or more.
15	Reserved
14	<i>Receiver Max Frame Enable</i> . When this bit is set to 1 and Receiver Jumbo Frame Enable is set to 0, the MAC receiver passes frames larger than the maximum legal frame length specified in IEEE 802.3-2008, provided they are smaller than the size specified in Receiver Max Frame Length. This is described in Maximum Permitted Frame Length . When set to 0, the MAC receiver only passes frames up to the legal maximum.
13:12	<i>Receiver Speed Configuration</i> 00 = 10 Mbps 01 = 100 Mbps 10 = 1 Gbps When the TEMAC solution is generated for 1 Gbps or 2.5 Gbps speed support, these inputs are unused. When the TEMAC solution is generated for 10 Mbps or 100 Mbps speed support, only Bit[12] is used to differentiate the speed: Bit[13] is unused. Note: Issue the core with a system-wide reset following a speed change.
11	Promiscuous Mode: When this bit is set to 1, the frame filter is set to operate in promiscuous mode. All frames are passed to the receiver client regardless of the destination address.
10	Reserved
9	Receiver Control Frame Length Check Disable When this bit is set so 1, the core does not mark control frames as bad if they are greater than the minimum frame length.
8	Receiver Length/Type Error Check Disable When this bit is 1, the core does not perform the length/type field error checks as described in Length/Type Field Error Checks . When it is set to 0, the length/type field checks are performed; this is normal operation.

Table 24: rx_configuration_vector Bit Definitions (cont'd)

Bits	Description
7	Reserved
6	Receiver Half-Duplex If 1, the receiver operates in half-duplex mode. If 0, the receiver operates in full-duplex mode. If the TEMAC has been generated without half-duplex support then this input to the core is unused.
5	Receiver Flow Control Enable. When this bit is 1, received flow control frames inhibit the transmitter operation as described in Receiving a Pause Control Frame . When it is 0, received flow frames are passed up to the user.
4	Receiver Jumbo Frame Enable. When this bit is 1, the MAC receiver passes frames larger than the maximum legal frame length specified in IEEE 802.3-2008. When set to 0, the maximum frame size is dependent on the setting of Receiver Max Frame Enable and Receiver Max Frame Length.
3	Receiver In-Band FCS Enable. When this bit is 1, the MAC receiver pass the FCS field to the user as described in User-Supplied FCS Passing . When it is 0, the MAC receiver does not pass the FCS field. In both cases, the FCS field is verified on the frame.
2	Receiver VLAN Enable. When this bit is set to 1, the receiver allows the reception of VLAN tagged frames up to 1,522 bytes in size.
1	Receiver Enable. When this bit is set to 1, the receiver is operational. When set to 0, the receiver is disabled.
0	Receiver Reset. When this bit is 1, the MAC receiver is held in reset. This signal is an input to the reset circuit for the receiver block.

Register Space

When the core is generated with a management interface, all control and Status registers are memory mapped. If no management interface is used, the key core parameters can be controlled through the configuration vectors as defined in [Configuration Vector Signal Definition](#) and [I/O Delay Calibration Ports for TEMAC RGMII Interface](#). After power-up or reset, you can reconfigure the core parameters from their defaults, such as flow control support. Configuration changes can be made at any time. Both the receiver and transmitter logic only sample configuration changes at the start of frame transmission/reception. The exceptions to this are the configurable resets that take effect immediately.

The AXI4-Lite register information such as register address, register name with bit position, mask value, access type, and their default values are provided in header (. h) file format. When the core is generated with the AXI4-Lite Management Interface, this file is present in the `header_files` folder. The following table shows the core register map.

Table 25: Core Registers

Address (Hex)	Description
0x000-0x1FC	Reserved

Table 25: Core Registers (cont'd)

Address (Hex)	Description
0x200	Received Bytes Counter Word 0
0x204	Received Bytes Counter Word 1 (if 64-bit width)
0x208	Transmitted Bytes Counter Word 0
0x20C	Transmitted Bytes Counter Word 1 (if 64-bit width)
0x210	Undersize Frames Counter Word 0
0x214	Undersize Frames Counter Word 1 (if 64-bit width)
0x218	Fragment Frames Counter Word 0
0x21C	Fragment Frames Counter Word 1 (if 64-bit width)
0x220	RX 64-Byte Frames Counter Word 0
0x224	RX 64-Byte Frames Counter Word 1 (if 64-bit width)
0x228	RX 65-127-Byte Frames Counter Word 0
0x22C	RX 65-127-Byte Frames Counter Word 1 (if 64-bit width)
0x230	RX 128-255-Byte Frames Counter Word 0
0x234	RX 128-255-Byte Frames Counter Word 1 (if 64-bit width)
0x238	RX 256-511-Byte Frames Counter Word 0
0x23C	RX 256-511-Byte Frames Counter Word 1 (if 64-bit width)
0x240	RX 512-1023-Byte Frames Counter Word 0
0x244	RX 512-1023-Byte Frames Counter Word 1 (if 64-bit width)
0x248	RX 1024-Max Frames Size Byte Frames Counter Word 0
0x24C	RX 1024-Max Frames Size Byte Frames Counter Word 1 (if 64-bit width)
0x250	RX Oversize Frames Counter Word 0
0x254	RX Oversize Frames Counter Word 1 (if 64-bit width)
0x258	TX 64-Byte Frames Counter Word 0
0x25C	TX 64-Byte Frames Counter Word 1 (if 64-bit width)
0x260	TX 65-127-Byte Frames Counter Word 0
0x264	TX 65-127-Byte Frames Counter Word 1 (if 64-bit width)
0x268	TX 128-255-Byte Frames Counter Word 0
0x26C	TX 128-255-Byte Frames Counter Word 1 (if 64-bit width)
0x270	TX 256-511-Byte Frames Counter Word 0
0x274	TX 256-511-Byte Frames Counter Word 1 (if 64-bit width)
0x278	TX 512-1023-Byte Frames Counter Word 0
0x27C	TX 512-1023-Byte Frames Counter Word 1 (if 64-bit width)
0x280	TX 1024-Max Frames Size Byte Frames Counter Word 0
0x284	TX 1024-Max Frames Size Byte Frames Counter Word 1 (if 64-bit width)
0x288	TX Oversize Frames Counter Word 0
0x28C	TX Oversize Frames Counter Word 1 (if 64-bit width)
0x290	RX Good Frames Counter Word 0
0x294	RX Good Frames Counter Word 1 (if 64-bit width)
0x298	RX Frame Check Sequence Errors Counter Word 0

Table 25: Core Registers (cont'd)

Address (Hex)	Description
0x29C	RX Frame Check Sequence Errors Counter Word 1 (if 64-bit width)
0x2A0	RX Good Broadcast Frames Counter Word 0
0x2A4	RX Good Broadcast Frames Counter Word 1 (if 64-bit width)
0x2A8	RX Good Multicast Frames Counter Word 0
0x2AC	RX Good Multicast Frames Counter Word 1 (if 64-bit width)
0x2B0	RX Good Control Frames Counter Word 0
0x2B4	RX Good Control Frames Counter Word 1 (if 64-bit width)
0x2B8	RX Length/Type Out of Range Errors Counter Word 0
0x2BC	RX Length/Type Out of Range Errors Counter Word 1 (if 64-bit width)
0x2C0	RX Good VLAN Tagged Frames Counter Word 0
0x2C4	RX Good VLAN Tagged Frames Counter Word 1 (if 64-bit width)
0x2C8	RX Good Pause Frames Counter Word 0
0x2CC	RX Good Pause Frames Counter Word 1 (if 64-bit width)
0x2D0	RX Bad Opcode Frames Counter Word 0
0x2D4	RX Bad Opcode Frames Counter Word 1 (if 64-bit width)
0x2D8	TX Good Frames Counter Word 0
0x2DC	TX Good Frames Counter Word 1 (if 64-bit width)
0x2E0	TX Good Broadcast Frames Counter Word 0
0x2E4	TX Good Broadcast Frames Counter Word 1 (if 64-bit width)
0x2E8	TX Good Multicast Frames Counter Word 0
0x2EC	TX Good Multicast Frames Counter Word 1 (if 64-bit width)
0x2F0	TX Underrun Errors Counter Word 0
0x2F4	TX Underrun Errors Counter Word 1 (if 64-bit width)
0x2F8	TX Good Control Frames Counter Word 0
0x2FC	TX Good Control Frames Counter Word 1 (if 64-bit width)
0x300	TX Good VLAN Frames Counter Word 0
0x304	TX Good VLAN Frames Counter Word 1 (if 64-bit width)
0x308	TX Good Pause Frames Counter Word 0
0x30C	TX Good Pause Frames Counter Word 1 (if 64-bit width)
0x310	TX Single Collision Frames Counter Word 0
0x314	TX Single Collision Frames Counter Word 1 (if 64-bit width)
0x318	TX Multiple Collision Frames Counter Word 0
0x31C	TX Multiple Collision Frames Counter Word 1 (if 64-bit width)
0x320	TX Deferred Frames Counter Word 0
0x324	TX Deferred Frames Counter Word 1 (if 64-bit width)
0x328	TX Late Collision Counter Word 0
0x32C	TX Late Collision Counter Word 1 (if 64-bit width)
0x330	TX Excess Collision Counter Word 0
0x334	TX Excess Collision Counter Word 1 (if 64-bit width)

Table 25: Core Registers (cont'd)

Address (Hex)	Description
0x338	TX Excess Deferral Counter Word 0
0x33C	TX Excess Deferral Counter Word 1 (if 64-bit width)
0x340	RX Alignment Errors Counter Word 0
0x344	RX Alignment Errors Counter Word 1 (if 64-bit width)
0x348	TX PFC Frames Counter Word 0
0x34C	TX PFC Frames Counter Word 1 (if 64-bit width)
0x350	RX PFC Frames Counter Word 0
0x354	RX PFC Frames Counter Word 1 (if 64-bit width)
0x358-0x364	User Defined Statistics Counters (if present)
0x368-0x3FC	Reserved
0x400	Receiver Configuration Word 0
0x404	Receiver Configuration Word 1
0x408	Transmitter Configuration
0x40C	Flow Control Configuration
0x410	Speed Configuration
0x414	RX Max Frame Configuration
0x418	TX Max Frame Configuration
0x41C-0x47C	Reserved
0x480	Priority 0 Quanta Register
0x484	Priority 1 Quanta Register
0x488	Priority 2 Quanta Register
0x48C	Priority 3 Quanta Register
0x490	Priority 4 Quanta Register
0x494	Priority 5 Quanta Register
0x498	Priority 6 Quanta Register
0x49C	Priority 7 Quanta Register
0x4A0	Legacy Pause Refresh Register
0x4A4-0x4F4	Reserved
0x4F8	ID Register
0x4FC	Ability Register
0x500	MDIO Setup
0x504	MDIO Control
0x508	MDIO Write Data
0x50C	MDIO Read Data
0x510-0x5FC	Reserved
0x600	Interrupt Status Register
0x604-0x60C	Reserved
0x610	Interrupt Pending Register
0x614-0x61C	Reserved

Table 25: Core Registers (cont'd)

Address (Hex)	Description
0x620	Interrupt Enable Register
0x624-0x62C	Reserved
0x630	Interrupt Clear Register
0x634-0x6FC	Reserved
0x700	Unicast Address Word 0
0x704	Unicast Address Word 1
0x708	Frame Filter Control
0x70C	Frame Filter Enable
0x710	Frame Filter Value Bytes 3-0
0x714	Frame Filter Value Bytes 7-4
0x718	Frame Filter Value Bytes 11-8
0x71C	Frame Filter Value Bytes 15-12
0x720	Frame Filter Value Bytes 19-16
0x724	Frame Filter Value Bytes 23-20
0x728	Frame Filter Value Bytes 27-24
0x72C	Frame Filter Value Bytes 31-28
0x730	Frame Filter Value Bytes 35-32
0x734	Frame Filter Value Bytes 39-36
0x738	Frame Filter Value Bytes 43-40
0x73C	Frame Filter Value Bytes 47-44
0x740	Frame Filter Value Bytes 51-48
0x744	Frame Filter Value Bytes 55-52
0x748	Frame Filter Value Bytes 59-56
0x74C	Frame Filter Value Bytes 63-60
0x750	Frame Filter Mask Value Bytes 3-0
0x754	Frame Filter Mask Value Bytes 7-4
0x758	Frame Filter Mask Value Bytes 11-8
0x75C	Frame Filter Mask Value Bytes 15-12
0x760	Frame Filter Mask Value Bytes 19-16
0x764	Frame Filter Mask Value Bytes 23-20
0x768	Frame Filter Mask Value Bytes 27-24
0x76C	Frame Filter Mask Value Bytes 31-28
0x770	Frame Filter Mask Value Bytes 35-32
0x774	Frame Filter Mask Value Bytes 39-36
0x778	Frame Filter Mask Value Bytes 43-40
0x77C	Frame Filter Mask Value Bytes 47-44
0x780	Frame Filter Mask Value Bytes 51-48
0x784	Frame Filter Mask Value Bytes 55-52
0x788	Frame Filter Mask Value Bytes 59-56

Table 25: Core Registers (cont'd)

Address (Hex)	Description
0x78C	Frame Filter Mask Value Bytes 63-60
0x790-0x7FC	Reserved
0x800-0xFFFC	Reserved
0x10000-0x100FC	RX PTP Buffer 0
0x10100-0x101FC	RX PTP Buffer 1
0x10200-0x102FC	RX PTP Buffer 2
0x10300-0x103FC	RX PTP Buffer 3
0x10400-0x104FC	RX PTP Buffer 4
0x10500-0x105FC	RX PTP Buffer 5
0x10600-0x106FC	RX PTP Buffer 6
0x10700-0x107FC	RX PTP Buffer 7
0x10800-0x108FC	RX PTP Buffer 8
0x10900-0x109FC	RX PTP Buffer 9
0x10A00-0x10AFC	RX PTP Buffer 10
0x10B00-0x10BFC	RX PTP Buffer 11
0x10C00-0x10CFC	RX PTP Buffer 12
0x10D00-0x10DFC	RX PTP Buffer 13
0x10E00-0x10EFC	RX PTP Buffer 14
0x10F00-0x10FFC	RX PTP Buffer 15
0x11000-0x110FC	TX PTP Buffer 0
0x11100-0x111FC	TX PTP Buffer 1
0x11200-0x112FC	TX PTP Buffer 2
0x11300-0x113FC	TX PTP Buffer 3
0x11400-0x114FC	TX PTP Buffer 4
0x11500-0x115FC	TX PTP Buffer 5
0x11600-0x116FC	TX PTP Buffer 6
0x11700-0x117FC	TX PTP Buffer 7
0x11800-0x11FFC	Reserved
0x12000	TX PTP Packet Buffer Control Register
0x12004	RX PTP Packet Control Register
0x12008	Reserved
0x1200C	TX Arbiter Send Slope Control Register
0x12010	TX Arbiter Idle Slope Control Register
0x12014-0x127FC	Reserved
0x12800	RTC Nanoseconds Field Offset
0x12804	Reserved
0x12808	RTC Seconds Field Offset[31:0]
0x1280C	RTC Seconds Field Offset[47:32]
0x12810	RTC Increment Value Control Register

Table 25: Core Registers (cont'd)

Address (Hex)	Description
0x12814	Current RTC Nanoseconds Value
0x12818	Current RTC Seconds Value Bits[31:0]
0x1281C	Current RTC Seconds Value Bits[47:32]
0x12820	RTC Interrupt Clear Register
0x12824	RTC Phase Adjustment Register
0x12828-0x13FFC	Reserved

Statistics Counters

The Statistics counters are defined to be either 32 or 64 bits wide with 64 bits set as the default. When defined as 64 bits wide, the counter values are captured across two registers. In all cases, a read of the lower 32-bit value causes the upper 32 bits to be sampled. A subsequent read of the upper 32-bit location returns this sampled value.

Note: If a different upper 32-bit location is read, an error is returned.

Table 26: Statistics Counter Definitions

Name	Increment Bit No.	Address	Description
Received bytes	NA	0x200-0x204	A count of bytes of frames received (destination address to frame check sequence inclusive).
Transmitted bytes	NA	0x208-0x20C	A count of bytes of frames transmitted (destination address to frame check sequence inclusive).
RX Undersize frames	NA	0x210-0x214	A count of the number of frames received that were fewer than 64 bytes in length but otherwise well formed.
RX Fragment frames	NA	0x218-0x21C	A count of the number of frames received that were fewer than 64 bytes in length and had a bad frame check sequence field.
RX 64-byte Frames	4	0x220-0x224	A count of error-free frames received 64 bytes in length.
RX 65-127-byte Frames	5	0x228-0x22C	A count of error-free frames received between 65 and 127 bytes in length.
RX 128-255-byte Frames	6	0x230-0x234	A count of error-free frames received between 128 and 255 bytes in length.
RX 256-511-byte Frames	7	0x238-0x23C	A count of error-free frames received between 256 and 511 bytes in length.
RX 512-1023-byte Frames	8	0x240-0x244	A count of error-free frames received between 512 and 1023 bytes in length.
RX 1024-MaxFrameSize byte Frames	9	0x248-0x24C	A count of error-free frames received between 1024 bytes and the specified IEEE 802.3-2008 maximum legal length.

Table 26: Statistics Counter Definitions (cont'd)

Name	Increment Bit No.	Address	Description
RX Oversize Frames	10	0x250-0x254	A count of otherwise error-free frames received that exceeded the maximum legal frame length specified in IEEE 802.3-2008. If Jumbo frames are enabled this counter increments when the received frame length exceeds the maximum legal length. If Jumbo frames are not enabled, it increments only if the received frame length is greater than the maximum legal length and less than or equal to the value specified in RX Maximum Frame Configuration Register
TX 64-byte Frames	11	0x258-0x25C	A count of error-free frames transmitted that were 64 bytes in length.
TX 65-127-byte Frames	12	0x260-0x264	A count of error-free frames transmitted between 65 and 127 bytes in length.
TX 128-255-byte Frames	13	0x268-0x26C	A count of error-free frames transmitted between 128 and 255 bytes in length.
TX 256-511-byte Frames	14	0x270-0x274	A count of error-free frames transmitted between 256 and 511 bytes in length.
TX 512-1023-byte Frames	15	0x278-0x27C	A count of error-free frames transmitted that were between 512 and 1023 bytes in length.
TX 1024-MaxFrameSize byte Frames	16	0x280-0x284	A count of error-free frames transmitted between 1024 and the specified IEEE 802.3-2008 maximum legal length.
TX Oversize Frames	17	0x288-0x28C	A count of otherwise error-free frames transmitted that exceeded the maximum legal frame length specified in IEEE 802.3-2008.
RX Good Frames	18	0x290-0x294	A count of error-free frames received.
RX Frame Check Sequence Errors	19	0x298-0x29C	A count of received frames that failed the CRC check and were at least 64 bytes in length.
RX Good Broadcast Frames	20	0x2A0-0x2A4	A count of frames successfully received and directed to the broadcast group address.
RX Good Multicast Frames	21	0x2A8-0x2AC	A count of frames successfully received and directed to a non-broadcast group address.
RX Good Control Frames	22	0x2B0-0x2B4	A count of error-free frames received that contained the special control frame identifier in the length/type field.
RX Length/Type Out of Range	23	0x2B8-0x2BC	A count of frames received that were at least 64 bytes in length where the length/type field contained a length value that did not match the number of MAC user data bytes received. The counter also increments for frames in which the length/type field indicated that the frame contained padding but where the number of MAC user data bytes received was greater than 64 bytes (minimum frame size). The exception is when the Length/Type Error Checks are disabled in the chosen MAC, in this case, the counter does not increment.
RX Good VLAN Tagged Frames	24	0x2C0-0x2C4	A count of error-free VLAN frames received. This counter only increments when the receiver is configured for VLAN operation.
RX Good Pause Frames	25	0x2C8-0x2CC	A count of error-free frames received that contained the MAC Control type identifier 88-08 in the length/type field, contained a destination address that matches either the MAC Control multicast address or the configured source address of the MAC, the PAUSE opcode, and were acted upon by the MAC.

Table 26: Statistics Counter Definitions (cont'd)

Name	Increment Bit No.	Address	Description
RX Bad Opcode	26	0x2D0-0x2D4	A count of error-free frames received that contained the MAC Control type identifier 88-08 in the Length/Type field but were received with an opcode other than the PAUSE (legacy) opcode or the Priority Flow Control opcode (when the core is generated with Priority Flow Control support).
TX Good Frames	27	0x2D8-0x2DC	A count of error-free frames transmitted.
TX Good Broadcast Frames	28	0x2E0-0x2E4	A count of error-free frames that were transmitted to the broadcast address.
TX Good Multicast Frames	29	0x2E8-0x2EC	A count of error-free frames that were transmitted to a group destination address other than broadcast.
TX Good Underrun Errors	30	0x2F0-0x2F4	A count of frames that would otherwise be transmitted by the core but could not be completed due to the assertion of TX_UNDERRUN during the frame transmission.
TX Good Control Frames	31	0x2F8-0x2FC	A count of error-free frames transmitted that contained the MAC Control Frame type identifier 88-08 in the length/type field.
TX Good VLAN Tagged Frames	32	0x300-0x304	A count of error-free VLAN frames transmitted. This counter only increments when the transmitter is configured for VLAN operation.
TX Good Pause Frames	33	0x308-0x30C	A count of error-free PAUSE frames generated and transmitted by the MAC in response to an assertion of pause_req.
TX Single Collision Frames	34	0x310-0x314	A count of frames involved in a single collision but subsequently transmitted successfully (half-duplex mode only).
TX Multiple Collision Frames	35	0x318-0x31C	A count of frames involved in more than one collision but subsequently transmitted successfully (half-duplex mode only).
TX Deferred	36	0x320-0x324	A count of frames whose transmission was delayed on its first attempt because the medium was busy (half-duplex mode only).
TX Late Collisions	37	0x328-0x32C	A count of the times that a collision has been detected later than one slot Time from the start of the packet transmission. A late collision is counted twice—both as a collision and as a late Collision (half-duplex mode only).
TX Excess Collisions	38	0x330-0x334	A count of the frames that, due to excessive collisions, are not transmitted successfully (half-duplex mode only).
TX Excess Deferral	39	0x338-0x33C	A count of frames that deferred transmission for an excessive period of time (half-duplex mode only).
RX Alignment Errors	40	0x340-0x344	Asserted for received frames of size 64-bytes and greater which contained an odd number of received nibbles and which also contained an invalid FCS field (half-duplex mode only).
TX Good PFC Frames	41	0x348-0x34C	A count of error-free frames transmitted that contained the MAC Control Frame type identifier 88-08 in the length/type field and PFC opcode 01-01.
RX Good PFC Frames	42	0x350-0x354	A count of error-free frames received that contained the MAC Control type identifier 88-08 in the length/type field, a destination address that matched either the MAC Control multicast address or the configured source address of the MAC, the PFC opcode, and were acted upon by the MAC.

Notes:

1. All bits are read-only.

MAC Configuration Registers

Configuration of the MAC core is performed using a register bank accessed through the management interface. The configuration registers available in the core are detailed in the following table.

Table 27: Configuration Registers

Address (Hex)	Description
0x400	Table 28: Receiver Configuration Word 0 (0x400)
0x404	Table 29: Receiver Configuration Word 1 (0x404)
0x408	Table 30: Transmitter Configuration Word (0x408)
0x40C	Table 31: Flow Control Configuration Word (0x40C)
0x410	Table 32: MAC Speed Configuration Word (0x410)
0x414	Table 33: RX Max Frame Configuration Word (0x414)
0x418	Table 34: TX Max Frame Configuration Word (0x418)
0x41C-0x47F	Reserved
0x480	Priority 0 Quanta Register
0x484	Priority 1 Quanta Register
0x488	Priority 2 Quanta Register
0x48c	Priority 3 Quanta Register
0x490	Priority 4 Quanta Register
0x494	Priority 5 Quanta Register
0x498	Priority 6 Quanta Register
0x49c	Priority 7 Quanta Register
0x4a0	Legacy Pause Refresh Register
0x4A4-0x4F4	Reserved
0x4F8	ID Register (0x4F8)
0x4FC	Table 38: Ability Register (0x4FC)

The contents of each configuration register are shown in the following tables.

Table 28: Receiver Configuration Word 0 (0x400)

Bits	Default Value	Type	Description
31:0	All 1s	R/W ¹	Pause frame MAC Source Address[31:0]: This address is used by the MAC to match against the destination address of any incoming flow control frames. It is also used by the flow control block as the source address (SA) for any outbound flow control frames. The address is ordered so the first byte transmitted/received is the lowest positioned byte in the register; for example, a MAC address of AA-BB-CC-DD-EE-FF would be stored in Address[47:0] as 0xFFEEDDCCBBAA.

Notes:

1. Read/Write.

Table 29: Receiver Configuration Word 1 (0x404)

Bits	Default Value	Type	Description
31	0	R/W	Reset: When this bit is set to 1, the receiver is reset. The bit then automatically reverts to 0. This reset also sets all of the receiver configuration registers to their default values.
30	0	R/W	Jumbo Frame Enable: When this bit is set to 1, the MAC receiver accepts frames over the specified IEEE 802.3-2008 maximum legal length. When this bit is 0, the MAC only accepts frames up to the specified maximum length.
29	0	R/W	In-band FCS Enable: When this bit is 1, the MAC receiver passes the FCS field up to the client as described in User-Supplied FCS Passing . When it is 0, the client is not passed to the FCS. In both cases, the FCS is verified on the frame.
28	1	R/W	Receiver Enable: If set to 1, the receiver block is operational. If set to 0, the block ignores activity on the physical interface RX port.
27	0	R/W	VLAN Enable: When this bit is set to 1, VLAN tagged frames are accepted by the receiver.
26	0	R/W	Half Duplex: If set to 1, the receiver operates in half-duplex mode. If 0, the receiver operates in full-duplex mode.
25	0	R/W	Length/Type Error Check Disable: When this bit is set to 1, the core does not perform the length/type field error checks as described in Length/Type Field Error Checks . When this bit is set to 0, the length/type field checks is performed: this is normal operation.
24	0	R/W	Control Frame Length Check Disable: When this bit is set to 1, the core does not mark control frames as bad if they are greater than the minimum frame length.
23:16	N/A	RO ¹	Reserved
15:0	All 1s	R/W	Pause frame MAC Source Address[47:32]: See description in the previous table.

Notes:

1. Read Only.

Table 30: Transmitter Configuration Word (0x408)

Bits	Default Value	Type	Description
31	0	R/W	Reset: When this bit is set to 1, the transmitter is reset. The bit then automatically reverts to 0. This reset also sets all of the transmitter configuration registers to their default values.
30	0	R/W	Jumbo Frame Enable: When this bit is set to 1, the MAC transmitter sends frames that are greater than the specified IEEE 802.3-2008 maximum legal length. When this bit is 0, the MAC only sends frames up to the specified maximum.
29	0	R/W	In-band FCS Enable: When this bit is 1, the MAC transmitter expects the FCS field to be passed by the client as described in User-Supplied FCS Passing . When this bit is 0, the MAC transmitter appends padding as required, computes the FCS and appends it to the frame.
28	1	R/W	Transmit Enable: When this bit is 1, the transmitter is operational. When it is 0, the transmitter is disabled.
27	0	R/W	VLAN Enable: When this bit is set to 1, the transmitter recognizes the transmission of VLAN tagged frames.

Table 30: Transmitter Configuration Word (0x408) (cont'd)

Bits	Default Value	Type	Description
26	0	R/W	Half Duplex: If 1, the transmitter operates in half-duplex mode.
25	0	R/W	Interframe Gap Adjust Enable: If 1, the transmitter reads the value on the port tx_ifg_delay at the start of frame transmission and adjusts the interframe gap following the frame accordingly (see Interframe Gap Adjustment – Full-Duplex Mode Only). If 0, the transmitter outputs a minimum interframe gap of at least twelve clock cycles, as specified in IEEE 802.3-2008.
24:0	N/A	RO	Reserved

Table 31: Flow Control Configuration Word (0x40C)

Bits	Default Value	Type	Description
31	N/A	RO	Reserved
30	1	R/W	Flow Control Enable (TX) : When this bit is 1, asserting the pause_req signal sends a flow control frame out of the transmitter as described in Transmitting a Pause Control Frame. When this bit is 0, asserting the pause_req signal has no effect. This mode should not be enabled at the same time as PFC (Bit[26]).
29	1	R/W	Flow Control Enable (RX) . When this bit is 1, received flow control frames inhibit the transmitter operation as described in Receiving a Pause Frame. When this bit is 0, received flow control frames are always passed up to the client. This mode should not be enabled at the same time as PFC (Bit[25]).
28:27	N/A	RO	Reserved
26	0	RW	Priority pause flow control enable (TX) . Only present when the core has been generated with PFC support. When this bit is 1, asserting an enabled TX PFC tvalid signal results in a PFC frame being sent from the transmitter. When this bit is 0, the TX PFC tvalid inputs are ignored. This mode should not be enabled at the same time as Flow Control (TX) (Bit[30]).
25	0	RW	Priority pause flow control enable (RX) . Only present when the core has been generated with PFC support. When this bit is 1, received PFC frames assert the relevant enabled RX PFC tvalid outputs as described in Receiving a PFC Frame. When this bit is 0, received PFC frames are ignored and passed to the client. This mode should not be enabled at the same time as Flow Control (RX) (Bit[29]).
24:21	N/A	RO	Reserved
20	1	RW	TX Auto XON . Only present when the core has been generated with PFC support – this bit defaults to 0 if PFC is not supported. Send a flow control or PFC frame with the relevant quanta set to zero (XON frame) when the relevant, enabled pause request is dropped.
19:16	N/A	RO	Reserved
15	1	R/W	TX Priority 7 pause enable . Only present when the core has been generated with PFC support- this bit defaults to 0 if PFC is not supported. When this bit is 1, and TX PFC is enabled, assertion or de-assertion of the TX PFC tvalid signal results in a PFC frame being transmitted. When this bit is 0, tx_pfc_p7_tvalid is ignored.

Table 31: Flow Control Configuration Word (0x40C) (cont'd)

Bits	Default Value	Type	Description
14	1	R/W	TX Priority 6 pause enable . Only present when the core has been generated with PFC support- this bit defaults to 0 if PFC is not supported. When this bit is 1, and TX PFC is enabled, assertion or de-assertion of the TX PFC tvalid signal results in a PFC frame being transmitted. When this bit is 0, tx_pfc_p6_tvalid is ignored.
13	1	R/W	TX Priority 5 pause enable. Only present when the core has been generated with PFC support- this bit defaults to 0 if PFC is not supported. When this bit is 1, and TX PFC is enabled, assertion or de-assertion of the TX PFC tvalid signal results in a PFC frame being transmitted. When this bit is 0, tx_pfc_p5_tvalid is ignored.
12	1	R/W	TX Priority 4 pause enable . Only present when the core has been generated with PFC support- this bit defaults to 0 if PFC is not supported. When this bit is 1, and TX PFC is enabled, assertion or de-assertion of the TX PFC tvalid signal results in a PFC frame being transmitted. When this bit is 0, tx_pfc_p4_tvalid is ignored.
11	1	R/W	TX Priority 3 pause enable . Only present when the core has been generated with PFC support- this bit defaults to 0 if PFC is not supported. When this bit is 1, and TX PFC is enabled, assertion or de-assertion of the TX PFC tvalid signal results in a PFC frame being transmitted. When this bit is 0, tx_pfc_p3_tvalid is ignored.
10	1	R/W	TX Priority 2 pause enable . Only present when the core has been generated with PFC support- this bit defaults to 0 if PFC is not supported. When this bit is 1, and TX PFC is enabled, assertion or de-assertion of the TX PFC tvalid signal results in a PFC frame being transmitted. When this bit is 0, tx_pfc_p2_tvalid is ignored.
9	1	R/W	TX Priority 1 pause enable. Only present when the core has been generated with PFC support- this bit defaults to 0 if PFC is not supported. When this bit is 1, and TX PFC is enabled, assertion or de-assertion of the TX PFC tvalid signal results in a PFC frame being transmitted. When this bit is 0, tx_pfc_p1_tvalid is ignored.
8	1	R/W	TX Priority 0 pause enable . Only present when the core has been generated with PFC support- this bit defaults to 0 if PFC is not supported. When this bit is 1, and TX PFC is enabled, assertion or de-assertion of the TX PFC tvalid signal results in a PFC frame being transmitted. When this bit is 0, tx_pfc_p0_tvalid is ignored.
7	1	R/W	RX Priority 7 pause enable. Only present when the core has been generated with PFC support- this bit defaults to 0 if PFC is not supported. When this bit is 1 and RX PFC is enabled, reception of a PFC frame with a valid quanta for priority 7 is processed as described in Receiving a PFC Frame. When this bit is 0, the rx_pfc_p7_tvalid remains at 0.
6	1	R/W	RX Priority 6 pause enable. Only present when the core has been generated with PFC support- this bit defaults to 0 if PFC is not supported. When this bit is 1 and RX PFC is enabled, reception of a PFC frame with a valid quanta for priority 6 is processed as described in Receiving a PFC Frame. When this bit is 0, the rx_pfc_p6_tvalid remains at 0.
5	1	R/W	RX Priority 5 pause enable. Only present when the core has been generated with PFC support- this bit defaults to 0 if PFC is not supported. When this bit is 1 and RX PFC is enabled, reception of a PFC frame with a valid quanta for priority 5 is processed as described in Receiving a PFC Frame. When this bit is 0, the rx_pfc_p5_tvalid remains at 0.

Table 31: Flow Control Configuration Word (0x40C) (cont'd)

Bits	Default Value	Type	Description
4	1	R/W	RX Priority 4 pause enable. Only present when the core has been generated with PFC support- this bit defaults to 0 if PFC is not supported. When this bit is 1 and RX PFC is enabled, reception of a PFC frame with a valid quanta for priority 4 is processed as described in Receiving a PFC Frame. When this bit is 0, the rx_pfc_p4_tvalid remains at 0.
3	1	R/W	RX Priority 3 pause enable. Only present when the core has been generated with PFC support- this bit defaults to 0 if PFC is not supported. When this bit is 1 and RX PFC is enabled, reception of a PFC frame with a valid quanta for priority 3 is processed as described in Receiving a PFC Frame. When this bit is 0, the rx_pfc_p3_tvalid remains at 0.
2	1	R/W	RX Priority 2 pause enable. Only present when the core has been generated with PFC support- this bit defaults to 0 if PFC is not supported. When this bit is 1 and RX PFC is enabled, reception of a PFC frame with a valid quanta for priority 2 is processed as described in Receiving a PFC Frame. When this bit is 0, the rx_pfc_p2_tvalid remains at 0.
1	1	R/W	RX Priority 1 pause enable. Only present when the core has been generated with PFC support- this bit defaults to 0 if PFC is not supported. When this bit is 1 and RX PFC is enabled, reception of a PFC frame with a valid quanta for priority 1 is processed as described in Receiving a PFC Frame. When this bit is 0, the rx_pfc_p1_tvalid remains at 0.
0	1	R/W	RX Priority 0 pause enable. Only present when the core has been generated with PFC support- this bit defaults to 0 if PFC is not supported. When this bit is 1 and RX PFC is enabled, reception of a PFC frame with a valid quanta for priority 0 is processed as described in Receiving a PFC Frame. When this bit is 0, the rx_pfc_p0_tvalid remains at 0.

Table 32: MAC Speed Configuration Word (0x410)

Bits	Default Value	Type	Description
31:30	10	R/W	MAC Speed Configuration 00 = 10 Mbps 01 = 100 Mbps 10 = 1 Gbps When the TEMAC solution has been generated for 1 Gbps or 2.5 Gbps speed support, bits[31:30] are hard-coded to the value 10. When the TEMAC solution has been generated for 10 Mbps and 100 Mbps speed support, bits[31:30] only accept the values of 00 to configure for 10 Mbps operation, or 01 to configure for 100 Mbps operation.
29:0	N/A	RO	Reserved

Notes:

1. A reset does not affect the setting of the MAC Speed Configuration register.

Table 33: RX Max Frame Configuration Word (0x414)

Bits	Default Value	Type	Description
31:17	N/A	RO	Reserved
16	0	R/W	RX Max Frame Enable: When Low, the MAC assumes the use of the standard 1518/1522 depending upon the setting of VLAN enable. When High, the MAC allows frames up to RX Max Frame Length irrespective of the value of VLAN enable. If Jumbo Enable is set, this register has no effect. See Maximum Permitted Frame Length .
15	N/A	RO	Reserved
14:0	0x7D0	R/W	RX Max Frame Length

Table 34: TX Max Frame Configuration Word (0x418)

Bits	Default Value	Type	Description
31:17	N/A	RO	Reserved
16	0	R/W	TX Max Frame Enable: When Low, the MAC assumes use of the standard 1518/1522 depending upon the setting of VLAN enable . When High, the MAC allows frames up to TX Max Frame Length irrespective of the value of VLAN enable . If Jumbo Enable is set, this register has no effect. See Maximum Permitted Frame Length .
15	N/A	RO	Reserved
14:0	0x7D0	R/W	TX Max Frame Length

Table 35: Per Priority Quanta/Refresh Register (0x480/0x49C)

Bits	Default Value	Type	Description
31:16	0xFF00	RW	Pause Quanta refresh value . This register is only present when PFC is enabled at the core customization time. When enabled, this register controls how frequently a PF quanta is refreshed by the transmission of a new PFC frame. When a refresh occurs, all currently active (TX PFC tvalid is High and enabled) priorities are refreshed.
15:0	0xFFFF	RW	Pause Quanta value . This register is only present when PFC is enabled at core customization time. When enabled, this register sets the quanta value to be inserted in the PFC frame for this priority.

Notes:

1. This register is repeated for the eight priorities, priority 0 to priority 7.
2. These registers only exist when the core is generated with PFC support.

Table 36: Legacy Pause Refresh Register (0x4A0)

Bits	Default Value	Type	Description
31:16	0xFF00	RW	Pause Quanta refresh value . This register is only present when PFC is enabled at the core customization time. When PFC is supported, the 802.3 pause request can also support XON/XOFF Extended Functionality. This controls the frequency of the automatic pause refresh.

Table 36: Legacy Pause Refresh Register (0x4A0) (cont'd)

Bits	Default Value	Type	Description
15:0	0x0	RO	Reserved

Notes:

- These registers only exist when the core is generated with PFC support.

ID Register (0x4F8)

Table 37: ID Register (0x4F8)

Bits	Default Value	Type	Description
31:24	z ¹	RO	Major Rev
23:16	y ¹	RO	Minor Rev
15:8	N/A	RO	Reserved
7:0	x ¹	RO	Patch Level (0-No patch, 1-Rev1)

Notes:

- The default values depend upon the version of the core being used.

Table 38: Ability Register (0x4FC)

Bits	Default Value	Type	Description
31:17	N/A	RO	Reserved
16	0	RO	PFC Support . This bit indicates that the core has been generated with PFC support.
15:11	N/A	RO	Reserved
10	1 ¹	RO	Frame filter available
9	1 ¹	RO	Half duplex capable
8	1 ¹	RO	Statistics Counters available
7:4	N/A	RO	Reserved
3	1 ¹	RO	2.5G Ability: If set, the core is 2.5G capable
2	1 ¹	RO	1G Ability: If set, the core is 1G capable
1	1 ¹	RO	100M Ability: If set, the core is 100M capable
0	1 ¹	RO	10M Ability: If set, the core is 10M capable

Notes:

- Depends on core abilities selected.

MDIO

Access to the MDIO interface through the management interface is entirely register mapped. A list of the MDIO registers is shown in the following table. See [MDIO Interface](#) for more details. These registers are available only when the core is generated with the optional MDIO interface.

MDIO Configuration Registers

Table 39: MDIO Configuration Registers

Address (Hex)	Description
0x500	Table 40: MDIO Setup Word (0x500)
0x504	Table 41: MDIO Control Word (0x504)
0x508	Table 42: MDIO Write Data (0x508)
0x50C	Table 43: MDIO Read Data (0x50C)

The contents of each configuration register are shown in the following tables.

Table 40: MDIO Setup Word (0x500)

Bits	Default Value	Type	Description
31:7	N/A	RO	Reserved
6	0x0	R/W	MDIO Enable: When this bit is 1, the MDIO interface can be used to access attached PHY devices. When this bit is 0, the MDIO interface is disabled and the MDIO signals remain inactive. A write to this bit only takes effect if the Clock Divide is set to a non-zero value.
5:0	0x0	R/W	Clock Divide[5:0]: See Accessing PHY Configuration Registers, through MDIO Using the Management Interface .

Table 41: MDIO Control Word (0x504)

Bits	Default Value	Type	Description
31:29	N/A	RO	Reserved
28:24	0x0	R/W	TX_PHYAD: This controls the PHY address being accessed.
23:21	N/A	RO	Reserved
20:16	0x0	R/W	TX_REGAD: This controls the register address being accessed.
15:14	0x0	R/W	TX_OP: This field controls the type of access performed when a 1 is written to initiate. 01-Write Access 10-Read Access
13:12	N/A	RO	Reserved
11	0x0	WO	Initiate: Writing a 1 to this bit starts an MDIO transfer.
10:8	N/A	RO	Reserved

Table 41: MDIO Control Word (0x504) (cont'd)

Bits	Default Value	Type	Description
7	0x0	RO	MDIO ready: When set, the MDIO is enabled and ready for a new transfer. This is also used to identify when a previous transaction has completed (for example, Read data is valid).
6:0	N/A	RO	Reserved

Table 42: MDIO Write Data (0x508)

Bits	Default Value	Type	Description
31:16	N/A	RO	Reserved
15:0	0x0000	R/W	Write Data

Table 43: MDIO Read Data (0x50C)

Bits	Default Value	Type	Description
31:17	N/A	RO	Reserved
16	0x0	RO	MDIO Ready: This is a copy of Bit[7] of the MDIO Control Word.
15:0	0x0000	RO	Read Data: Only valid when MDIO ready is sampled High.

Interrupt Controller

Table 44: Interrupt Controller Configuration

Address (Hex)	Default Value	Type	Description
0x600	0x00	RO	Interrupt Status Register. Indicates the status of an interrupt.
0x610	0x00	RO	Interrupt Pending Register. Indicates the pending status of an interrupt. Bits in this register are only set when the corresponding bits in IER and ISR are set.
0x620	0x00	R/W	Interrupt Enable Register. Indicates the enable state of an interrupt. Writing a 1 to any bit enables that particular interrupt.
0x630	0x00	WO	Interrupt Clear Register. Writing a 1 to any bit of this register clears that particular interrupt.

Bit[0] of all interrupt registers is used to indicate the MDIO Transaction complete interrupt and is asserted only when the core is generated with the MDIO interface. Bits[3:1] are used for AVB interrupts and are asserted only when the core is generated with AVB feature. Bit[1] is used to indicate `interrupt_ptp_tx`, Bit[2] is used to indicate `interrupt_ptp_rx`, and Bit[3] is used to indicate `interrupt_ptp_timer`. Bits[31:4] are Reserved.

Frame Filter Configuration

The following tables describe the registers used to access the optional frame filter configuration when the TEMAC solution is implemented with a frame filter. In addition to the unicast address, broadcast address, and pause addresses, the frame filter can optionally be generated to respond to up to 16 additional configurable frame filter matches. These are stored in an address table within the frame filter. See [Frame Filter](#).

If no frame filter is present, these registers do not exist and return 0s for a read from the stated addresses.

The following table shows the frame filter configuration registers.

Table 45: Frame Filter Configuration

Address (Hex)	Description
0x700	Table 46: Unicast Address (Word 0) (0x700)
0x704	Table 47: Unicast Address (Word 1) (0x704)
0x708	Table 48: Frame Filter Control (0x708)
0x70C	Table 49: Frame Filter Enable (0x70C)
0x710	Frame Filter Value Bytes 3-0
0x714	Frame Filter Value Bytes 7-4
0x718	Frame Filter Value Bytes 11-8
0x71C	Frame Filter Value Bytes 15-12
0x720	Frame Filter Value Bytes 19-16
0x724	Frame Filter Value Bytes 23-20
0x728	Frame Filter Value Bytes 27-24
0x72C	Frame Filter Value Bytes 31-28
0x730	Frame Filter Value Bytes 35-32
0x734	Frame Filter Value Bytes 39-36
0x738	Frame Filter Value Bytes 43-40
0x73C	Frame Filter Value Bytes 47-44
0x740	Frame Filter Value Bytes 51-48
0x744	Frame Filter Value Bytes 55-52
0x748	Frame Filter Value Bytes 59-56
0x74C	Frame Filter Value Bytes 63-60
0x750	Frame Filter Mask Value Bytes 3-0
0x754	Frame Filter Mask Value Bytes 7-4
0x758	Frame Filter Mask Value Bytes 11-8
0x75C	Frame Filter Mask Value Bytes 15-12
0x760	Frame Filter Mask Value Bytes 19-16
0x764	Frame Filter Mask Value Bytes 23-20
0x768	Frame Filter Mask Value Bytes 27-24

Table 45: Frame Filter Configuration (cont'd)

Address (Hex)	Description
0x76C	Frame Filter Mask Value Bytes 31-28
0x770	Frame Filter Mask Value Bytes 35-32
0x774	Frame Filter Mask Value Bytes 39-36
0x778	Frame Filter Mask Value Bytes 43-40
0x77C	Frame Filter Mask Value Bytes 47-44
0x780	Frame Filter Mask Value Bytes 51-48
0x784	Frame Filter Mask Value Bytes 55-52
0x788	Frame Filter Mask Value Bytes 59-56
0x78C	Frame Filter Mask Value Bytes 63-60

The contents of each configuration register are shown in the following tables.

Table 46: Unicast Address (Word 0) (0x700)

Bits	Default Value	Type	Description
31:0	unicast_addresses[31:0]	R/W	Frame filter unicast address[31:0]: This address is used by the MAC to match against the destination address of any incoming frames. The address is ordered so the first byte transmitted/received is the lowest positioned byte in the register. For example, a MAC address of AA-BB-CC-DD-EE-FF would be stored in Address[47:0] as 0xFFEEDDCCBBAA.

Table 47: Unicast Address (Word 1) (0x704)

Bits	Default Value	Type	Description
31:16	N/A	RO	Reserved
15:0	unicast_addresses[47 downto 32]	R/W	Frame filter unicast address[47:32]: See description in See Unicast Address (Word 0) (0x700) .

Table 48: Frame Filter Control (0x708)

Bits	Default Value	Type	Description
31	1	R/W	Promiscuous Mode: If this bit is set to 1, the frame filter is set to operate in promiscuous mode. All frames are passed to the receiver client regardless of the destination address.
30:9	N/A	RO	Reserved
8	0	R/W	AVB Select : If the AVB Endpoint is present. This is used to indicate that the filter to be selected is one of the three dedicated filters.
7:4	N/A	RO	Reserved

Table 48: Frame Filter Control (0x708) (cont'd)

Bits	Default Value	Type	Description
3:0	0	R/W	<i>Filter Index</i> : All frame filters are mapped to the same location with the filter index and AVB Select, specifying which physical filter is to be accessed. When an AVB filter is being selected only indexes of 0-2 are allowed.

Table 49: Frame Filter Enable (0x70C)

Bits	Default Value	Type	Description
31:1	N/A	RO	Reserved
0	1	R/W	Filter Enable: This enable relates to the physical frame filter pointed to by the Filter index and take the value of AVB Select into account. If clear, the filter passes all packets.

Table 50: Frame Filter Value (0x710-0x74C)

Bits	Default Value	Type	Description
31:0	Bits[47:0] = 1 All other = 0	R/W	Filter Value All filter value registers have the same format. The lower 31 bits of filter value, at address 0x710, relating to the filter at physical Frame Filter index that is to be written to the address table. The value is ordered so that the first byte transmitted/received is the lowest positioned byte in the register. For example, a MAC address of AA-BB-CC-DD-EE-FF would be stored in Filter Value[47:0] as 0xFFEEDDCCBBAA. By default, the frame filters are configured to match against the broadcast address.

Table 51: Frame Filter Mask Value (0x750-0x790)

Bits	Default Value	Type	Description
31:0	Bits[47:0] = 1 All other = 0	R/W	Mask Value. All mask value registers have the same format. If a mask bit is set to 1, the corresponding bit of the Filter Value is compared by the frame filter. For example, if a basic Destination address comparison was desired, Bits[47:0] should be written to 1 and all other bits to 0.

AVB Specific Frame Filters

This section is only applicable if the TEMAC solution is implemented with the optional Ethernet AVB Endpoint option.

PTP Frame Filter

The PTP frame filter is at AVB index 0 and is initialized to match the following:

Destination Address = 0x0E0000C28001

Type = 0xF788

This translates to the following register settings:

Table 52: PTP Frame Filter Value (0x710-0x74C)

Address (Hex)	Default Value	Description
0x710	0x00C28001	First four bytes of the AVB Special address
0x714	0x0000E00	Final two bytes of the AVB Special Address and first two bytes of the source address
0x718	0x0	Final four bytes of the source address
0x71C	0x0000F788	Type field
0x720-0x74C	0x0	Not Used

Table 53: PTP Frame Filter Mask Value (0x750-0x790)

Address (Hex)	Default Value	Description
0x750	0xFFFFFFFF	Match against the Destination address
0x754	0x0000FFFF	Match against the destination address but not the source address.
0x758	0x0	Do not match against source address.
0x75C	0x0000FFFF	Match against the Type field
0x760-0x790	0x0	Not Used

SR Classes A and B Frame Filters

Table 54: SR Class A (Index 1) Frame Filter Value (0x710-0x74C)

Address (Hex)	Default Value	Description
0x710-0x718	0x0	Not Used
0x71C	0x02600081	Match against VLAN field with PCP field set to 3 and ID field set to 2
0x720-0x74C	0x0	Not Used

Table 55: SR Class A (Index 1) Frame Filter Mask Value (0x750-0x790)

Address (Hex)	Default Value	Description
0x750-758	0x0	Not Used
0x75C	0xFFFFFFFF	Match against Type field and type info field
0x760-0x790	0x0	Not Used

Table 56: SR Class B (Index 2) Frame Filter Value (0x710-0x74C)

Address (Hex)	Default Value	Description
0x710-0x718	0x0	Not Used
0x71C	0x02400081	Match against VLAN field with PCP field set to 2 and ID field set to 2
0x720-0x74C	0x0	Not Used

Table 57: SR Class B (Index 2) Frame Filter Mask Value (0x750-0x790)

Address (Hex)	Default Value	Description
0x750-758	0x0	Not Used
0x75C	0xFFFFFFFF	Match against Type field and type info field
0x760-0x790	0x0	Not Used

AVB Endpoint

This section describes the registers used for setting up and operating the optional AVB Endpoint functionality.

RX PTP Packet Buffer Address Space

The RX PTP Packet buffers are only available when the AVB functionality is included in the TEMAC core. The Address space of the [RX PTP Packet Buffer](#) is 4K in total. This represents the size of a single FPGA block RAM pair (4K). Every byte of this block RAM can be read.

This address space is divided equally into 16 separate buffers of 256 bytes, each of which is capable of storing a unique PTP frame. When received, a PTP frame is written into one of these buffers. The buffer write pointer increments and points to the next buffer in preparation for the subsequent PTP frame reception.

Within each buffer, the entire PTP frame is written (from MAC Destination Address to the last byte from the data field), starting at the base address of that buffer. Following PTP frame reception, the RX timestamp captured for that frame is written into the top 4 bytes of the buffer used. A list of the RX PTP Buffers is shown in the following table.

Table 58: RX PTP Buffers

Address (Hex)	Description
0x10000-0x100FC	RX PTP Buffer 0
0x10100-0x101FC	RX PTP Buffer 1
0x10200-0x102FC	RX PTP Buffer 2
0x10300-0x103FC	RX PTP Buffer 3
0x10400-0x104FC	RX PTP Buffer 4
0x10500-0x105FC	RX PTP Buffer 5

Table 58: RX PTP Buffers (cont'd)

Address (Hex)	Description
0x10600-0x106FC	RX PTP Buffer 6
0x10700-0x107FC	RX PTP Buffer 7
0x10800-0x108FC	RX PTP Buffer 8
0x10900-0x109FC	RX PTP Buffer 9
0x10A00-0x10AFC	RX PTP Buffer 10
0x10B00-0x10BFC	RX PTP Buffer 11
0x10C00-0x10CFC	RX PTP Buffer 12
0x10D00-0x10DFC	RX PTP Buffer 13
0x10E00-0x10EFC	RX PTP Buffer 14
0x10F00-0x10FFC	RX PTP Buffer 15

TX PTP Packet Buffer Address Space

The TX PTP Packet buffers are only available when the AVB functionality is included in the TEMAC core.

The Address space of the [TX PTP Packet Buffer](#) is 2K in total, representing the size of a single FPGA block 18K RAM. Every byte of this block RAM is accessible by the CPU. This address space is divided equally into eight separate buffers of 256 bytes, each of which is capable of storing a unique PTP frame. Seven of these buffer locations are pre-initialized with standard PTP frame syntax. However, each byte can be modified if desired.

Within each single buffer, the initial byte is used as a length field used to indicate to the core logic the number of bytes to be transmitted for that frame. An entire PTP frame (from MAC Destination Address through to the last byte from the data field) is then stored, starting at the eighth address of that particular buffer. Following PTP frame transmission from one of these buffers, the TX Timestamp captured for that frame is written into the top four bytes of the buffer used. See [TX PTP Packet Buffer](#) for more details. A list of the TX PTP Buffers is shown in the following table.

Table 59: TX PTP Buffers

Address (Hex)	Description
0x11000-0x110FC	TX PTP Buffer 0 – Initialized for a SYNC frame.
0x11100-0x111FC	TX PTP Buffer 1 – Initialized for a Follow up frame.
0x11200-0x112FC	TX PTP Buffer 2 – Initialized for a Pdelay request frame.
0x11300-0x113FC	TX PTP Buffer 3 – Initialized for a Pdelay response frame.
0x11400-0x114FC	TX PTP Buffer 4 – Initialized for a Pdelay response follow up frame.
0x11500-0x115FC	TX PTP Buffer 5 – Initialized for an Announce frame.
0x11600-0x116FC	TX PTP Buffer 6 – Initialized for a Signaling frame.
0x11700-0x117FC	TX PTP Buffer 7

AVB Configuration

The AVB configuration registers are only present when the AVB is included in the TEMAC core. These registers are used by the software drivers to control the AVB functionality.

A list of the AVB Configuration registers is shown in the following table.

AVB Configuration Registers

Table 60: AVB Configuration Registers

Address (Hex)	Description
0x12000	TX PTP Packet Buffer Control Register
0x12004	RX PTP Packet Buffer Control Register
0x12008	Reserved
0x1200C	TX Arbiter Send Slope Control Register
0x12010	TX Arbiter Idle Slope Control Register
0x12014-0x127FC	Reserved

The contents of each configuration register are shown in the following tables.

TX PTP Packet Buffer Control Register

The following table defines associated control register of the TX PTP Packet Buffers used by the software to request the transmission of the PTP frames.

Table 61: TX PTP Packet Buffer Control Register (0x12000)

Bits	Default Value	Type	Description
31:19	0	RO	Reserved
18:16	0	RO	tx_packet. Indicates the number (block RAM bin position) of the most recently transmitted PTP packet.
15:8	0	RO	tx_frame_waiting Indication. The TX PTP Packet Buffer is split into eight regions of 256 bytes, each of which can contain a separate PTP frame. There is one tx_frame_waiting bit for each of the eight regions. When logic is set as 1, each bit indicates that a request has been made for frame transmission to the TX Arbiter, but that a grant has not yet occurred. When the frame has been successfully transmitted, the bit is set to logic 0. This bit allows the microprocessor to run off a polling implementation as opposed to the Interrupts.

Table 61: TX PTP Packet Buffer Control Register (0x12000) (cont'd)

Bits	Default Value	Type	Description
7:0	0	WO	tx_send_frame Bits. The TX PTP Packet Buffer is split into eight regions of 256 bytes, each of which can contain a separate PTP frame. There is one tx_send_frame bit for each of the eight regions. When each bit is written to one, causes a request to be made to the TX Arbiter. When access is granted the frame contained within the respected region is transmitted. If read, always returns 0.

Notes:

1. A read or a write to this register clears the interrupt_ptp_tx interrupt (asserted after each successful PTP packet transmission).

RX PTP Packet Buffer Control Register

The following table defines the associated control register of the RX PTP Packet Buffers used by the software to monitor the position of the most recently received PTP frame.

Table 62: RX PTP Packet Buffer Control Register (0x12004)

Bits	Default Value	Type	Description
31:12	0	RO	Reserved
11:8	0	RO	rx_packet. Indicates the number (block RAM bin position) of the most recently received PTP packet.
7:1	0	RO	Reserved
0	0	WO	rx_clear. When written with a one, forces the buffer to empty, in practice moving the write address to the same value as the read address. If read, always returns 0.

Notes:

1. A read or a write to this register clears the interrupt_ptp_rx interrupt (asserted after each successful PTP packet reception).

TX Arbiter Send Slope Control Register

The SendSlope variable is defined in IEEE802.1Qav-2009 to be the rate of change of credit in bits per second when the value of credit is decreasing (during AV packet transmission). The [TX Arbiter Idle Slope Control Register](#), [RTC Nanoseconds Field Offset Control](#), and [RTC Seconds Field Offset Control](#) registers define the maximum limit of the bandwidth reserved for AV traffic, as enforced by the TX Arbiter. The default values allow the maximum bandwidth proportion of 75% for the AV traffic. See the *IEEE 802.3-2008 specification* for further information.

Table 63: TX Arbiter Send Slope Control Register (0x1200C)

Bits	Default Value	Type	Description
31:20	0	RO	Reserved
19:0	2048	R/W	Value of sendSlope

TX Arbiter Idle Slope Control Register

The idleSlope variable is defined in IEEE802.1Qav-2009 to be the rate of change of credit in bits per second when the value of credit is increasing (whenever there is no AV packet transmission). The [TX Arbiter Send Slope Control Register](#), [RTC Nanoseconds Field Offset Control](#), and [RTC Seconds Field Offset Control](#) registers define the maximum limit of the bandwidth reserved for AV traffic, as enforced by the TX Arbiter. The default values allow the maximum bandwidth proportion of 75% for the AV traffic. See the *IEEE 802.3-2008 specification* for further information.

Table 64: TX Arbiter Idle Slope Control Register (0x12010)

Bits	Default Value	Type	Description
31:20	0	RO	Reserved
19:0	6144	R/W	Value of idleSlope

RTC Configuration

The RTC configuration registers are only present when the AVB is included in the TEMAC core. These registers are used by the software drivers to control the RTC functionality.

A list of the RTC Configuration registers is shown in the following table.

Table 65: RTC Configuration Registers

Address (Hex)	Description
0x12800	RTC Nanoseconds Field Offset Control
0x12804	Reserved
0x12808	Table 67: Seconds Field Offset Bits[31:0] (0x12808)
0x1280C	Table 68: Seconds Field Offset Bits[47:32] (0x1280C)
0x12810	RTC Increment Value Control Register
0x12814	Table 70: Current RTC Nanoseconds Value (0x12814)
0x12818	Table 71: Current RTC Seconds Field Value Bits [31:0] (0x12818)
0x1281C	Table 72: Current RTC Seconds Field Value Bits [47:32] (0x1281C)
0x12820	RTC Interrupt Clear Register
0x12824	RTC Phase Adjustment Register

Table 65: RTC Configuration Registers (cont'd)

Address (Hex)	Description
0x12828–0x13FFC	Reserved

The contents of each configuration register are shown in the following tables.

RTC Nanoseconds Field Offset Control

The following table describes the offset control register for the nanoseconds field of the RTC used to force step changes into the counter. When in PTP clock master mode, this can be used to set the initial value following power-up. While in PTP clock slave mode, the software drivers use this register to implement the periodic step corrections.

This register and the registers defined in [RTC Seconds Field Offset Control](#) are linked. These three offset values are loaded into the RTC counter logic simultaneously following a write to this nanosecond offset register.

Table 66: RTC Nanoseconds Field Offset (0x12800)

Bits	Default Value	Type	Description
31:30	0	RO	Reserved
29:0	0	R/W	30-bit offset value for the RTC nanoseconds. Used by the microprocessor to initialize the RTC, then to perform the regular RTC corrections (In slave mode).

RTC Seconds Field Offset Control

The following table describes the offset control register for the lower 32 bits of seconds field of the RTC, used to force step changes into the counter. When in PTP clock master mode, this can be used to set the initial value following power-up. When in PTP clock slave mode, the software drivers use this register to implement the periodic step corrections.

These register and the register defined in [RTC Nanoseconds Field Offset Control](#) are linked. These three offset values are loaded into the RTC counter logic simultaneously following a write to the nanosecond offset register defined in the following table.

Table 67: Seconds Field Offset Bits[31:0] (0x12808)

Bits	Default Value	Type	Description
31:0	0	R/W	32-bit offset value for the RTC seconds field (Bits[31:0]). Used by the microprocessor to initialize the RTC, then to perform the regular RTC corrections (when in slave mode).

The following table describes the offset control register for the upper 16 bits of seconds field of the RTC used to force step changes into the counter. When in PTP clock master mode, this can be used to set the initial value following power-up. When in PTP clock slave mode, the software drivers use this register to implement the periodic step corrections.

These register and the registers defined in [RTC Nanoseconds Field Offset Control](#) are linked. These three offset values are loaded into the RTC counter logic simultaneously following a write to the nanosecond offset register defined in [RTC Nanoseconds Field Offset Control](#).

Table 68: Seconds Field Offset Bits[47:32] (0x1280C)

Bits	Default Value	Type	Description
31:16	0	RO	Reserved
15:0	0	R/W	16-bit offset value for the RTC seconds field (Bits[47:32]). Used by the microprocessor to initialize the RTC, then to perform the regular RTC corrections (when in slave mode).

RTC Increment Value Control Register

The following table describes the RTC Increment Value Control Register, which provides a configurable increment rate for the RTC counter. This increment register should take the value of the clock period being used to increment the RTC. However, the resolution of this increment register is very fine (in units of 1/1048576 (1/220) fraction of one nanosecond). Therefore, the RTC increment rate can be adjusted to a very fine degree of accuracy, thus providing the following features:

- The RTC can be incremented from any available clock frequency that is greater than the IEEE802.1AS defined minimum of 25 MHz.
- When acting as a clock slave, the rate adjustment of the RTC can be matched to that of the network clock master to an exceptional level of accuracy.

Table 69: RTC Increment Value Control Register (0x12810)

Bits	Default Value	Type	Description
31:26	0	RO	Reserved
25:0	0	R/W	Per gtx_clk clock period Increment Value for the RTC.

Current RTC Value Registers

The following table describes the nanoseconds field value register for the nanoseconds field of the RTV. When read, this returns the latest value of the counter. The registers defined in this section are linked. When this nanoseconds value register is read, the entire RTC (including the seconds field) is sampled.

Table 70: Current RTC Nanoseconds Value (0x12814)

Bits	Default Value	Type	Description
31:30	0	RO	Reserved
29:0	0	RO	Current Value of the synchronized RTC nanoseconds field. Note: A read from this register samples the entire RTC counter (synchronized) so that the Epoch and Seconds field are held static for a subsequent read.

The following table describes the lower 32 bits of the seconds value register for the seconds field of the RTC. When read, this returns the latest value of the counter. The registers defined in this section are linked. When the nanoseconds value register is read (see [Table 70: Current RTC Nanoseconds Value \(0x12814\)](#)), the entire RTC is sampled.

Table 71: Current RTC Seconds Field Value Bits [31:0] (0x12818)

Bits	Default Value	Type	Description
31:0	0	RO	Sampled Value of the synchronized RTC Seconds field (Bits[31:0]).

The following table describes the upper 16 bits of the seconds value register for the seconds field of the RTC. When read, this returns the latest value of the counter. The registers defined in this section are linked. When the nanoseconds value register is read (see [Table 70: Current RTC Nanoseconds Value \(0x12814\)](#)), the entire RTC is sampled.

Table 72: Current RTC Seconds Field Value Bits [47:32] (0x1281C)

Bits	Default Value	Type	Description
31:16	0	RO	Reserved
15:0	0	RO	Sampled Value of the synchronized RTC Seconds field (Bits[47:32]).

RTC Interrupt Clear Register

The following table describes the control register defined for the `interrupt_ptp_timer` signal, the periodic interrupt signal that the RTC raises.

Table 73: RTC Interrupt Clear Register (0x12820)

Bits	Default Value	Type	Description
31:1	0	RO	Reserved
0	0	WO	Write ANY value to Bit[0] of this register to clear the <code>interrupt_ptp_timer</code> Interrupt signal. This bit always returns 0 on read.

RTC Phase Adjustment Register

The following table describes the Phase Adjustment Register that has units of nanoseconds. This value is added to the synchronized value of the RTC nanoseconds field, and the RTC timing signals are then derived from the result. This phase offset is therefore applied to the `clk8k` signal. As an example, writing the value of the decimal 62500 (half of an 8 kHz clock period) to this register would invert the `clk8k` signal with respect to a value of 0. Therefore, this register can provide fine grained phase alignment of these signals to a 1 ns resolution.

Table 74: RTC Phase Adjustment Register (0x12824)

Bits	Default Value	Type	Description
31:30	0	RO	Reserved
29:0	0	R/W	ns value relating to the phase offset for all RTC derived timing signals (clk8k).

System Requirements

For operating system requirements, see the *Vivado Design Suite User Guide: Release Notes, Installation, and Licensing* ([UG973](#)).

Designing with the Core

This chapter includes guidelines and additional information to facilitate designing with the core.

General Design Guidelines

This section describes the steps required to turn the TEMAC solution into a fully-functioning design integrated with user application logic.



IMPORTANT! *It is important to recognize that not all designs require all the design steps defined in this chapter. The following sections discuss the design steps required for various implementations; follow the logic design guidelines carefully.*

Design Steps

Generate the core using the AMD Vivado™ Design Suite. See [Chapter 5: Design Flow Steps](#).

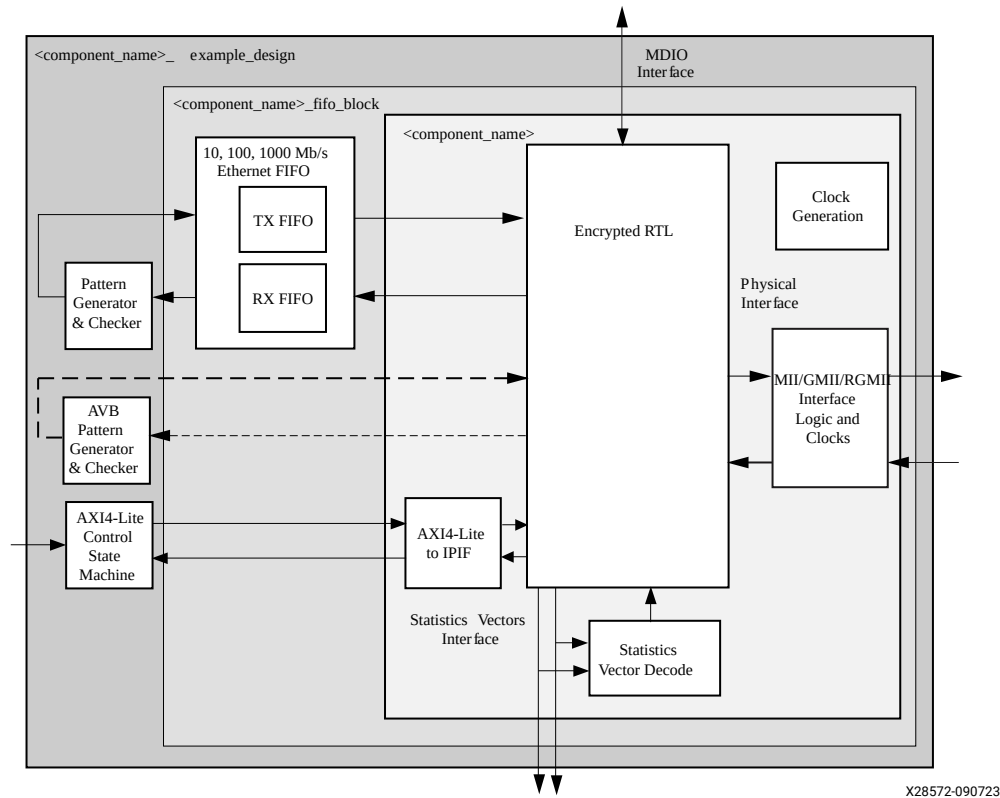
Using the Example Design as a Starting Point

The core is delivered through the Vivado Design Suite with an HDL example design built around the core which when placed on a suitable board allows the functionality of the core to be demonstrated using either a simulation package or in hardware. The following figure shows a block diagram of the example design. For details about the Vivado example design, see [Chapter 6: Example Design](#).

The example design shows how to:

- Instantiate the core from HDL.
- Source and use the user-side interface ports of the core from application logic.
- Connect the physical-side interface of the core (GMII/MII or RGMII) to device IOBs creating an external interface. (See the Physical Interface chapters in this document)
- Derive the clock logic required for the core (See the Physical Interface sections in this document).

Figure 6: Tri-Mode Ethernet MAC Core Example Design



Using the example design as a starting point enables you to do the following:

- Edit the HDL top-level of the example design file to:
 - Change the clocking scheme.
 - Add/remove IOBs as required.
 - Replace the basic pattern generator logic with your specific application logic.
 - Adapt the 10 Mbps, 100 Mbps, and 1 Gbps Ethernet FIFO to suit your specific application (see [10, 100, 1000 Mbps Ethernet FIFO](#)).
 - Remove the AXI4-Lite Control State machine and directly drive the AXI4-Lite bus from a processor.
- Synthesize the entire design.
- Implement the entire design.
- After the completion of implementation you can also create a bitstream that can be downloaded to an AMD device.
- Simulate the entire design using the provided demonstration test bench.
- Download the bitstream to a target device.

Implementing the Tri-Mode Ethernet MAC

The example design can be studied as an example of how to do the following:

- Instantiate the core from HDL.
- Source and use the user-side interface ports of the core from application logic.
- Connect the physical-side interface of the core (GMII/MII or RGMII) to device IOBs to create an external interface.
- Derive the required clock logic.

After working with the example design and this product guide, you can write your own HDL application, using single or multiple instances of the core.

Care must be taken to constrain the design correctly, and the constraints provided with the core should be used as the basis for your own purpose. For Vivado Design Suite constraints, see [Constraining the Core](#).

You can simulate the entire design and download the bitstream to the target device.

Keep it Registered

To simplify timing and increase system performance in an FPGA, keep all inputs and outputs registered between your application and the core. This means that all inputs and outputs from your application should come from, or connect to a flip-flop. While registering signals might not be possible for all paths, it simplifies timing analysis and makes it easier for the AMD tools to place-and-route the design.

Recognize Timing Critical Signals

The constraints provided with the example design identify the applicable critical signals and timing constraints. For Vivado Design Suite constraints, see [Constraining the Core](#).

Make Only Allowed Modifications

Do not modify the core. Any modifications can affect the system timing and protocol compliance adversely. When the core is generated, the supported user configurations of the core can only be made by selecting the options in the customization IP window.

Shared Logic

Up to version 7.0 of the core, the RTL hierarchy for the core was fixed. This resulted in some difficulty because shareable clocking and reset logic are to be extracted from the core example design for use with a single instance or multiple instances of the core.

Shared Logic is a feature that provides a flexible architecture that works both as a standalone core and as a part of a larger design with one or more core instances. This minimizes the amount of required HDL modifications. But at the same time, retains the flexibility to address more uses of the core.

★ **IMPORTANT!** *IDELAYCTRL is the only sharable component for AMD UltraScale™ architecture devices. Even though all the permutations of the core are self-contained using the available XIPHY resources. Making IDELAYCTRL a shared resource makes the core more flexible when its I/Os need to share the XIPHY Byte with another module I/Os.*

Figure 7: Shared Logic Included in Core

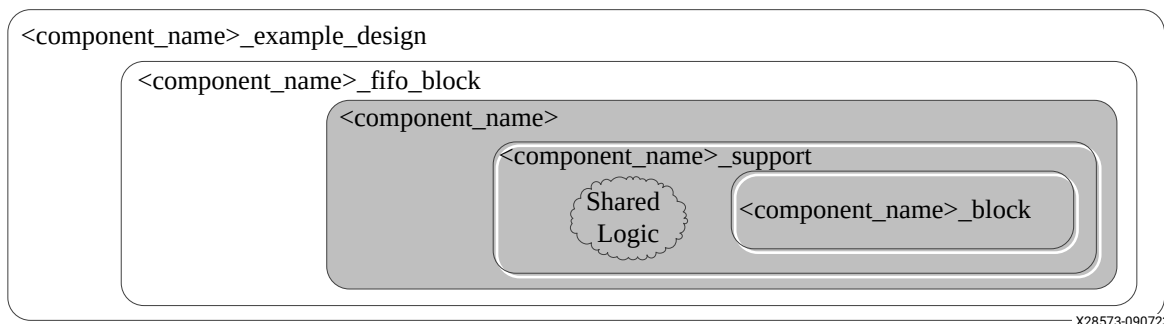
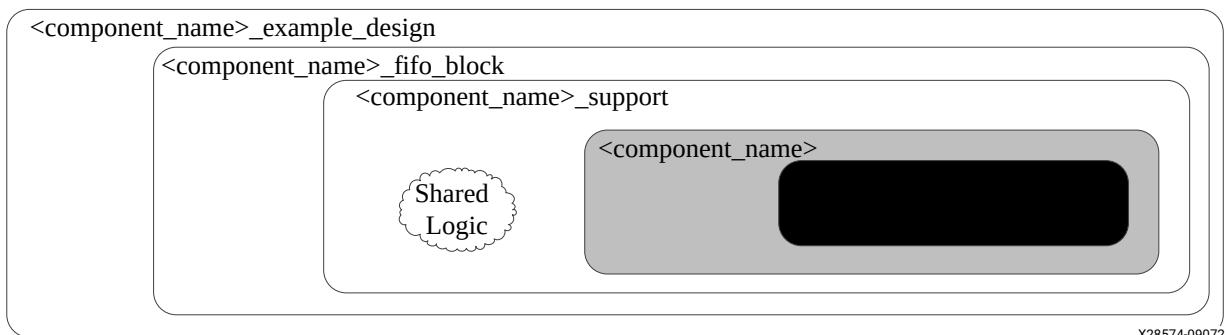


Figure 8: Shared Logic Included in Example Design

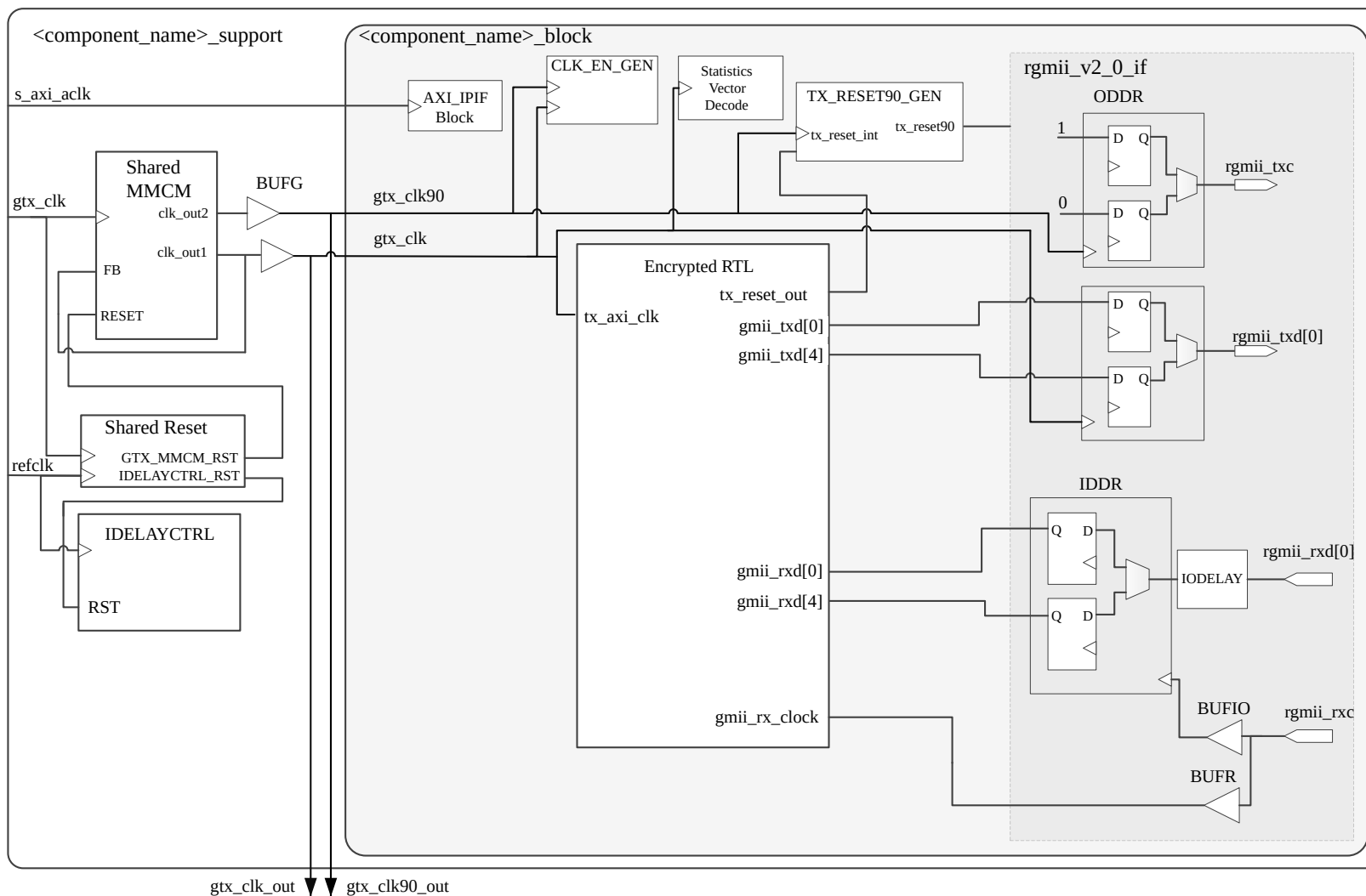


The Shared Logic level of the hierarchy is called `<component_name>_support`. The following figures show two hierarchies where the shared logic block is contained either in the core or in the example design. In these figures, `<component_name>` is the name of the generated core. The difference between the two hierarchies is the boundary of the core. It is controlled using the Shared Logic option in the Vivado IDE (see [Figure 101: Vivado Customize IP IDE – Interface Tab](#)).

The contents of the shared logic depend upon the physical interface and the target device. Port `refclk` is not part of the `<component_name>_block` level because the `IDELAYCTRL` has been moved to the `<component_name>_support` level—this is demonstrated in [Figure 9: RGMII Logic for 7 Series Using HR I/O Devices](#) to [Figure 11: GMII Logic for 7 Series Devices](#). The `<component_name>` level is not shown for simplification.

RGMII for 7 Series Devices

Figure 9: RGMII Logic for 7 Series Using HR I/O Devices

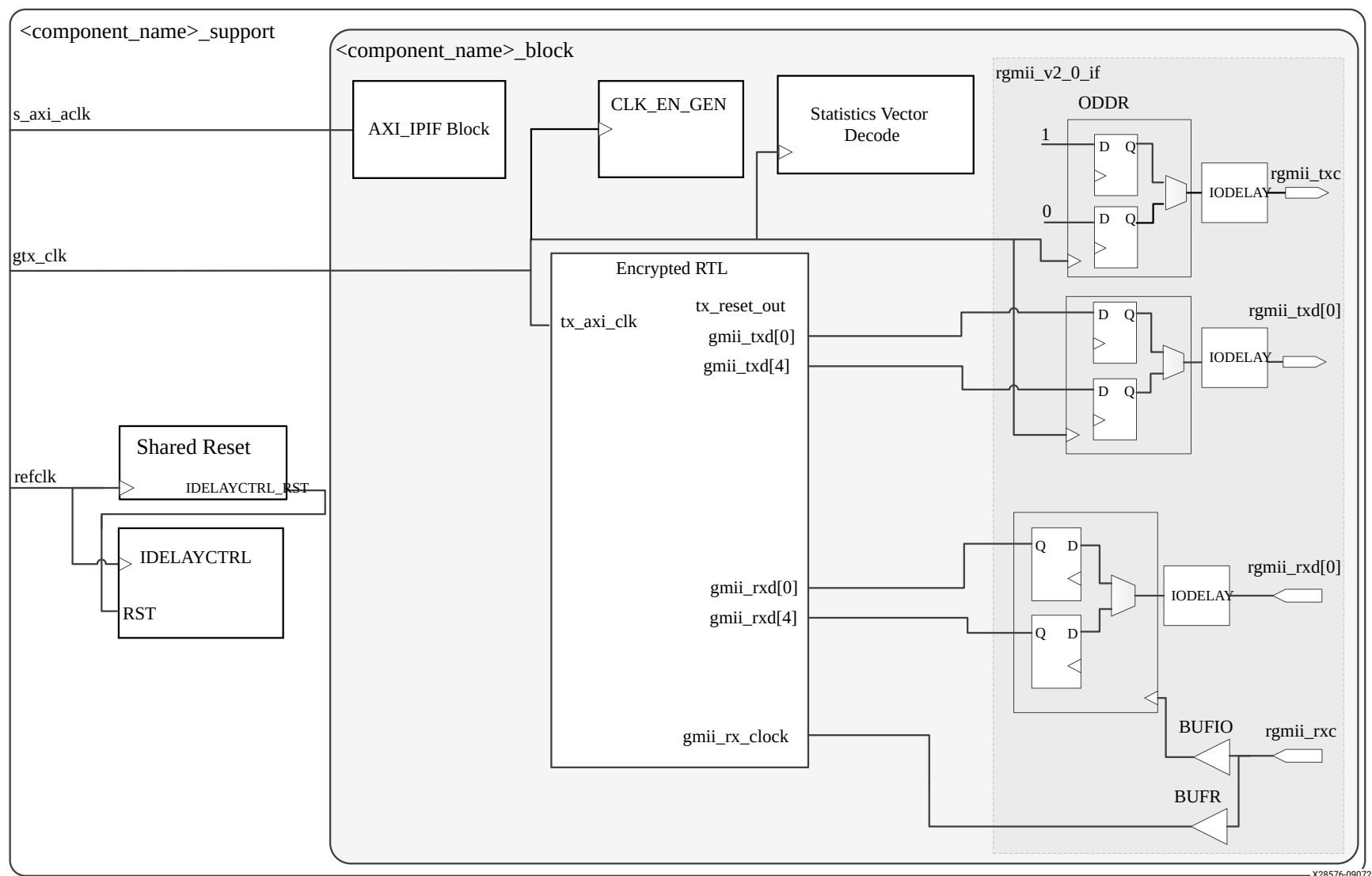


X28575-090723

The previous figure shows that the IDELAYCTRL module is instantiated at the `<component_name>_support` level. If the Shared Logic level option is not selected, ensure that an IDELAYCTRL is instantiated elsewhere in the user design.

In the previous figure two output ports `gtx_clk_out` and `gtx_clk90_out` are present. These can be directly connected to `gtx_clk` and `gtx_clk90` clock inputs for secondary core instances, if they are generated with the “Shared logic included in example design” option selected in the Vivado IDE.

Figure 10: RGMII Logic for 7 Series Using HP I/O Devices



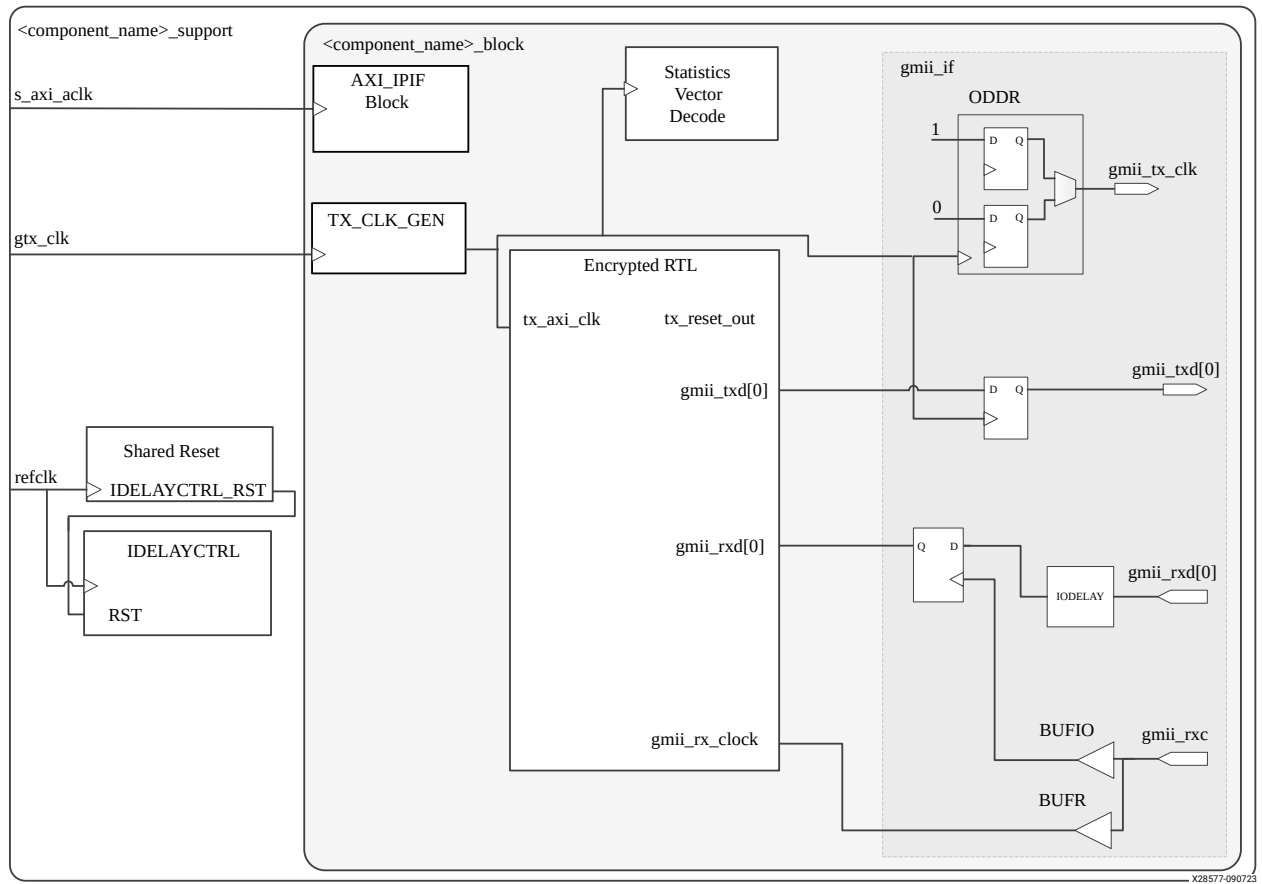
X28576-090723

The previous figure shows that the IDELAYCTRL module is instantiated at the `<component_name>_support` level. If the Shared Logic level is not selected, ensure that an IDELAYCTRL is instantiated elsewhere in the user design.

GMII for 7 Series Devices

The following figure shows that the IDELAYCTRL module is instantiated at the `<component_name>_support` level. If Shared Logic level is not selected, ensure that an IDELAYCTRL is instantiated elsewhere in the user design.

Figure 11: GMII Logic for 7 Series Devices



MII or Internal Interface for 7 Series Devices

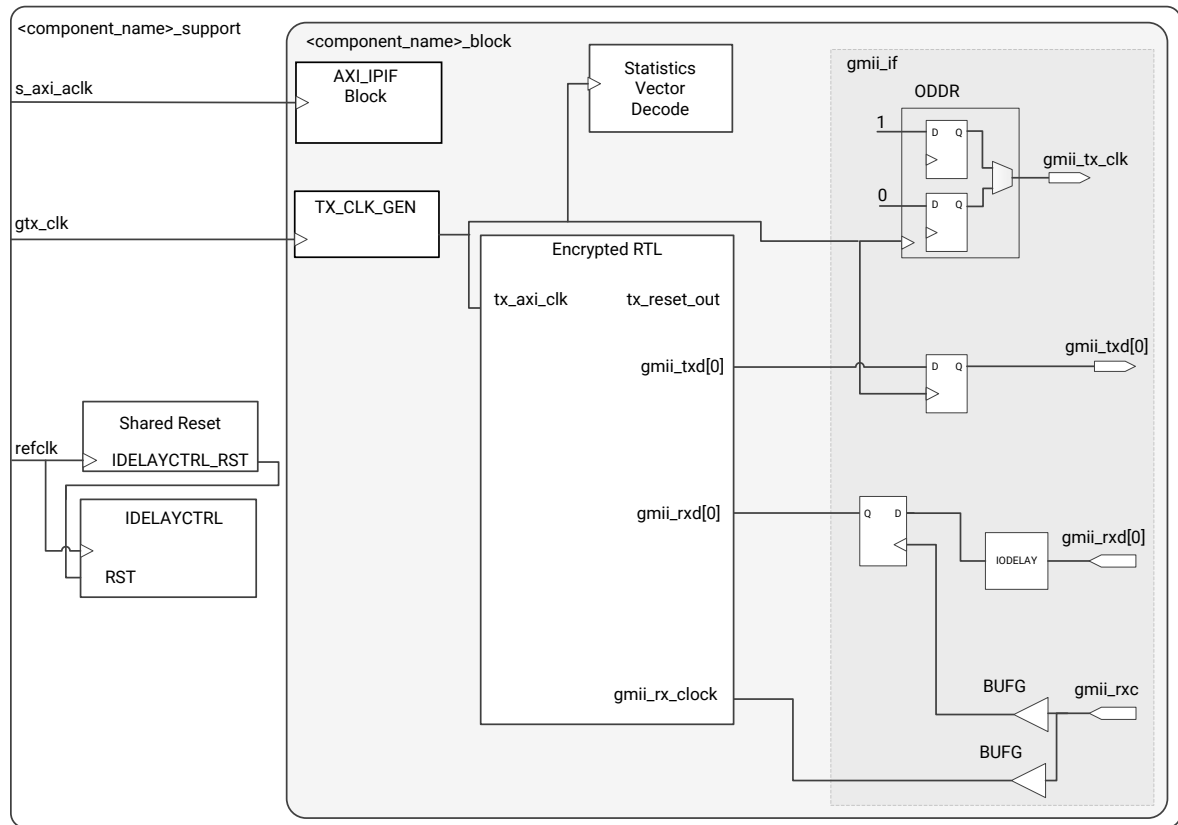
If the core is generated with MII or internal physical interface, `<component_name>_support` becomes a logic-less wrapper without using resources because there is nothing that can be shared.

GMII/RGMII for UltraScale Devices

The following figures show the IDELAYCTRL module instantiated at the `<component_name>_support` level for GMII and RGMII physical interfaces, respectively. For UltraScale devices, it is mandatory to have an IDELAYCTRL instance for every XIPHY Byte to which the I/Os are mapped. Ensure the following:

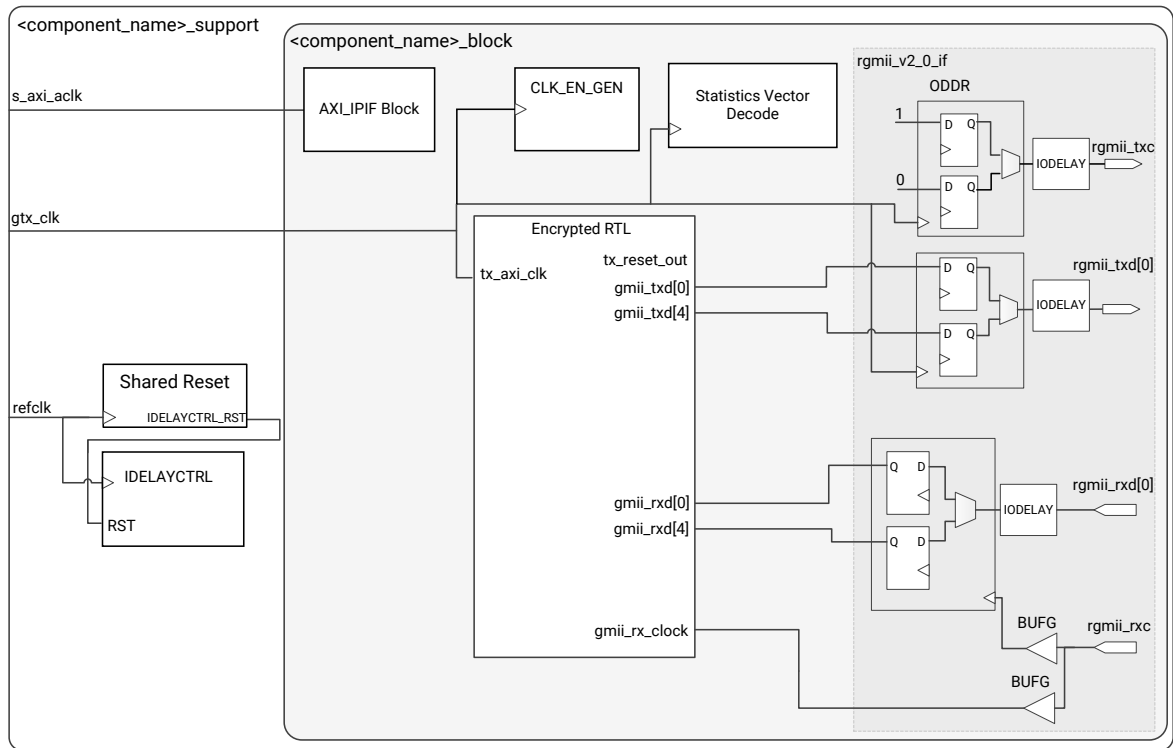
1. If the core instance I/Os are the only I/Os to be mapped to the XIPHY Byte. Ensure that the IDELAYCTRL component is present by generating the core instance with “Shared logic in core.”
2. If the core instance I/Os are sharing the XIPHY Byte with other instance(s) and if these instance(s) happen to instantiate IDELAYCTRL, generate the core instance with “Shared logic in example design.”

Figure 12: GMII Logic for UltraScale Devices



X25629-080321

Figure 13: RGMII Logic for UltraScale Devices



X25630-080321

Another scenario is when the core instance I/Os are spread across multiple XIPHY Bytes. In this case, the Vivado tool copies the IDELAYCTRL instance and the associated reset to all these XIPHY Bytes.

MII or Internal Interface for UltraScale Devices

If the core is generated with MII or internal physical interface `<component_name>_support` becomes a logic-less wrapper using no resources because there is nothing that can be shared.

Clocking

The TEMAC solution has a complicated clocking structure that varies depending upon the specific configuration and the selected FPGA family. The majority of these changes are specific to the physical interface and selected shared logic option. This is described in the following sections:

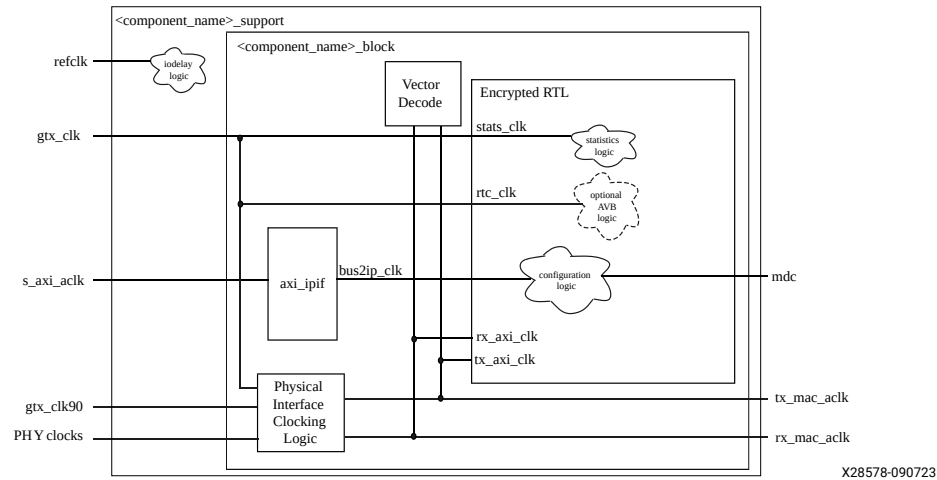
- [Physical Interface for 7 Series and Zynq 7000 Devices](#)
- [Physical Interface for UltraScale Devices](#)

The remainder of the clocking for the TEMAC solution is shown in the following figure . These clocks are all dependent on the core configuration:

- `s_axi_aclk` is present only if the Management type is set to AXI4-Lite.
- `mdc` is present if the Management type is set to AXI4-Lite and the core is generated with MDIO.
- `refclk` is only present for GMII or RGMII and if the Shared Logic option “Include shared logic in core” option is selected.
- `gtx_clk` is not present when physical interface is MII and statistics counters are not enabled.

When the core is generated with the internal interface, it is assumed that it is connected to the [Ethernet 1G/2.5G BASE-X PCS/PMA or SGMII](#) core. See the *1G/2.5G Ethernet PCS/PMA or SGMII LogiCORE IP Product Guide* ([PG047](#)).

Figure 14: Clocking Architecture (Not Including the Physical Interface Clocking)

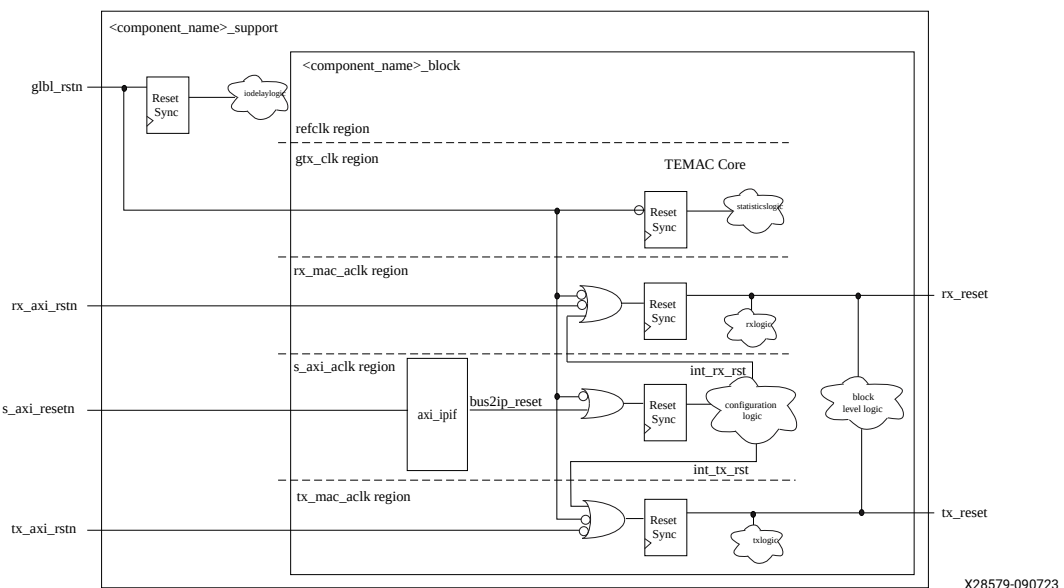


Resets

Due to the number of clock domains in this core, the reset structure is not simple and involves many separate reset regions, with the number of regions being dependent upon the particular parameterization of the core.

The following figure shows the most common reset structure for the core. Because the `rx_reset` and `tx_reset` outputs have dependencies on the `glbl_rstn` and `tx/rx_axi_rstn` inputs they cannot be used in their creation.

Figure 15: Reset Architecture



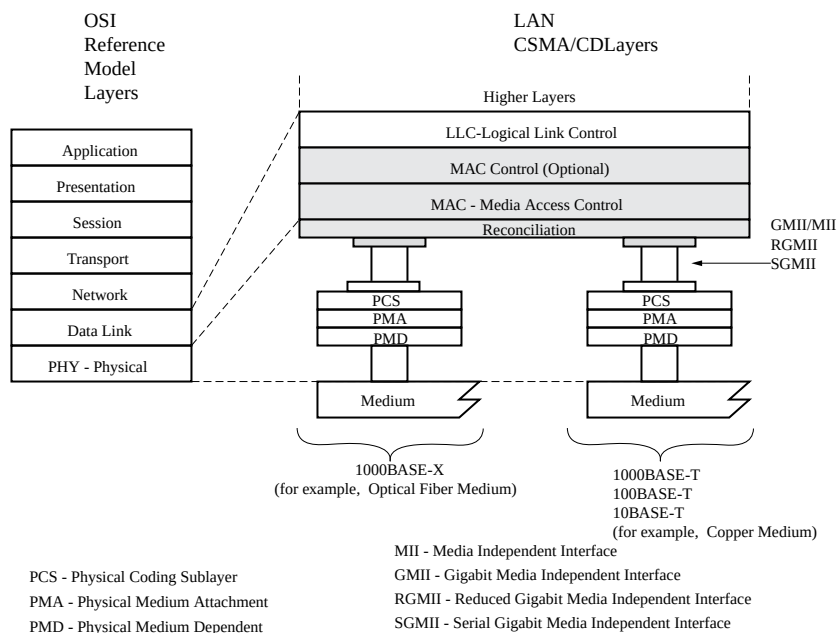
Protocol Description

This section gives an overview of where the Ethernet MAC fits into an Ethernet system and describes some basic Ethernet terminology.

Ethernet Sublayer Architecture

The following figure illustrates the relationship between the Open Systems Interconnection (OSI) reference model and the Ethernet MAC, as defined in IEEE 802.3-2008 specification. The grayed-in layers show the functionality that the Ethernet MAC handles. The following figure also shows where the supported physical interfaces fit into the architecture.

Figure 16: IEEE Std 802.3-2008 Ethernet Model



X28602-091323

MAC and MAC CONTROL Sublayer

The Ethernet MAC is defined in IEEE Std 802.3-2008 clauses 2, 3, and 4. A MAC is responsible for the Ethernet framing protocols described in [Ethernet Data Format](#) and error detection of these frames. The MAC is independent of and can connect to any type of physical layer device.

The MAC CONTROL sublayer is defined in IEEE Std 802.3-2008 clause 31. This provides real-time flow control manipulation of the MAC sublayer.

Ethernet MAC provides MAC CONTROL and MAC sublayers, in all modes of operation.

Physical Sublayers PCS, PMA, and PMD

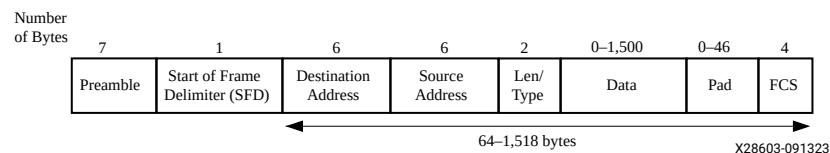
The combination of the Physical Coding Sublayer (PCS), Physical Medium Attachment (PMA), and Physical Medium Dependent (PMD) sublayer constitute the physical layers for the protocol. Two main physical standards are specified:

- BASE-T PHYs provide a link between the MAC and copper mediums. This functionality is not offered within the TEMAC. However, external BASE-T PHY devices are readily available on the market. These can connect to the Ethernet MAC using GMII/MII, RGMII, or by additionally using the [Ethernet 1G/2.5G BASE-X PCS/PMA or SGMII LogiCORE](#), SGMII interfaces.
- BASE-X PHYs provide a link between the MAC and (usually) fiber optic mediums. The TEMAC is capable of supporting the 1 Gbps BASE-X standard; 1G/2.5G PCS and PMA sublayers can be offered by connecting the TEMAC to the [Ethernet 1G/2.5G BASE-X PCS/PMA or SGMII LogiCORE](#).

Ethernet Data Format

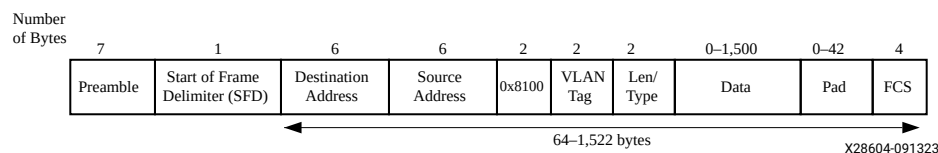
Ethernet data is encapsulated in frames as shown in the following figure for standard Ethernet frames. The fields in the frame are transmitted from left to right. The bytes within the fields are transmitted from left to right (from least significant bit to most significant bit unless specified otherwise). The Ethernet MAC can handle jumbo Ethernet frames where the data field can be much larger than 1,500 bytes.

Figure 17: Standard Ethernet Frame Format



The Ethernet MAC can also accept Virtual LAN (VLAN) frames. The VLAN frame format is shown in the following figure. If the frame is a VLAN type frame, the Ethernet MAC accepts four additional bytes.

Figure 18: Ethernet VLAN Frame Format



Ethernet PAUSE/flow control frames can be transmitted and received by the Ethernet MAC. [Figure 39: MAC Control Frame Format](#) shows how a PAUSE/flow control frame differs from the standard Ethernet frame format. The following subsections describe the individual fields of an Ethernet frame and some basic functionality of the Ethernet MAC.

Preamble

For transmission, this field is automatically inserted by the Ethernet MAC. The preamble field was historically used for synchronization and contains seven bytes with the pattern `0x55` transmitted from left to right. For the reception, this field is always stripped from the incoming frame before the data is passed to you. The Ethernet MAC can receive Ethernet frames even if the preamble does not exist as long as a valid start of frame delimiter is available.

Start of Frame Delimiter

The start of a frame delimiter field marks the start of the frame and must contain the pattern `0xD5`. For transmission on the physical interface, this field is automatically inserted by the Ethernet MAC. For reception, this field is always stripped from the incoming frame before the data is passed to you.

MAC Address Fields

MAC Address

The least significant bit of the first octet of a MAC address determines if the address is an individual/unicast (0) or group/multicast (1) address. Multicast addresses are used to group logically related stations. The broadcast address (destination address field is all 1s) is a multicast address that addresses all stations on the Local Area Network (LAN). The Ethernet MAC supports the transmission and reception of unicast, multicast, and broadcast packets.

The address is transmitted in an Ethernet frame's least significant bit first. So, the bit representing an individual or group address is the first bit to appear in an address field of the Ethernet frame.

Destination Address

This MAC Address field is the first field of the Ethernet frame that is always provided in the packet data for transmissions and is always retained in the received packet data. It provides the MAC address of the intended recipient on the network.

Source Address

This MAC Address field is the second field of the Ethernet frame that is always provided in the packet data for transmissions and is always retained in the received packet data. It provides the MAC address of the frame initiator on the network.

For transmission, you should always provide the source address of the Ethernet frame because it is unmodified by the TEMAC.

Length/Type

The value of this field determines if it is interpreted as a length or a type field, as defined by IEEE Std 802.3-2008. A value of 1,536 decimals or greater is interpreted by the Ethernet MAC as a type field.

When used as a length field, the value in this field represents the number of bytes in the following data field. This value does not include any bytes that can be inserted in the pad field following the data field.

A length/type field value of `0x8100` indicates that the frame is a VLAN frame and a value of `0x8808` indicates a PAUSE MAC control frame.

For the transmission, the Ethernet MAC does not perform any processing of the length/type field.

For the reception,

- If this field is a length field, the Ethernet MAC receive engine interprets this value and removes any padding in the pad field when necessary.
- If the field is a length field and length/type checking is enabled, the Ethernet MAC compares the length against the actual data field length and flags an error if a mismatch occurs.
- If the field is a type field, the Ethernet MAC ignores the value and passes it along with the packet data with no further processing.

The length/type field is always retained in the receive packet data.

Data

The data field can vary from 0 to 1,500 bytes in length for a normal frame. The Ethernet MAC can handle jumbo frames of any length.

This field is always provided in the packet data for transmissions and is always retained in the receive packet data.

Pad

The pad field can vary from 0 to 46 bytes in length. This field is used to ensure that the frame length is at least 64 bytes in length (the preamble and SFD fields are not considered part of the frame for this calculation), which is required for successful CSMA/CD operation. The values in this field are used in the frame check sequence calculation but are not included in the length field value when it is used. The length of this field and the data field combined must be at least 46 bytes. If the data field contains 0 bytes, the pad field is 46 bytes. If the data field is 46 bytes or more, the pad field has 0 bytes.

For the transmission, this field can be inserted automatically by the Ethernet MAC or supplied by you. If the pad field is inserted by the Ethernet MAC, the FCS field is calculated and inserted by the Ethernet MAC. If the pad field is supplied by you, the FCS can either be inserted by the Ethernet MAC or provided by you, as indicated by a configuration register bit.

For the reception, if the length/type field has a length interpretation, any pad field in the incoming frame is not passed to you unless the Ethernet MAC is configured to pass the FCS field on to you.

FCS

The value of the FCS field is calculated over the destination address, source address, length/type, data, and pad fields using a 32-bit Cyclic Redundancy Check (CRC) as defined in IEEE Std 802.3-2008 section 3.2.9:

$$G(x) = x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x^1 + x^0$$

The CRC bits are placed in the FCS field with the x^{31} term in the left-most bit of the first byte, and the x^0 term is the rightmost bit of the last byte (that is, the bits of the CRC are transmitted in the order $x^{31}, x^{30}, \dots, x^1, x^0$).

For the transmission, this field can be inserted automatically by the Ethernet MAC or supplied by you, as indicated by a configuration register bit.

For reception, an incoming FCS value is verified on every frame. If an incorrect FCS value is received, the Ethernet MAC indicates that it has received a bad frame. The FCS field can either be passed on to you or be dropped by the Ethernet MAC, as indicated by a configuration register bit.

Frame Transmission and Interframe Gap

Frames are transmitted over the Ethernet medium with an interframe gap of 96-bit times (9.6 μ s for 10 Mbps, 0.96 μ s for 100 Mbps, and 96 ns for 1 Gbps), as specified by the IEEE Std 802.3-2008. This value is a minimum and can be increased with a resulting decrease in throughput. The process for frame transmission is different for half-duplex and full-duplex systems.

Half-Duplex Frame Transmission

In a half-duplex system, the CSMA/CD media access method defines how two or more stations share a common medium.

1. Even when it has nothing to transmit, the Ethernet MAC monitors the Ethernet medium for traffic by watching the carrier sense signal (CRS) from the external PHY. Whenever the medium is busy ($CRS = 1$), the Ethernet MAC defers to the passing frame by delaying any pending transmission of its own.

2. After the last bit of the passing frame (when the carrier sense signal changes from TRUE to FALSE), the Ethernet MAC starts the timing of the interframe gap.
3. The Ethernet MAC resets the interframe gap timer if the carrier sense becomes TRUE during the period defined by “interframe gap part 1 (IFG1).” IEEE Std 802.3-2008 states that this should be the first 2/3 of the interframe gap timing interval (64-bit times) but it can be shorter and as small as zero. The purpose of this option is to support a possible brief failure of the carrier sense signal during a collision condition and is described in paragraph 4.2.3.2.1 of the IEEE standard.
4. To ensure fair access to the bus, the Ethernet MAC does not reset the interframe gap timer if carrier sense becomes TRUE during the period defined by “interframe gap part 2 (IFG2).” IEEE Std 802.3-2008 states that this should be the last 1/3 of the interframe gap timing interval.

After initiating a transmission, if the message collides with the message of another station (COL = 1), each transmitting station intentionally continues to transmit (jam) for an additional predefined period (32-bit times for 10/100 Mbbs) to ensure the propagation of the collision throughout the system. The station remains silent for a random amount of time (back off) before attempting to transmit again.

A station can experience a collision during the beginning of its transmission (the collision window) before its transmission has had time to propagate to all stations on the bus. After the collision window is passed, a transmitting station acquires the bus. Subsequent collisions (late collisions) are avoided because all other (properly functioning) stations are assumed to have detected the transmission and trying to defer it.

Full-Duplex Frame Transmission

In a full-duplex system, there is a point-to-point dedicated connection between two Ethernet devices, capable of simultaneously transmit and receive with no possibility of collisions. The Ethernet MAC does not use the carrier sense signal from the external PHY because the medium is not shared and the Ethernet MAC only needs to monitor its own transmissions. After the last bit of an Ethernet MAC frame transmission, the Ethernet MAC starts the interframe gap timer and defer transmissions until the IFG count completes. The minimum value supported for the IFG depends on the TEMAC solution options and the current mode of operation.

If the TEMAC solution has been built with half-duplex support, the IFG delay is 96-bit times. When IFG Adjustment is enabled, greater than 64-bit times, the value presented on `tx_ifg_delay`.

If the TEMAC solution has been built with only full-duplex support, the IFG delay is 96-bit times. When IFG Adjustment is enabled, greater than 32-bit times, the value presented on `tx_ifg_delay`.

AXI4-Stream User Interface

This section provides a detailed description of the AXI4-Stream user-side interface. This interface must be used by the user-side logic to initiate frame transmission and accept frame reception to and from the core. The definitions and abbreviations used in this chapter are described in the following table.

Table 75: Abbreviations Used in Timing Diagrams

Abbreviation	Definition
DA	Destination address; 6 bytes
SA	Source address; 6 bytes
L/T	Length/type field; 2 bytes
FCS	Frame check sequence; 4 bytes

Receiving Inbound Frames

Received Ethernet frames are presented to the user logic on the receiver subset of the AXI4-Stream interface. For port definition, see [Receiver Interface](#). All receiver signals are synchronous to the `rx_mac_aclk` clock.

Normal Frame Reception

[Figure 19: Normal Frame Reception at 1 Gbps](#) shows the timing of a normal inbound frame transfer at 1 Gbps. [Figure 20: Normal Frame Reception at 10/100 Mbps](#) shows the timing at 10/100 Mbps when the core is configured for MII/GMII or RGMII. For the Internal option the timing for 100 Mbps is shown in [Figure 21: Normal Frame Reception at 100 Mbps for Internal Interface](#). For 10 Mbps the `rx_axis_mac_tvalid` is only enabled once every 100 cycles. You must be prepared to accept data at any time; there is no buffering within the MAC to allow for latency in the receive logic. When the frame reception begins, the data is transferred on consecutive validated cycles to the receive logic until the frame is complete. The MAC asserts the `rx_axis_mac_tlast` signal to indicate that the frame has completed with `rx_axis_mac_tuser` signal being used to indicate any errors.

Figure 19: Normal Frame Reception at 1 Gbps

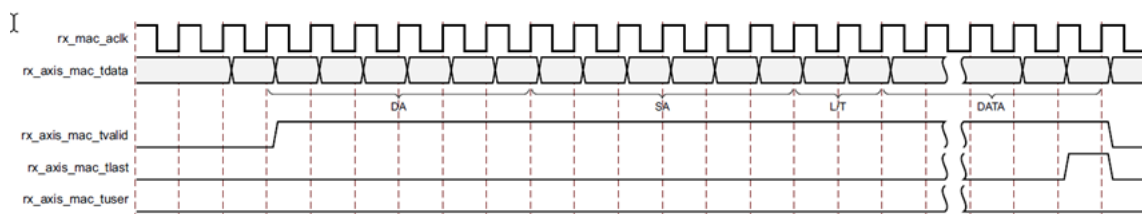


Figure 20: Normal Frame Reception at 10/100 Mbps

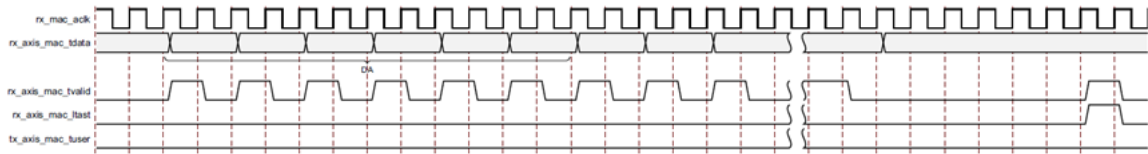
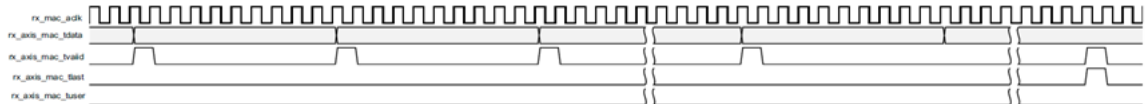


Figure 21: Normal Frame Reception at 100 Mbps for Internal Interface



Frame parameters (destination address, source address, length/type, and optionally FCS) are supplied on the data bus according to the timing diagram. The abbreviations are described in [Table 75: Abbreviations Used in Timing Diagrams](#).

If the length/type field in the frame has the length interpretation and this indicates that the inbound frame has been padded to meet the Ethernet minimum frame size specification, this padding is not passed to you in the data payload. However, this is excepted when the FCS passing is enabled. See [User-Supplied FCS Passing](#).

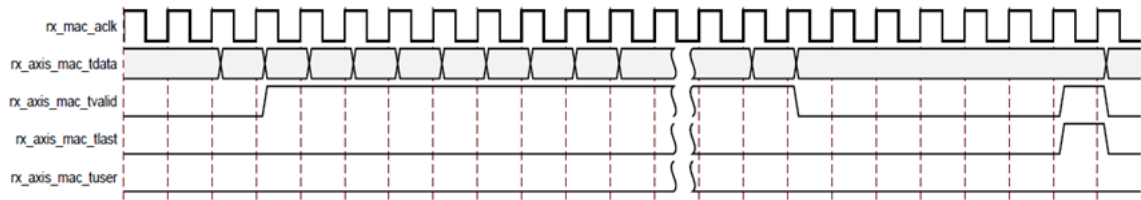
When user-supplied FCS passing is disabled, `rx_axis_mac_tvalid = 0` between frames for the duration of the padding field (if present), the FCS field, carrier extension (if present), the interframe gap following the frame, and the preamble field of the next frame. When user-supplied FCS passing is enabled, `rx_axis_mac_tvalid = 0` between frames for the duration of carrier extension (if present), the interframe gap, and the preamble field of the following frame.

rx_axis_mac_tlast and rx_axis_mac_tuser Timing

Although [Figure 19: Normal Frame Reception at 1 Gbps](#) illustrates the `rx_axis_mac_tlast` signal asserted immediately after a cycle containing valid data on `rx_axis_mac_tdata`, this is not usually the case.

The `rx_axis_mac_tlast` and `rx_axis_mac_tuser` signals are asserted along with the final byte of the transfer, only after all frame checks are completed. This is after the FCS field has been received (and after reception of carrier extension, if present). This is shown in the following figure.

Figure 22: Frame Reception TLast Timing

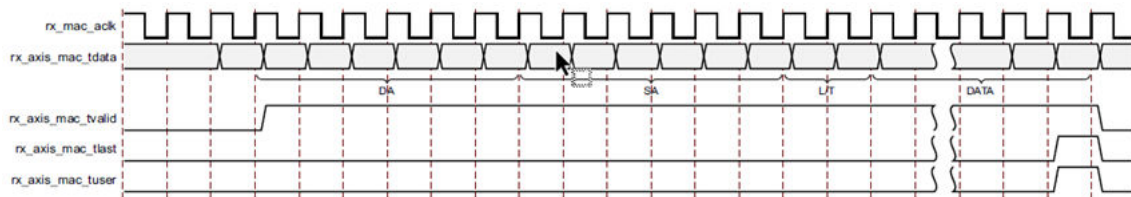


Therefore, `rx_axis_mac_tlast` and possibly `rx_axis_mac_tuser` are asserted following frame reception at the beginning of the interframe gap.

Frame Reception with Errors

The following figure illustrates an unsuccessful frame reception (for example, a fragment frame or a frame with an incorrect FCS). In this case, the `rx_axis_mac_tuser` signal is asserted to you at the end of the frame. It is your responsibility to drop the data already transferred for this frame.

Figure 23: Frame Reception with Error



The following conditions cause the assertion of `rx_axis_mac_tuser`:

- Occurrence of FCS errors.
- Packets are shorter than 64 bytes (undersized or fragment frames).
- Jumbo frames are received when jumbo frames are not enabled.
- VLAN frames of length 1,519-1,522 are received when VLAN frames are not enabled.
- A frame above the programmed Maximum Frame Size is received when Maximum Frame Length checking is enabled.
- A value of 0x0000 to 0x002D is in the type/length field. In this situation, the frame should be padded to minimum length. If it is not padded to exactly minimum frame length, the frame is marked as bad (when length/type checking is enabled).
- A value of 0x002E to 0x0600 is in the type/length field, but the real length of the received frame does not match this value (when length/type checking is enabled).
- Any control frame that is received is not exactly the minimum frame length (unless control frame length checks are disabled: see [Receiving a Pause Control Frame](#)).

- An error is indicated on the PHY interface at any point during frame reception.
- An error code is received in the 1 Gb frame extension field.
- A valid pause frame addressed to the MAC, is received when flow control is enabled. See [Flow Control Overview](#).
- A frame does not match against any of the enabled frame filters, if present.
- A valid Priority Flow Control (PFC) frame addressed to the Ethernet MAC, is received when PFC is enabled. This frame is only marked as an error because it has been used by the MAC PFC logic and has now served its purpose.

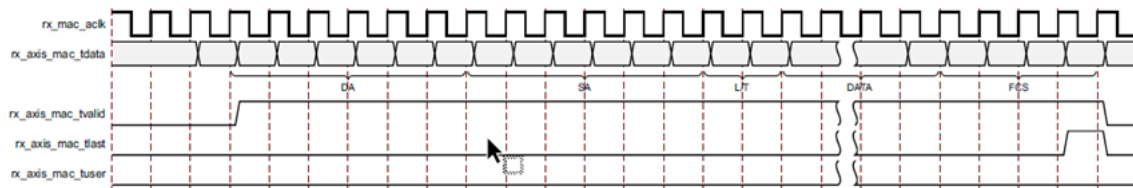
User-Supplied FCS Passing

If the MAC core is configured to pass the FCS field to you. It is handled as displayed in the following figure.

In this case, any padding inserted into the frame to meet Ethernet minimum frame length specifications is left intact and passed to you.

Even though the FCS is passed to you, it is also verified by the MAC core and `rx_axis_mac_tuser` is asserted if the FCS check fails.

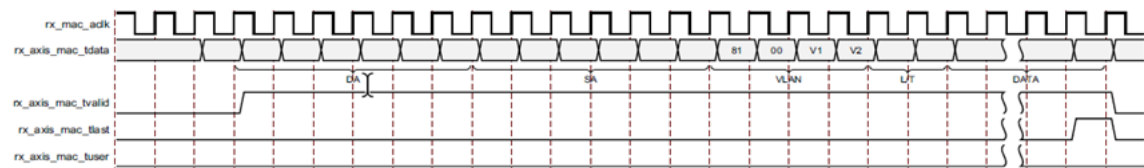
Figure 24: Frame Reception with In-Band FCS Field



VLAN Tagged Frames

The reception of a VLAN tagged frame can be seen in the following figure. This frame is identified as being a VLAN frame by the inclusion of the VLAN type tag (81-00), located in the first two bytes following the Source Address. This is followed by the Tag Control Information bytes, V1 and V2. The length/type field after the tag control information is not checked for errors. More information on the interpretation of these bytes can be found in the IEEE 802.3-2008 standard.

Figure 25: Reception of a VLAN Tagged Frame



Maximum Permitted Frame Length

The maximum legal length of a frame specified in IEEE 802.3-2008 is 1,518 bytes for non-VLAN tagged frames. VLAN tagged frames can be extended to 1,522 bytes. When jumbo frame handling is disabled and the core receives a frame that exceeds the maximum legal length, `rx_axis_mac_tuser` is asserted. When jumbo frame handling is enabled, frames that are longer than the legal maximum are received in the same way as shorter frames.

It is also possible to specify a different maximum frame size. If this is enabled and the frame exceeds the configured value, the frame is rejected, that is, `rx_axis_mac_tuser` is asserted at the end of the frame. In this case, VLAN frames are not treated separately. If jumbo frame handling is enabled, that takes precedence and the configured value is ignored.

Length/Type Field Error Checks

- **Enabled:** The default operation is with the length/type error checking enabled. In this mode, the following checks are made on all frames received. If either of these checks fails, the frame is marked as bad.
 - A value in the length/type field that is $\geq$ decimal 46 but less than decimal 1,536 (a Length interpretation) is checked against the actual data length received.
 - A value in the length/type field that is $<$ decimal 46 is checked to see that the data field is padded to exactly 46 bytes (so that the resultant frame is minimum frame size: 64 bytes total in length).

Furthermore, if padding is indicated (the length/type field is less than decimal 46) and [User-Supplied FCS Passing](#) is disabled, the length value in the length/type field is used to de-assert `rx_axis_mac_tvalid` after the indicated number of data bytes that removes the padding bytes from the frame.

- **Disabled:** When the length/type error checking is disabled and the length/type field has a length interpretation, the MAC does not check the length value against the actual data length received. A frame containing only this error is marked as good. However, if the length/type field is less than decimal 46, the MAC marks a frame as bad if it is not the minimum frame size of 64 bytes. Furthermore, if padding is indicated and [User-Supplied FCS Passing](#) is disabled, a length value in the length/type field is not used to de-assert `rx_axis_mac_tvalid`. Instead, `rx_axis_mac_tvalid` is de-asserted before the start of the FCS field. In this way, no padding is removed from the frame.

Frame Filter

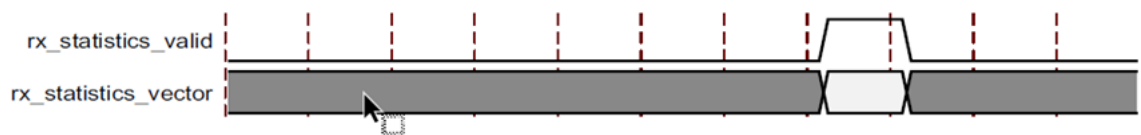
If the optional frame filter is included in the core, the MAC is able to reject frames that do not match against a recognized pattern, that is, a specified destination address. If a frame is rejected within the destination address, the `rx_axis_mac_tvalid` signal is not asserted for the duration of the frame. The statistics vectors are still output with a valid pulse at the end of the rejected frame. If a frame is not rejected during the destination address, `rx_axis_mac_tvalid` is asserted as normal though the frame can still be rejected at a later point through the assertion of `rx_axis_mac_tuser` at the end of the frame. This is described in more detail in [Frame Filter](#).

Receive Statistics Vector

The statistics for the frame received are contained within the `rx_statistics_vector` output. [Table 9: Bit Definition for the Receiver Statistics Vector](#) defines the bit field for the vector.

All bit fields, with the exception of `BYTE_VALID` are valid only when the `rx_statistics_valid` is asserted, as illustrated in the following figure. `BYTE_VALID` is significant on every validated receiver cycle.

Figure 26: Receiver Statistics Vector Timing



Transmitting Outbound Frames

Ethernet frames to be transmitted are presented to the user logic on the transmitter subset of the AXI4-Stream user-side interface. For port definition, see [Transmitter Interface](#). All transmitter signals are synchronous to the `tx_mac_aclck` clock if present or `gtx_clk` if not.

Normal Frame Transmission

The timing of a normal outbound frame transfer at 1 Gbps can be seen in [Figure 27: Normal Frame Transmission at 1 Gbps Across User Interface](#) with the timing at 100 Mbps shown in [Figure 28: Normal Frame Transmission at 100 Mbps Across User Interface \(MII/GMII\)](#) and [Figure 29: Normal Frame Transmission at 100 Mbps Across User Interface \(RGMII\)](#). When you want to transmit a frame, it places the first column of data onto the `tx_axis_mac_tdata` port and asserts a 1 onto `tx_axis_mac_tvalid`.

The TEMAC core accepts the first two bytes of data by asserting `tx_axis_mac_tready` and then waits until it is allowed to transmit and it then accepts the remainder of the frame. You must be capable of supplying new data on the following cycle when data has been taken, indicated by the assertion of `tx_axis_mac_tready`. The end of frame is signaled to the MAC core by asserting `tx_axis_mac_tlast` on the final byte of the frame.

At 1 Gbps, data can be taken every 8 ns; at 100 Mbps, data is taken every 80 ns; at 10 Mbps, data is taken every 800 ns (on an average). In all the cases, `tx_axis_mac_tready` qualifies when data is taken by the MAC. [Figure 28: Normal Frame Transmission at 100 Mbps Across User Interface \(MII/GMII\)](#) shows the use of `tx_axis_mac_tready` to throttle the data when the core has been generated with either an MII or GMII interface. In this mode, the timing at 100 Mbps and 10 Mbps is identical as `tx_mac_aclk` is sourced by the PHY at the required frequency (25 MHz or 2.5 MHz).

When the core is generated with an RGMII interface or the Internal interface the timing at 10/100 Mbps is very different as shown in [Figure 29: Normal Frame Transmission at 100 Mbps Across User Interface \(RGMII\)](#). In this mode, the `tx_mac_aclk` remains at 125 MHz at all MAC speeds and the `tx_axis_mac_tready` is activated once every 10 cycles as shown in [Figure 29: Normal Frame Transmission at 100 Mbps Across User Interface \(RGMII\)](#) or once every 100 cycles at 10 Mbps. This is not true for the first 2 bytes of frame data where the data pipeline fills at full rate.

For maximum flexibility in switching applications, the Ethernet frame parameters (destination address, source address, length/type and optionally FCS) are encoded within the same data stream that the frame payload is transferred upon rather than on separate ports.

Figure 27: Normal Frame Transmission at 1 Gbps Across User Interface

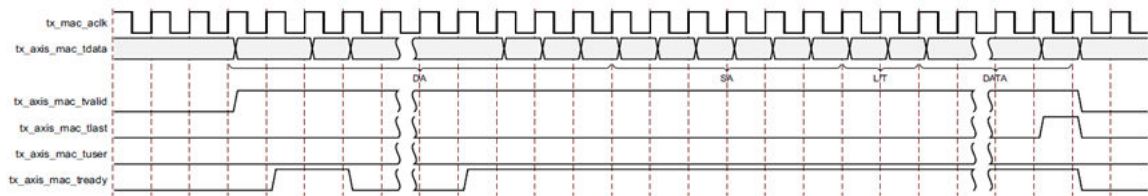


Figure 28: Normal Frame Transmission at 100 Mbps Across User Interface (MII/GMII)

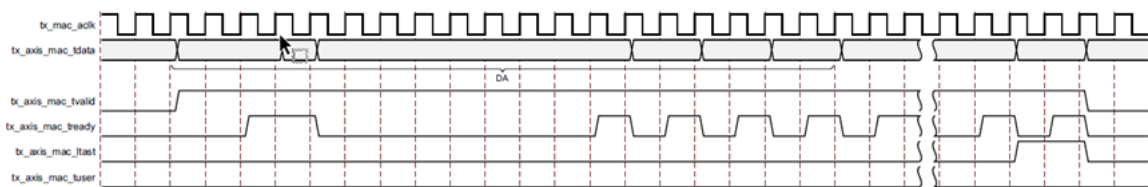
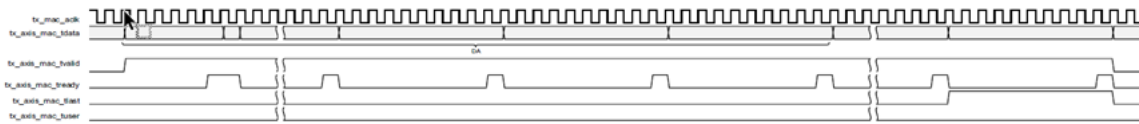


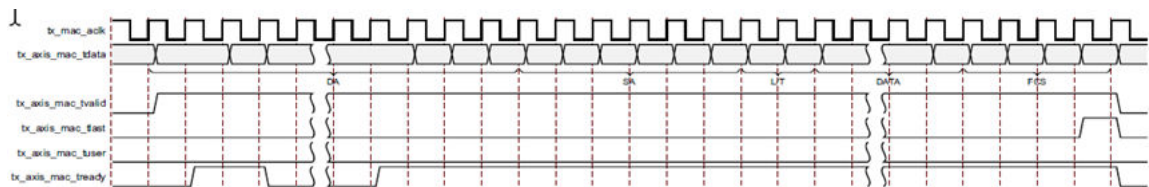
Figure 29: Normal Frame Transmission at 100 Mbps Across User Interface (RGMII)

Padding

When fewer than 46 bytes of data are supplied by you to the MAC core, the transmitter module adds padding up to the minimum frame length. The exception to this is when the MAC core is configured for user-passed FCS. In this case, you must also supply the padding to maintain the minimum frame length.

User-Supplied FCS Passing

If the MAC core is configured to have the FCS field passed in by you, the transmission timing is as depicted in the following figure. In this case, it is your responsibility to ensure that the frame meets the Ethernet minimum frame length requirements. If frame length requirements are not met, the core appends zeros at the end of the supplied frame to meet the minimum frame length. Although this does not cause the [Transmit Statistics Vector](#) to indicate a bad frame, it results in an errored frame as received by the link partner MAC (due to the detection of an FCS error).

Figure 30: Frame Transmission with User-Supplied FCS

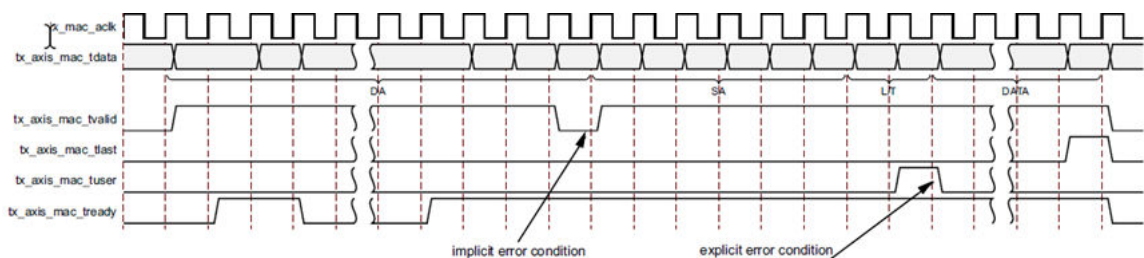
User Error Indication

The following figure shows an example of the timing for an aborted transfer. This can occur, for example, if a FIFO connected to the AXI4-Stream TX interface empties before a frame is completed. When you assert `tx_axis_mac_tuser` during a frame transmission, the MAC core inserts an error code to corrupt the current frame and then falls back to idle transmission. It is your responsibility to re-queue the aborted frame for transmission. It is also possible to abort a frame by de-asserting `tx_axis_mac_tvalid` before the final byte of the frame. It is classed by the MAC as a frame underrun because it does not buffer the data and any gap is passed directly to the PHY. To avoid incorrect data being output, this is classed as an implicit error condition and the frame is aborted.

The `tx_axis_mac_tuser` signal can be asserted at any time during active frame transmission. If it occurs prior to the MAC accepting the third byte of the frame, indicating it is actively transmitting to the PHY, it is possible to provide new frame data to the MAC and avoid the transmission of the aborted frame entirely. If this new data is not provided or arrives too late, a minimum sized errored frame is output. There is a possibility of MAC locking up if you present a stream of very short length errored frames on the TX AXI4-Stream interface. In such cases, ensure that a minimum of four bytes frame data has been accepted by the MAC before asserting `tuser`.

If the very first byte of the transfer is aborted, the MAC will not assert `tready` because there is no input for the MAC to perform the task. Ensure that your designs do not present such inputs to the MAC.

Figure 31: Frame Transmission with Underrun



When `tuser` is asserted before the third byte, the frame is aborted and the next frame can be sent out. When `tuser` is asserted in between the packet transition, the PHY error signal is asserted for the corresponding frame. If a collision occurs and the retransmission signal is asserted, `tvalid` must be asserted within 6 clk cycles from the retransmission assert signal to resend the packet. Otherwise, the retransmit count is reset.

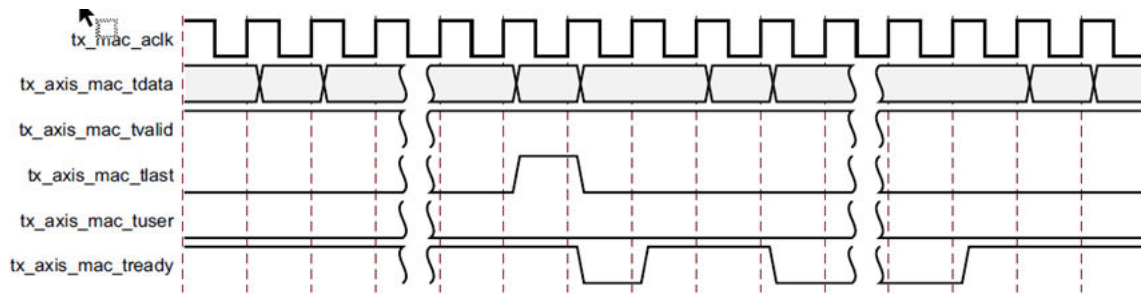
Back-to-Back Transfers

The following figure shows the MAC user immediately ready to transmit a second frame of data following completion of its first frame. In this figure, the end of the first frame is shown on the left with the assertion of `tx_axis_mac_tlast`. On the cycle, immediately following the final byte of the first frame, `tx_axis_mac_tvalid` remains High to indicate that the first byte of the destination address of the second frame is on `tx_axis_mac_tdata` awaiting transmission.

When the MAC core is ready to accept data, `tx_axis_mac_tready` is asserted and the transmission continues in the same manner as in the case of the single frame. The MAC core defers the assertion of `tx_axis_mac_tready` appropriately to comply with inter-packet gap requirements and flow control requests.

If the MAC core is operating at 1 Gbps in half-duplex mode, the timing shown in the following figure is required to take advantage of frame bursting. The MAC core is only guaranteed to retain control of the medium if the `tx_axis_mac_tvalid` signal is High immediately after the end of the previous packet. For details on frame bursting, see IEEE 802.3-2008.

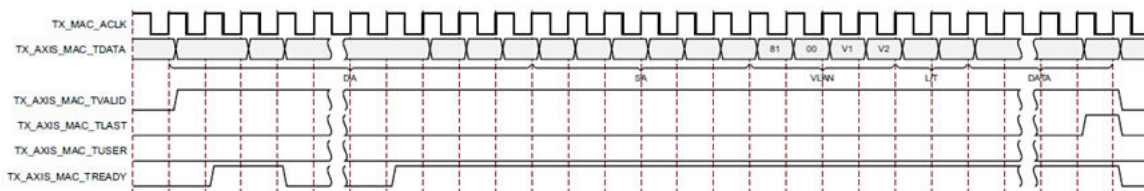
Figure 32: Back-to-Back Frame Transmission



VLAN Tagged Frames

Transmission of a VLAN tagged frame (if enabled) can be seen in the following figure. The handshaking signals across the interface do not change. However, the VLAN type tag 81-00 must be supplied by you to signify that the frame is VLAN tagged. You also supply the two bytes of Tag Control Information, V1 and V2, at the appropriate times in the data stream. More information on the contents of these two bytes can be found in IEEE 802.3-2008.

Figure 33: Transmission of a VLAN Tagged Frame



Maximum Permitted Frame Length

The maximum legal length of a frame specified in IEEE 802.3-2008 is 1,518 bytes for non-VLAN tagged frames. VLAN tagged frames can be extended to 1,522 bytes. When jumbo frame handling is disabled and you attempt to transmit a frame that exceeds the maximum legal length, the MAC core inserts an error code to corrupt the current frame and the frame is truncated to the maximum legal length. When jumbo frame handling is enabled, frames that are longer than the legal maximum are transmitted error-free.

It is also possible to specify a different maximum frame size. If this is enabled and the frame exceeds the configured value then the frame is corrupted. In this case, VLAN frames are not treated separately. If jumbo frame handling is enabled, it takes precedence and the configured value is ignored.

Packets < 64 bytes are considered undersized according to the Ethernet 802.3-2012 specification. Undersized packets might cause the core to lock up and this must be avoided.

Frame Collisions – Half-Duplex Operation Only

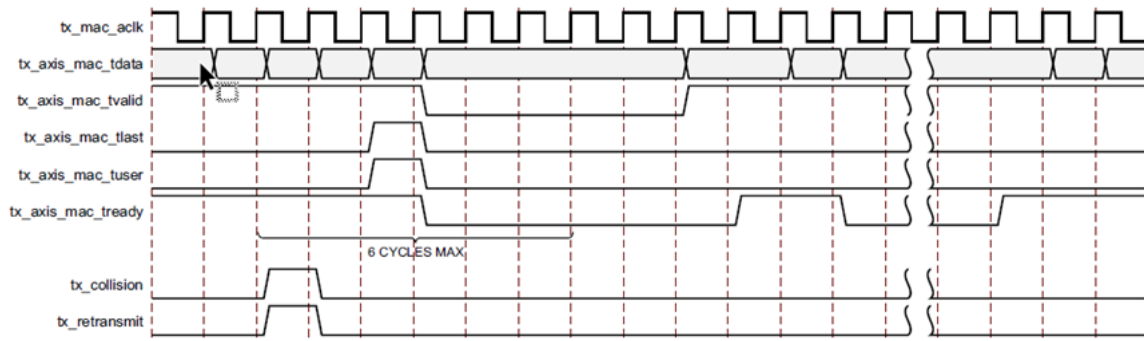
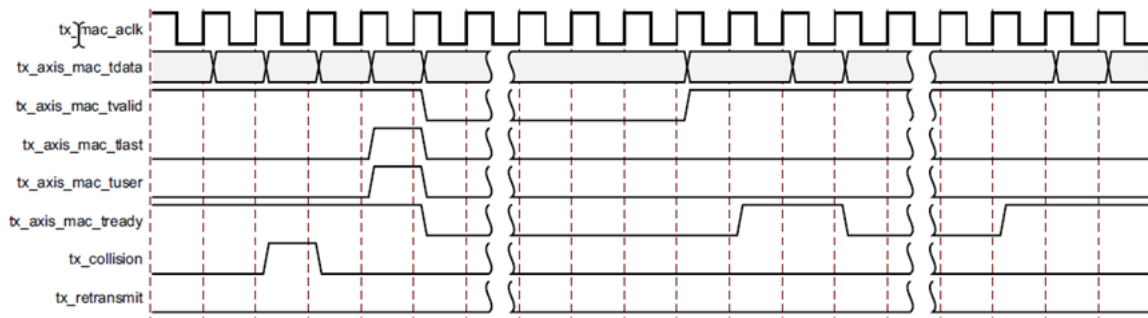
In half-duplex Ethernet operation, collisions occur on the medium as a matter of course; this is how the arbitration algorithm is fulfilled. In the case of a collision, the MAC core signals to you that data might need to be resupplied as follows.

- If there is a collision, the `tx_collision` signal is set to 1 by the MAC core. If a frame is in progress, you must abort the transfer asserting `tx_axis_mac_tlast` and `tx_axis_mac_tuser`.
- If the `tx_retransmit` signal is 1 in the same cycle that the `tx_collision` signal is 1, you must then resubmit the previous frame to the MAC core for retransmission, `tx_axis_mac_tvalid` must be asserted to the MAC core within six cycles of the `tx_retransmit` signal. If `tx_axis_mac_tvalid` is asserted later than this, the MAC assumes that the frame is not retransmitted and the number of retransmission attempts counter within the MAC is reset. This case is illustrated in [Figure 34: Collision Handling – Frame Retransmission Required](#).

If any frame presented to the user interface is shorter than the collision window (slot time) as defined in IEEE Std 802.3-2008, a retransmission request can occur after the end of the frame as observed on the user interface. Therefore, the user logic (which might have queued a subsequent frame for transmission) might then have to rewind back to the previous frame. In this case, the current frame has to be aborted by asserting `tx_axis_mac_tlast` with `tx_axis_mac_tuser` and the previous frame data should be re-supplied on the `tx_axis_mac_tdata[7:0]` port within the same six cycles illustrated in [Figure 34: Collision Handling – Frame Retransmission Required](#).

For reference, the collision window (slot time) is 64 validated clock cycles when operating at 10 Mbps and 100 Mbps speeds (corresponding to 64-bytes of frame data), and 512 clock cycles when operating at 1 Gbps speed (corresponding to 512-bytes of frame data).

- If the `tx_retransmit` signal is 0 in the same cycle that the `tx_collision` signal is 1, the number of retries for this frame has exceeded the Ethernet specification or the collision has been classed as late, and the frame should be dropped by you. You can then make any new frame available to the MAC for transmission without timing restriction. This case is illustrated in [Figure 35: Collision Handling – No Frame Retransmission Required](#).

Figure 34: Collision Handling – Frame Retransmission Required**Figure 35: Collision Handling – No Frame Retransmission Required**

Identifying the frame that has undergone collision can be difficult for certain traffic scenarios. For example, if the MAC asserts the `tx_collision` signal when a back-to-back frame transmission is in progress or when a short frame (that is, less than the slot time) is followed immediately by another frame for which the MAC has already accepted two bytes. Here you might not recognize if it was the first or the second frame that has undergone collision and thus not know which frame to present again to MAC for the retransmission. In such cases, you can monitor `tx_statistic_vector[0]` when collision is indicated to confirm if the first frame has been successfully transmitted. If it has, the collision indication is for the second frame and vice versa.

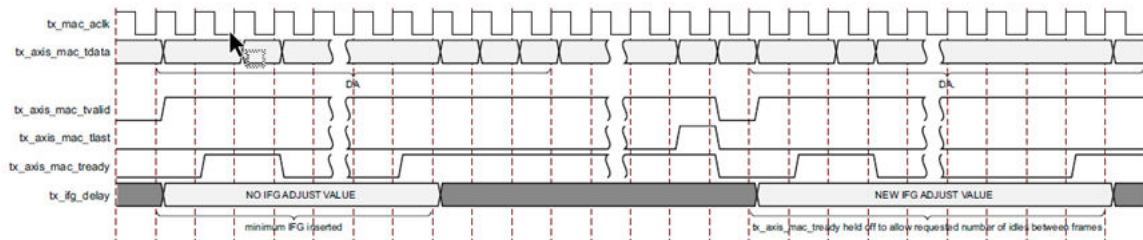
Interframe Gap Adjustment – Full-Duplex Mode Only

A configuration bit in the transmitter control register allows you to control the length of the interframe gap transmitted by the MAC on the physical interface. If this function is selected, the MAC exerts back pressure on the user interface to delay the transmission of the next frame until the requested number of idle cycles has elapsed. The number of idle cycles is controlled by the value on the `tx_ifg_delay` port seen at the start of frame transmission on the user interface. The following figure shows the MAC operating in this mode.

The minimum interframe gap supported is dependent upon the support of half-duplex operation. If half-duplex is supported, the minimum IFG possible is eight bytes of transmitted data. If the MAC only supports full-duplex operation then this reduces the minimum possible IFG to four bytes of transmitted data.

In both cases, the interframe gap used when the Interframe Gap Adjust Enable bit set to 0 is the minimum value as specified in the IEEE 802.3-2008 standard. This corresponds to 12 bytes of transmitted data on the GMMI/MII interface. The value on the tx_ifg_delay port must be equal to or larger than four or eight to have an effect as described previously.

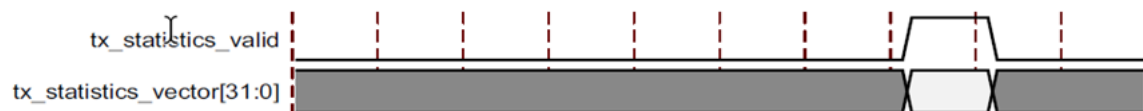
Figure 36: Interframe Gap Adjustment



Transmit Statistics Vector

The statistics for the frame transmitted are contained within the tx_statistics_vector output. The bit field definition for the Vector is defined in [Table 5: Bit Definition for the Transmitter Statistics Vector](#). All bit fields, except BYTE_VALID, are valid only when the tx_statistics_valid is asserted, as illustrated in the following figure. BYTE_VALID is significant on every transmitter cycle that the clock enable is High. tx_statistics_vector bits 28 down to 20 inclusive are for half-duplex only and are set to logic 0 when operating in full-duplex mode.

Figure 37: Transmitter Statistics Vector Timing

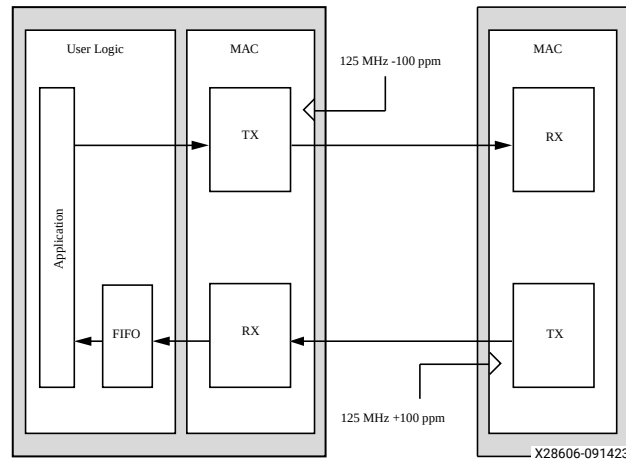


Flow Control Using IEEE 802.3

This section describes the operation of the flow control logic of the TEMAC solution. The flow control block is designed based on the clause 31 of the IEEE 802.3-2008 standard. In full duplex mode, the MAC can be configured to transmit pause requests and to act on their reception, these modes of operation can be independently enabled or disabled.

Flow Control Overview

Figure 38: Requirement for Flow Control



The previous figure illustrates the requirement for Flow Control at 1 Gbps. The MAC on the right side of the figure has a reference clock slightly faster than the nominal 125 MHz. The MAC on the left side of the figure has a reference clock slightly slower than the nominal 125 MHz. This results in the MAC on the left side of the figure not being able to match the full line rate of the MAC on the right side (due to clock tolerances). The MAC at the left is illustrated as performing a loopback implementation, which results in the FIFO filling up over time. Without Flow Control, this FIFO eventually fills and overflows, resulting in the corruption or loss of Ethernet frames. Flow Control is one solution to this issue.

Flow Control Basics

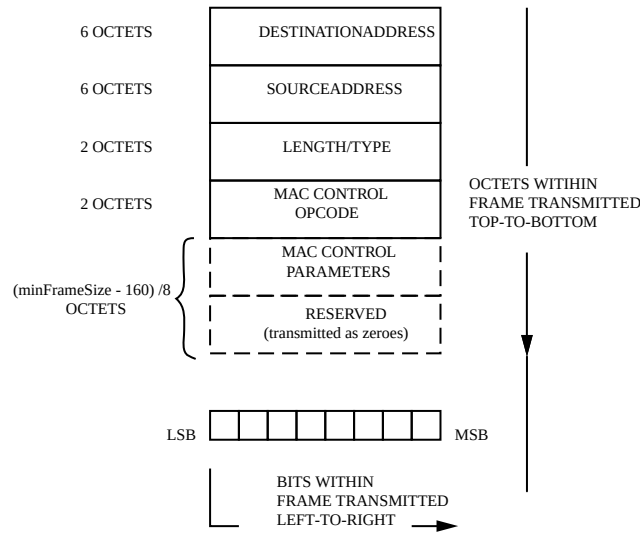
A MAC can transmit a Pause Control frame to request that its link partner cease transmission for a specific period of time. For example, the left MAC in the previous figure can initiate a pause request when its user FIFO (illustrated) reaches a nearly full state.

A MAC should respond to received Pause Control frames by ceasing transmission of frames for the period defined in the received pause control frame. For example, the right MAC of the previous figure can cease transmission after receiving the Pause Control frame transmitted by the left MAC. In a well designed system, the right MAC ceases transmission before the user FIFO of the left MAC overflows to provide time to empty the FIFO to a safe level before resuming normal operation. This practice safeguards the system against FIFO overflow conditions and frame loss.

Pause Control Frames

Control frames are a special type of Ethernet frame defined in clause 31 of the IEEE 802.3 standard. Control frames are identified from other frame types by a defined value placed into the length/type field (the MAC Control Type code). The following figure illustrates control frame format.

Figure 39: MAC Control Frame Format



X28608-091423

A Pause Control frame is a special type of Control frame identified by a defined value placed into the MAC Control opcode field.

Note: MAC Control OPCODES other than for Pause (Flow Control) frames have also been defined for Ethernet Passive Optical Networks.

The MAC Control Parameter field of the Pause Control frame contains a 16-bit field that contains a binary value directly relating to the duration of the pause. This defines the number of pause_quantum (512-bit times of the particular implementation). At 1 Gbps, a single pause_quantum corresponds to 512 ns. At 100 Mbps, a single pause_quantum corresponds to 5,120 ns, and at 10 Mbps, a single pause_quantum corresponds to 51,200 ns.

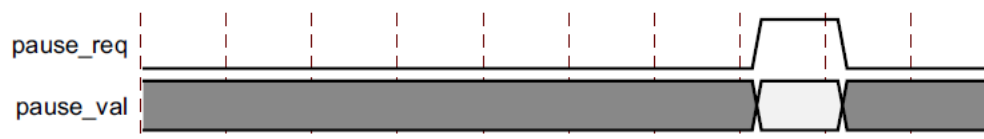
Flow Control Operation of the TEMAC

Transmitting a Pause Control Frame

Core-Initiated Pause Request

If the core is configured to support transmit flow control, you can initiate a flow control frame by asserting `pause_req` while the pause value is on the `pause_val` bus. The following figure illustrates this timing. Pause request signals are synchronous to the `gtx_clk` clock.

Figure 40: Pause Request Timing



This action causes the core to construct and transmit a Pause Control frame on the link with the following MAC Control frame parameters (see [Figure 39: MAC Control Frame Format](#)):

- The destination address used is an IEEE 802.3 globally assigned multicast address (which any Flow Control capable MAC responds to).
- The source address used is the configurable Pause Frame MAC Address.
- The value sampled from the `pause_val[15:0]` port at the time of the `pause_req` assertion is encoded into the MAC Control Parameter field to select the duration of the pause (in units of `pause_quantum`).

If the transmitter is currently inactive at the time of the pause request, this Pause Control frame is transmitted immediately. If the transmitter is currently busy, the current frame being transmitted is allowed to complete. The Pause Control frame then follows in preference to any pending user supplied frame. A Pause Control frame initiated by this method is transmitted even if the transmitter itself has ceased transmission in response to receiving an inbound pause request.

- Only a single pause control frame request is stored by the transmitter. If the `pause_req` signal is asserted numerous times in a short time period (before the control pause frame transmission has had a chance to begin), only a single pause control frame is transmitted. The `pause_val[15:0]` value used is the most recent value sampled.

If the PFC feature is included, the flow control includes additional logic to enable it to be used as an XON/XOFF (transmission ON/ transmission OFF) interface. This is described in more detail in [XON/XOFF Extended Functionality](#).

User-Initiated Pause Request

For maximum flexibility, the flow control logic can be disabled in the core and alternatively implemented in the user logic connected to the core. Any type of Control frame can be transmitted through the core by the TX AXI4-Stream interface using the same transmission procedure as a standard Ethernet frame (see [Transmitting Outbound Frames](#)).

XON/XOFF Extended Functionality

If the Ethernet MAC has been generated with PFC functionality included but configured for IEEE 802.3 functionality then the Ethernet MAC supports XON/XOFF functionality. If, as shown in the following figure, the `pause_req` signal is asserted, the Ethernet MAC generates a new 802.3 pause frame. If it is subsequently held High for more than one clock cycle, the Ethernet MAC automatically generates a new pause frame each time the internal quanta count of the Ethernet MAC reaches the number of quanta specified by the legacy refresh value (in either the register or the configuration vector). This is shown as an XOFF refresh frame in the following figure. When the `pause_req` is de-asserted, an XON frame (standard pause frame with the pause quanta forced to zero) is automatically generated if the auto XON feature is enabled. This functionality is shown in the following figure. If auto XON is not enabled the remaining quanta are left to expire naturally at the link partner.

Figure 41: XON/XOFF Frame Transmission



- To enable an Auto XON after a single pause request, ensure that the spacing between these two pause requests is at least equal to the End of the current frame + IFG + Pause frame duration for a single pause.

Receiving a Pause Control Frame

Core-Initiated Response to a Pause Request

An error-free Control frame is a received frame matching the format of [Figure 39: MAC Control Frame Format](#). It must pass all standard receiver frame checks (for example, FCS field checking). In addition, the control frame received must be exactly 64 bytes in length (from destination address through to the FCS field inclusive). This is minimum legal Ethernet MAC frame size and the defined size for control frames.

Any Control frame received that does not conform to these checks contains an error, and it is passed to the RX AXI4-Stream as an errored packet (`rx_axis_mac_tuser` asserted).

Pause Frame Reception Disabled

When pause control reception is disabled, an error-free control frame is received through the RX AXI4-Stream interface. In this way, the frame is passed to the user logic for interpretation (see [User-Initiated Response to a Pause Request](#)).

Pause Frame Reception Enabled

When pause control reception is enabled and an error-free frame is received by the MAC core, the following frame decoding functions are performed:

- The destination address field is matched against the IEEE 802.3 globally assigned control multicast address (01-80-C2-00-00-01) or the configurable Pause Frame MAC Address.
- The length/type field is matched against the MAC Control Type code.
- If the second match is TRUE, the OP CODE field contents are matched against the Ethernet MAC control OP CODE for pause frames.

If all the previously listed checks are TRUE, and the frame is of minimum legal size OR larger and control frame length checking is disabled, the 16-bit binary value in the MAC control parameters field of the control frame is then used to inhibit transmitter operation for the required number of pause_quantum. This inhibit is implemented by delaying the assertion of `tx_axis_mac_tready` at the TX AXI4-Stream interface until the requested pause duration has expired. Because the received pause frame has been acted upon, it is passed to the RX AXI4-Stream interface as an errored packet to indicate that it can now be dropped.

If the second match is TRUE and the frame is not exactly 64 bytes in length (when control frame length checking is enabled), the reception of any frame is considered to be an invalid control frame. This frame is ignored by the flow control logic and passed to the RX AXI4-Stream interface as an errored frame. In this case, the frame is errored even if the flow control is not enabled.

If any of the previously listed checks are FALSE, the frame is ignored by the Flow Control logic and passed up to the user logic for interpretation by marking it as a good frame. It is then the responsibility of the MAC user logic to decode, act on (if required), and drop this control frame.

- Any frame in which the length/type field contains the MAC Control Type in the length/type field should be dropped by the receiver user logic. All Control frames are indicated by `rx_statistics_vector` bit 19 (see [Receive Statistics Vector](#)).

User-Initiated Response to a Pause Request

For maximum flexibility, the flow control logic can be disabled in the core and alternatively implemented in the user logic connected to the core. Any type of error-free Control frame is then passed through the core without error.

In this way, the frame is passed to you for interpretation. It is your responsibility to drop this control frame and to act on it by ceasing transmission through the core, if applicable.

Flow Control Implementation Example

This explanation is intended to describe a simple (but crude) example of a Flow Control implementation to introduce the concept. Consider the system shown in [Figure 38: Requirement for Flow Control](#). To summarize the example, the MAC on the left-hand side of the figure cannot match the full line rate of the right-hand MAC due to clock tolerances. Over time, the FIFO illustrated fills and overflows. The aim is to implement a Flow Control method, which over a long time period, reduces the full line rate of the right-hand MAC to average that of the lesser full line rate capability of the left-hand MAC.

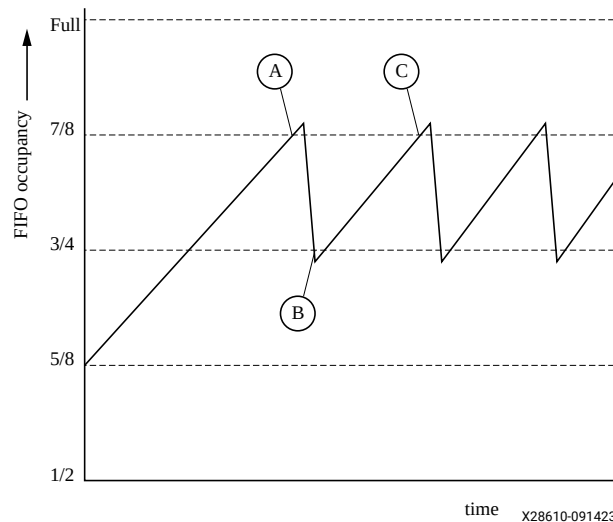
Method

1. Choose a FIFO nearly to full occupancy threshold ($7/8$ occupancy is used in this description). When the occupancy of the FIFO exceeds this occupancy, initiate a single pause control frame with `0xFFFF` used as the `pause_quantum` duration (`0xFFFF` is placed on `pause_val[15:0]`). This is the maximum pause duration. This causes the right-hand MAC to cease transmission and the FIFO of the left-hand MAC starts to empty.
2. Choose a second FIFO occupancy threshold ($3/4$ is used in this description). When the occupancy of the FIFO falls below this occupancy, initiate a second pause control frame with `0x0000` used as the `pause_quantum` duration (`0x0000` is placed on `pause_val[15:0]`). This indicates a zero pause duration, and upon receiving this pause control frame, the right-hand MAC immediately resumes transmission (it does not wait for the original requested pause duration to expire). This pause control frame can therefore be considered a “pause cancel” command.

Operation

The following figure illustrates the FIFO occupancy over time.

Figure 42: Flow Control Implementation Triggered from FIFO Occupancy



The following text describes the sequence of flow control operations in this example.

1. The average FIFO occupancy of the left-hand MAC gradually increases over time due to the clock tolerances. At point A, the occupancy has reached the threshold of $7/8$ occupancy. This triggers the maximum duration pause control frame request.
2. Upon receiving the pause control frame, the right-hand MAC ceases transmission.

3. After the right-hand MAC ceases transmission, the occupancy of the FIFO attached to the left-hand MAC rapidly empties. The occupancy falls to the second threshold of 3/4 occupancy at point B. This triggers the zero duration pause control frame request (the pause cancel command).
4. Upon receiving this second pause control frame, the right-hand MAC resumes transmission.
5. Normal operation resumes and the FIFO occupancy again gradually increases over time. At point C, this cycle of Flow Control repeats.

Using Priority Flow Control

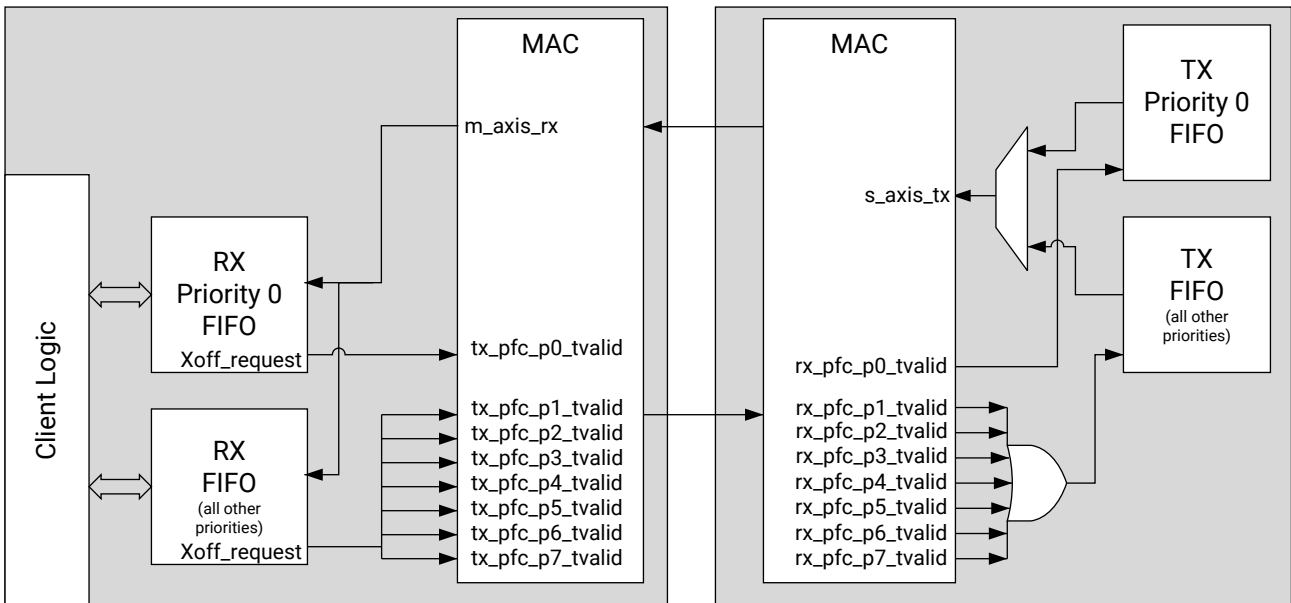
This section describes the operation of the priority flow control logic of the core. Priority-based flow control is defined in the *IEEE Standard 802.1Qbb*. The Ethernet MAC can be configured to transmit and receive priority-based flow control requests. These modes of operation can be independently enabled or disabled.

See [MAC Configuration Registers](#). Operation of the MAC with both PFC and 802.3 flow control enabled is not supported as these modes of operation are mutually exclusive.

Priority Flow Control Requirement

As described in [Flow Control Using IEEE 802.3](#), the basic requirement for flow control is to avoid dropping frames if a receiving port cannot process frames at the rate at which they are being provided. IEEE 802.3 pause frames operate on the entire link, which is not ideal when there are multiple priority queues awaiting transmission with each queue originating from a separate FIFO. In this case, the transmission of one or more specific queues (up to eight) can be inhibited using priority-based flow control with the eight frame priorities as defined by IEEE 802.1p. The following figure illustrates an implementation of two different priority queues.

Figure 43: Priority Flow Control Requirement



X25631-080321

A MAC can transmit a PFC frame to request that its link partner cease transmission of one or more of the eight frame priority queues for a defined period of time. For example, the Ethernet MAC at the left side of the previous figure can initiate a PFC request if its RX priority 0 FIFO reaches a nearly full state, this is considered to be an XOFF request. To make this request, the RX Priority 0 FIFO drives the `tx_pfc_p0_tvalid` signal High, and holds it High until it is ready to accept data again (this is one of the modes of operation supported by the MAC). If this FIFO remains nearly full for an extended period of time, the Ethernet MAC automatically generates a new PFC request to prevent the link partner from restarting transmission of this frame class when the initially requested duration (the original requested number of pause quanta) has expired. When the FIFO level drops to a specified level and the FIFO is able to accept data again, the requirement to inhibit transmission of that frame class can be removed by driving the `tx_pfc_p0_tvalid` signal Low.

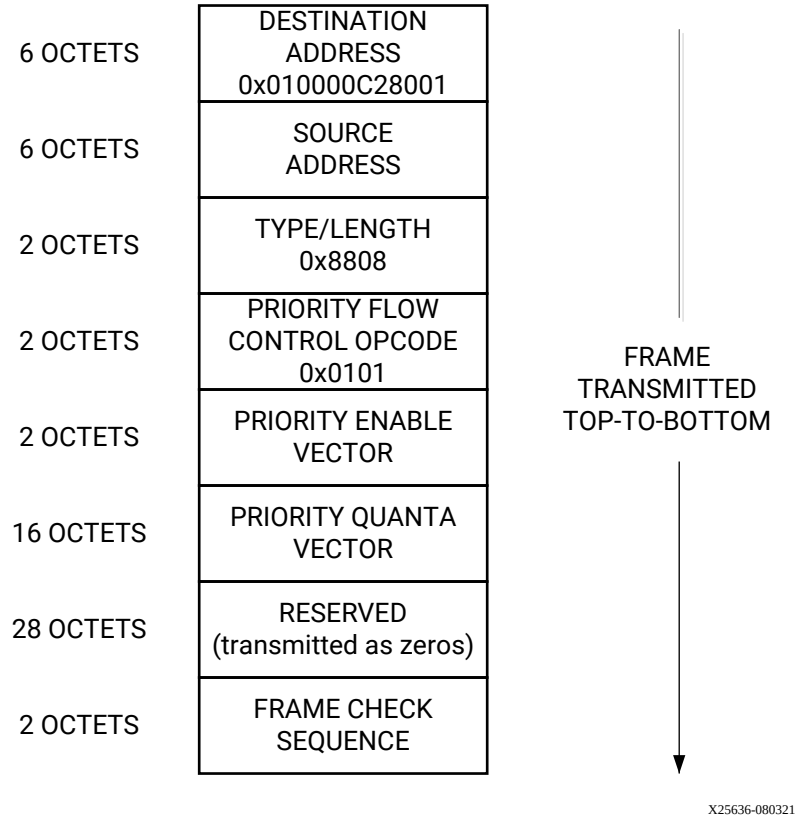
Now, the Ethernet MAC core can be optionally configured to generate a PFC frame to cancel any remaining pause quanta (duration) by placing a zero pause quanta value into the PFC frame for priority 0, this is considered to be an XON request.

When the Ethernet MAC core receives priority-based flow control frames it cannot directly cease transmission of the specified priority (or priorities) without ceasing transmission of all frames. To enable specific priorities to be paused the MAC has a per-priority pause request output which can be asserted to inhibit transmission of specific priorities. For example, the Ethernet MAC at the right side of the previous figure might cease transmission from its TX priority 0 FIFO queue after receiving the PFC frame transmitted by the left-hand MAC. In a well-designed system, the right side MAC would cease transmission before the RX priority 0 FIFO of the left side MAC overflowed. This provides time for the FIFO to be emptied to a safe level before normal operation resumes and safeguards the system against FIFO overflow conditions and frame loss.

Priority-Based Flow Control Frames

Priority-based flow control frames are a special type of Ethernet frame defined in IEEE 802.1Qbb. Control frames are identified from other frame types by a defined value placed into the length/type field (the MAC Control Type code). The priority-based flow control frame format is shown in the following figure.

Figure 44: MAC Priority Control Flow Frame Format



A PFC frame is a special type of control frame identified by a defined value placed into the OPCODE field. This is shown in the previous figure.

The 16-bit priority enable vector contains eight priority enable bits with all other bits set to zero. The priority vector field contains eight 16-bit quanta values, one for each priority, with priority 0 being the first. This defines the number of pause_quantum (512-bit times of the particular implementation) for the priority pause duration request. For 10 Gb Ethernet, a single pause_quantum corresponds to 51.2 ns.

Transmitting a PFC Frame

Core-Initiated Request

There are three methods of generating a PFC frame, all of which assume that TX PFC is enabled and any actively used priority has the relevant TX priority enable set to 1.

1. The client asserts one or more of the eight `tx_pfc_p[0-7]_tvalid` signals. If the `tx_pfc_p[0-7]_tvalid` signal is asserted for more than a single cycle then this is considered to be an XOFF request and the MAC refreshes the relevant quanta at the link partner whenever a new PFC frame is transmitted until the `tx_pfc_p[0-7]_tvalid` is de-asserted. If `tx_pfc_p[0-7]_tvalid` is asserted for a single cycle, it is assumed that any required refresh is directly controlled by a subsequent reassertion of the respective `tvalid` as required.
2. The client keeps a priorities `tx_pfc_p[0-7]_tvalid` signal High and the internal quanta count of the Ethernet MAC reaches the pre-programmed refresh value for that priority. On reaching this refresh value, the MAC “refreshes” the priority pause request by resending a new PFC frame with the pre-programmed pause value for the given priority. For each of the eight priorities, there is a configuration register that contains both the pause duration and the refresh value. (see [Table 35: Per Priority Quanta/Refresh Register \(0x480/0x49C\)](#))
3. The client has held a `tx_pfc_p[0-7]_tvalid` High for more than one cycle this is then de-asserted and the TX auto XON feature is enabled. This results in a new PFC frame with the relevant priorities quanta being both enabled and forced to zero. This is considered to be an XON request for that priority.

When any new PFC frame is transmitted it also resends the quanta for any currently active, enabled priority, effectively refreshing each priority quanta at the link partner. This also restarts the internal quanta count.

The eight `tx_pfc_p[0-7]_tvalid` signals are synchronous with respect to `tx_clk0`. The various transmit methods can be seen in the following figure. In the following figure, the `tx_pfc_p0_tvalid` is asserted and held High. This results in a PFC frame with only the P0 quanta enabled (set to 0xFFFF). Several cycles later, `tx_pfc_p2_tvalid` is asserted for a single cycle and this results in a new PFC frame with both the P0 and P2 quantas enabled (and restarts the internal quanta count).

Any control frame received that does not conform to these checks contains an error and is passed to the receiver client with the `rx_axis_tuser` signal asserted on the cycle that `rx_axis_tlast` is asserted.

PFC Frame Reception Disabled

When PFC reception is disabled (see [MAC Configuration Registers](#)), an error free control frame is received through the client interface with the `rx_axis_tuser` signal de-asserted. In this way, the frame is passed to the client logic for interpretation (see [User-Initiated Response to a Pause Request](#)).

Pause Frame Reception Enabled

When pause control reception is enabled (see [MAC Configuration Registers](#)) and an error-free frame is received by the Ethernet MAC core, the frame decoding functions are performed:

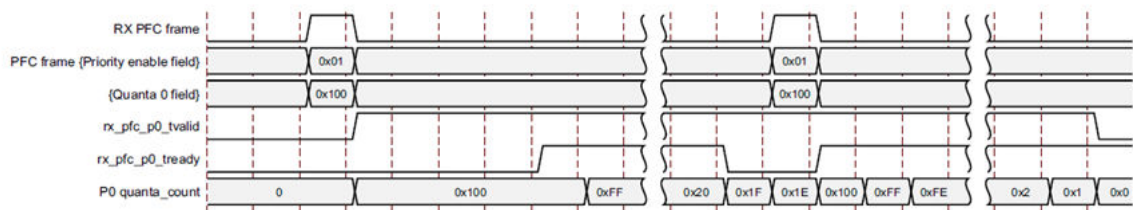
- The destination address field is matched against the MAC control multicast address or the configured source address for the Ethernet MAC. (see [MAC Configuration Registers](#))
- The length/type field is matched against the MAC Control type code 88-08.
- The opcode field contents are matched against the priority-based flow control opcode.

If any of these checks are FALSE or the MAC receiver PFC is disabled, the frame is ignored by the PFC logic and passed to the client.

If the frame passes these checks, that is, minimum legal size or the Control Frame Length Check Disable is set, and the MAC receiver PFC is enabled, then the priority enable field and per priority quanta values are extracted from the frame. If a particular priority is enabled in the frame with a non-zero pause quanta value, the respective `rx_pfc_p[0-7]_tvalid` output is asserted and the requested quanta value loaded into the control logic for that priority. This control logic consists of a counter, which is loaded with the requested pause quanta value and decrements down to zero. The `rx_pfc_p[0-7]_tvalid` remains asserted while the local quanta counter is non zero. The local quanta counter does not start to decrement until `rx_pfc_p[0-7]_tready` is High.

Subsequent de-assertion of `rx_pfc_p[0-7]_tready` does not stop the quanta expiration. If the quanta counts down to zero, and not refreshed by reception of a new PFC frame with the respective priorities quanta enabled and non-zero, `rx_pfc_p[0-7]_tvalid` is de-asserted. An example of this is shown in the following figure, which shows the case where only one priority is active in a frame.

Figure 46: RX PFC Frame Reception



If at any time a new PFC frame is received with a priorities quanta enabled and set to zero, this is loaded into the local quanta count which results in the respective `rx_pfc_p[0-7]_tvalid` signal being de-asserted immediately to re-enable transmission of this particular priority queue.

Because the received PFC frame has been acted on, it is passed to the client with `rx_axis_tuser` asserted to indicate that it should be dropped.

- Any frame in which the length/type field contains the MAC Control Type should be dropped by the receiver client logic. All control frames are indicated by `rx_statistic_vector` bit 20 (see [Receive Statistics Vector](#)).

Client-Initiated Response to a Pause Request

For maximum flexibility, flow control logic can be disabled in the core (see [MAC Configuration Registers](#)) and alternatively implemented in the client logic connected to the core. Any type of the error free control frame is then passed through the core with the `rx_axis_tuser` signal de-asserted. In this way, the frame is passed to the client for interpretation. It is the responsibility of the client to drop this control frame and to act on it by ceasing transmission of the appropriate priorities through the core, if applicable.

PFC Implementation Example

This section describes a simple example of a PFC implementation.

Consider the system illustrated in [Figure 43: Priority Flow Control Requirement](#). The client logic connected to the Ethernet MAC on the left side is unable to service the data in the RX priority 0 FIFO for an extended period of time. Over time, the RX priority 0 FIFO illustrated fills and overflows. The aim is to implement a PFC method which applies back pressure on only the priority 0 traffic to ensure no frames are dropped while allowing the other priority queues to continue.

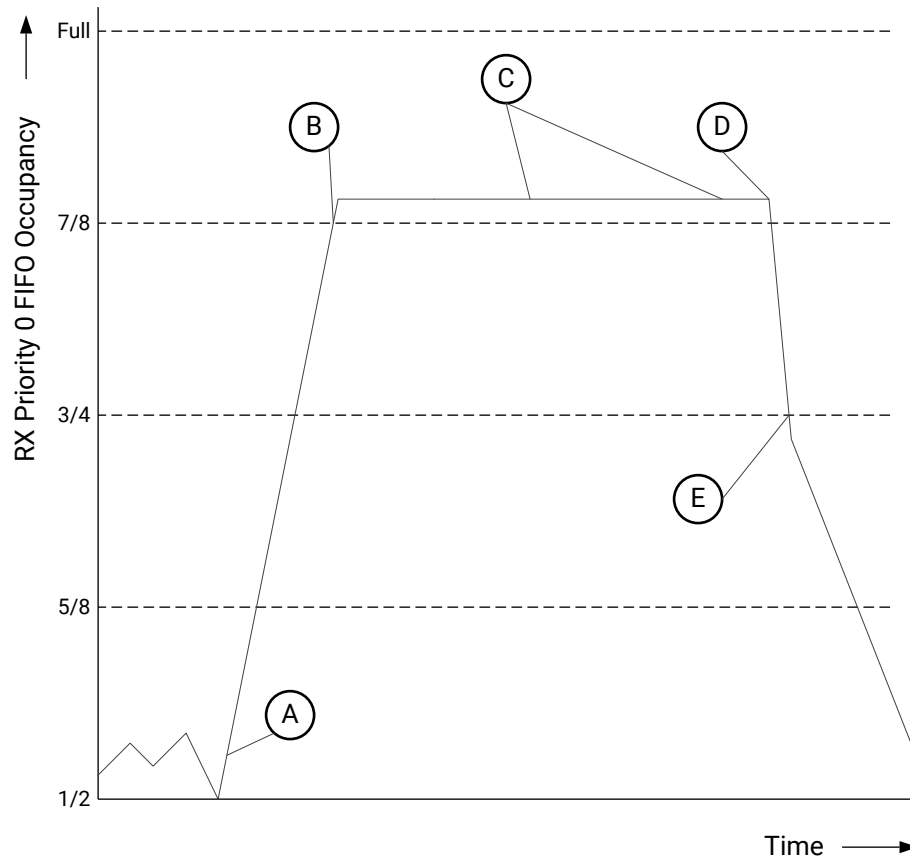
Method

1. Choose a RX priority 0 FIFO nearly-full occupancy threshold (7/8 occupancy is used in this description but the choice of threshold is implementation-specific). When the FIFO exceeds this occupancy, assert the XOFF request signal to initiate a PFC frame with priority 0 enabled and 0xFFFF used as the priority 0 `pause_quantum` duration (0xFFFF is the default value of the priority 0 quanta register). This is the maximum pause duration. This causes the left-hand MAC to transmit a PFC frame, which in turn causes the right-hand MAC to assert its `rx_pfc_p0_tvalid` to request that the TX priority 0 FIFO on the right-hand side stops transmission. If the client logic of the left-hand MAC continues to be unable to service the RX priority 0 FIFO then the left-hand Ethernet MAC automatically re-sends the PFC frame each time 0xFF00 quanta expires (this is the default value of the priority 0 refresh), that is, before the previously sent quanta has expired.
2. Choose a RX priority 0 FIFO occupancy threshold (3/4 is used in this description but the choice of threshold is implementation-specific). When the occupancy of the FIFO falls below this occupancy, send an XON request by de-asserting the `tx_pfc_p0_tvalid` signal. If the TX auto XON feature is enabled this initiates a PFC frame with priority 0 enabled and the priority 0 quanta set to 0x0000. This indicates a zero pause duration (XON), and upon receiving this PFC frame, the right-hand MAC de-asserts the `rx_pfc_p0_tvalid` allowing the TX priority 0 FIFO on the right to resume transmission (it does not wait for the original requested quantum duration to expire). If the TX auto XON feature is not enabled then no PFC frame is sent and transmission does not restart until the requested quantum duration has expired.

Operation

The following figure shows the Priority 0 FIFO occupancy over time.

Figure 47: Priority Flow Control Implementation Triggered from FIFO Occupancy



X25635-080321

1. The FIFO occupancy is maintained at a low level as the client logic is able to service the frames. At point A, the client logic is unable to service the FIFO and the occupancy increases. At point B, the FIFO has reached the threshold of $7/8$ occupancy. This triggers the XOFF request assertion, a PFC frame is generated and requested priority 0, traffic is stopped.
2. Upon receiving the PFC frame, the right-hand priority 0 FIFO ceases transmission.
3. The client logic remains unable to service the priority 0 FIFO for an extended duration, and subsequent PFC frames are automatically generated at C to ensure the Priority 0 FIFO on the right does not restart.
4. The client begins to service the priority 0 FIFO at point D, and the FIFO empties. The occupancy falls to the second threshold of $3/4$ occupancy at point E. This triggers the XON PFC frame request.
5. Upon receiving this PFC frame, the right-hand MAC de-asserts the `rx_pfc_p0_tvalid` and the priority 0 FIFO resumes transmission.
6. Normal operation resumes.

Statistics Counters

The Statistics counters ([Table 26: Statistics Counter Definitions](#)) are only available when the management interface is enabled. These counters can be defined to be either 32 or 64 bits wide, with 64 bits being the default. Two registers capture the counter values when defined as 64 bits wide. In all cases, a read of the lower 32-bit value causes the upper 32 bits to be sampled. A subsequent read of the upper 32-bit location returns this sampled value. If an upper location for a different counter is read, the access returns an error condition to indicate that the returned data value is incorrect.

Note: All statistics counters are Read Only. A write to any location is ignored and returns an error.

The Statistics counters can be optionally reset upon a global reset, with this being enabled using the Vivado Integrated Design Environment (IDE). They do not reset upon a read and wrap around when the maximum count value is reached. Ensure the counters are read frequently enough to guarantee it doesn't miss a wraparound.

The TEMAC core always outputs RX and TX statistics vectors. When the statistics counters are present, these are decoded in the Vector Decode block, provided in the Block level, which in turn, provides the "increment vector" bus to the TEMAC core. The contents of the increment vector specifies which statistic counters are to increment.

The Vector Decode block is provided in plain text HDL, allowing you the ability to customize the provided statistic counters. Therefore, the following sections describe the operation of the statistic counter logic and its interfaces in detail to allow for a custom implementation. To use the provided default statistic counters, these sections can be skipped.

Increment Interface Overview

The Increment Interface has two main logical sections:

- A low-frequency increment component controlled by the increment vector input. This accommodates the majority of the statistical counters, which only increment at (or less frequently than) a standard minimum Ethernet frame period. These are decoded in the Vector Decode block and therefore can be edited if desired though this is not recommended for the pre-defined counters.
- A high-frequency increment component. These are generated internally to the Ethernet MAC core level and as such cannot be edited or monitored by you. These are used to accommodate counters which can increment on every cycle, and captured in counters 0-3.

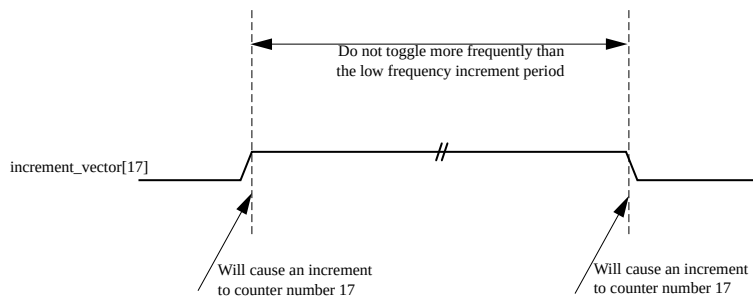
Low-Frequency Statistical Counters

The `increment_vector[4:43]` is an input bus signal which provides the predefined counters as described in [Table 26: Statistics Counter Definitions](#) and three user-defined counters.

This accommodates the majority of the statistical counters that only increment at (or less frequently than) a minimum Ethernet frame period.

The following figure illustrates the `increment_vector` bus provided by the vector decode block. There is an increment bit for each counter from counter four upwards. A toggle on a particular increment bit causes the corresponding counter to increment. The mapping of the increment vector bits to the various register-mapped counter is shown in [Table 26: Statistics Counter Definitions](#).

Figure 48: Increment Vector Timing Diagram



X28615-091523

The `increment_vector` is input to the core, and edge detection circuitry (toggle detection) is placed on each bit. The toggle detection circuitry is synchronous to `stats_ref_clk`. The distributed memory stores the current counter values within the core. The Statistics core accesses each of the counters within this memory in a round-robin fashion. If an increment is requested because, in the last access, the relative value is incremented before being written back to memory.

Bandwidth Requirements

The frequency of `stats_ref_clk` is flexible but depends upon both the number of counters and the maximum frequency supported by the MAC. The low-frequency increment vectors can update at a maximum rate of one per minimum sized Ethernet frame. This translates to 584 ns when running at 1 Gbps (64 bytes of minimum Ethernet frame size, plus 1-byte of minimum received preamble, plus eight bytes of minimum received interframe gap, at a byte rate of 1-byte per 8 ns).

With `stats_ref_clk` set to 125 MHz, 36 statistical counters can be safely updated between successive Ethernet frames (584 ns divided by the 8 ns clock period of `stats_ref_clk`, divided by two because a counter update requires two accesses). As this is less than the provided 44 counters, extra decode logic is included to take advantage of the frame size counters' one-hot status (that is, only one of the seven RX and one of the seven TX frame size counters can update on a per packet basis).

The round-robin function that controls which counter is being accessed, only accesses the required frame size counter and skips the other five. This means the 44 counters supported only require 32 counter accesses. However, this does mean that the `stats_ref_clk` should be at least as fast as the clock used for the maximum rate supported by the MAC (125 MHz at 1 Gbps or 12.5 MHz at 10/100 Mbps).

Frame Filter

The MAC can be configured with an optional frame filter. This is available irrespective of the use of the management interface but has much reduced functionality if no management interface is present. This use model is described in [Configuration Vector](#).

The frame filter performs two functions:

- Checking if any received packet matches one of the predefined Destination Address values: Pause Address, Broadcast Address, User Defined Unicast Address and the special multicast Pause Address.
- Comparing the first 64 bytes of a received packet against a user defined pattern.

In the case of the Destination Address comparisons, the results are used in other blocks within the MAC, such as flow control and in the generation of statistics vectors.

The other function of the frame filter is much more flexible and allows you to specify any match pattern within the first 64 bytes while ignoring any other byte or bit values. This method is extremely flexible as it allows the filtering of packets based on almost any header field or combination of header fields. Each frame filter contains two 512-bit registers (64 bytes):

- Frame Filter Value register: This pattern is compared to the first 512 bits received in a frame with bit 0 being the first bit within a frame to be received.
- Frame Filter Mask Value register: Each bit within this register refers to the same bit number within the Frame Filter Value register. When a bit in the Mask Value Register is set to:
 - `logic 1`, The same bit number within the Frame Filter Value register is compared with the respective bit in the received frame and must match if the overall frame filter is to obtain a match.
 - `logic 0`, The same bit number within the Frame Filter Value register is not compared. This effectively turns the respective bit in the Frame Filter Value register into a don't care bit. The overall frame filter is capable of obtaining a match even if this bit does not match.

This is described in more detail in [Using the Frame Filter](#).

You can specify up to 16 frame filters in the Ethernet MAC core level, with an additional three being used if the AVB Endpoint is present. Each frame is accessed through an address mapped register. However, because each filter requires 32 registers to access the pattern and mask values there is a control register that specifies which of the filters is being accessed with all filters being accessed through the same register set. When one or more frame filters are specified, the `rx_axis_filter_tuser` output is available from the Ethernet MAC core level. This is sized depending upon the number of filters selected and provides a direct pass/fail indication on a per-filter basis. This does not include AVB specific filters.

For more details, see [Using the Frame Filter](#). [Table 45: Frame Filter Configuration](#) shows the Frame Filter Configuration registers.

Using the Frame Filter

The Configurable frame filters can be used to perform simple Destination Address filtering, Multicast Group matching, Source Address matching, and VLAN field matching. The use of the Mask register enables any field or combination of fields in the first 64 bytes to be isolated and matched. The Frame Filter Control register, shown in [Table 48: Frame Filter Control \(0x708\)](#), allows you to enable or disable the frame filter by setting the promiscuous Mode bit which has the following functionality:

- When enabled, all good frames are marked as good.
- When disabled, only frames which match one or more of the pre-defined Destination Address filters, or the configurable frame filters are marked good.

When more than one frame filter is generated, it is necessary to write to the Frame Filter Control register. This write specifies which frame filter is being accessed before writing to any of the filter specific registers. It is then possible to enable each frame filter individually. While a particular filter is disabled, it does not match any packets. AMD recommends that frame filters are disabled prior to updating the match pattern to avoid acceptance of unexpected packets.

By default, all non-AVB frame filters are configured with all 1s in the bottom, 48 bits in both the Frame Filter Value registers, and the Frame Filter Mask Value registers. This results in a broadcast frame match, which has no effect as they are already accepted by the pre-defined Destination Address filters. After the frame filter value and mask value have been updated to the desired values, the filter should be enabled.

[Figure 49: Received Frame with a Match on Filter 0](#) shows the reception of an error free frame which matches against filter 0. In this example, the `rx_axi_filter_tuser` is treated as a “bad frame” indicator and is asserted when you want the frame to be dropped (that is, when an appropriate filter did not match against the frame). When one or more filters are generated, the `rx_axis_filter_tuser` bus is generated with one extra bit. For example, if four filters are selected, `rx_axis_filter_tuser` would be a 5-bit bus. This extra bit (always the upper bit) is used to provide the “else” case. It is asserted if any of the user-defined filters match a frame to indicate that the frame should be dropped.

Usage of the `rx_axis_filter_tuser` outputs would mean the frame is being serviced by a dedicated output and should therefore be dropped by the else output. This is shown in the [Figure 50: Received Frame with No Match on Configurable Frame Filters](#), a good frame has matched against a pre-defined Destination Address filter but failed to match any of the configurable filters.

Figure 49: Received Frame with a Match on Filter 0

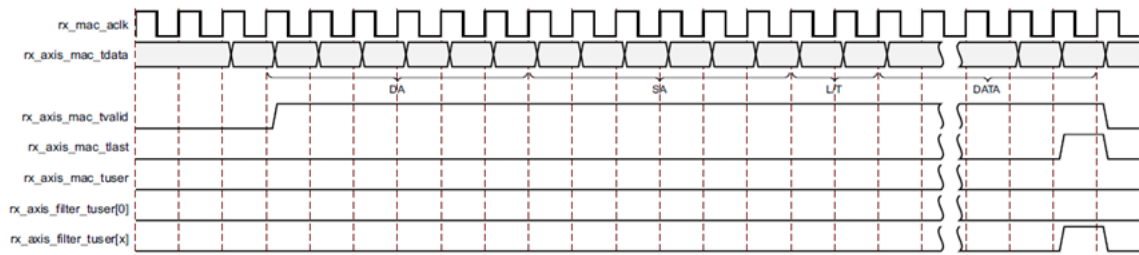
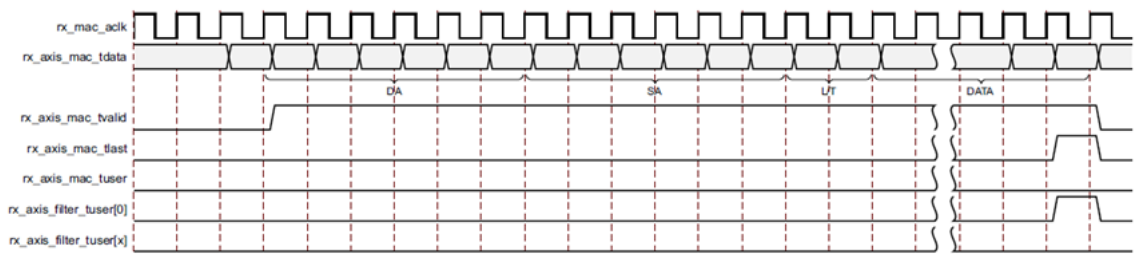


Figure 50: Received Frame with No Match on Configurable Frame Filters



This extra bit allows the `rx_axis_filter_tuser` bus to be used to directly drive FIFOs for particular filter matches, such as VLAN priority. When the frame filter is configured in Promiscuous Mode the `rx_axis_filter_tuser` bits continue to operate as normal.

Frame Filter Example Application

This section describes the usage of the frame filter to implement VLAN priority based filtering.

Using the AVB Specific Frame Filters

This section is only applicable if the TEMAC solution is implemented with the optional Ethernet AVB Endpoint functionality.

The three AVB frame filters are initialized to identify PTP frames and AV frames of both priorities. These initial values can be adjusted to change the VLAN field values used by the software drivers as required.

The Frame Filter Control register ([Table 48: Frame Filter Control \(0x708\)](#)) allows you to access each of the AVB frame filters by setting Bit[8] and AVB Select. When set, Bits[1:0] are used to specify the required AVB frame filter with the value of 3 is ignored. See [AVB Specific Frame Filters](#) for the register definitions of the PTP Frame filter.

SR Classes A and B Frame Filters

Two frame filters are provided to identify frames belonging to either SR Class A or SR Class B. In order to access the SR Class A filter, the Filter Index shall set to 1 in the Frame Filter Control Register (see [Table 48: Frame Filter Control \(0x708\)](#)). The SR Class B frame filter can be accessed by setting Filter Index to 2.

The output of the two filters is combined so that a match against either filter allows AV traffic to pass. If only one SR Class is supported then either disable the undesired filter or set both filters to the same value.

The default behavior of the SR Class frame filters is to match both the default VLAN PCP and VLAN ID values for that SR Class, as follows:

SR Class A:

Type = 0x0081 (VLAN)

Type_info = 0x0260 (VLAN PCP = 3 VLAN ID = 2)

SR Class B:

Type = 0x0081 (VLAN)

Type_info = 0x0240 (VLAN PCP = 2 VLAN ID = 2)

This translates to the register settings in [Table 54: SR Class A \(Index 1\) Frame Filter Value \(0x710-0x74C\)](#) to [Table 57: SR Class B \(Index 2\) Frame Filter Mask Value \(0x750-0x790\)](#).

Ethernet AVB Endpoint

This section provides information about the key functional blocks which are introduced when the optional AVB Endpoint is included in the core.



IMPORTANT! It is actually the “controlled frequency RTC” nanoseconds field which is sampled by the timestamping logic rather than the synchronized RTC (see [Figure 57: Increment of Sub-Nanoseconds and Nanoseconds Field](#)). This is important when operating as a clock slave: the controlled frequency RTC always acts as a smooth counter and the synchronized RTC might suffer from occasional step changes (whenever a new offset adjustment is periodically applied by the software drivers). These step changes are avoided by using the controlled frequency RTC, could otherwise lead to errors in the various PTP calculations which are performed by the software drivers.



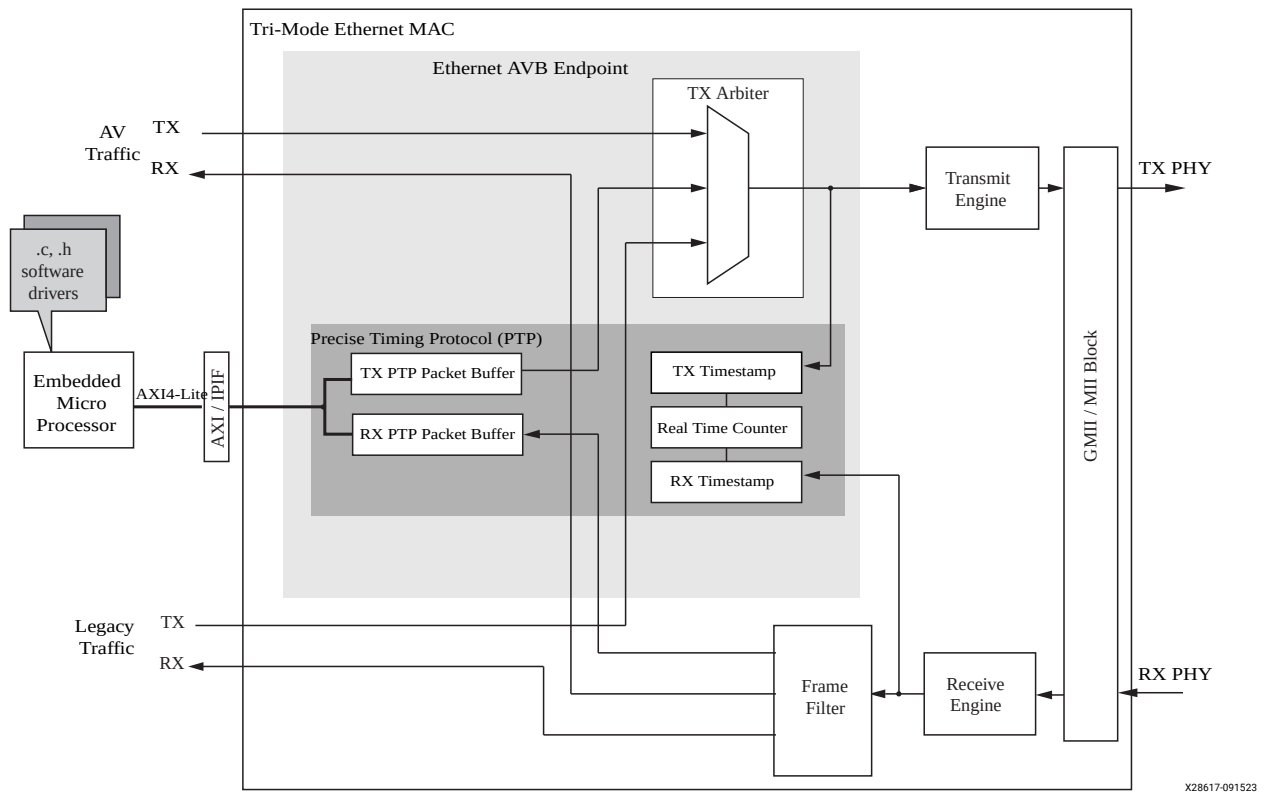
TIP: The software drivers can themselves obtain (when required) the local synchronized RTC value by summing the captured timestamp with the current nanoseconds offset value of the [RTC Nanoseconds Field Offset Control](#) (effectively performing the step 2 calculation of [Figure 57: Increment of Sub-Nanoseconds and Nanoseconds Field](#) in software).

Ethernet AVB Endpoint Transmission

As illustrated in the following figure, data for transmission over an AVB network can be obtained from three types of sources:

1. AV Traffic – For transmission from the [TX AV Traffic Interface](#) of the core.
2. Precise Timing Protocol (PTP) Packets – Initiated by the software drivers using the dedicated hardware [TX PTP Packet Buffer](#).
3. Legacy Traffic – For transmission from the [TX Legacy Traffic Interface](#) of the core.

Figure 53: Ethernet AVB Endpoint Data Path



TX Legacy Traffic Interface

The legacy traffic interface is maintained for best effort Ethernet data: Ethernet as it is known today (for example, a PC surfing the Internet). The signals forming the TX Legacy Traffic interface are defined in [Table 2: Transmit Interface AXI4-Stream Signal Pins](#). The timing of this interface is as described in [Transmitting Outbound Frames](#), with the interface functionality limited to full-duplex, no jumbo support, VLAN frames enabled, and flow control disabled.

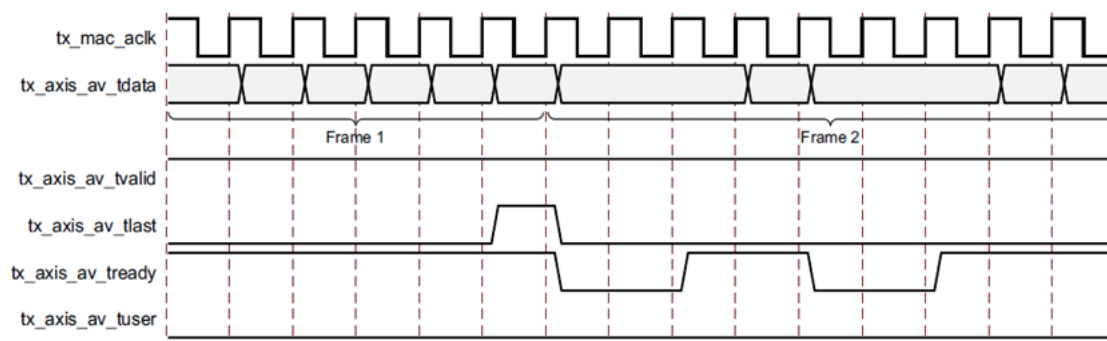
TX AV Traffic Interface

The AV traffic interface is intended for the Quality of Service audio/video data. The Ethernet AVB Endpoint gives priority to the AV traffic interface over the legacy traffic interface, as dictated by IEEE 802.1Q 75% bandwidth restrictions. The signals forming the TX AV Traffic interface are defined in [Table 4: Transmit Interface AXI4-Stream AV Signal Pins](#). The timing of this interface is exactly the same as that of the TX Legacy Traffic with the only difference being how the `tvalid` signal is handled between frames.

In the following figure, the end of frame transmission, the `tx_axis_av_tvalid` signal is held High, which indicates to the [Credit-Based Traffic Shaping Algorithm](#) that another AV frame is queued. Unless the configurable bandwidth restrictions have been exceeded, this parks the [Credit-Based Traffic Shaping Algorithm](#) onto the AV traffic queue and the following frame can immediately be taken. However, if no further AV traffic frames are queued, the `tx_axis_av_tvalid` signal should be set to Low immediately following the end of frame transmission. This then allows the [Credit-Based Traffic Shaping Algorithm](#) to schedule legacy traffic transmission (if any legacy frames are queued).

If following the end of frame reception, the bandwidth allocation for AV traffic has been exceeded, the [Credit-Based Traffic Shaping Algorithm](#) switches to service the legacy traffic regardless of the state of the `tx_axis_av_tvalid` signal.

Figure 54: TX AV Traffic Timing



Transmitter AXI4-Stream AV Specifics

One of the key functions of the Ethernet AVB Endpoint is the configurable bandwidth allocation for the AV user data. Because this bandwidth is managed over time, this is done using credits which are gained when non-AV data is sent and lost, when AV data is sent with a positive or zero balance of credits enabling the AV path. When no data is present at the AV input, any credits available are removed thus preventing bursty AV traffic from getting an artificially high bandwidth. The `tx_axis_av_tvalid` indicates that data is available, but at the end of a frame, if another frame is available, the `tx_axis_av_tvalid` should remain asserted and the first byte of the new frame should be presented. This is shown in the previous figure. If `tx_axis_av_tvalid` is dropped between frames then any positive credit balance is lost and a negative balance remains, which results in a lower overall bandwidth allowance for the AV path.

TX Arbiter

Overview

As illustrated in [Figure 53: Ethernet AVB Endpoint Data Path](#), data for transmission over an AVB network can be obtained from three types of sources.

The transmitter (TX) arbiter selects from these three sources in the following manner:

- If there is AV data available and the programmed AV bandwidth limitation is not exceeded, then the AV packet is transmitted
- otherwise, the TX arbiter checks to see if there are any PTP packets to be transmitted
- otherwise, if there is an available legacy packet, this is transmitted.

The Ethernet AVB Endpoint uses configuration registers to set up the percentage of available Ethernet bandwidth reserved for AV traffic. To comply with the IEEE802.1Q specification these should not be configured to exceed 75%. The arbiter then polices this bandwidth restriction for the AV traffic and ensures that on average, it is never exceeded. Consequently, despite the AV traffic having a higher priority than the legacy traffic, there is always remaining bandwidth available to schedule legacy traffic.

The relevant configuration registers for programming the bandwidth percentage dedicated to AV traffic are defined in [Configuration and Status](#) and are:

- [TX Arbiter Send Slope Control Register](#)
- [TX Arbiter Idle Slope Control Register](#)

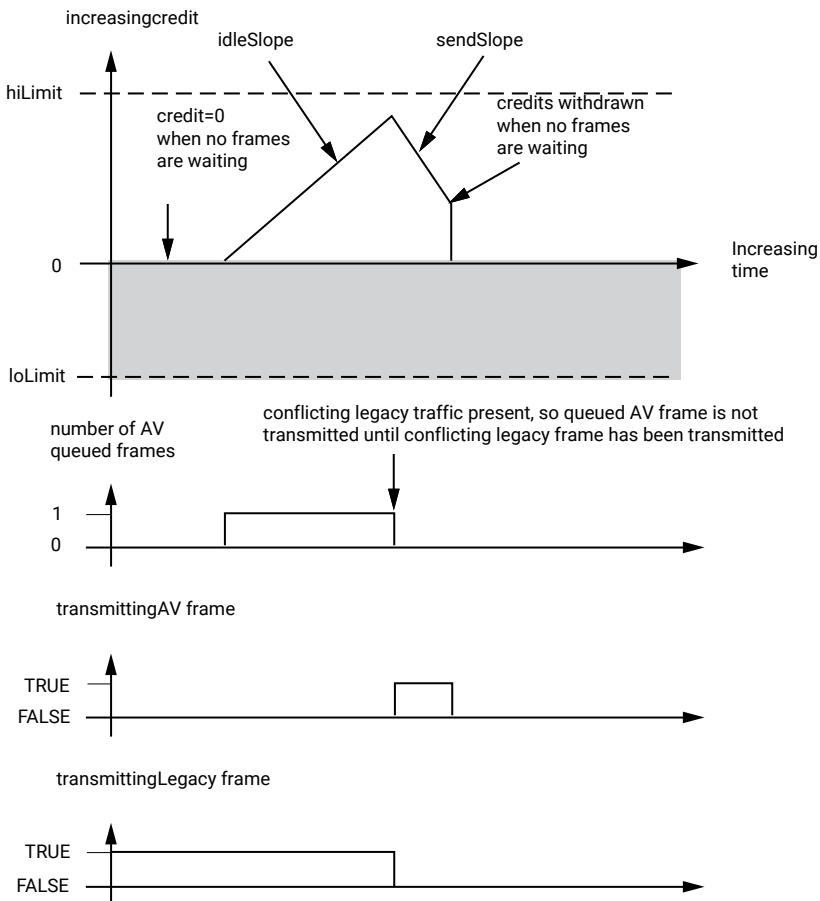
These registers default to values which dedicate up to 75% of the overall bandwidth to the AV traffic. This is the maximum legal percentage that is defined in the IEEE802.1 AVB standards.

In many implementations, it might be unnecessary to change these register values. Correct use of the `tx_axis_av_tvalid` signal, as defined in [TX AV Traffic Interface](#), allows the TX Arbiter to share the bandwidth allocation efficiently between the AV and Legacy sources (even in the situations where the AV traffic requires less than 75% of the overall bandwidth). However, for the cases that require less than 75% of the overall bandwidth, a careful configuration can result in a smoother (less bursty) transmission of the AV traffic, which should prevent frame bunching across the AVB network.

Credit-Based Traffic Shaping Algorithm

To enforce the bandwidth policing of the AV Traffic, a credit-based shaper algorithm is implemented in the TX Arbiter. The following figure illustrates the basic operation of the algorithm and indicates how the TX Arbiter decides which Ethernet frame to transmit.

Figure 55: Credit-Based Shaper Operation



X28618-091523

The previous figure illustrates the key features of the credit-based algorithm, which are:

- If the algorithm is in credit (≥ 0), the TX Arbiter schedules queued transmission from the [TX AV Traffic Interface](#). If there is less than 0 credit (not shown in the previous figure, but the credit can sink below 0), the TX Arbiter does not allow AV traffic to be transmitted; legacy traffic is scheduled instead when queued.
- When no AV traffic is queued, any positive credit is lost and the credit is reset to 0.
- When AV traffic is queued, and until the time at which the TX Arbiter is able to schedule it (while waiting for an in-progress legacy frame to complete transmission), credit can be gained at a rate defined by the `idleSlope`.
- During AV traffic transmission, credit is removed at a rate defined by the `sendSlope`.
- The `hiLimit` and `loLimit` settings impose a fixed range on the possible values of credit. If the available credit hits one of these limits, it does not exceed but saturates at the magnitude of that limit. These limits are fixed at the Ethernet MAC core level to ensure that the interface is not used incorrectly.

The overall intention of the two settings `idleSlope` and `sendSlope` is to spread out the AV traffic transmission as evenly as possible over time, preventing periods of bursty AV transmission surrounded by idle AV transmission periods. No further background information is provided in this document with regard to the credit-based algorithm. The remainder of this section describes the `idleSlope` and `sendSlope` variables from the perspective of the TX Arbiter.

TX Arbiter Bandwidth Control

The configuration register settings used for setting the cores local definitions of `idleSlope` and `sendSlope`, are described in general, and then from the point of view of a single example which describes the calculations made to set the register default values. This example dedicates up to 75% of the overall bandwidth to be reserved for the AV traffic (leaving at least 25% for the Legacy Traffic).

The calculations described are independent of Ethernet operating speed (no re-calculation is required when changing between Ethernet speeds of 1 Gbps and 100 Mbps).

`idleSlope`

The general equation is:

```
idleSlopeValue=(AV percentage / 100) x 8192
```

In this example, dedicating up to 75% of the total bandwidth to the AV traffic:

```
idleSlopeValue=(75 / 100) x 8192 = 6144
```

The calculated value for the `idleSlopeValue` should be written directly to the [TX Arbiter Send Slope Control Register](#). This provides a per-byte increment value when relating this to Legacy Ethernet frame transmission.

sendSlope

The general equation is:

```
sendSlopeValue=((100 - AV percentage) / 100) x 8192
```

In this example, dedicating up to 75% of the total bandwidth to the AV traffic, obtain:

```
sendSlopeValue=((100 - 75) / 100) x 8192 = 2048
```

The calculated value for the sendSlopeValue should be written directly to the [TX Arbiter Idle Slope Control Register](#). This provides a per-byte decrement value when relating this to AV Ethernet frame transmission.

Ethernet AVB Endpoint Reception

When the AVB Endpoint is present, the optional frame filter is always present with three dedicated filters for the identification of AVB specific frames. As shown in [Figure 53: Ethernet AVB Endpoint Data Path](#), the received data from an AVB network can be of three types:

- Precise Timing Protocol (PTP) Packets – Routed to the dedicated hardware [RX PTP Packet Buffer](#) which can be accessed by the software drivers. PTP packets are identified by searching for a specific MAC Destination Address and Type field.
- AV Traffic – Routed to the [RX AV Traffic Interface](#) of the core. These packets are identified by searching for MAC packets containing a MAC VLAN field with one of two possible configurable VLAN PCP and VID combinations.
- Legacy Traffic – Routed to the [RX Legacy Traffic Interface](#) of the core. All packet types which are not identified as PTP or AV Traffic are considered legacy traffic.

RX Legacy Traffic Interface

The signals forming the RX Legacy Traffic Interface are defined in [Table 6: Receive Interface AXI4-Stream Signal Pins](#). The timing of this interface is as described in the [Receiving Inbound Frames](#), with the interface functionality limited to full-duplex, no jumbo support, VLAN frames enabled, and no flow control.

RX AV Traffic Interface

The signals forming the RX AV Traffic Interface are defined in [Table 8: Receive Interface AXI4-Stream AV Signal Pins](#). The timing of this interface is exactly the same as for the RX Legacy Traffic (there is a one-to-one correspondence between signal names when the axis_mac is exchanged for axis_av).

The RX AV traffic is identified using the dedicated AVB frame filters. These are only present when the AVB Endpoint is included in the TEMAC core and initialized to match against the default AV VLAN p and VLAN Q values. These are described in more detail in [Using the AVB Specific Frame Filters](#).

Real-Time Clock and Timestamping

This chapter describes two closely related logical components that are partially responsible for the AVB timing synchronization protocol.

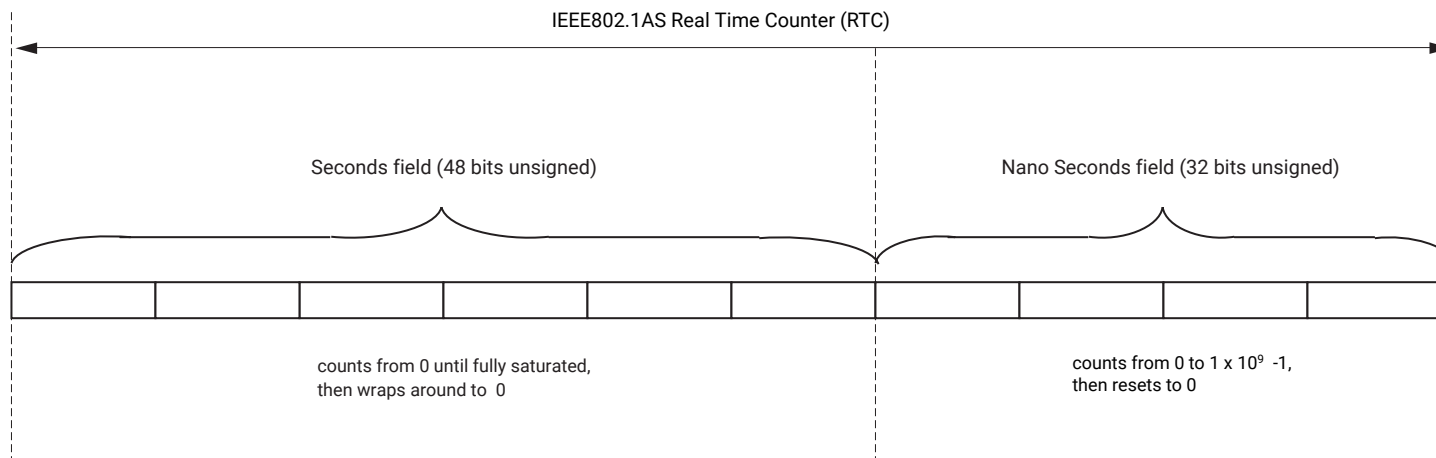
- [See Real-Time Clock](#)
- [Timestamping Logic](#)

Real-Time Clock

A significant component of the PTP network wide timing synchronization mechanism is the Real-Time Clock (RTC), which provides the common time of the network. Every device on the network maintains its own local version.

The RTC is effectively a large counter which consists of a 32-bit nanoseconds field (the unit of this field is one nanosecond and this field counts the duration of exactly one second, then resets back to zero), and a 48-bit seconds field (the unit of this field is one second, and this field increments when the nanosecond field saturates at one second). The second field only wraps around when its count fully saturates. The entire RTC is therefore designed never to wrap around in this lifetime. The RTC is summarized in the following figure.

Figure 56: Real-Time Clock (RTC)



X28619-091523

Conceptually, the RTC is not related to the frequency of the clock used to increment it. The [RTC Increment Value Control Register](#) is a configuration register within the core that provides a configurable increment rate for the RTC counter. It is programmed with the value of the RTC reference clock period which is being used to increment the RTC. The resolution of this increment register is very fine (in units of $1/1,048,576$ ($1/220$) fraction of one nanosecond). Therefore, the RTC increment rate can be adjusted to a very fine degree of accuracy which provides the following features:

- The RTC can be incremented from any available clock frequency that is greater than the AVB standards defined minimum of 25 MHz. However, the faster the frequency of the clock, the smaller the step increment, and the smoother the overall RTC increment rate.

Note: AMD recommends clocking the RTC logic at 125 MHz because this is a readily available clock source. This frequency significantly exceeds the minimum performance of the IEEE 802.1AS specification.

- When acting as a clock slave, the rate adjustment of the RTC can be matched to that of the network clock master to an exceptional level of accuracy (by slightly increasing or decreasing the value within the [RTC Increment Value Control Register](#)). The software drivers (available separately) periodically calculate the increment rate error between themselves and the master, and update the RTC increment value accordingly.

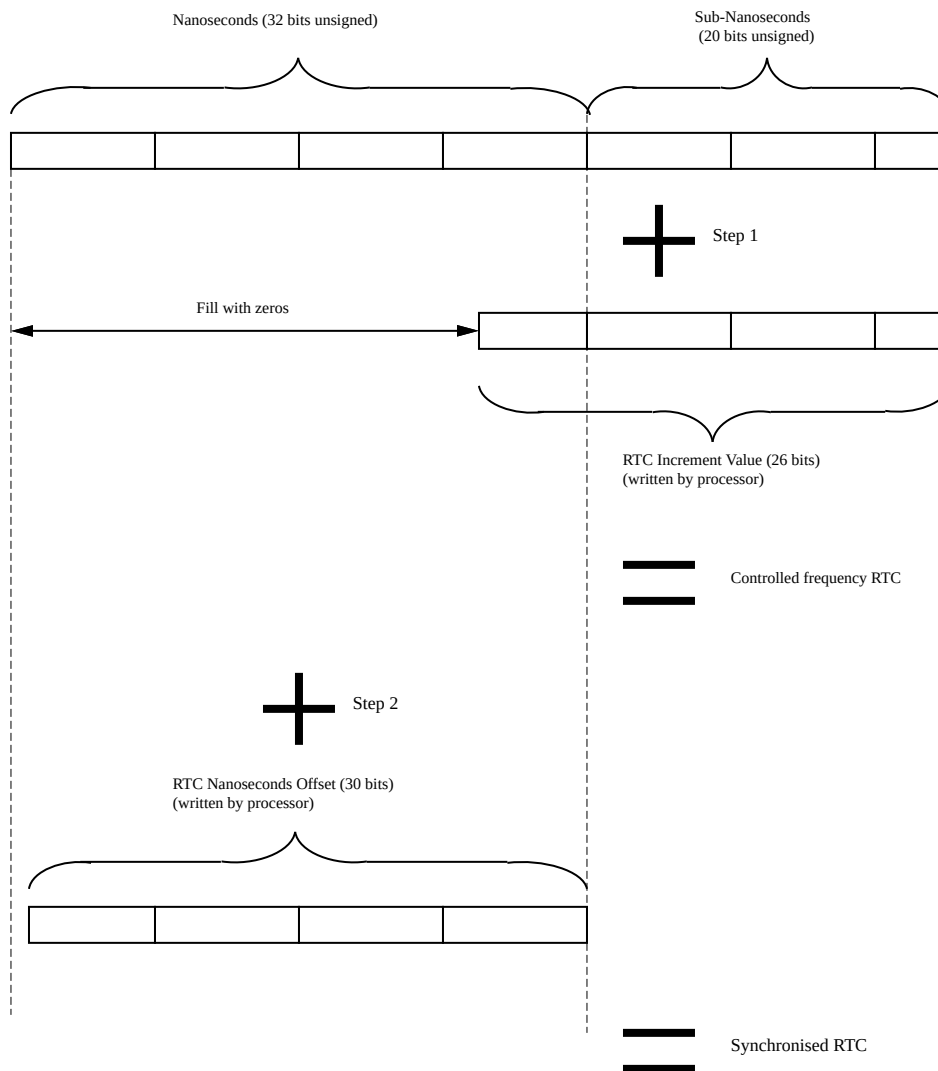
The core also contains configuration registers, [RTC Seconds Field Offset Control](#) and [RTC Nanoseconds Field Offset Control](#), which allow a large step change to be made to the RTC. This can be used to initialize the RTC, after power-up. It is also used to make periodic corrections, as required, by the software drivers when operating as a clock slave. However, if the increment rates are closely matched, these periodic step corrections are small.

RTC Implementation

Increment of Nanoseconds Field

The following figure shows the implementation used to create the RTC nanoseconds field. This is performed by the use of an implementation-specific 20-bit sub-nanoseconds field. The nanoseconds and sub-nanoseconds fields can be considered to be concatenated together. All RTC logic within the core is synchronous to the RTC Reference clock, `gtx_clk`.

Figure 57: Increment of Sub-Nanoseconds and Nanoseconds Field



X28620-091523

There are two stages to the implementation:

1. Controlled Frequency RTC

The RTC Increment Value illustrated in the previous figure is set directly from the [RTC Increment Value Control Register](#). The upper six bits of this register align with the lower six bits of the RTC nanoseconds field. The lower 20 bits of the RTC Increment Value align with the 20-bit sub-nanoseconds field. It is assumed that the frequency of the RTC reference clock is known by the processor to program the increment value correctly. For example, if the RTC is being clocked from a 125 MHz clock source, a nominal increment value of 8 ns should be programmed (by writing the value 0x800000 into the [RTC Increment Value Control Register](#)). However, if the microprocessor determines that this clock is drifting with respect to the grand master clock, it can revise this nominal 8 ns up or down by a very fine degree of accuracy.

The “step 1” addition illustrated in the previous image (of current counter value plus increment) occurs on every clock cycle of the RTC reference clock. The result from this addition forms the new value of the “controlled frequency RTC” nanoseconds field. This controlled frequency RTC initializes to zero, following a reset, and continues to increment smoothly on every RTC reference clock cycle by the current value contained in the RTC Increment Value Control Register.

The previous image illustrates that 26 bits have been reserved for the Increment Value, the upper 6 bits of which overlap into the nanoseconds field. For this reason, the largest per-cycle increment = $1 \text{ ns} \times 26 = 64 \text{ ns}$. The lowest clock period which is expected to increment this counter is 40 ns (corresponding to the 25 MHz MAC clock used at 100 Mbps speeds). So, this should satisfy all allowable clock periods.

1. Synchronized RTC

The value contained in the [RTC Seconds Field Offset Control](#) and [RTC Nanoseconds Field Offset Control](#) written by the microprocessor, is then applied to the free running “controlled frequency RTC” counter. This is used by the microprocessor to:

- Initialize the power-up value of the Synchronized RTC.
- Apply step corrections to the Synchronized RTC (when a slave), based on the timing of PTP packets received from the Grand Master Clock RTC.

The “step 2” addition illustrated in the previous image (of controlled frequency RTC value plus offset) occurs on every clock cycle of the RTC reference clock. The result from this addition forms the new value of the Synchronized RTC nanoseconds field. It is this version of the RTC nanoseconds field which is made available as an output of the core – the `rtc_nanosec_field[31:0]` port.

Increment of the Seconds Field

Conceptually, the RTC seconds field is implemented in a similar way to the nanoseconds field. The seconds field should be incremented by a value of one whenever the synchronized RTC nanoseconds field saturates at one second. The [RTC Seconds Field Offset Control](#) and [RTC Nanoseconds Field Offset Control](#) allow the software to make large step corrections to the seconds field in a similar manner. Again, the step correction capability can be used to either initialize the RTC counter following reset or to synchronize the local RTC to that of the Grand Master Clock (when the local device is acting as a clock slave).

Clock Outputs Based on the Synchronized RTC Nanoseconds Field

The clk8k (8 kHz clock) output, derived from the Synchronized RTC, is provided as an output from the core. The synchronized RTC counter, unlike the controlled frequency version, has no long-term drift (assuming the provided software drivers are used correctly). Therefore, the clk8k signal is synchronized exactly to the network RTC frequency.

The 8 kHz clock is the period of the shortest class measurement interval for an SR class as specified in IEEE802.1Q. This clock could also be useful for external applications (for example, a 1722 implementation of the AV traffic).

Timestamping Logic

Note: TEMAC supports IEEE 1588 timestamping.

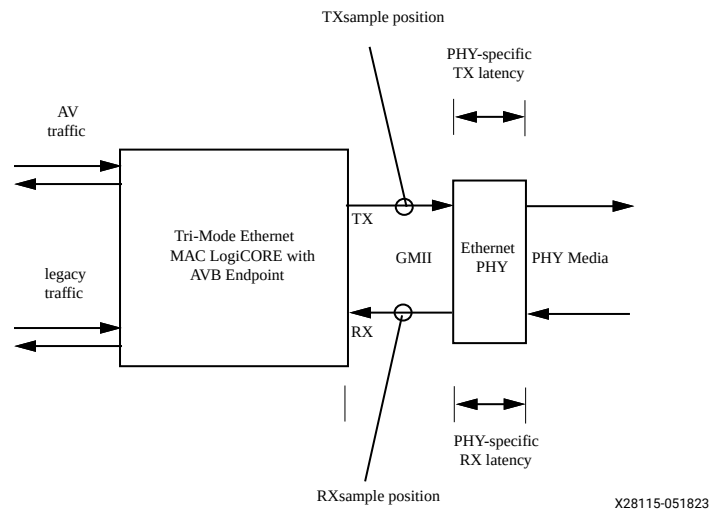
Whenever a PTP packet, used with the Precise Timing Protocol (PTP), is transmitted or received (see [Precise Timing Protocol Packet Buffers](#)), a sample of the current value of the RTC is taken and made available for the software drivers to read. The hardware makes no distinction between frames carrying events or general PTP messages (as defined in IEEE 802.1AS). It always stores a timestamp value for Ethernet frames containing the Ethertype specified for PTP messages.

This timestamping of packets is a key element of tight timing synchronization across the AVB network wide RTC, and these samples must be performed in hardware for accuracy. The hardware in this core therefore samples and captures the local nanoseconds RTC field for every PTP frame transmitted or received. These captured timestamps are stored in the [Precise Timing Protocol Packet Buffers](#) alongside the relevant PTP frame, and are read and used by the PTP software drivers.

Timestamp Sampling Position of MAC Frames

A timestamp value should be sampled at the beginning of the first symbol following the Start of Frame Delimiter (SFD) of the Ethernet MAC frame.

Figure 58: Timestamping Position



The previous figure illustrates the timestamp sampling position that is used by the core. Timestamps are taken after the MAC frame SFD is seen on the GMII.

- If the PHY specific latency values are available the software drivers can use them to adjust the timestamps and improve overall system accuracy.

IEEE1722 Real-Time Clock Format

The IEEE1722 specification defines the `avbtp_timestamp` field. This is derived by sampling the IEEE802.1 AS Real-Time Clock and converting the low order time to nanoseconds. This conversion is performed in the core and an alternative RTC using the 1722 format. The output produces on the `rtc_nanosec_field_1722[31:0]` port.

This port contains a 32-bit word representing nanosecond values. Unlike the IEEE802.1 AS nanosecond field (which resets back to zero when it reaches 1 second), the IEEE1722 nanosecond field counts fully to `0xFFFFFFFF` before wrapping around. Therefore, the field wraps around approximately every 4 seconds.

If the system is using the IEEE1722 functionality, this port can be sampled to create the `avbtp_timestamp` field. Otherwise, this port can be ignored.

Precise Timing Protocol Packet Buffers

This chapter describes two closely related logical components which are partly responsible for the AVB timing synchronization protocol.

- [TX PTP Packet Buffer](#)
- [RX PTP Packet Buffer](#)

TX PTP Packet Buffer

The TX PTP packet buffer is illustrated in the following figure. This packet buffer provides working memory to hold the PTP frames which are required for transmission. The software drivers, through the AXI4-Lite configuration bus, can read/modify/write the PTP frame contents, and request transmission of the appropriate PTP frames, when required.

The PTP packet buffer is implemented in dual-port block RAM. Port A of the block RAM is connected to the configuration bus and all addresses in the buffer are read/writable. Port B of the block RAM is connected to the TX Arbiter module, allowing PTP frames to be read out of the block RAM and transmitted.

The TX PTP Packet Buffer is divided into eight identical buffer sections as illustrated. Each section contains 256 bytes, which are formatted as follows:

- The first byte, at address zero, contains a frame length field. This indicates how many bytes make up the PTP frame that is to be transmitted from a particular PTP buffer.
- The next seven bytes from addresses 1 to 7 are reserved for future use.
- The PTP frame data itself is stored from address 8 onwards. The amount of addresses used is dependent on the indicated frame length field, which is different for each PTP frame type. Each PTP buffer provides a maximum of 244 bytes (more than that required for the largest PTP frame). Each PTP frame holds the entire MAC frame (with the exception of any required MAC padding or CRC—these are automatically inserted by the transmit logic) from the Destination Address field onwards.
- The top four addresses of each buffer, from address 0xFC to 0xFF are reserved for a timestamp field. At the beginning of PTP frame transmission from any of the eight buffers, the [Timestamping Logic](#) samples the [Real-Time Clock](#). Following the end of the PTP frame transmission, this captured timestamp is automatically written into this location to accompany the frame for which it was taken.

Despite the logic and formatting of each individual PTP buffer being identical, the block RAM is pre-initialized at device configuration to hold template copies of each of the PTP frames, as indicated in the following figure. This shows that the first seven memory segments are in use. PTP Buffer number 8 is currently unused and could therefore be used by proprietary applications.

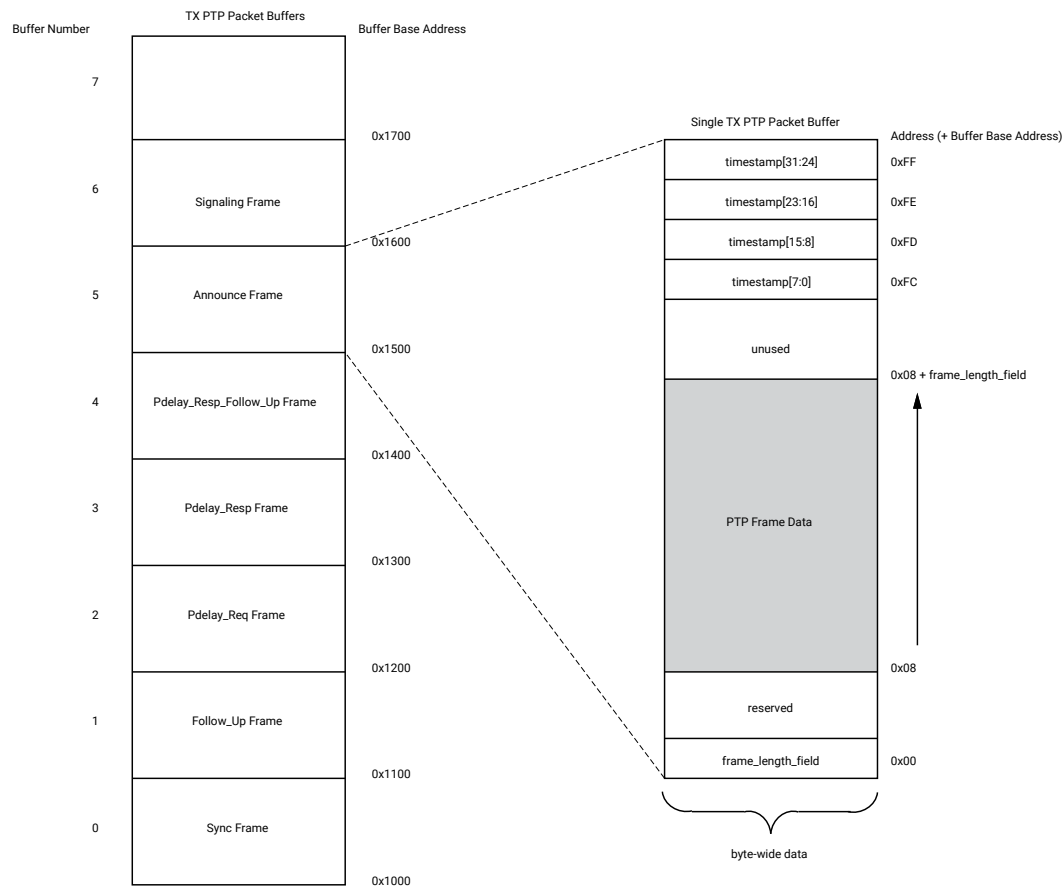
The [TX PTP Packet Buffer Control Register](#) is defined for the purpose of requesting which of the eight TX PTP Buffers are to be transmitted. It is possible to request more than a single frame at one time (indeed it is possible to request all 8). When more than one frame is requested, the TX PTP Buffer logic gives a priority order to the lowest PTP Buffer Number that has been requested.

The [TX PTP Packet Buffer Control Register](#) also contains a frame waiting field. This can be read by the software drivers to determine which of the previously requested PTP frames have been sent, and which are still queued.

Following transmission completion of each requested PTP frame, a dedicated interrupt signal, `interrupt_ptp_tx`, is generated by the core. On the assertion of the interrupt, the captured timestamp is already available in the upper four bytes of the buffer, and the `tx_packet` field of the [TX PTP Packet Buffer Control Register](#) indicates the most recently transmitted Buffer Number.

The software drivers available from AMD, using the AXI4-Lite and dedicated interrupts, use this interface too, as defined by the IEEE802.1AS protocol, periodically update specific fields within the PTP packets, and request transmission of these packets.

Figure 59: TX PTP Packet Buffer Structure



X28621-091523

RX PTP Packet Buffer

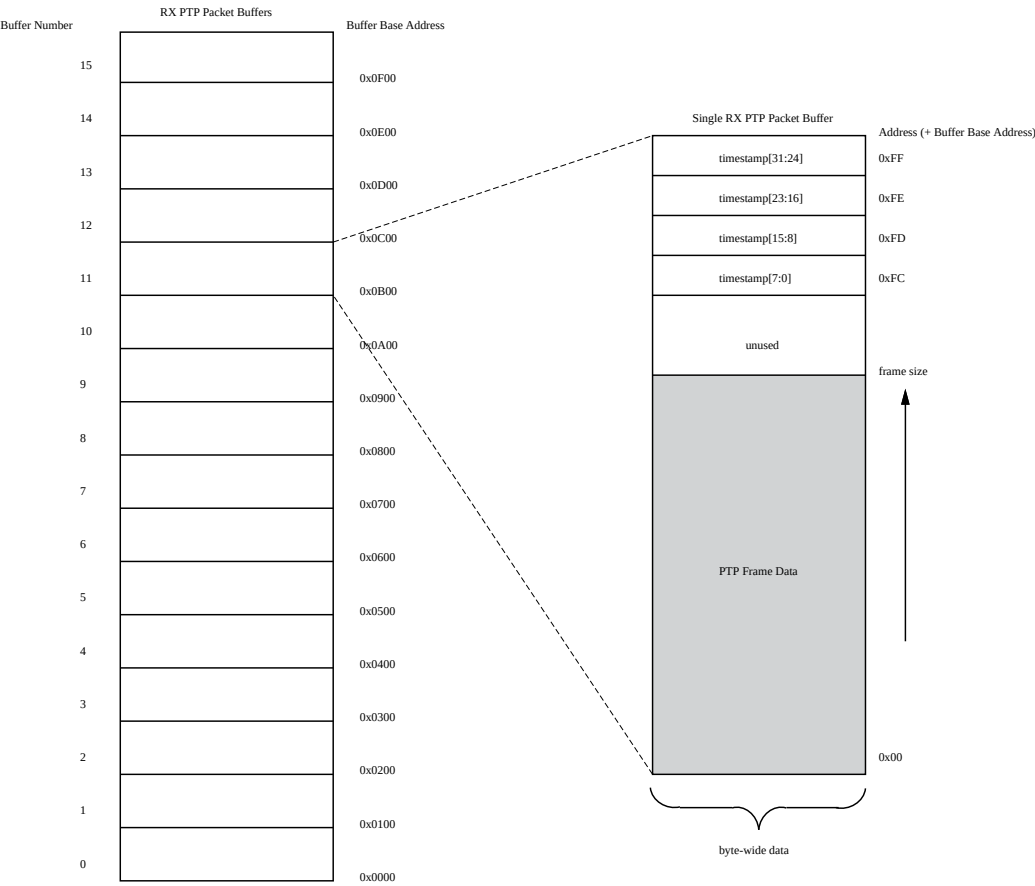
The RX PTP packet buffer is illustrated in the following figure. This provides working memory to hold each received PTP frame. The software drivers, using the AXI4-Lite configuration bus, can then read and decode the contents of the received PTP frames. The PTP packet buffer is implemented in dual-port block RAM. Port A of the block RAM is connected to the configuration bus and all addresses in the buffer can be read (writes are not allowed). Port B of the block RAM is connected to the PTP frame filter, which routes all received PTP frames into the RX PTP Packet Buffer. The RX PTP Packet Buffer is divided into 16 identical buffer sections as illustrated. Each section contains 256 bytes, which are formatted as follows:

- The PTP frame data itself is stored from address 0 onwards: the entire MAC frame from the Destination Address onwards is written (with the exception of the FCS field which has been removed by the receive logic). The number of addresses used is dependent on the particular PTP frame size, which is different for each PTP frame type. Each PTP buffer provides a maximum of 252 bytes (more than that required for the largest PTP frame). When an oversized PTP frame is received, the first 252 bytes is captured and stored, and other bytes are lost.
- The top four addresses of each buffer, from address 0xFC to 0xFF are reserved for a timestamp field. At the beginning of PTP frame reception, the [Timestamping Logic](#) samples the [Real-Time Clock](#). Following the end of PTP frame reception, this captured timestamp is automatically written into this location to accompany the frame for which it was taken.

Following reset, the first received PTP frame is written into Buffer Number 0. The next subsequent received PTP frame is written into the next available buffer—in this case, number 1. This process continues with buffer numbers 2, 3, then 4,... being used. After receiving the 16th PTP frame (which would have been stored in buffer number 15), the count is reset, and then buffer number 0 is overwritten with the next received PTP frame. For this reason, at any one time the RX PTP Packet Buffer is capable of storing the most recently received 16 PTP frames.

Following the completion of PTP frame reception, a dedicated interrupt signal, `interrupt_ptp_rx`, is generated by the core. On the assertion of the interrupt, the captured timestamp is already available in the upper four bytes of the buffer, and the `rx_packet` field of the [RX PTP Packet Buffer Control Register](#) indicates the most recently filled Buffer Number. The software drivers available from AMD using the AXI4-Lite and dedicated interrupt, use this interface to decode, and then act on the received PTP packet information.

Figure 60: RX PTP Packet Buffer



X28622-091523

Configuration and Status

This section provides general guidelines for configuring and monitoring the core, including a detailed description of the user-side management interface. It also describes the alternative to the optional management interface which is the Configuration Vector. See the appropriate section:

- [Management Interface](#)
- [Configuration Vector](#)

Management Interface

The management interface uses the industry standard AXI4-Lite to allow access to the Ethernet MAC core level. This interface is used for these operations:

- Configuring the MAC core
- Configuring the frame filter
- Configuring the interrupts
- Accessing statistics information
- Providing access to the MDIO interface to configure Ethernet PHY devices

[Table 12: Optional AXI4-Lite Signal Pinout](#) describes the optional signals used by you to access the Ethernet MAC core level.

The following figures show the basic AXI4-Lite transactions supported by the MAC solution. Illegal access results in an error indication. For more information about the standard, see *AMBA AXI4-Stream Protocol Specification* ([ARM IHI 0051A](#)) (Arm

Figure 61: Management Register Write Timing

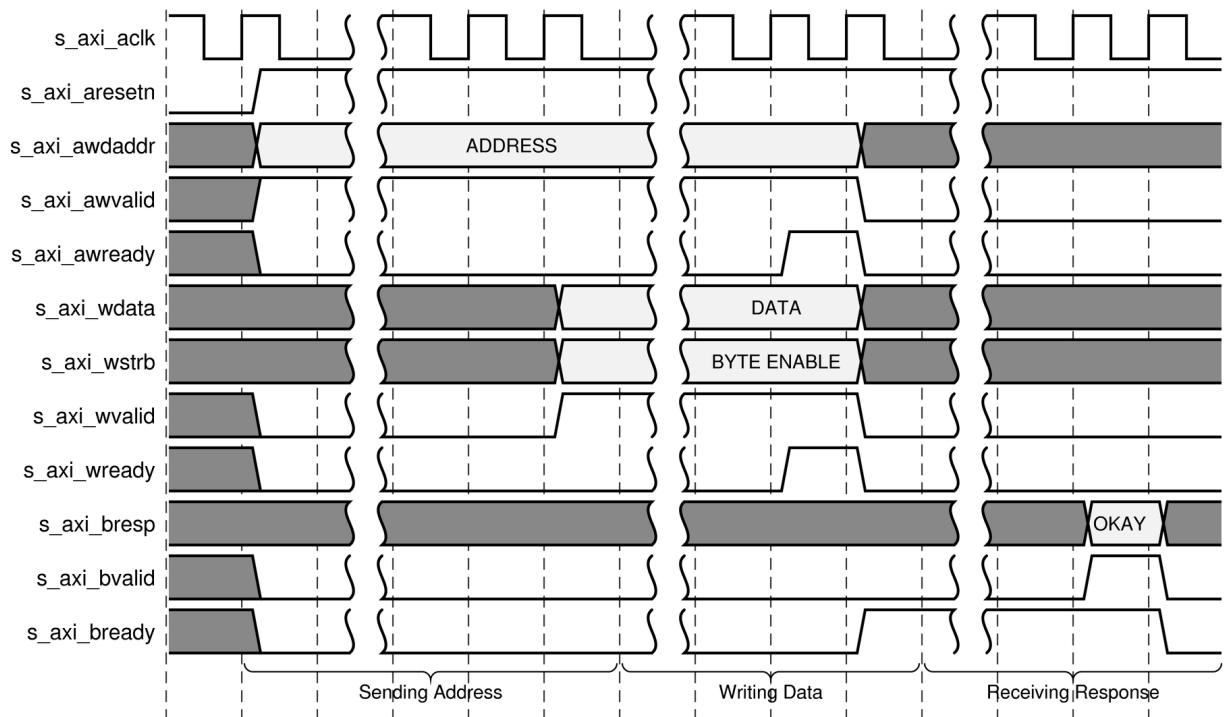
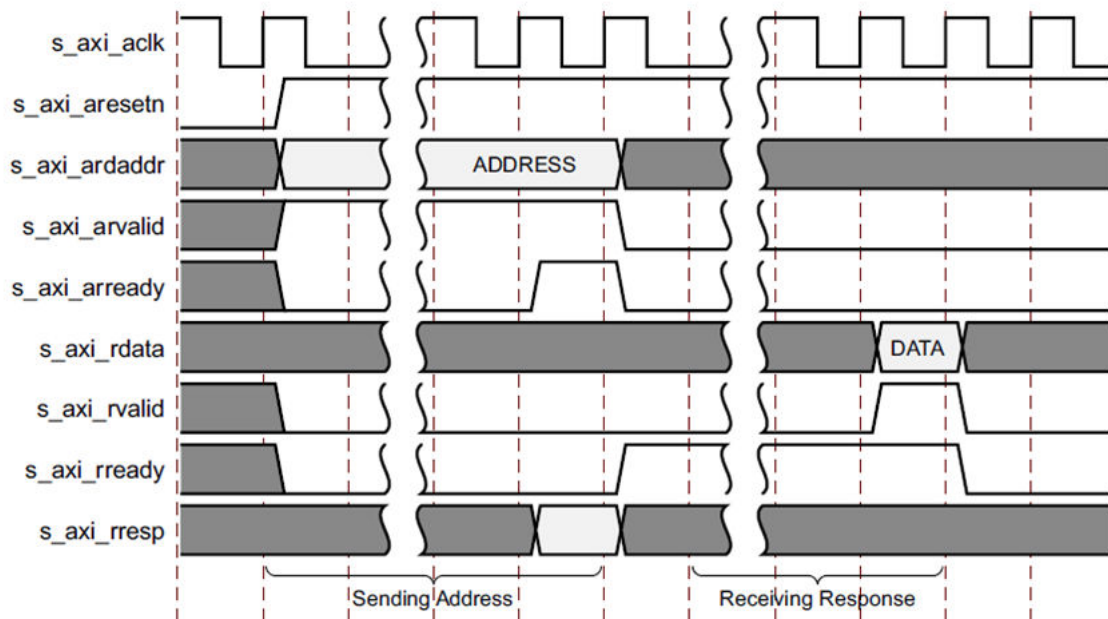


Figure 62: Management Register Read Timing



Address Map

The MAC address map, as shown in the following table, has been updated to fully memory map all registers and provide the same interface in all cases. Where possible, gaps have been left to allow for future expansion. The Address Map has been split into distinct functional blocks. Each of these blocks is described in more detail in the following sections. For full details of the registers, see [Register Space](#).

Table 76: MAC Registers

Address (Hex)	Description
0x000-0x1FC	Reserved
0x200-0x3FC	Statistics Counters
0x400-0x4FC	MAC Configuration
0x500-0x5FC	MDIO Interface
0x600-0x6FC	Interrupt Controller
0x700-0x7FC	Frame Filter
0x800-0xFFFC	Reserved
0x10000-0x10FFC	RX PTP Packet Buffer Address Space
0x11000-0x117FC	TX PTP Packet Buffer Address Space
0x11800-0x11FFC	Reserved
0x12000-0x127FC	AVB Configuration
0x12800-0x13FFC	RTC Configuration

MAC Configuration

After the core is powered up and reset, you can reconfigure some of the core parameters from their defaults, such as flow control support and RX/TX VLAN support. Configuration changes can be written at any time. However, both receiver and transmitter configuration changes only take effect during interframe gaps. The configurable soft resets are an exception, which take effect immediately. See [MAC Configuration Registers](#).

MDIO Interface

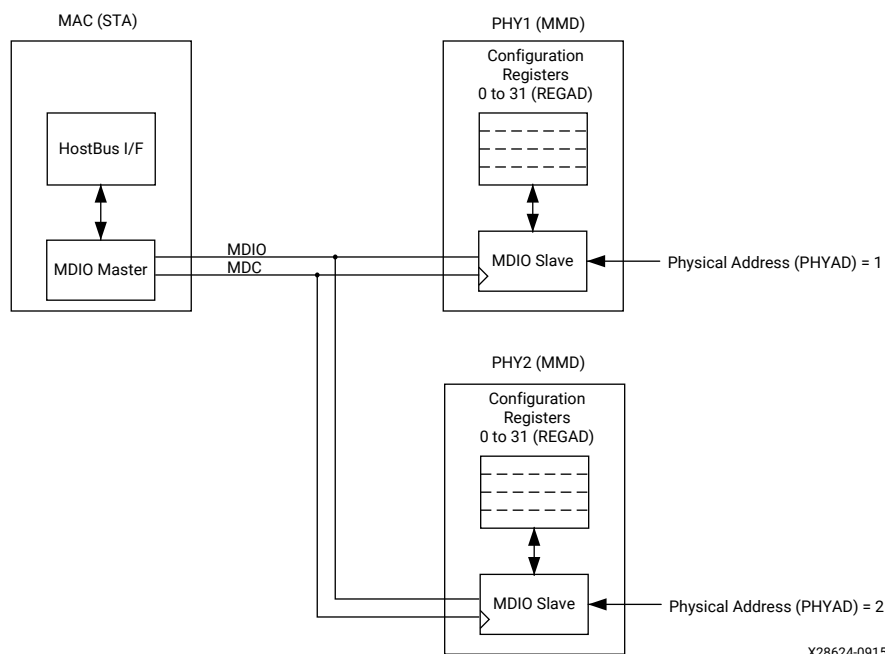
The Management Interface is also used to access the MDIO Interface of the TEMAC core. This interface is used to access the Managed Information Block of the PHY components attached to the TEMAC core and only available when the management interface and MDIO are enabled.

MDIO Bus System

The Management Data Input/Output (MDIO) interface for access to Ethernet PHY devices for 1 Gbps operation and slower speeds is defined in IEEE 802.3, clause 22. This two-wire interface consists of a Management Data Clock (MDC) and a shared serial data line (MDIO). The maximum permitted frequency of MDC is set at 2.5 MHz. The following figure illustrates an example MDIO bus system.

An Ethernet MAC is shown as the MDIO bus master (the Station Management (STA) entity). Two PHY devices are shown connected to the same bus, both devices are MDIO slaves (MDIO Managed Device (MMD) entities).

Figure 63: A Typical MDIO-Managed System



X28624-091523

The MDIO bus system is a standardized interface for accessing the configuration and status registers of Ethernet PHY devices. In the example illustrated, the Management Bus Interface of the Ethernet MAC is able to access the Configuration and Status registers of two PHY devices using the MDIO bus.

MDIO Transactions

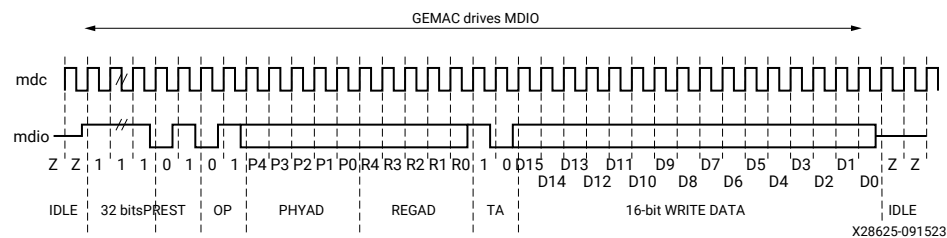
All transactions, read or write, are initiated by the MDIO master. All MDIO slave devices must respond when addressed. MDIO transactions take the form of an MDIO frame, containing fields for the transaction type, address and data. This MDIO frame is transferred across the MDIO wire synchronously to MDC. The abbreviations used in this section are explained in the following table.

Table 77: Abbreviations and Terms

Abbreviation	Term
PRE	Preamble
ST	Start of frame
OP	Operation code
PHYAD	Physical address
REGAD	Register address
TA	Turnaround

Write Transaction

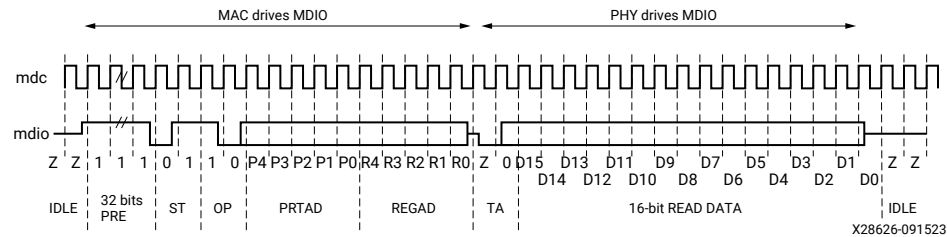
Figure 64: MDIO Write Transaction



The previous figure shows a Write transaction across the MDIO which is defined by OP = 01. The addressed PHY (PHYAD) device takes the 16-bit word in the data field and writes it to the register at REGAD.

Read Transaction

Figure 65: MDIO Read Transaction



The previous figure shows a Read transaction, defined by OP = 10. The addressed PHY (PHYAD) device returns the 16-bit word from the register at REGAD. For details of the register map of PHY layer devices and a fuller description of the operation of the MDIO interface itself, see IEEE 802.3-2008 specification.

MDIO Port Options

The core provides an option to automatically add I/O buffers for the MDIO Interface signals. This option is set by default when the physical interface is MII, GMII or RGMII. When the physical interface is Internal, this option is not set by default.

Connecting the MDIO to an Internally Integrated PHY

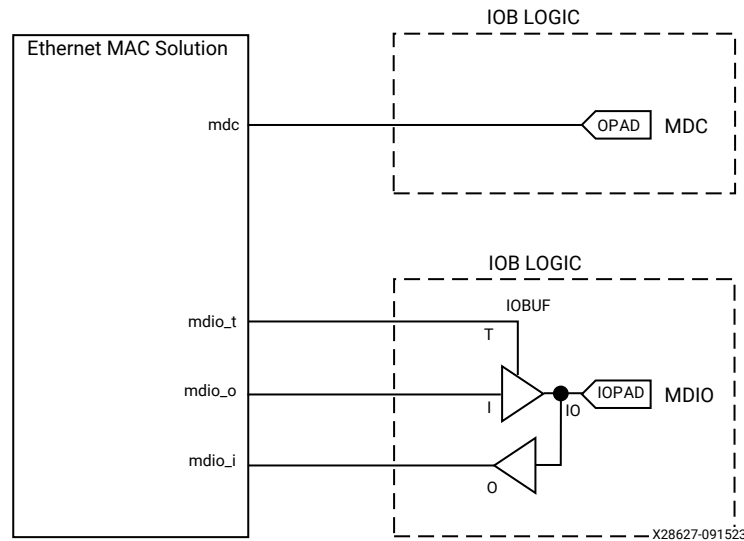
The MDIO ports of the core (see [Table 17: MDIO Interface Signal Pinout](#)) can be connected to the MDIO ports of an internally integrated physical layer device. For example, the MDIO ports of the Ethernet 1G/2.5G PCS/PMA or SGMII from AMD (see [Interfacing to Other AMD Ethernet Cores](#)).

Connecting the MDIO to an External PHY

When the core is used to connect to an external PHY device using GMII/MII or RGMII, it is expected that the MDIO of the core is also connected to the external PHY. This allows the configuration registers of the PHY to be accessed through the Management interface of the core.

In this situation, mdio_i, mdio_o, and mdio_t must be connected to a 3-state buffer to create a bidirectional wire, mdio. This 3-state buffer can be either external to the FPGA, or internally integrated by using an IOB IOBUF component with an appropriate SelectIO™ an interface standard for the external PHY (see the following figure).

Figure 66: External MDIO Interface

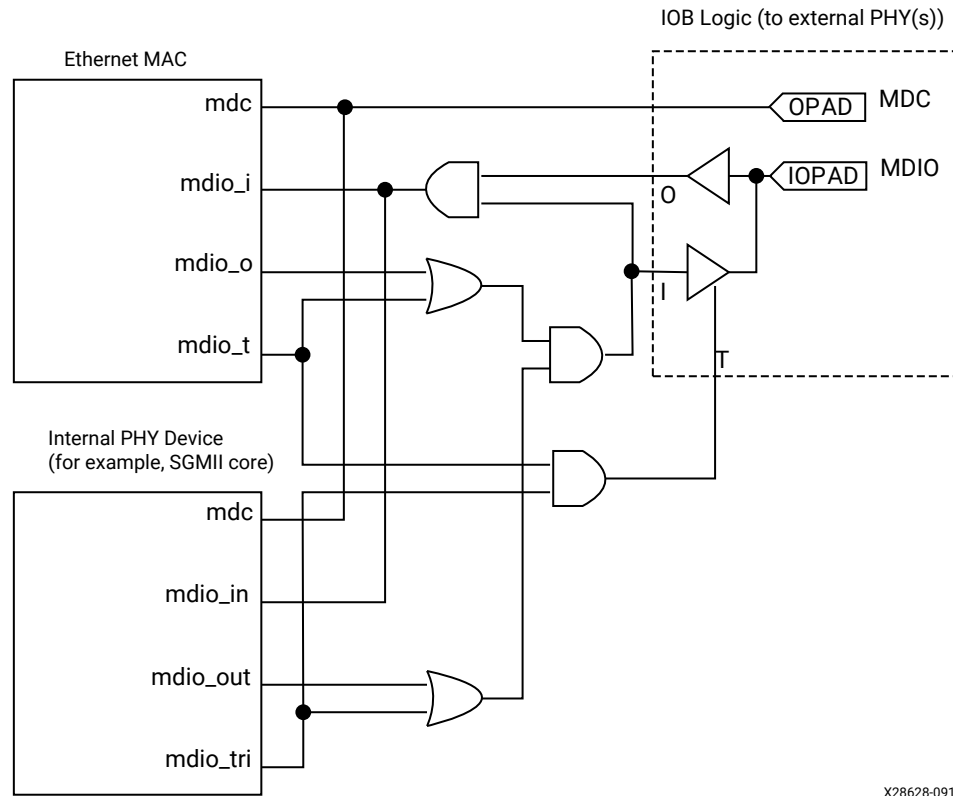


Connecting the MDIO to an External and Internal PHY

The MDIO can connect to more than one device. If an internal PHY is present (such as the [Ethernet 1G/2.5G BASE-X PCS/PMA or SGMII cores](#)), performing as SGMII, and you want to connect the MDIO to an external PHY device, then an arbitration circuit is required. An example circuit is shown in the following figure. Both PHY devices must be assigned a unique non-zero physical address (PHYAD). This description is included for reference, and is not expected to be common.

The SGMII specification contains a method of transferring PHY configuration information across the SGMII link. Therefore, all relevant PHY information can be obtained directly from the internal SGMII core.

Figure 67: Internal and External MDIO Interfaces



X28628-091523

Accessing PHY Configuration Registers, through MDIO Using the Management Interface

The Management Interface is also used to access the MDIO interface of the core. The MDIO interface supplies a clock to the connected PHY, mdc. This clock is derived from the `s_axi_aclk` signal using the value in the Clock Divide[5:0] configuration register. The frequency of mdc is given:

Equation 1: Equation 1

$$f_{MDC} = \frac{f_{s_axi_aclk}}{(1 + \text{Clock Divide}[5:0] \times 2)}$$

The frequency of mdc given by the previous equation should not exceed 2.5 MHz to comply with the IEEE 802.3-2008 specification for this interface. To prevent mdc from being out of specification, the `Clock Divide[5:0]` value powers up at 00000, and while this value is in the register, it is impossible to enable the MDIO interface.

For details of the register map of PHY layer devices and a fuller description of the operation of the MDIO interface itself, see IEEE 802.3-2008.

MDIO Configuration and Control

Access to the MDIO interface through the management interface is entirely register mapped. To perform an MDIO write, the write data must first be written to the MDIO Write Data register, shown in [Table 42: MDIO Write Data \(0x508\)](#). The MDIO write transaction is then initiated by a write to the MDIO Control Word register, shown in [Table 41: MDIO Control Word \(0x504\)](#), with Initiate (Bit[11]) set to 0x1, OP (Bits[15:14]) set to 0x1, and the PHYAD and REGAD set according to the PHY and Register being accessed. This triggers the MDIO Ready bit to de-assert and it remains de-asserted until the MDIO transaction is completed.

To perform an MDIO read, the read transaction is initiated by a write to the MDIO Control Word register, shown in [Table 41: MDIO Control Word \(0x504\)](#), with Initiate (Bit[11]) set to 0x1, OP (Bits[15:14]) set to 0x2, and the PHYAD and REGAD set according to the PHY and Register being accessed. This triggers the MDIO Ready bit to de-assert and it remains de-asserted until the MDIO transaction is completed. When the MDIO Ready is re-asserted the read data is ready to be read from the MDIO Read data register, shown in [Table 43: MDIO Read Data \(0x50C\)](#).

- It is possible to either poll the MDIO Control register or the MDIO Read Data register to check the status of MDIO Ready. Alternatively, the MAC interrupt can be used (see [Interrupt Controller](#)).

Interrupt Controller

An Interrupt Block is implemented in the Tri-Mode Ethernet MAC solution to assert an interrupt when a pending MDIO transaction is completed. Interrupt registers are shown in [Table 44: Interrupt Controller Configuration](#).

Configuration Vector

If the optional management interface is omitted from the core, all of the relevant configuration signals are brought out of the core. These signals are bundled into the `rx_configuration_vector` and the `tx_configuration_vector` signals. The bit mapping of these signals is defined in [Table 23: tx_configuration_vector Bit Definitions](#) and [Table 24: rx_configuration_vector Bit Definitions](#).

You can permanently set the vector bits to logic 0 or 1 or change the configuration vector signals at any time. However, with the exception of the reset signals, they do not take effect until the current frame has completed transmission or reception.

Frame Filter

When the frame filter is selected with no management interface, only a subset of its functionality is available. Because there is no user access to internal registers it is not possible to update the configurable frame filters, these are therefore not generated as part of the core. However, the basic Destination Address filtering is still available and enables the MAC to identify/filter the Broadcast address, a User supplied Pause/Unicast Address and the Special Pause Multicast Address. In this configuration, it is assumed that the user-supplied Pause address is the same as the MAC Unicast address. A packet matching this filter is only treated as a pause frame if it meets all other criteria to identify a pause frame.

TEMAC Configuration Settings

This section discusses unusual configuration options. These can be set by either method (Management Interface or the Configuration Vector).

Half-Duplex Configuration Settings

When the core is generated with half-duplex capability, the transmitter and receiver can be independently configured between full and half-duplex modes. This functionality is made available for full flexibility in unusual applications, for example, Ethernet protocol testers. However, for legal and predictable behavior in Ethernet networks, always configure transmitter and receiver duplex modes identically.

Half-Duplex and Flow Control Configuration Settings

The Flow Control functionality (legacy), as defined in IEEE Standard 802.3, and the Priority Flow Control functionality, as defined in the IEEE standard 802.1Qbb, are specified by the respective standards for full-duplex applications only.

Configuration of the TEMAC allows Legacy Flow Control/Priority Flow Control functionality and Duplex mode to be configured independently. However, Legacy Flow Control/Priority Flow Control is enabled only in full-duplex mode. Ensure that the following conditions are met:

- When operating half-duplex mode, always disable Legacy Flow Control/Priority Flow Control.
- When operating in full-duplex mode, Flow Control/Priority Flow Control can optionally be enabled.

MAC Address Settings

Under all core generation settings, the core contains a configurable Pause frame MAC Source Address (see [MAC Configuration Registers](#)) and the use of configuration vectors allows this to be set independently for RX and TX. This [MAC Address](#) is used by the flow control logic, received pause frames are matched against this address appearing in the Destination Address field, pause frames initiated by the core place this MAC Address into the Source Address field of a transmitted pause frame.

When the TEMAC solution is generated with the optional frame filter, the core contains a configurable Unicast Address (see [MAC Configuration Registers](#)). This is used by the frame filter to match against this address appearing in the Destination Address field of all received frames. The core, for full flexibility, allows the Pause frame MAC Source Address and the Unicast Address (when present) to be configured independently.

However, under standard network operating conditions the Pause frame MAC Source Address should be set to the Unicast Address.

The core, when generated without a management interface, contains independent RX and TX Pause Frame MAC source Address control which should be set to the same value.

AVB Endpoint

When enabling AVB Endpoint operation, disable flow control and jumbo mode and use only in full-duplex mode.

Note: The AVB Endpoint is only available if the AXI4-Lite management interface is included and therefore there is no configuration vector control available.

Physical Interface for 7 Series and Zynq 7000 Devices

For UltraScale architecture device information, see [Physical Interface for UltraScale Devices](#).

10 or 100 Mbps Ethernet MAC Core Interfaces

The 10 Mbps or 100 Mbps core provides an MII interface. This is typically used to connect the MAC to an external PHY device. The MII, defined in IEEE Std 802.3-2008, clause 22 is a parallel interface that connects a 10 Mbps and/or 100 Mbps capable MAC to the physical sublayers.

For 7 series and AMD Zynq™ 7000 families, the voltage standard used depends on the type of I/O used: HR I/O supports MII at 3.3 V or lower and HP I/O only supports 1.8 V or lower. Therefore, an external voltage converter is required to interoperate with any multi-standard PHY.

MII Transmitter Interface

The logic required to implement the MII transmitter logic is illustrated in [Figure 68: MII Transmitter, Receiver and Clock Logic For 7 Series and Zynq 7000 Devices](#). `mii_tx_clk` is provided by the external PHY device connected to the MII. As shown, this is placed onto regional (BUFR) clock routing to provide the clock for all transmitter logic, both within the core and for the user-side logic which connects to the TX AXI4-Stream interface of the core. Alternatively, for fully flexible I/O placement the BUFR in [Figure 68: MII Transmitter, Receiver and Clock Logic For 7 Series and Zynq 7000 Devices](#) can be replaced with a global clock buffer (BUFG). To perform this, edit the core instance unencrypted HDL file, `<component_name_mii_if>` present in the core instance synth/physical directory, and replace the BUFR instance on `mii_tx_clk` input with a BUFG. For more details on editing core instance unencrypted HDL files, see the [Synthesis and Implementation](#).

To match the user data rate (which uses an 8-bit datapath) and the MII (which uses a 4-bit datapath), the TX AXI4-Stream interface is throttled, using `tx_axis_mac_tready`, under control of the MAC to limit data transfers to every other cycle.

[Figure 68: MII Transmitter, Receiver and Clock Logic For 7 Series and Zynq 7000 Devices](#) also illustrates how to use the physical transmitter interface of the core to create an external MII. The signal names and logic shown in this figure exactly match those delivered with the core. [Figure 68: MII Transmitter, Receiver and Clock Logic For 7 Series and Zynq 7000 Devices](#) shows that the output transmitter signals are registered in device IOBs before driving them to the device pads.

MII Receiver Interface

The logic required to implement the MII receiver logic is also illustrated in [Figure 68: MII Transmitter, Receiver and Clock Logic For 7 Series and Zynq 7000 Devices](#). `mii_rx_clk` is provided by the external PHY device connected to the MII. As illustrated, this is placed on regional (BUFR) clock routing to provide the clock for all receiver logic, both within the core and for the user-side logic which connects to the RX AXI4-Stream interface of the TEMAC.

Alternatively, for fully flexible I/O placement, the BUFR in [Figure 68: MII Transmitter, Receiver and Clock Logic For 7 Series and Zynq 7000 Devices](#) can be replaced with a global clock buffer (BUFG). To perform this edit the core instance unencrypted HDL file, `<component_name_mii_if>` present in the core instance synth/physical directory, and replace the BUFR instance on `mii_rx_clk` input with a BUFG. For more details on editing core instance unencrypted HDL files, see the [Synthesis and Implementation](#).

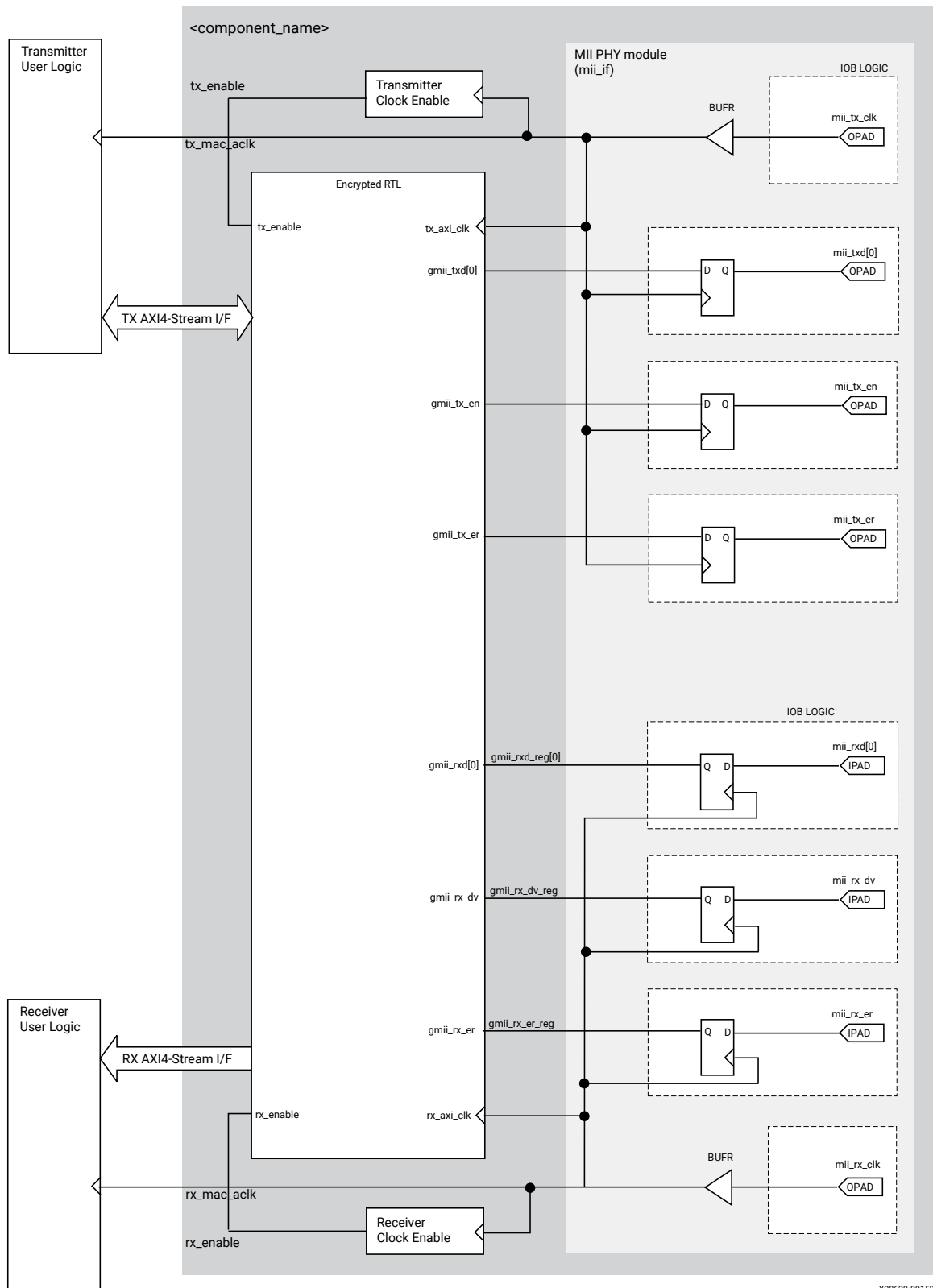
To match the user data rate, which uses an 8-bit datapath and the MII, which uses a 4-bit datapath, the RX AXI4-Stream interface is throttled, using `rx_axis_mac_tvalid`, under control of the MAC to limit data transfers to every other cycle.

[Figure 68: MII Transmitter, Receiver and Clock Logic For 7 Series and Zynq 7000 Devices](#) also illustrates how to use the physical receiver interface of the core to create an external MII. The signal names and logic shown in this figure exactly match those delivered with the example design. [Figure 68: MII Transmitter, Receiver and Clock Logic For 7 Series and Zynq 7000 Devices](#) shows that the input receiver signals are registered in device IOBs before routing them to the core.

Multiple Core Instances with the MII

Because both `mii_tx_clk` and `mii_rx_clk` are sourced by the external PHY device connected to the MII, it is not possible to share transmitter or receiver clock resources across multiple instantiations of the core. Each instance of the core requires its own independent clocking resources. Therefore, the logic of [Figure 68: MII Transmitter, Receiver and Clock Logic For 7 Series and Zynq 7000 Devices](#) must be duplicated for each instance of the core.

Figure 68: MII Transmitter, Receiver and Clock Logic For 7 Series and Zynq 7000 Devices



X28629-091523

1 Gbps Ethernet MAC Core Interfaces

The core supplied for 1 Gbps only operation provides either a GMII or RGMII interface. These are typically used to connect the MAC to an external PHY device. GMII defined in IEEE Std 802.3-2008, clause 35, is used to connect a 1 Gbps capable MAC to the physical sublayers.

For 7 series and Zynq 7000 families, the voltage standard used depends on the type of I/O used: HR I/O supports GMII at 3.3V or lower and HP I/O only supports 1.8V or lower. Therefore an external voltage converter is required to interoperate with any multi-standard PHY for GMII.

RGMII is an alternative to GMII and achieves a 50% reduction in the pin count compared with GMII. Therefore, this is often favored over GMII by Printed Circuit Board (PCB) designers. This configuration is achieved with the use of double-data-rate (DDR) flip-flops.

For 7 series and Zynq 7000 families, the voltage standard used depends on the type of I/O used: HR I/O supports RGMII at 2.5V or lower and HP I/O only supports 1.8V or lower. Despite this being the defined RGMII voltage, most PHYs require 2.5V and therefore an external voltage converter is required to interoperate with any multi-standard PHY for RGMII.

GMII Transmitter Interface

The logic required to implement the GMII transmitter logic is shown in the following figure. The `gtx_clk` is a user-supplied 125 MHz reference clock source. As illustrated, this is placed on global clock routing to provide the clock for all transmitter logic, both within the core and for the user-side logic that connects to the TX AXI4-Stream interface of the core.

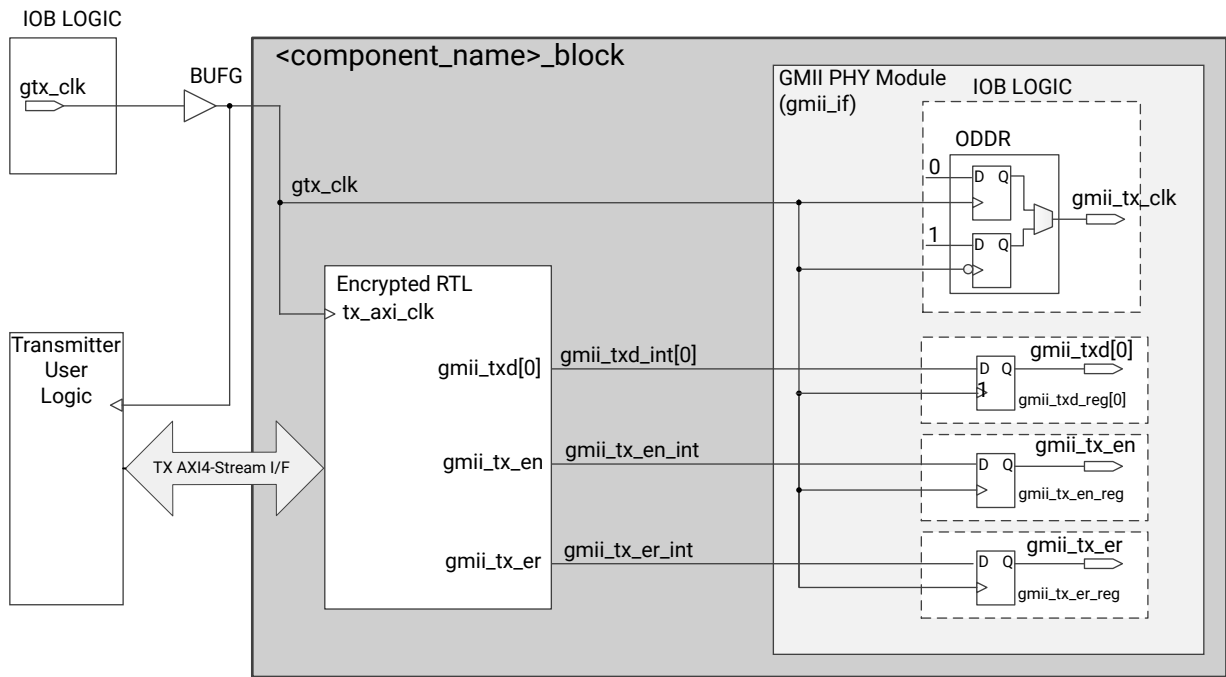
The following figure illustrates how to use the physical transmitter interface of the core to create an external GMII. The signal names and logic shown in this figure exactly match those delivered with the core for an AMD Virtex™ 7 device when the GMII is selected. If other families are chosen, equivalent primitives specific to that family are used in the example design.

The following figure shows that the output transmitter signals are registered in device IOBs before driving them to the device pads. The logic required to forward the transmitter clock is also shown. This logic uses an IOB output Double-Data-Rate (DDR) register so that the clock signal produced incurs exactly the same delay as the data and control signals.

This clock signal, `gmii_tx_clk`, is inverted with respect to `gtx_clk` so that the rising edge of `gmii_tx_clk` occurs in the center of the data valid window, therefore maximizing setup and hold times across the interface.

The half-duplex signals `gmii_col` and `gmii_crs` are asynchronous to the transmit clock. These are routed through PADs and IOBs and then input to the core.

Figure 69: 1 Gbps GMII Transmitter and Clock Logic



X25642-080521

GMII Receive Interface

In this implementation, a BUFIO is used to provide the lowest form of clock routing delay from the input clock to input GMII RX signal sampling at the device IOBs. This creates placement constraints: a BUFIO capable clock input pin must be selected, and all other input GMII RX signals must be placed in the respective BUFIO region. The *7 Series FPGAs Clocking Resources User Guide* ([UG472](#)) should be consulted.

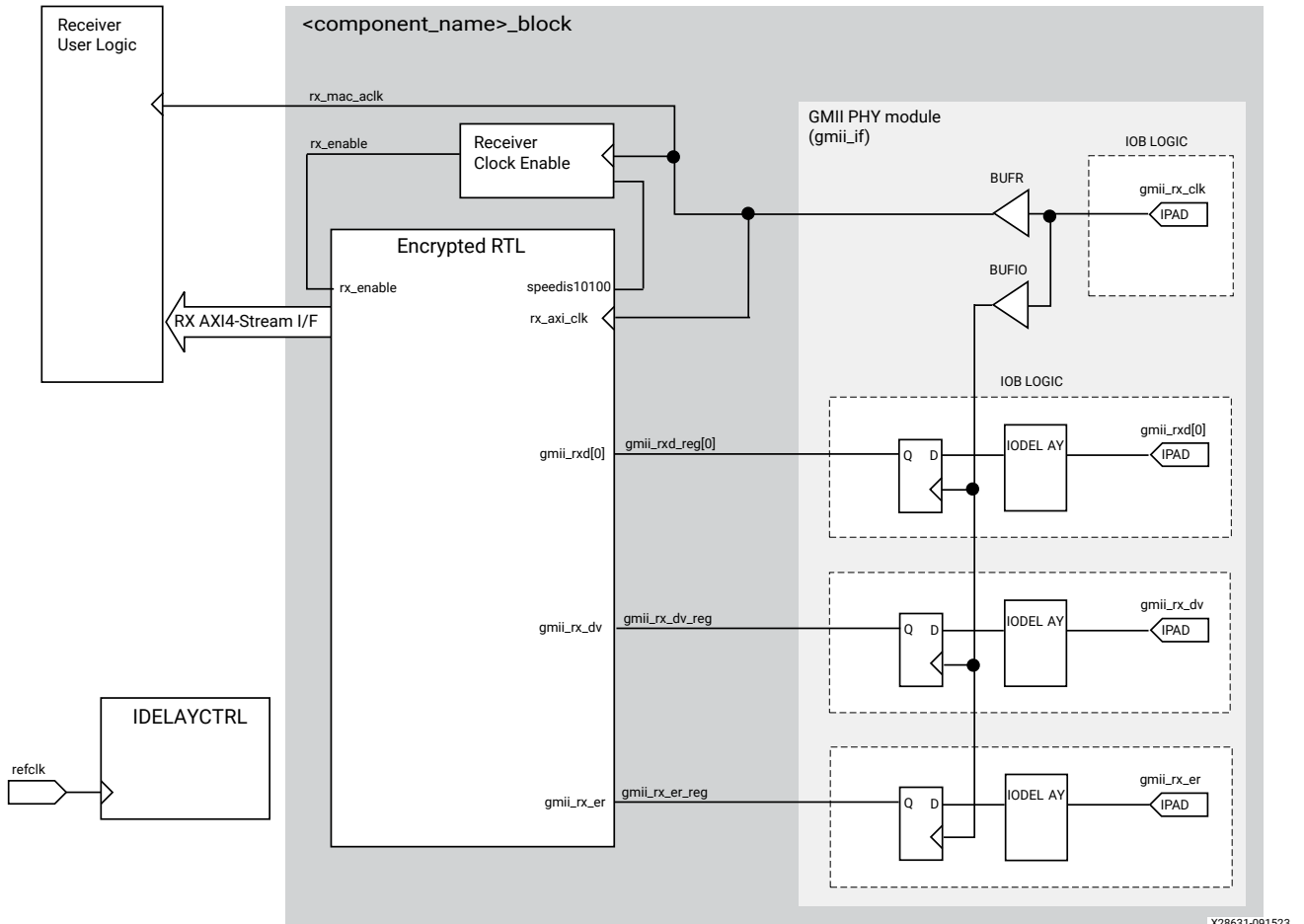
The input clock is also placed onto regional clock routing using the BUFR component as illustrated in [Figure 70: 1 Gbps GMII Receiver and Clock Logic for 7 Series Devices](#). This regional clock then provides the clock for all receiver logic, both within the core and for the user-side logic which connects to the receiver AXI4-Stream interface of the core.

Alternatively, for fully flexible I/O placement, the BUFR in [Figure 70: 1 Gbps GMII Receiver and Clock Logic for 7 Series Devices](#) can be replaced with a global clock buffer (BUFG). To perform this, edit the core instance unencrypted HDL file, `<component_name_gmii_if>` present in the core instance synth/implementation directory, and replace the BUFR instance on `gmii_rx_clk` with a BUFG. For more details on editing core instance unencrypted HDL files, see [Synthesis and Implementation](#). The IODELAY elements can be adjusted to fine-tune the setup and hold times at the GMII IOB input flip-flops. The delay is applied to the IODELAY element using constraints in the XDC which can be edited if desired. See [Constraining the Core](#).

IDELAYCTRL

The following figure shows that IODELAY elements are used by the receiver logic. An IDELAYCTRL module must therefore be instantiated in the design. For the GMII, this is included in the `<component_name>_support` level, present in the core, if the Shared Logic option is selected.

Figure 70: 1 Gbps GMII Receiver and Clock Logic for 7 Series Devices



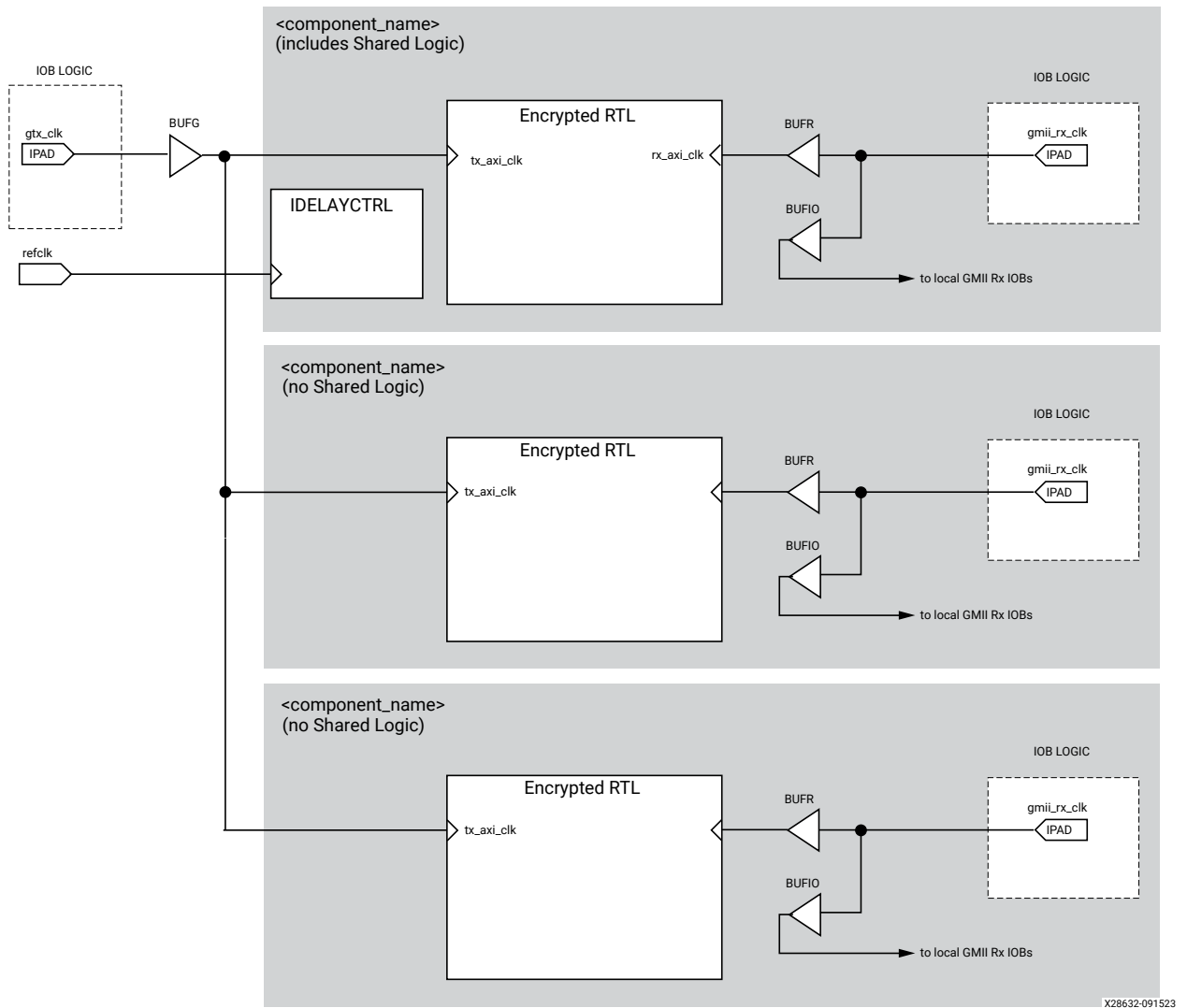
Clock Sharing across Multiple Cores with GMII for 1 Gbps Operation

When multiple instances of the core are instantiated in a design, transmitter clock resources can be shared across all core instances, receiver clock resources cannot be shared and are independent for each core instance.

The following figure shows clock resource sharing across multiple instantiations of the core when using GMII at 1 Gbps. For all instantiations, `gtx_clk` can be shared between multiple cores, resulting in a common clock domain across the device. The receiver clocks cannot be shared. Each core is provided with its own local version of `gmi_rx_clk` from the connected external PHY device as shown in the following figure .

The upper core instance has been generated with the Shared Logic option enabled, and for the GMII this instantiates an IDELAYCTRL as illustrated. The other core instances have been generated without the Shared Logic option enabled (because only a single instance of an IDELAYCTRL is usually required per design). The following figure illustrates three cores. However, more can be added using the same principle. This is done by instantiating further cores without the Shared Logic option and sharing `gtx_clk` across all instantiations. The receiver clock, which cannot be shared, is unique for every instance of the core.

Figure 71: Clock Resource Sharing for 1 Gbps GMII in 7 Series Devices



X28632-091523

RGMII

The logic required to implement the RGMII transmitter logic is shown in [Figure 72: 1 Gbps RGMII Transmitter and Clock Logic for 7 Series Using HP I/O](#). The *gtx_clk* is a user-supplied 125 MHz reference clock source which is placed on global clock routing to provide the clock for all transmitter logic within the core, for the user-side logic which connects to the transmitter AXI4-Stream interface of the core.

[Figure 72: 1 Gbps RGMII Transmitter and Clock Logic for 7 Series Using HP I/O](#) shows how to use the physical transmitter interface of the core to create an external RGMII. The signal names and logic shown in this figure exactly match those delivered with the core. [Figure 72: 1 Gbps RGMII Transmitter and Clock Logic for 7 Series Using HP I/O](#) shows that the output transmitter signals are registered in device IOBs, using DDR registers, before driving them to the device pads.

For 7 series and Zynq 7000 families, the voltage standard used depends on the type of I/O used: HR I/O supports RGMII at 2.5V or lower and HP I/O only supports 1.8V or lower. Despite this being the defined RGMII voltage, most PHYs require 2.5V and therefore an external voltage converter is required to interoperate with any multi-standard PHY when using 7 series HP I/O.

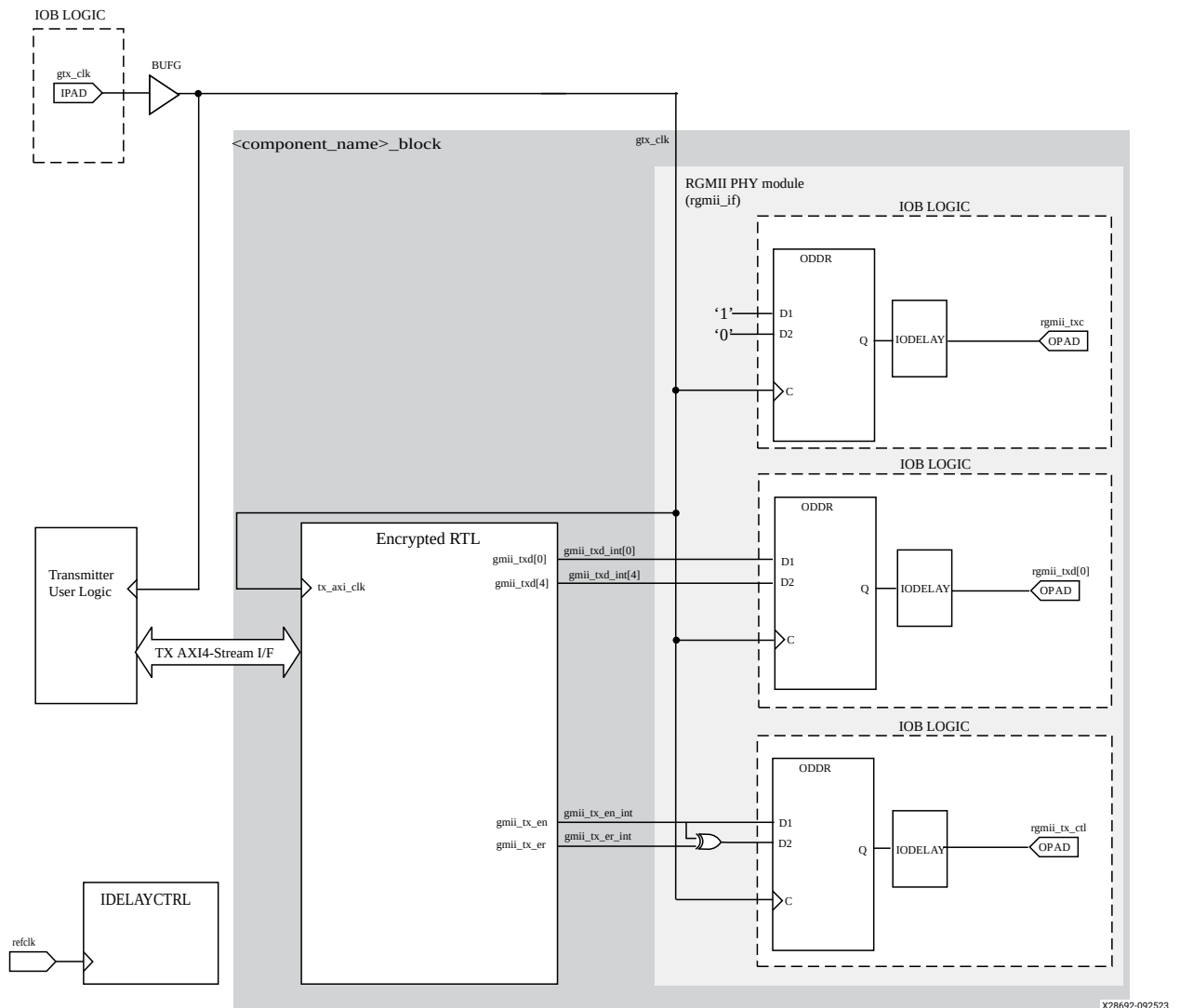
Note: If the target FPGA family is AMD Artix™ 7 or AMD Kintex™ 7, the physical interface logic generated by default is for HR I/O. For Kintex 7 devices, which can contain both HR and HP I/O banks, the HP I/O scheme can be used when the RGMII interface pins are mapped to HP I/O banks.

The logic required to forward the transmitter clock is also shown. This logic uses an IOB output Double-Data-Rate (DDR) register. So, the clock signal produced incurs the same delay as the data and control signals. However, the clock signal is then routed through an output delay element (IODELAY) before connecting to the device pad. The result of this is to create a 2 ns delay, which places the *rgmii_txc* forwarded clock in the center of the data valid window for forwarded RGMII data and control signals.

IDELAYCTRL

The following figure shows that IODELAY elements are used by the transmitter logic. An IDELAYCTRL module must therefore be instantiated in the design. For the RGMII, this is included in the `<component_name>_support` level, included in the core, if the Shared Logic option is selected.

Figure 72: 1 Gbps RGMII Transmitter and Clock Logic for 7 Series Using HP I/O



Transmitter Logic for 7 Series Using HR I/O

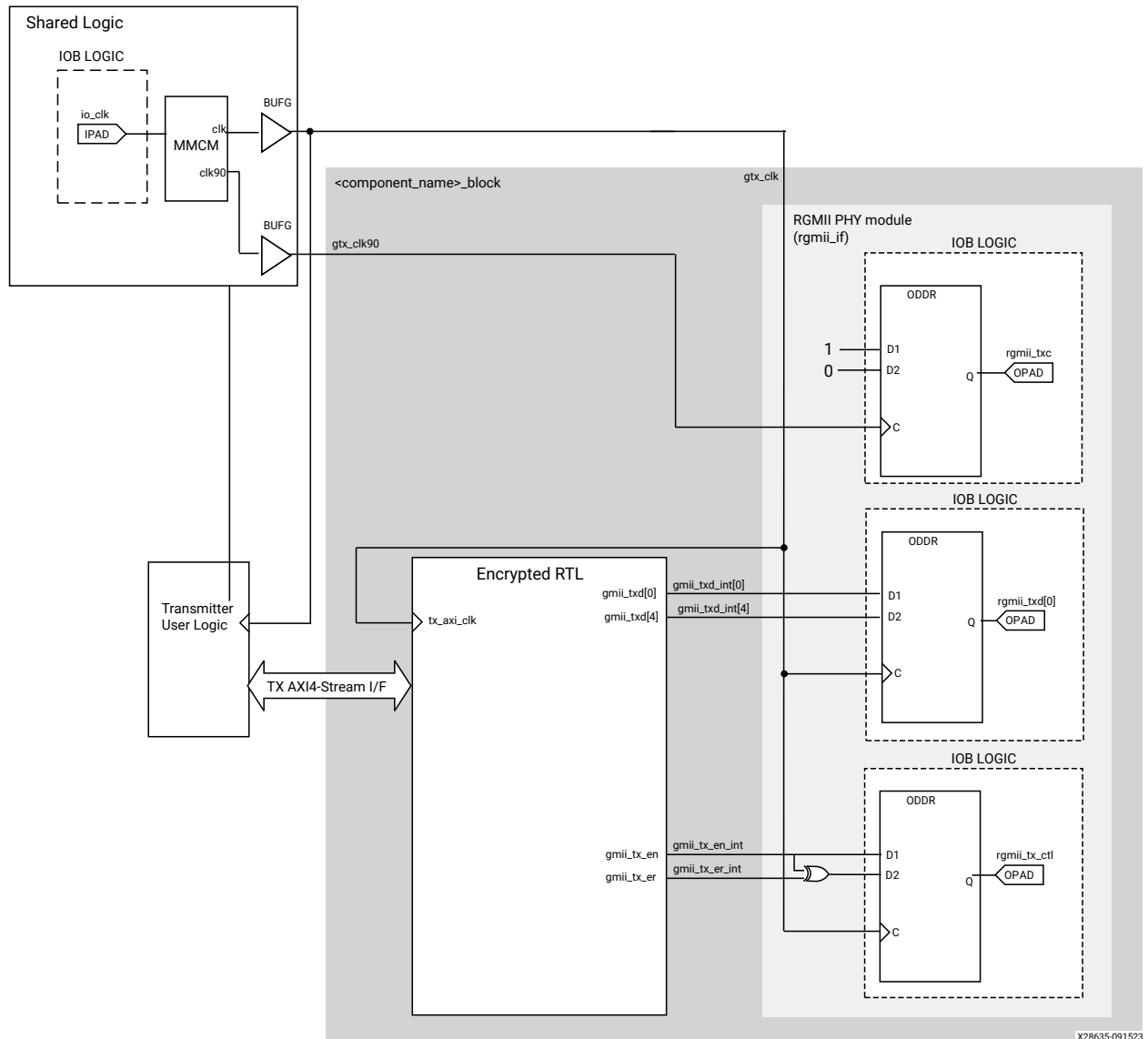
HR I/O do not include ODELAY components and another method is required to introduce the required 2 ns offset between the clock and data.

The logic required to implement the RGMII transmitter logic is illustrated in the following figure . The `gtx_clk` and `gtx_clk90` signals are 125 MHz reference clock sources with 0° and 90° phase shift respectively. These can be created using an MMCM driving BUFGs and as illustrated, this clocking logic can be provided in the core by the Shared Logic option. The `gtx_clk` is used as the clock for the RGMII data and control. It is used both within the core and for the user-side logic which connects to the transmitter AXI4-Stream interface of the core. The `gtx_clk90` is used only for the RGMII clock.

The following figure shows how to use the physical transmitter interface of the core to create an external RGMII. The signal names and logic shown in this figure exactly match those delivered with the core. The following figure shows that the output transmitter signals are registered in device IOBs using DDR registers, before driving them to the device pads.

The logic required to forward the transmitter clock is also shown. This logic uses an IOB output Double-Data-Rate (DDR) register so that the clock signal produced incurs the same delay as the data and control signals. However, the clock signal uses the 90° phase shifted version of the clock. The result of this is to create a 2 ns delay, which places the `rgmii_txc` forwarded clock in the center of the data valid window for forwarded RGMII data and control signals.

Figure 73: 1 Gbps RGMII Transmitter and Clock Logic for 7 Series Using HR I/O



Receiver Logic

In this implementation, a BUFIO is used to provide the lowest form of clock routing delay from the input clock to input RGMII RX signal sampling at the device IOBs. This creates placement constraints: a BUFIO capable clock input pin must be selected, and all other input RGMII RX signals must be placed in the respective BUFIO region. Refer the *7 Series FPGAs Clocking Resources User Guide* ([UG472](#)).

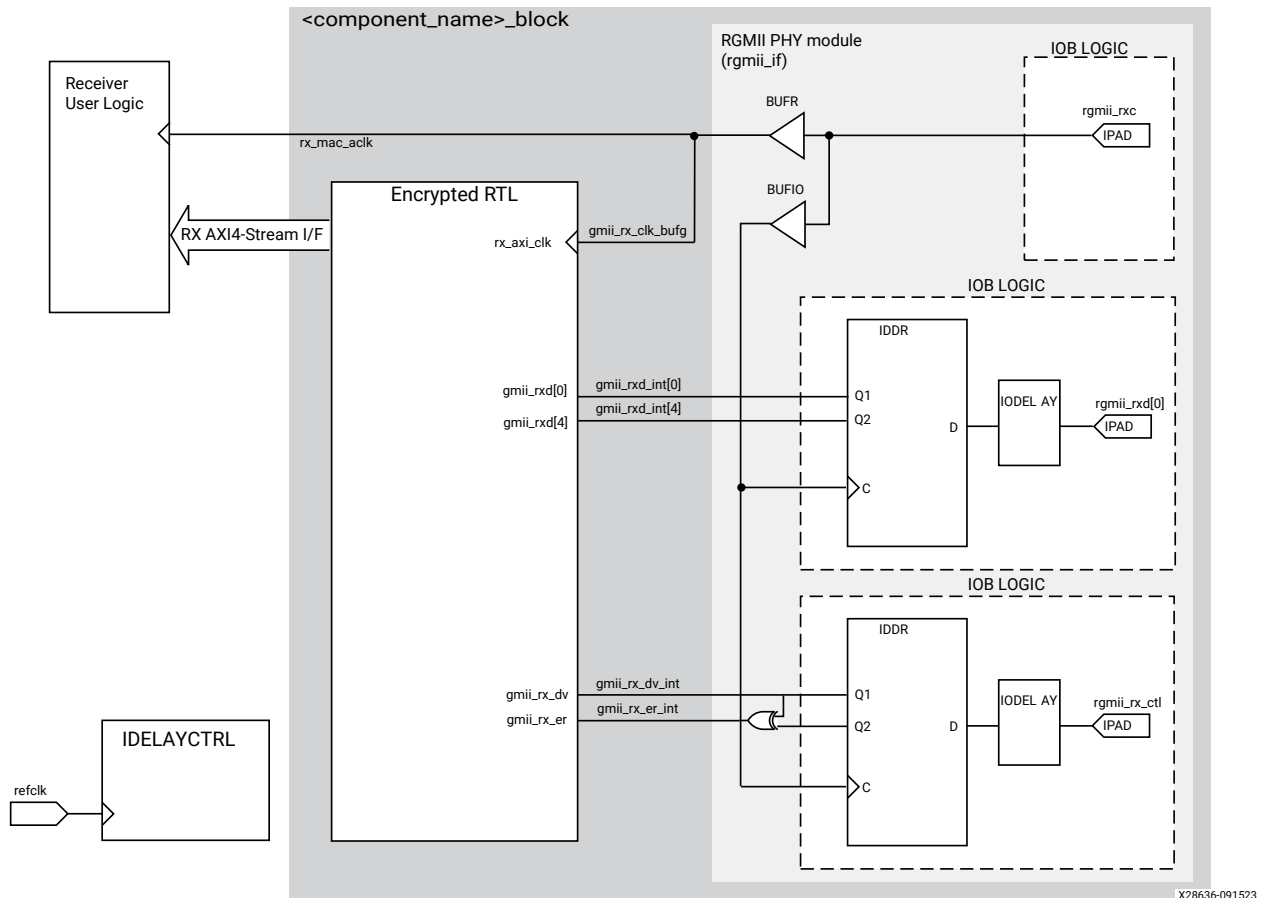
The input clock is also placed onto regional clock routing using the BUFR component as shown in [Figure 74: 1 Gbps RGMII Receiver and Clock Logic for 7 Series Devices](#). This regional clock then provides the clock for all receiver logic, both within the core and for the user-side logic which connects to the receiver AXI4-Stream interface of the core.

Alternatively, for fully flexible I/O placement, the BUFR in [Figure 74: 1 Gbps RGMII Receiver and Clock Logic for 7 Series Devices](#) can be replaced with a global clock buffer (BUFG). To perform this, edit the core instance unencrypted HDL file, `<component_name_rgmii_v2_0_if>` present in the core instance synth/implementation directory, and replace the BUFR instance on `rgmii_rxc` with a BUFG. For more details on editing core instance unencrypted HDL files, see [Synthesis and Implementation](#). The IODELAY elements can be adjusted to fine-tune the setup and hold times at the RGMII IOB input flip-flops. The delay is applied to the IODELAY element using constraints in the XDC which can be edited if desired. See [Constraining the Core](#).

IDELAYCTRL

The following figure shows that IODELAY elements are used by the receiver logic. An IDELAYCTRL module must therefore be instantiated in the design. For the RGMII, this is included in the `<component_name>_support` level, present in the core, if the Shared Logic option is selected.

Figure 74: 1 Gbps RGMII Receiver and Clock Logic for 7 Series Devices



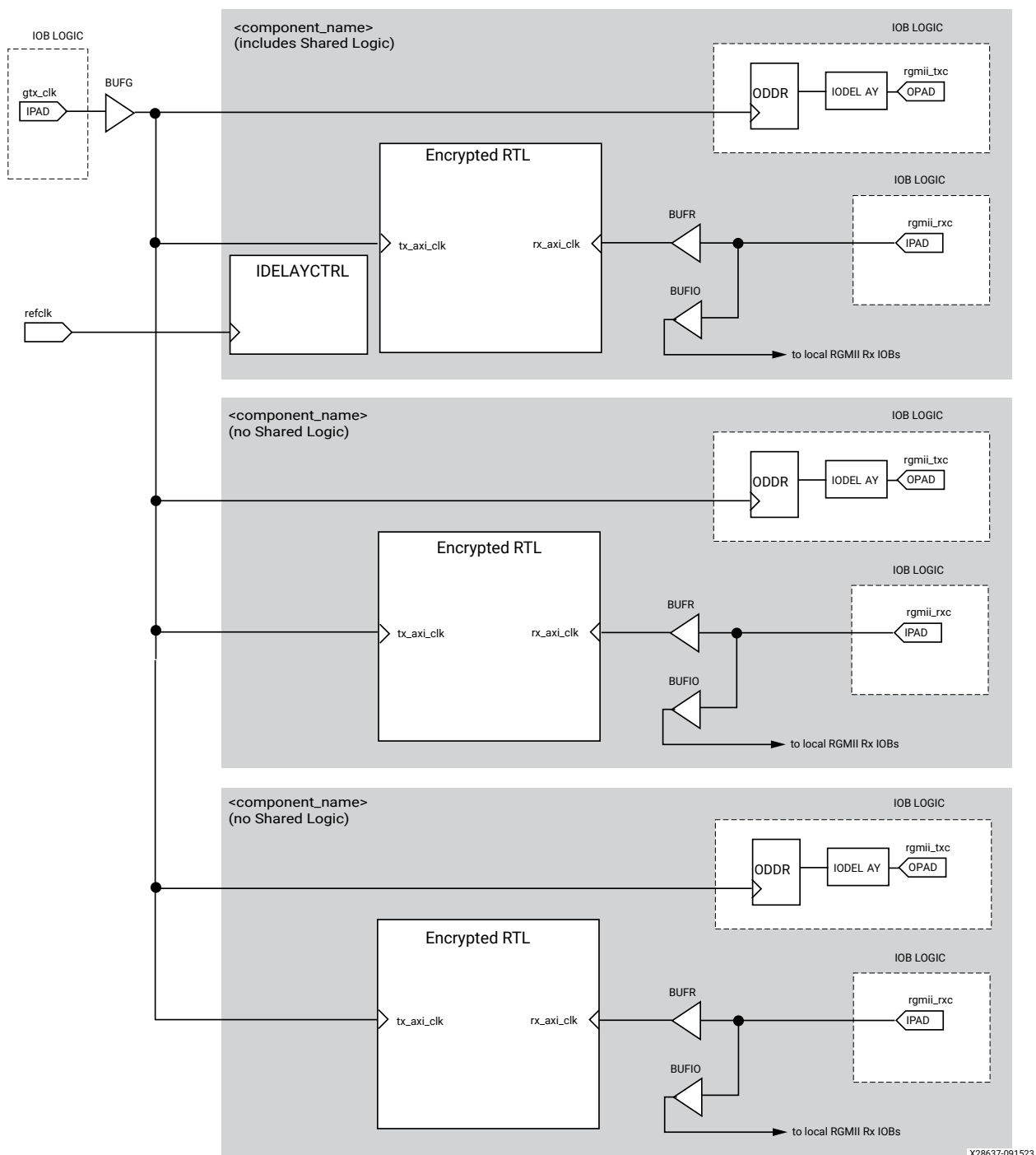
Clock Resource Sharing across Multiple Cores with RGMII for 1 Gbps Operation

[Figure 75: Clock Resource Sharing for 1 Gbps RGMII in 7 Series Using HP I/O](#) illustrates clock resource sharing across multiple instantiations of the core when using RGMII at 1 Gbps in 7 series devices using HP I/O. [Figure 76: Clock Resource Sharing for 1 Gbps RGMII in 7 Series Using HR I/O](#) illustrates clock resource sharing across multiple instantiations of the core when using RGMII at 1 Gbps in 7 series devices using HR I/O. For all instantiations, `gtx_clk` and `gtx_clk90`, where present, can be shared between multiple cores, resulting in a common clock domain across the device. The receiver clocks cannot be shared. Each core is provided with its own local version of `rgmii_rxc` from the connected external PHY device as shown in the following figures.

In both figures, the upper core instance has been generated with the Shared Logic option enabled including an IDELAYCTRL as illustrated. The other core instances have been generated without the Shared Logic option enabled.

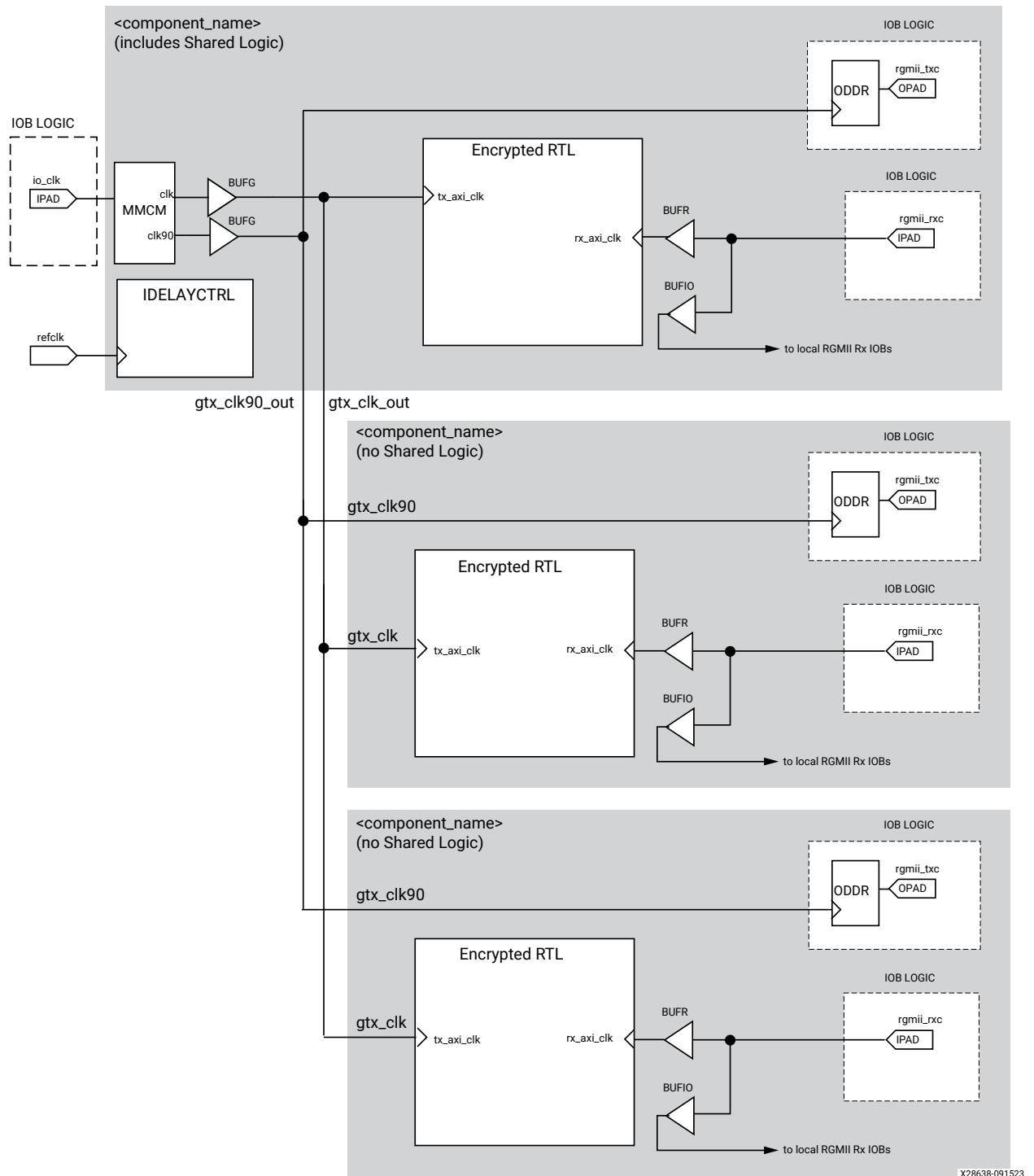
The following figures illustrate three cores. However, more can be added using the same principle. This is done by instantiating further cores without the Shared Logic option and sharing `gtx_clk` across all instantiations. The receiver clock, which cannot be shared, is unique for every instance of the core.

Figure 75: Clock Resource Sharing for 1 Gbps RGMII in 7 Series Using HP I/O



X28637-091523

Figure 76: Clock Resource Sharing for 1 Gbps RGMII in 7 Series Using HR I/O



Tri-Speed Ethernet MAC Core Interfaces

The HDL example design supplied with the core for tri-speed operation (10 Mbps, 100 Mbps, and 1000 Mbps) provides either a GMII or RGMII interface. These are typically used to connect the MAC to an external PHY device.

The MII defined in IEEE 802.3-2008 specification clause 22, is a parallel interface that connects a 10 Mbps and/or 100 Mbps capable MAC to the physical sublayers. The GMII defined in IEEE 802.3-2008 specification clause 35, is an extension of the MII and used to connect a 1 Gbps capable MAC to the physical sublayers. MII can be considered a subset of GMII. As a result, GMII/MII can carry Ethernet traffic at 10 Mbps, 100 Mbps and 1 Gbps.

For 7 series and Zynq 7000 families, the voltage standard used depends on the type of I/O used: HR I/O supports GMII at 3.3 V or lower and HP I/O only supports 1.8 V or lower. Therefore, an external voltage converter is required to interoperate with any multi-standard PHY for GMII.

The RGMII is an alternative to the GMII/MII. RGMII can carry Ethernet traffic at 10 Mbps, 100 Mbps and 1 Gbps and achieves a 50% reduction in the pin count compared with GMII, this is achieved with the use of double-data-rate (DDR) flip-flops. RGMII is therefore often favored over GMII by PCB designers. A further advantage of the RGMII implementation is that, unlike GMII/MII, clock resources for the transmitter can be shared across multiple core instances. This results in significant clock resource savings when implementing multiple cores in a design.

For 7 series and Zynq 7000 families, the voltage standard used depends on the type of I/O used: HR I/O supports RGMII at 2.5 V or lower and HP I/O only supports 1.8 V or lower. Despite this being the defined RGMII voltage most PHYs require 2.5 V. Therefore, an external voltage converter is required to interoperate with any multi-standard PHY for RGMII.

GMII Transmitter Interface

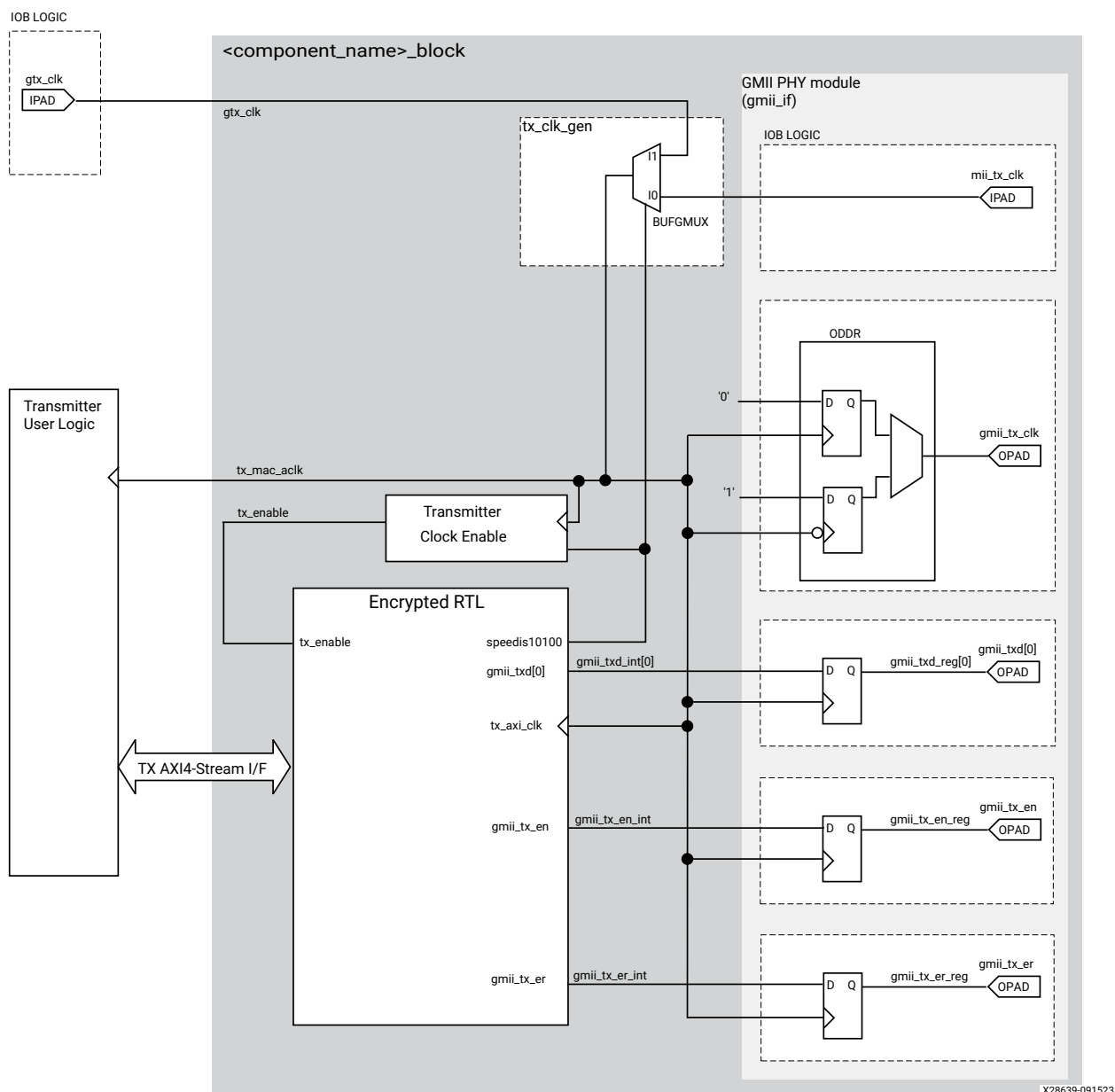
The logic required to implement the GMII transmitter logic is illustrated in the following figure. The `gtx_clk` is a user-supplied 125 MHz reference clock source for use at 1 Gbps. `mii_tx_clk` is sourced by the external PHY device for use at 10 Mbps and 100 Mbps speeds. Consequently, a global clock multiplexer, a BUFGMUX, is used to switch the clock source depending on the operating speed. The output from this BUFGMUX provides the transmitter clock for the core and user logic as illustrated in the following figure .

Closely linked to the clock logic is the use of the `tx_enable` clock enable derivation. This must be provided to the Ethernet MAC core level. All user logic uses the AXI4-Stream interface handshaking to throttle the data to allow for the differing data widths between the 4-bit MII and the cores 8-bit user datapath.

The following figure also illustrates how to use the physical transmitter interface of the core to create an external GMII. The signal names and logic shown in this figure exactly match those delivered with the core for a Virtex 7 device when the GMII is selected. If other families are chosen, equivalent primitives specific to that family are used in the example design.

As shown in the following figure , the output transmitter signals are registered in device IOBs before driving them to the device pads. The logic required to forward the transmitter clock for 1 Gbps operation is also shown. This logic uses an IOB output Double-Data-Rate (DDR) register so that the clock signal produced incurs exactly the same delay as the data and control signals. This clock signal, `gmii_tx_clk` , is inverted with respect to `gtx_clk` so that the rising edge of `gmii_tx_clk` occurs in the center of the data valid window, therefore maximizing setup and hold times across the interface.

Figure 77: Tri-Speed GMII Transmitter and Clock Logic



GMII Receive Interface

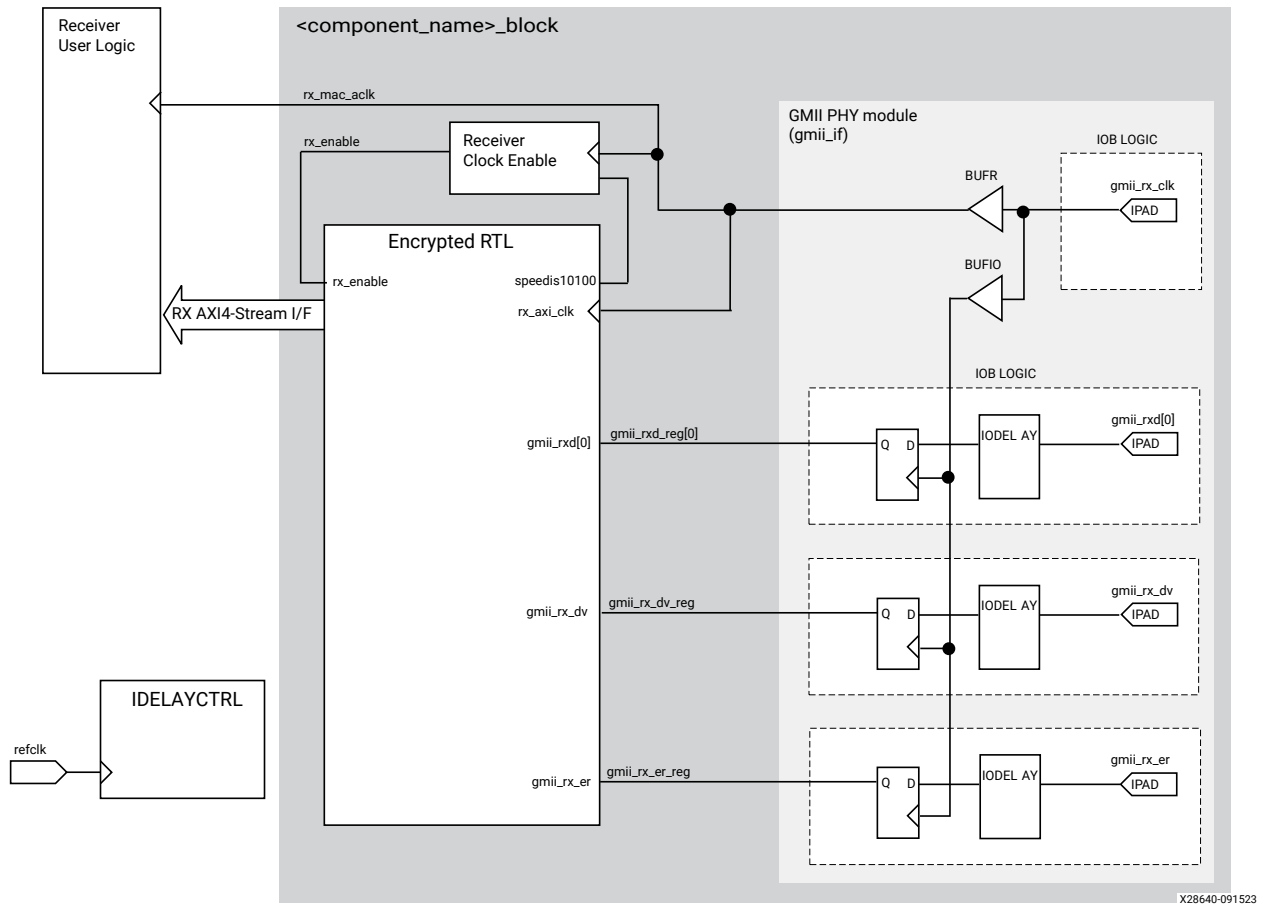
In this implementation, a BUFIO is used to provide the lowest form of clock routing delay from the input clock to input GMII RX signal sampling at the device IOBs. However, this creates placement constraints, a BUFIO capable clock input pin must be selected, and all other input GMII RX signals must be placed in the respective BUFIO region. Refer the *7 Series FPGAs Clocking Resources User Guide* ([UG472](#)).

The input clock is also placed onto regional clock routing using the BUFR component as illustrated in [Figure 78: Tri-Speed GMII Receiver and Clock Logic for 7 Series Devices](#). This regional clock then provides the clock for all receiver logic, both within the core and for the user-side logic which connects to the receiver AXI4-Stream interface of the core.

Alternatively, for fully flexible I/O placement, the BUFR in [Figure 78: Tri-Speed GMII Receiver and Clock Logic for 7 Series Devices](#) can be replaced with a global clock buffer (BUFG). To perform this, edit the core instance unencrypted HDL file, `<component_name_gmii_if>` present in the core instance synth/implementation directory, and replace the BUFR instance on `gmii_rx_clk` with a BUFG. For more details on editing core instance unencrypted HDL files, see [Synthesis and Implementation](#).

The IODELAY elements can be adjusted to fine-tune the setup and hold times at the GMII IOB input flip-flops. This meets the input setup and hold constraints at all three Ethernet speeds. The delay is applied to the IODELAY element using constraints in the XDC which can be edited if desired. See [Constraining the Core](#). Closely linked to the clock logic is the use of the `rx_enable` clock enable derivation. This must be provided to the Ethernet MAC core level. All user logic uses the AXI4-Stream interface handshaking to throttle the data to allow for the differing data widths between the 4-bit MII and the core 8-bit user datapath.

Figure 78: Tri-Speed GMII Receiver and Clock Logic for 7 Series Devices



IDELAYCTRL

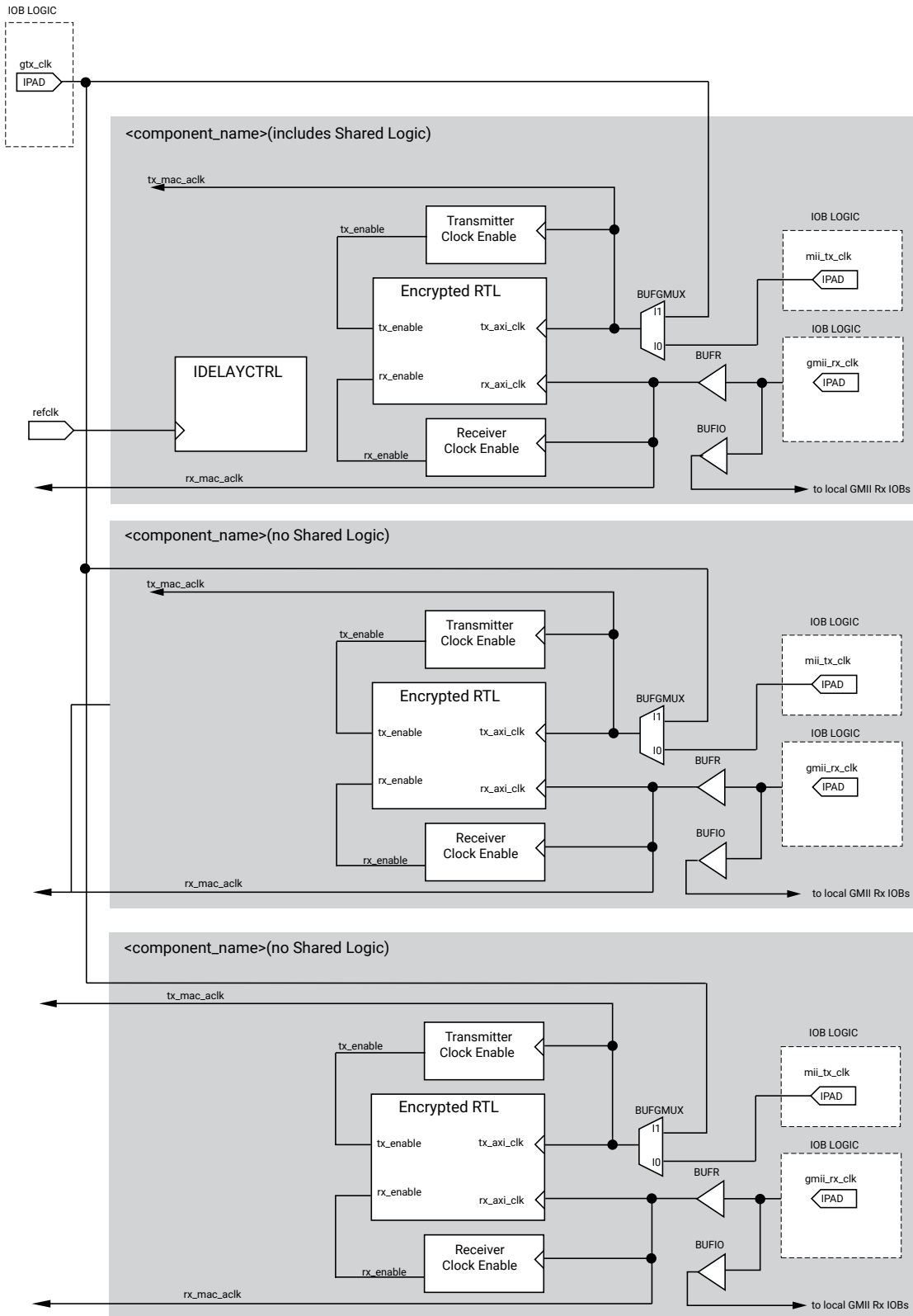
The following figure shows that IODELAY elements are used by the receiver logic. An IDELAYCTRL module must therefore be instantiated in the design. For the GMII, this is included in the `<component_name>_support` level, present in the core, if the Shared Logic option is selected.

Clock Resource Sharing across Multiple Cores with GMII for Tri-Speed Operation

Because both `mii_tx_clk` and `gmii_rx_clk` are sourced by the external PHY device connected to the GMII/MII, it is not possible to share global transmitter or receiver clock resources across multiple instantiations of the core. Each instance of the core requires its own endpoint clocking resources. RGMII provides a more optimal solution because it does allow transmitter clock resources to be shared. See [RGMII](#).

The following figure illustrates that the upper core instance has been generated with the Shared Logic option enabled. For the GMII, this instantiates an IDELAYCTRL as illustrated. The other core instances have been generated without the Shared Logic option enabled (because only a single instance of an IDELAYCTRL is usually required per design). The following figure illustrates three cores. However, more can be added using the same principle. This is done by instantiating further cores without the Shared Logic option.

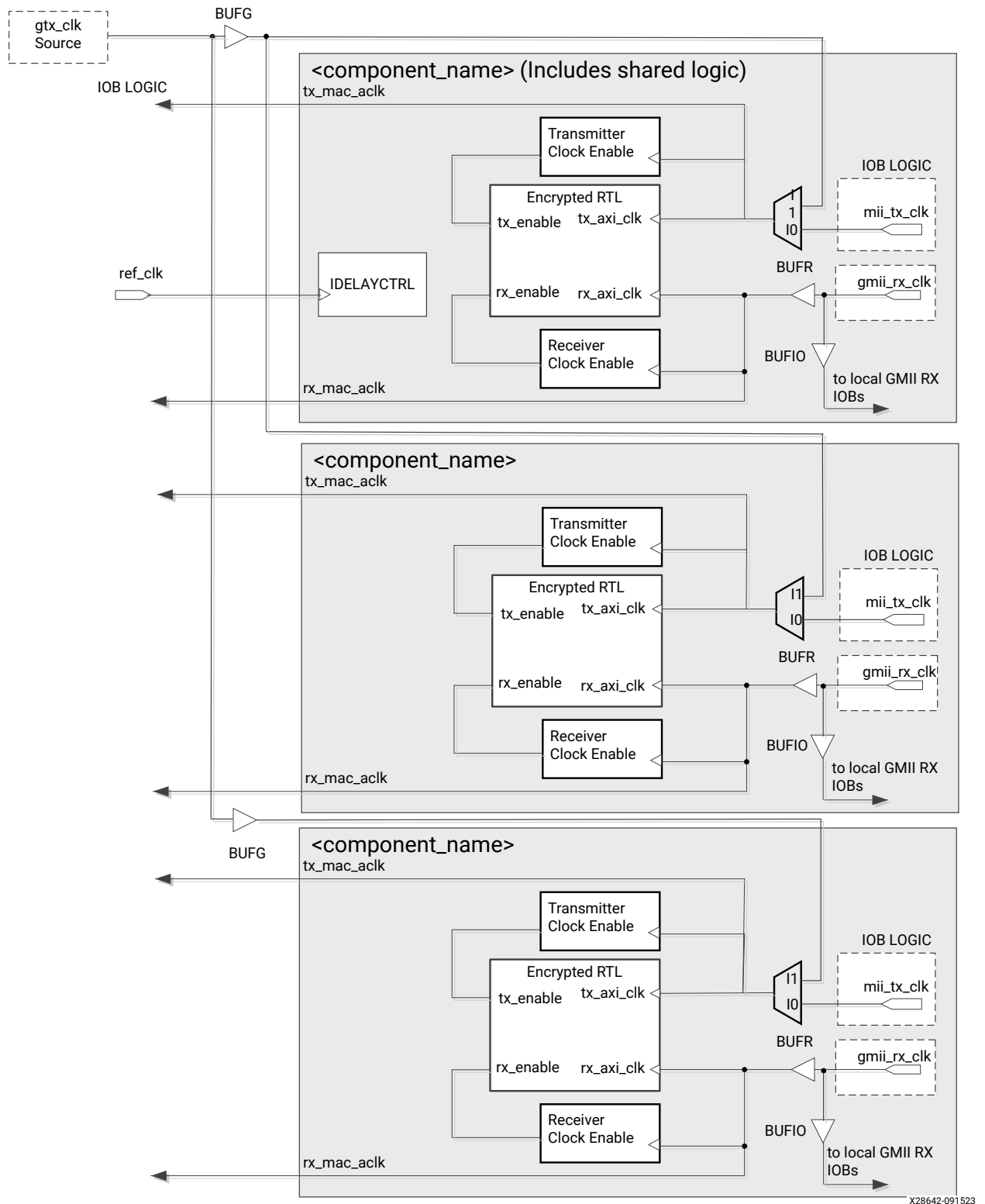
Figure 79: Clock Resource Sharing for Tri-Speed GMII in 7 Series Devices



X28641-091523

If the GTX clock (`gtx_clk`) is sourced from a BUFG then it should be connected to multiple instances of the TEMAC IP, using multiple BUFGs, as shown in the following figure . This is required to limit the quantity of BUFGs are cascaded together up to two. For more details, see the *7 Series FPGAs Clocking Resources User Guide* ([UG472](#)).

Figure 80: Clock Resource Sharing for Tri-Speed GMII in 7 Series Devices with GTX Clock BUFG



RGMII

The logic required to implement the RGMII transmitter logic for HP I/O is illustrated in [Figure 81: Tri-Speed RGMII Transmitter and Clock Logic for 7 Series Devices](#). The `gtx_clk` is a user-supplied 125 MHz reference clock source which is placed onto global clock routing to provide the clock for all transmitter logic, both within the core. For the user-side logic which connects to the transmitter AXI4-Stream interface of the TEMAC.

- If the target FPGA family is Artix 7 or Kintex 7, the physical interface logic generated by default is for HR I/O. For Kintex 7 devices, which can contain both HR and HP I/O banks, the HP I/O scheme can be used when the RGMII interface pins are mapped to HP I/O banks.

For RGMII, this global 125 MHz is used to clock transmitter logic at all three Ethernet speeds. The data rate difference between the three speeds is compensated for by the transmitter clock enable logic (the `enable_gen` module from the example design describes the required logic). The derived `tx_enable` signal must be supplied to the Ethernet MAC core level. All user logic uses the AXI4-Stream interfaces built-in handshaking to throttle the data appropriately, under control of the Ethernet MAC core level. At all speeds the MAC expects the user logic to supply/accept new data after each validated clock cycle. The generated `tx_enable` signal is always High at 1 Gbps, High for one in ten cycles at 100 Mbps and High for one in a hundred cycles at 10 Mbps. The advantage of this approach is that it allows common transmitter global clocks to be shared across any number of instantiated cores.

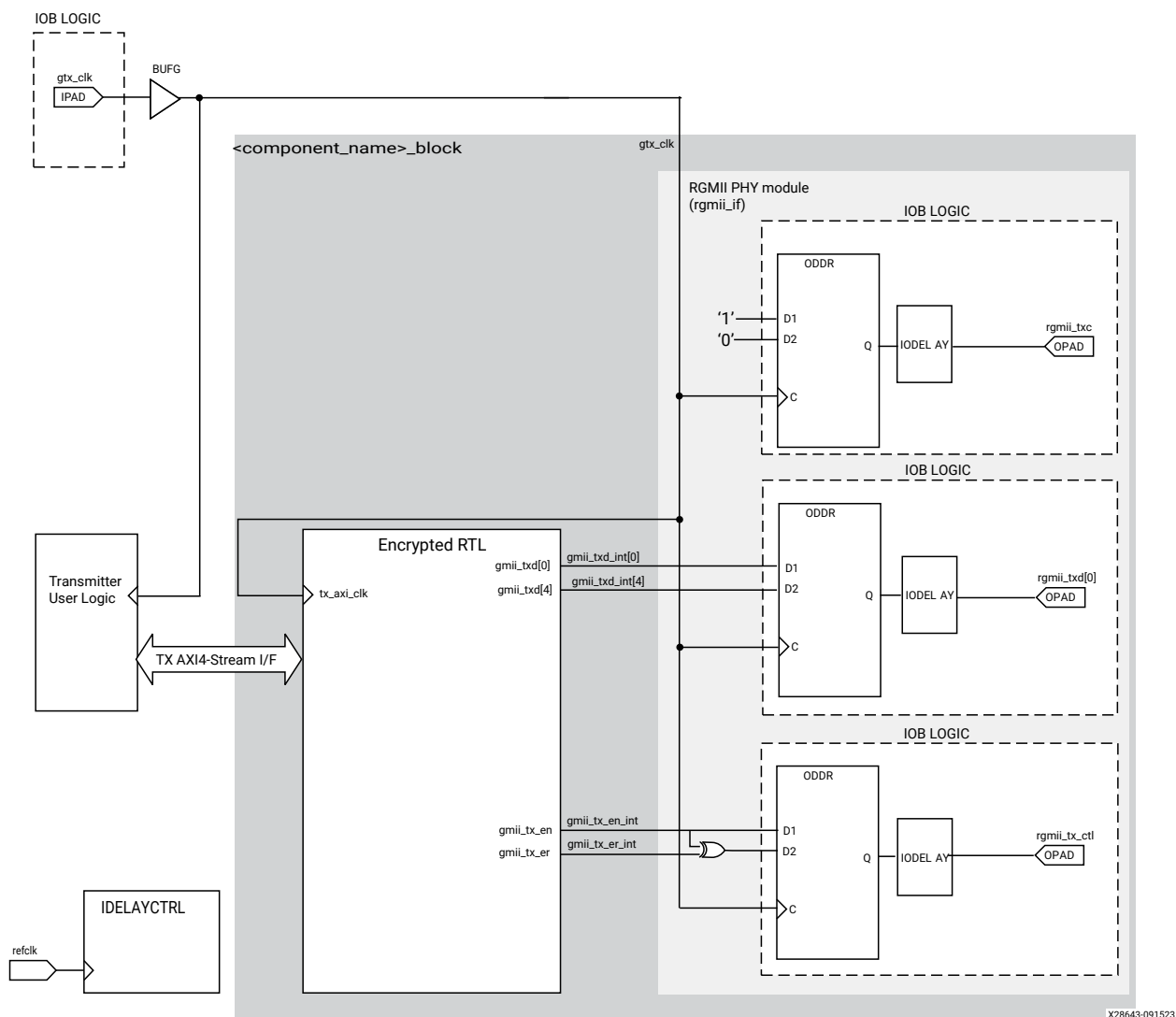
[Figure 81: Tri-Speed RGMII Transmitter and Clock Logic for 7 Series Devices](#) illustrates how to use the physical transmitter interface of the core to create an external RGMII. The signal names and logic shown in this figure exactly match those delivered with the core. [Figure 81: Tri-Speed RGMII Transmitter and Clock Logic for 7 Series Devices](#) shows that the output transmitter signals are registered in device IOBs, using DDR registers, before driving them to the device pads.

The logic required to forward the transmitter clock is also shown. This logic uses an IOB output Double-Data-Rate (DDR) register so that the clock signal produced incurs exactly the same delay as the data and control signals. However, the clock signal is then routed through an output delay element (IODELAY) before connecting to the device pad. The result of this is to create a 2 ns delay, which places the `rgmii_txc` forwarded clock in the center of the data valid window for forwarded RGMII data and control signals when operating at 1 Gbps Ethernet speed. At 10 Mbps and 100 Mbps speeds, the `enable_gen` module toggles the DDR input signals at the required frequency so that the forwarded `rgmii_txc` clock is always of the correct frequency for the forwarded data.

IDELAYCTRL

The following figure shows that IODELAY elements are used by the transmitter logic. An IDELAYCTRL module must therefore be instantiated in the design. For the RGMII, this is included in the `<component_name>_support` level, present in the core, if the Shared Logic option is selected.

Figure 81: Tri-Speed RGMII Transmitter and Clock Logic for 7 Series Devices



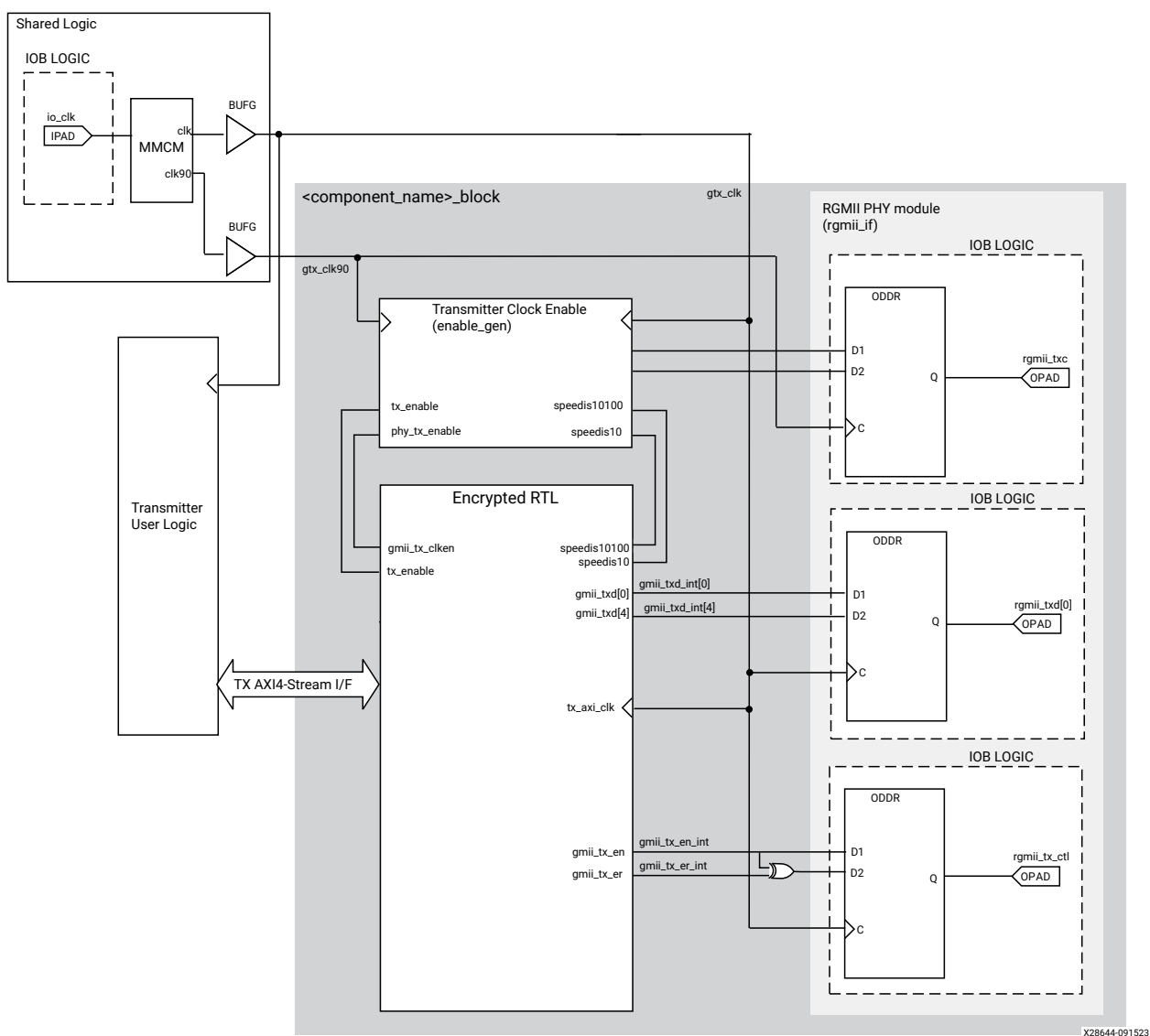
HR I/O do not include ODELAY components and another method is required to introduce the required 2 ns offset between the clock and data. The logic required to implement the RGMII transmitter logic is illustrated in the following figure. The `gtx_clk` and `gtx_clk90` signals are user-supplied 125 MHz reference clock sources with `gtx_clk90` having a 90° phase shift with respect to `gtx_clk`. These are placed onto global clock routing to provide the clocks for all transmitter logic. The `gtx_clk` is used for the RGMII data and control and clock source for the transmit datapath of the core. The `gtx_clk90` is used only for the RGMII clock.

For RGMII, this global 125 MHz is used to clock transmitter logic at all three Ethernet speeds. The data rate difference between the three speeds is compensated for by the transmitter clock enable logic (the `enable_gen` module from the example design describes the required logic). The derived `tx_enable` signal must be supplied to the Ethernet MAC core level. All user logic uses the AXI4-Stream interfaces built in handshaking to throttle the data appropriately, under the control of the Ethernet MAC core level. At all speeds, the MAC expects the user logic to supply/accept new data after each validated clock cycle. The generated `tx_enable` signal is always High at 1 Gbps, High for one in ten cycles at 100 Mbps, and High for one in a hundred cycles at 10 Mbps. The advantage of this approach is that it allows common transmitter global clocks to be shared across any number of instantiated cores.

The following figure illustrates how to use the physical transmitter interface of the core to create an external RGMII. The signal names and logic shown in this figure exactly match those delivered with the example design. The following figure shows that the output transmitter signals are registered in device IOBs, using DDR registers, before driving them to the device pads.

The logic required to forward the transmitter clock is also shown. This logic uses an IOB output Double-Data-Rate (DDR) register so that the produced clock signal incurs exactly the same delay as the data and control signals. However, the clock signal uses the 90° phase-shifted version of the clock. The result of this is to create a 2 ns delay, which places the `rgmii_txc` forwarded clock in the center of the data valid window for forwarded RGMII data and control signals when operating at 1 Gbps Ethernet speed. At 10 Mbps and 100 Mbps speeds, the `enable_gen` module toggles the DDR input signals at the required frequency so that the forwarded `rgmii_txc` clock is always of the correct frequency for the forwarded data.

Figure 82: Tri-Speed RGMII Transmitter and Clock Logic for 7 Series Devices Using HR I/O



Receiver Logic

In this implementation, a BUFIO is used to provide the lowest form of clock routing delay from the input clock to input RGMII RX signal sampling at the device IOBs. However, this creates placement constraints, a BUFIO capable clock input pin must be selected, and all other input RGMII RX signals must be placed in the respective BUFIO region. Refer the *7 Series FPGAs Clocking Resources User Guide* ([UG472](#)).

The input clock is also placed onto regional clock routing using the BUFR component as illustrated in the following figure . This regional clock then provides the clock for all receiver logic, both within the core and for the user-side logic which connects to the receiver AXI4-Stream interface of the core.

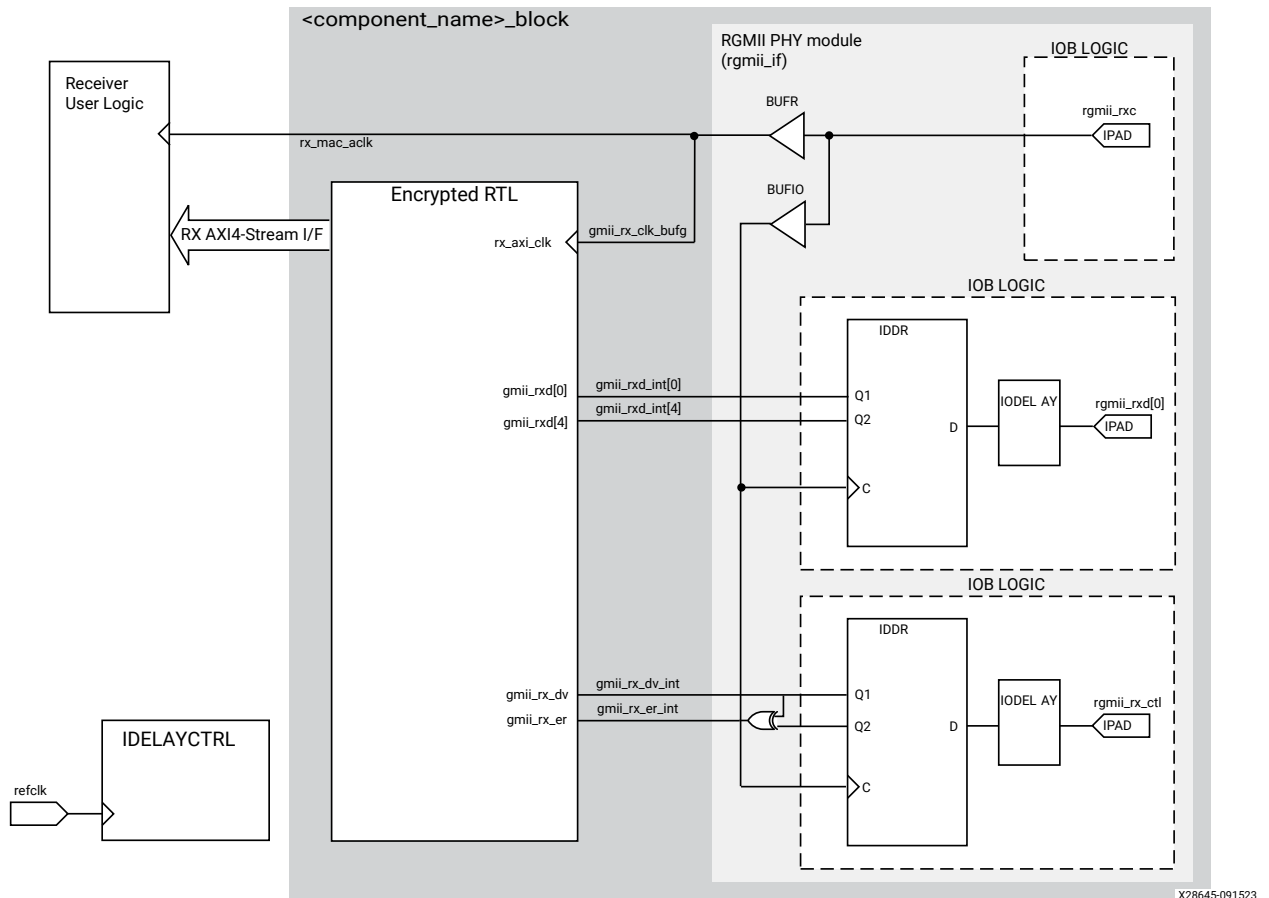
Alternatively, for fully flexible I/O placement, the BUFR in the following figure can be replaced with a global clock buffer (BUFG). To perform this, edit the core instance unencrypted HDL file, `<component_name_rgmii_v2_0_if>` present in the core instance synth/implementation directory, and replace the BUFR instance on `rgmii_rxc` with a BUFG. For more details on editing core instance unencrypted HDL files, see [Synthesis and Implementation](#).

The IODELAY elements can be adjusted to fine-tune the setup and hold times at the RGMII IOB input flip-flops. This meets the input setup and hold constraints at all three Ethernet speeds. The delay is applied to the IODELAY element using constraints in the XDC and these can be edited if desired. See [Constraining the Core](#).

Closely linked to the clock logic is the use of the `rx_enable` clock enable derivation. This must be provided to the Ethernet MAC core level. All user logic uses the AXI4-Stream interface handshaking to throttle the data as required at different speeds.

The following figure shows that IODELAY elements are used by the receiver logic. An IDELAYCTRL module must therefore be instantiated in the design. For the RGMII, this is included in the `<component_name>_support` level, present in the core when the Shared Logic option is selected.

Figure 83: Tri-Speed RGMII Receiver and Clock Logic for 7 Series Devices



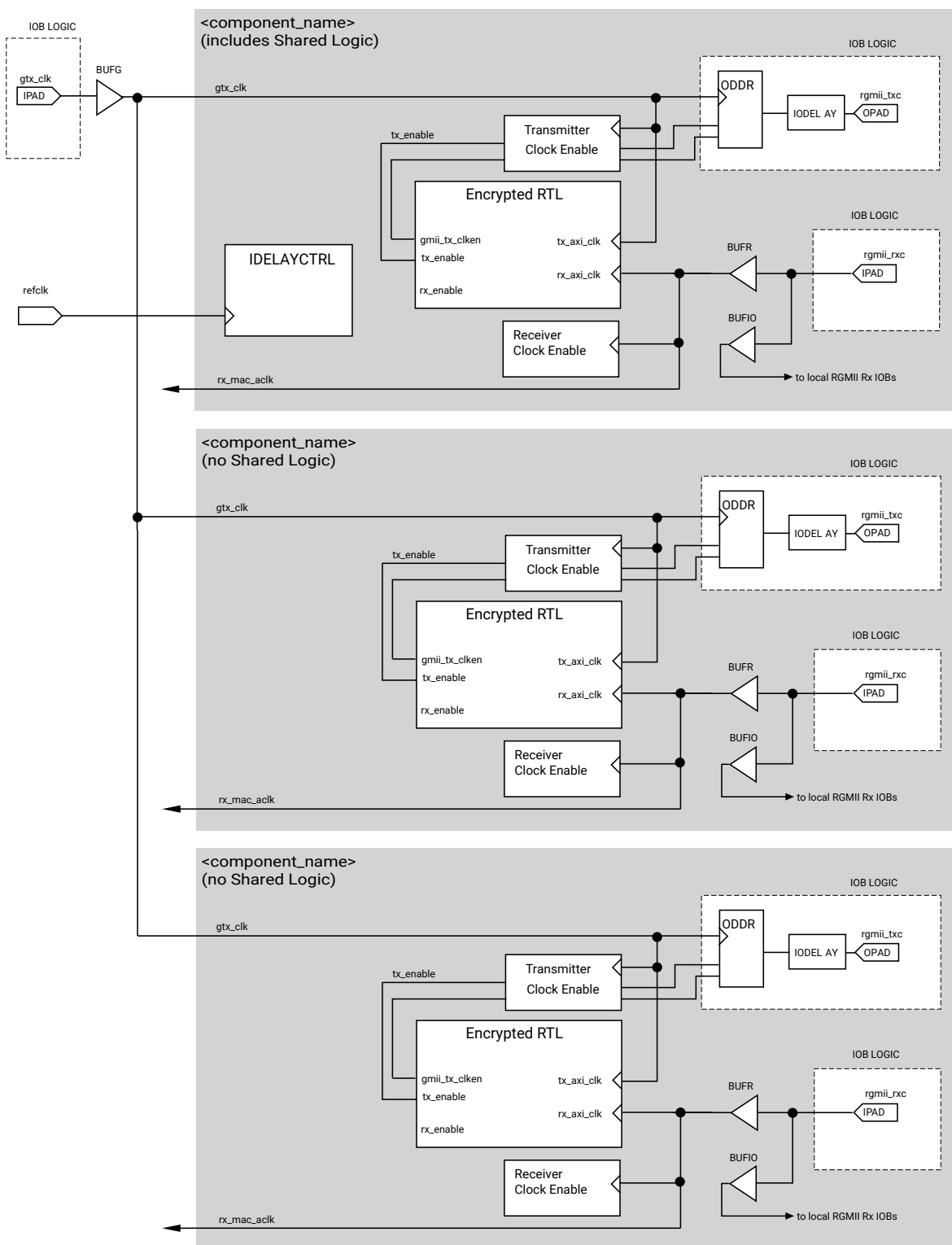
Clock Resource Sharing across Multiple Cores with RGMII for Tri-Speed Operation

Figure 84: Clock Resource Sharing for Tri-Speed RGMII in 7 Series Using HP I/O illustrates clock resource sharing across multiple instantiations of the core when using RGMII in 7 series devices through HP I/O. Figure 85: Clock Resource Sharing for Tri-Speed RGMII in 7 Series Devices Using HR I/O illustrates clock resource sharing across multiple instantiations of the core when using RGMII in 7 series devices through HR I/O. For all instantiations, `gtx_clk` and `gtx_clk90`, where present, can be shared between multiple cores, resulting in a common clock domain across the device. The receiver clocks cannot be shared. Each core is provided with its own local version of `rgmii_rxc` from the connected external PHY device as shown.

In both figures, the upper core instance has been generated with the Shared Logic option enabled. In both cases, this includes an IDELAYCTRL as illustrated. The other core instances have been generated without the Shared Logic option enabled.

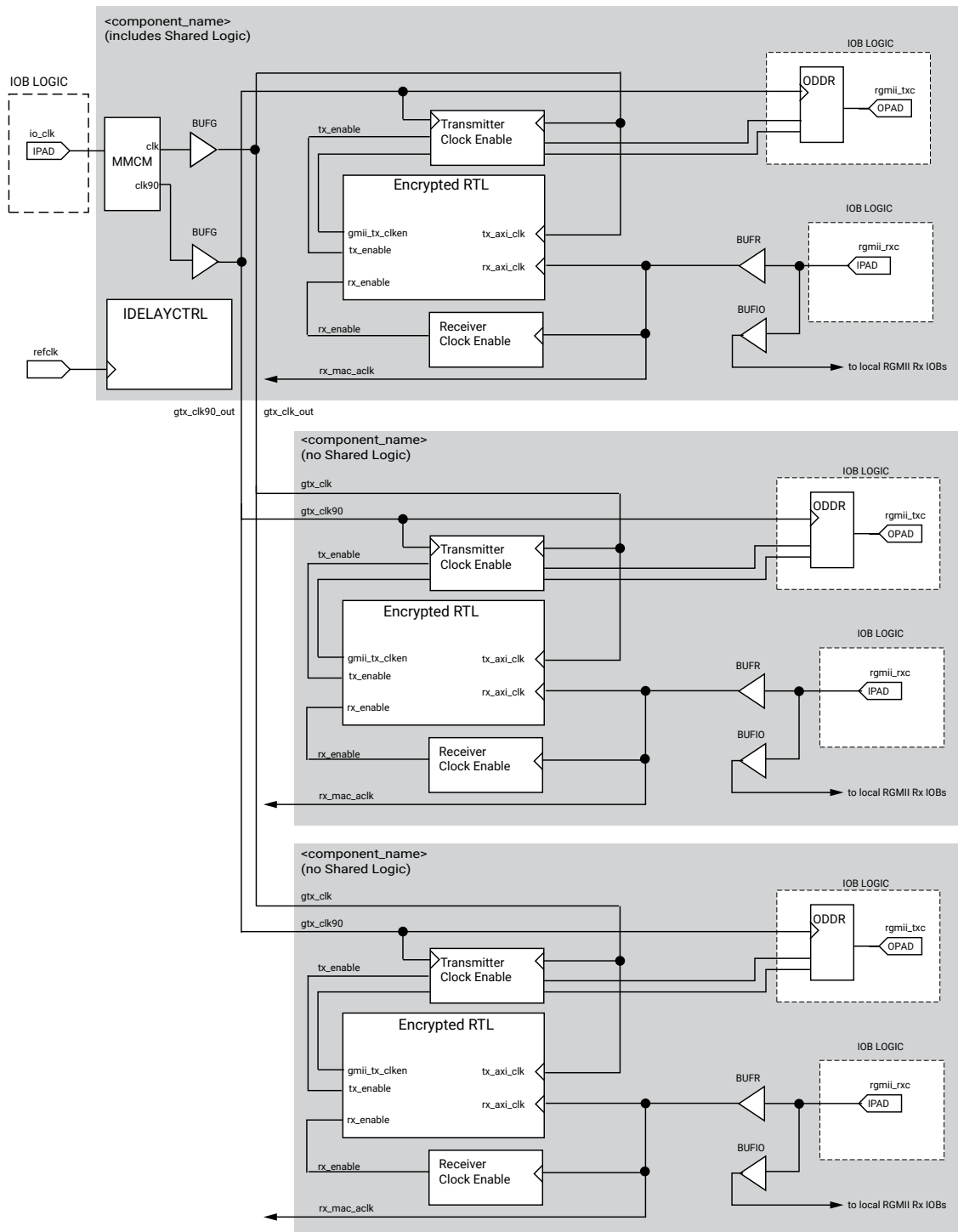
The following figures show three cores. However, more can be added using the same principle. This is done by instantiating further cores without the Shared Logic option and sharing `gtx_clk` across all instantiations. The receiver clock, which cannot be shared, is unique for every instance of the core.

Figure 84: Clock Resource Sharing for Tri-Speed RGMII in 7 Series Using HP I/O



X28646-091523

Figure 85: Clock Resource Sharing for Tri-Speed RGMII in 7 Series Devices Using HR I/O



X28647-091523

Physical Interface for UltraScale Devices

10 or 100 Mbps Ethernet MAC Core Interfaces

The 10 Mbps or 100 Mbps core provides an MII interface. This is typically used to connect the MAC to an external PHY device. The MII defined in IEEE Std 802.3-2008 clause 22, is a parallel interface that connects a 10 Mbps and/or 100 Mbps capable MAC to the physical sublayers.

The voltage standard used depends on the type of I/O used: HR I/O supports MII at 3.3 V or lower and HP I/O only supports 1.8 V or lower. Therefore, an external voltage converter is required to interoperate with any multi-standard PHY.

MII Transmitter Interface

The logic required to implement the MII transmitter logic is illustrated in [Figure 86: MII Transceiver, Receiver and Clock Logic for UltraScale Devices](#). `mii_tx_clk` is provided by the external PHY device connected to the MII. As shown, this is placed onto a global clock routing (BUFG) to provide the clock for all transmitter logic, both within the core and for the user-side logic which connects to the TX AXI4-Stream interface of the core.

To match the user data rate (which uses an 8-bit datapath) and the MII (which uses a 4-bit datapath), the TX AXI4-Stream interface is throttled, using `tx_axis_mac_tready`, under control of the MAC to limit data transfers to every other cycle. [Figure 86: MII Transceiver, Receiver and Clock Logic for UltraScale Devices](#) also illustrates how to use the physical transmitter interface of the core to create an external MII. The signal names and logic shown in this figure exactly match those delivered with the core. [Figure 86: MII Transceiver, Receiver and Clock Logic for UltraScale Devices](#) shows that the output transmitter signals are registered in device IOBs before driving them to the device pads.

MII Receiver Interface

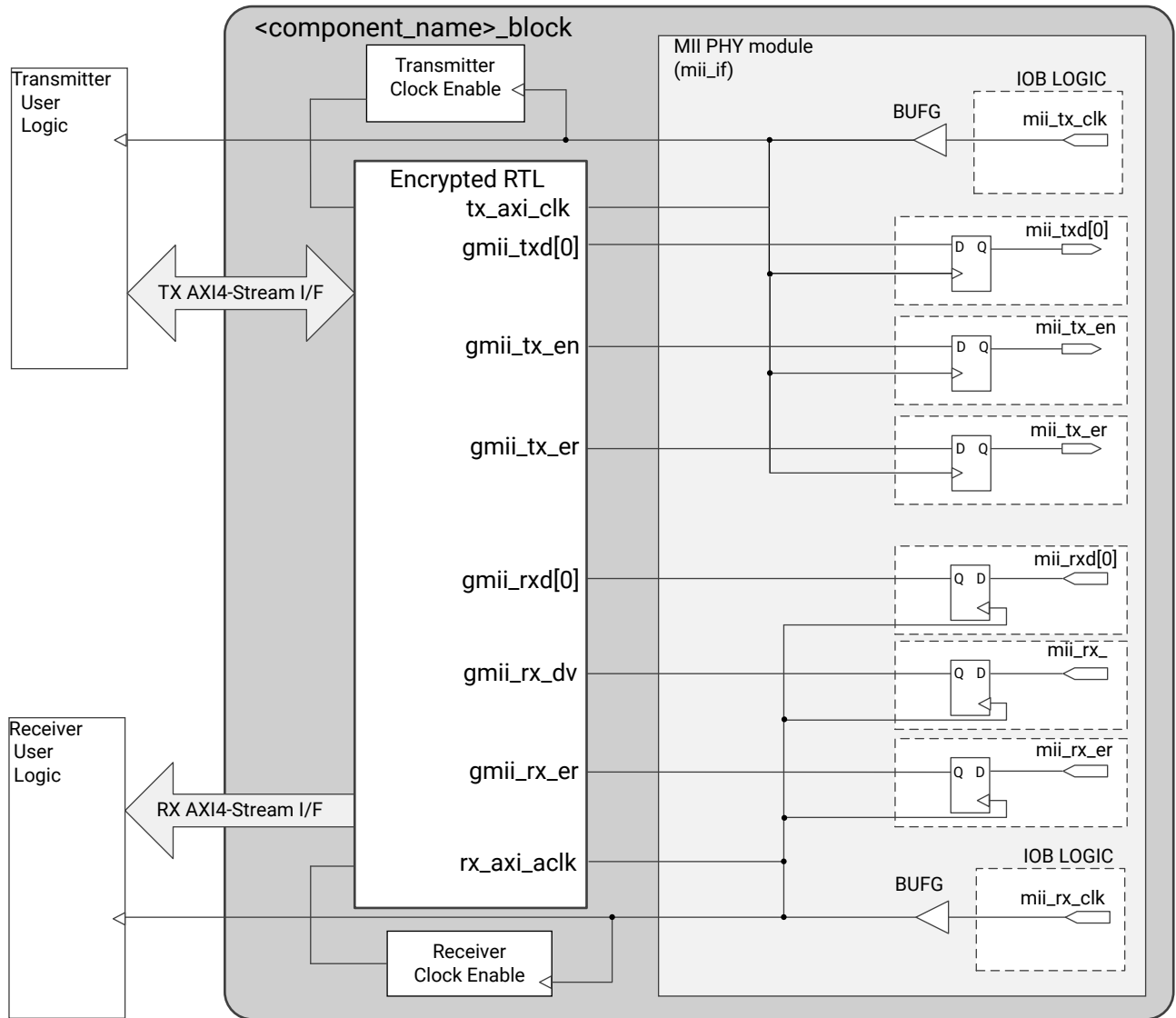
The logic required to implement the MII receiver logic is also illustrated in [Figure 86: MII Transceiver, Receiver and Clock Logic for UltraScale Devices](#). `mii_rx_clk` is provided by the external PHY device connected to the MII. As illustrated, this is placed onto global clock routing (BUFG) to provide the clock for all receiver logic, both within the core and for the user-side logic which connects to the RX AXI4-Stream interface of the TEMAC.

To match the user data rate, which uses an 8-bit datapath and the MII, which uses a 4-bit datapath, the RX AXI4-Stream interface is throttled, using `rx_axis_mac_tvalid`, under control of the MAC to limit data transfers to every other cycle. [Figure 86: MII Transceiver, Receiver and Clock Logic for UltraScale Devices](#) also illustrates how to use the physical receiver interface of the core to create an external MII. The signal names and logic shown in this figure exactly match those delivered with the example design. [Figure 86: MII Transceiver, Receiver and Clock Logic for UltraScale Devices](#) shows that the input receiver signals are registered in device IOBs before routing them to the core.

Multiple Core Instances with the MII

Because both `mii_tx_clk` and `mii_rx_clk` are sourced by the external PHY device connected to the MII, it is not possible to share transmitter or receiver clock resources across multiple instantiations of the core. Each instance of the core requires its own independent clocking resources. Therefore, the logic of the following figure must be duplicated for each instance of the core.

Figure 86: MII Transceiver, Receiver and Clock Logic for UltraScale Devices



X25641-080521

1 Gbps Ethernet MAC Core Interfaces

The core supplied for 1 Gbps operation provides either a GMII or RGMII interface. These are typically used to connect the MAC to an external PHY device. The GMII defined in IEEE Std 802.3-2008 clause 35, is used to connect a 1 Gbps capable MAC to the physical sublayers.

The voltage standard used depends on the type of I/O used: HR I/O supports GMII at 3.3 V or lower and HP I/O only supports 1.8 V or lower. Therefore an external voltage converter is required to interoperate with any multi-standard PHY for GMII. The RGMII is an alternative to the GMII and achieves a 50% reduction in the pin count compared with GMII. Therefore, this is often favored over GMII by Printed Circuit Board (PCB) designers. This configuration is achieved with the use of double-data-rate (DDR) flip-flops.

The voltage standard used depends on the type of I/O used: HR I/O supports RGMII at 2.5 V or lower and HP I/O only supports 1.8 V or lower. Despite this being the defined RGMII voltage, most PHYs require 2.5 V and therefore an external voltage converter is required to interoperate with any multi-standard PHY for RGMII.

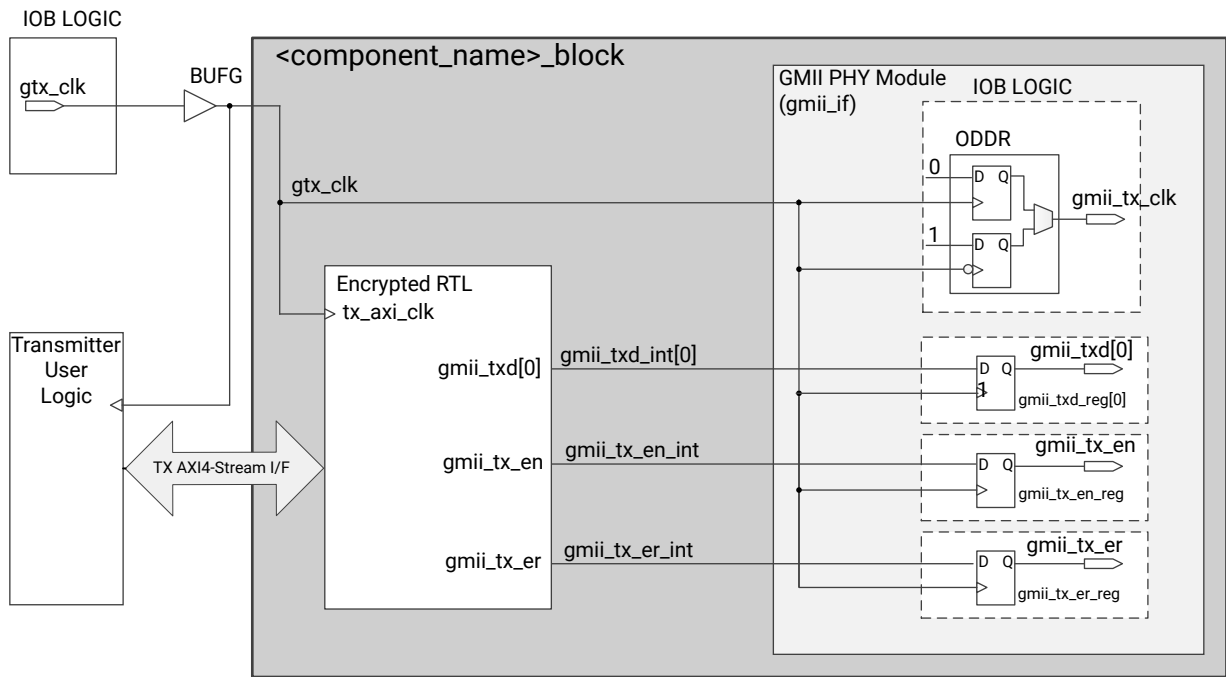
GMII Transmitter Interface

The logic required to implement the GMII transmitter logic is shown in the following figure. The `gtx_clk` is a user-supplied 125 MHz reference clock source. As illustrated, this is placed onto global clock routing to provide the clock for all transmitter logic, both within the core and for the user-side logic that connects to the TX AXI4-Stream interface of the core.

The following figure illustrates how to use the physical transmitter interface of the core to create an external GMII. The signal names and logic shown in this figure exactly match those delivered with the core for an UltraScale architecture-based device when the GMII is selected.

The following figure shows that the output transmitter signals are registered in device IOBs before driving shown. This logic uses a bit slice output Double-Data-Rate (DDR) register so that the clock signal produced incurs exactly the same delay as the data and control signals. This clock signal, `gmii_tx_clk`, is inverted with respect to `gtx_clk` so that the rising edge of `gmii_tx_clk` occurs in the center of the data valid window, therefore maximizing setup and hold times across the interface. The half-duplex signals `gmii_col` and `gmii_crs` are asynchronous to the transmit clock. These are routed through PADS and IOBs, and then input to the core.

Figure 87: 1 Gbps GMII Transmitter and Clock Logic



X25642-080521

GMII Receive Interface

In this implementation, the input clock is used for sampling the GMII RX signal at the device IOBs. This creates placement constraints: a clock capable input pin must be selected, and all other input GMII RX signals must be placed in the respective byte group. See the *UltraScale Architecture Clocking Resources User Guide* (UG572) for more information.

Figure 88: 1 Gbps GMII Receiver and Clock Logic shows that the input clock is passed through two parallel global clock routes using BUFG components. One global clock route is used to clock the receiver IOB logic and the other global clock route is for clocking user-side logic which connects to the receiver AXI4-Stream interface of the core. The reason for providing two parallel clock paths is to help Vivado to close timing on the input paths.

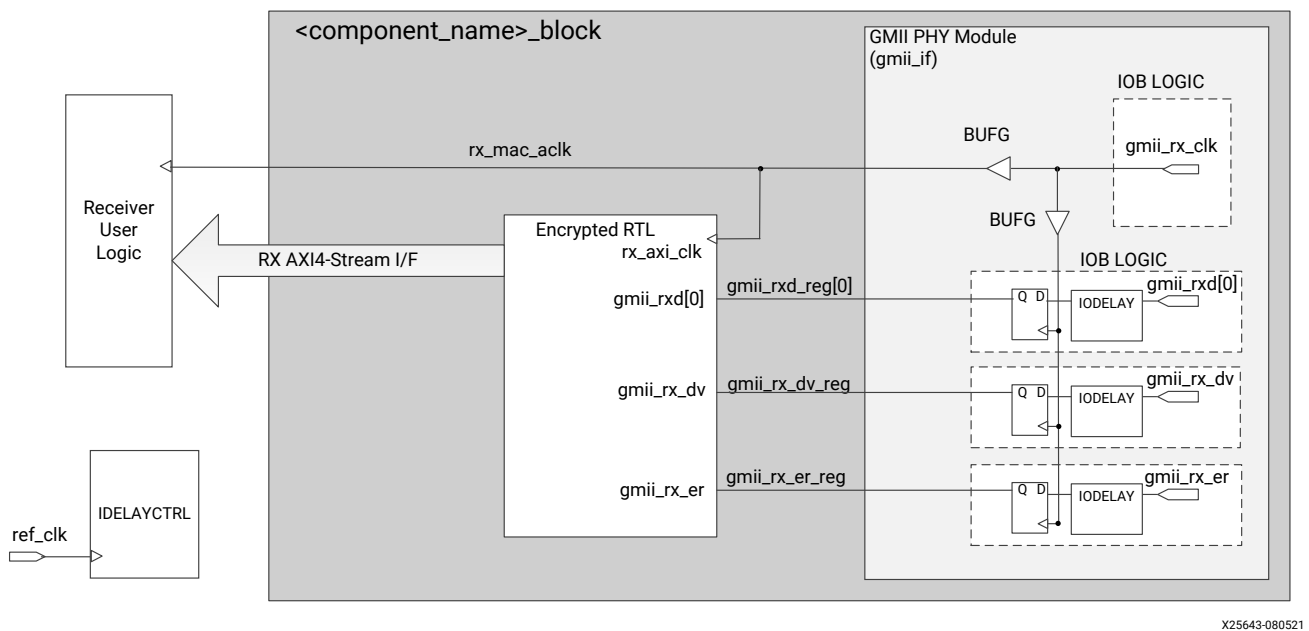
It has been observed that, for some devices, Vivado might not succeed in closing timing on the input paths if the input clock net is heavily loaded. Splitting the single clock route into two parallel routes reduces the load on the clock nets and thus helps in closing timing. However, you can modify the clocking structure to use only one global clock path if you can meet the input timing for your selected devices. To perform this, edit the core instance unencrypted HDL file, `<component_name_gmii_if>` present in the core instance `synth/implementation` directory.

The IODELAY elements can be adjusted to fine-tune the setup and hold times at the GMII IOB input flip-flops. The delay is applied to the IODELAY element using constraints in the XDC which can be edited if desired. See [Constraining the Core](#).

IDELAYCTRL

The following figure shows that IODELAY elements are used by the receiver logic. An IDELAYCTRL module must therefore be instantiated in the design. For the GMII, this is included in the `<component_name>_support` level, present in the core, if the Shared Logic option is selected.

Figure 88: 1 Gbps GMII Receiver and Clock Logic



Clock Sharing across Multiple Cores with GMII for 1 Gbps Operation

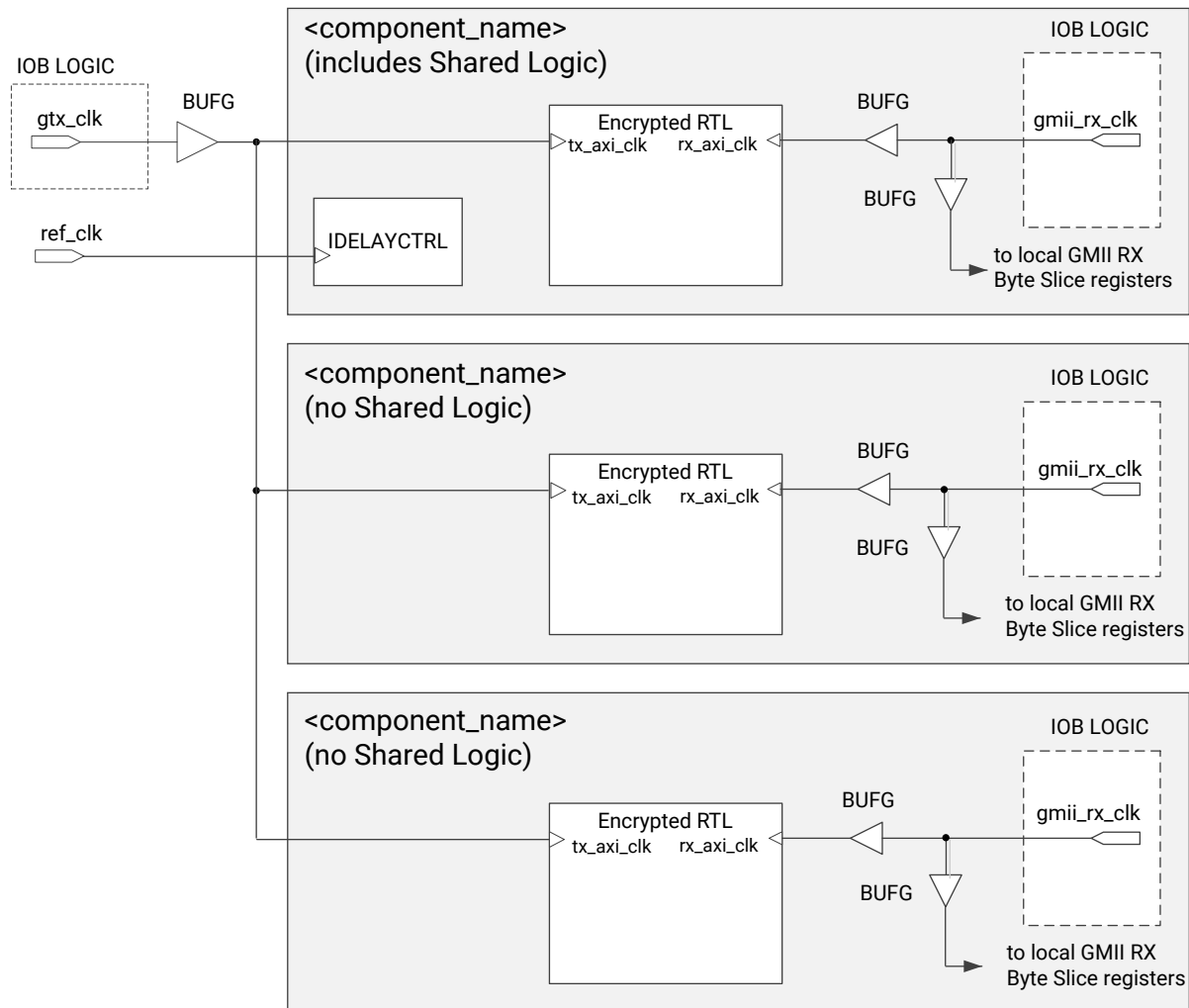
When multiple instances of the core are instantiated in a design, transmitter clock resources can be shared across all core instances, receiver clock resources cannot be shared and are independent for each core instance.

The following figure shows clock resource sharing across multiple instantiations of the core when using GMII at 1 Gbps. For all instantiations, `gtx_clk` can be shared between multiple cores, resulting in a common clock domain across the device. The receiver clocks cannot be shared. Each core is provided with its own local version of `gmii_rx_clk` from the connected external PHY device as shown in the following figure. The following figure illustrates three cores. However, more can be added using the same principle. The receiver clock, which cannot be shared, is unique for every instance of the core.

The upper core instance has been generated with the Shared Logic option enabled, and for the GMII this instantiates an IDELAYCTRL as illustrated. The other core instances have been generated without the Shared Logic option enabled (because only a single instance of an IDELAYCTRL is usually required per design).

The following figure illustrates three cores. However, more can be added using the same principle. This is done by instantiating further cores without the Shared Logic option and sharing `gtx_clk` across all instantiations. The receiver clock, which cannot be shared, is unique for every instance of the core.

Figure 89: Clock Resource Sharing for 1 Gbps GMII



X25644-080521

RGMII

The logic required to implement the RGMII transmitter logic is shown in [Figure 90: 1 Gbps RGMII Transmitter and Clock Logic](#). The `gtx_clk` is a user-supplied 125 MHz reference clock source which is placed onto global clock routing to provide the clock for all transmitter logic, both within the core and for the user-side logic which connects to the transmitter AXI4-Stream interface of the core.

[Figure 90: 1 Gbps RGMII Transmitter and Clock Logic](#) shows how to use the physical transmitter interface of the core to create an external RGMII. The signal names and logic shown in this figure exactly match those delivered with the core. [Figure 90: 1 Gbps RGMII Transmitter and Clock Logic](#) shows that the output transmitter signals are registered in a device bit slice using output DDR registers, before driving them to the device pads.

The voltage standard used depends on the type of I/O used: HR I/O supports RGMII at 2.5 V or lower and HP I/O only supports 1.8 V or lower. Despite this being the defined RGMII voltage, most PHYs require 2.5 V and therefore an external voltage converter is required to interoperate with any multi-standard PHY when using HP I/O. The logic required to forward the transmitter clock is also shown. This logic uses a bit slice output Double-Data-Rate (DDR) register so that the clock signal produced incurs exactly the same delay as the data and control signals. However, the clock signal is then routed through an output delay element (ODELAY cascaded with IDELAY) before connecting to the device pad. The result of this is to create a 2 ns delay, which places the `rgmii_txc` forwarded clock in the center of the data valid window for forwarded RGMII data and control signals.



CAUTION! *TXC cannot be placed at BITSlice0 of the lower nibble. This is because TXC is routed through a delay chain consisting of ODELAY cascaded with IDELAY. Given that, cascading always happens in a downward direction, until the byte boundary. TXC placed at BITSlice0 of the lower nibble cannot be cascaded with IDELAY.*

RX Data/Control pins or any input pin which needs to be routed through IDELAY cannot be placed at BITSlice below TXC. This is because of the cascaded delay chain on TXC which occupies the IDELAY of the lower BITSlice.

For further information, see *UltraScale Architecture Clocking Resources User Guide* ([UG572](#)).

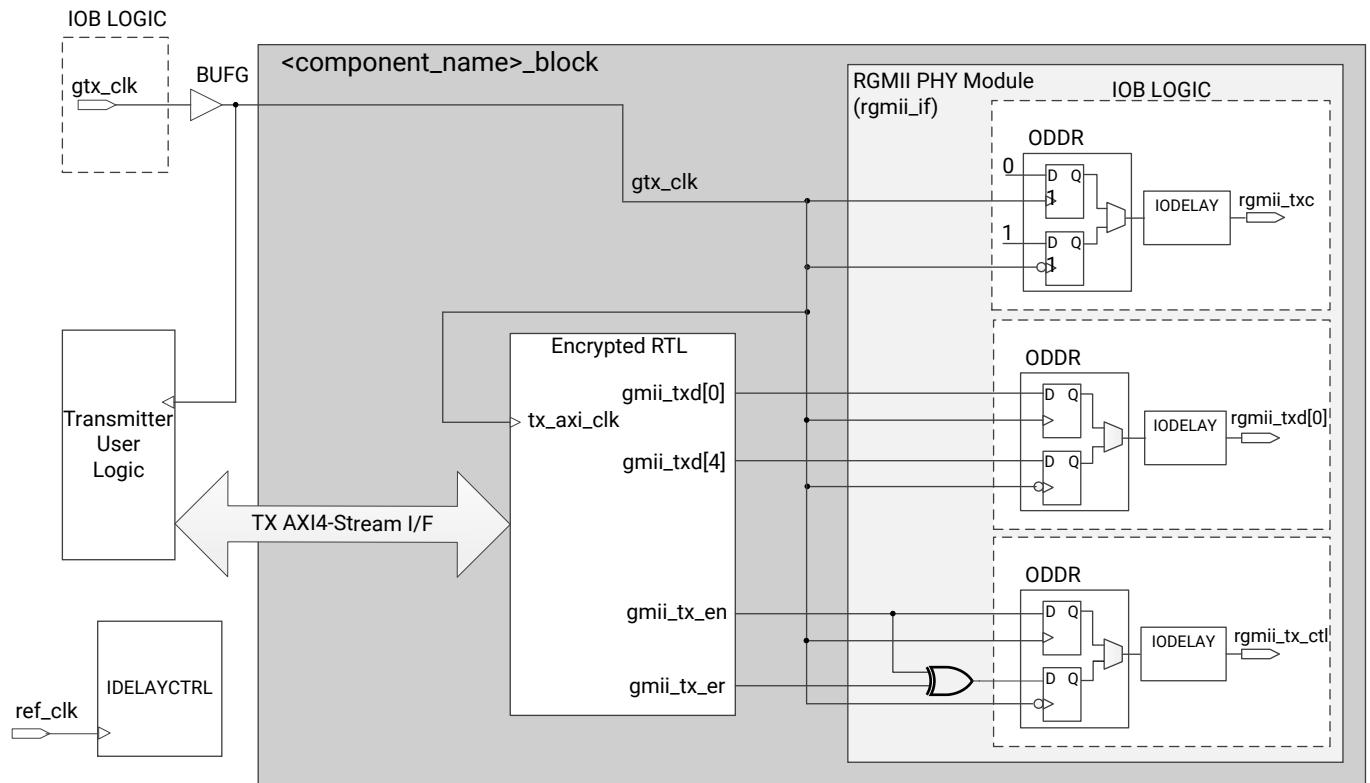


CAUTION! *For UltraScale+ devices, RGMII interface logic uses only the IDELAY/ODELAY components to provide skew between clock and data lines. This requirement restricts you from placing the RGMII interface pins in HD I/O banks as these banks do not have any IDELAY/ODELAY elements.*

IDELAYCTRL

The following figure shows that IODELAY elements are used by the transmitter logic. An IDELAYCTRL module must therefore be instantiated in the design. For the RGMII, this is included in the `<component_name>_support` level, included in the core, if the Shared Logic option is selected.

Figure 90: 1 Gbps RGMII Transmitter and Clock Logic



Receiver Logic

In this implementation, the input clock is used for sampling the RGMII RX signal at the device IOBs. This creates placement constraints: a clock capable input pin must be selected, and all other input RGMII RX signals must be placed in the respective byte group. See the *UltraScale Architecture Clocking Resources User Guide (UG572)* for more information.

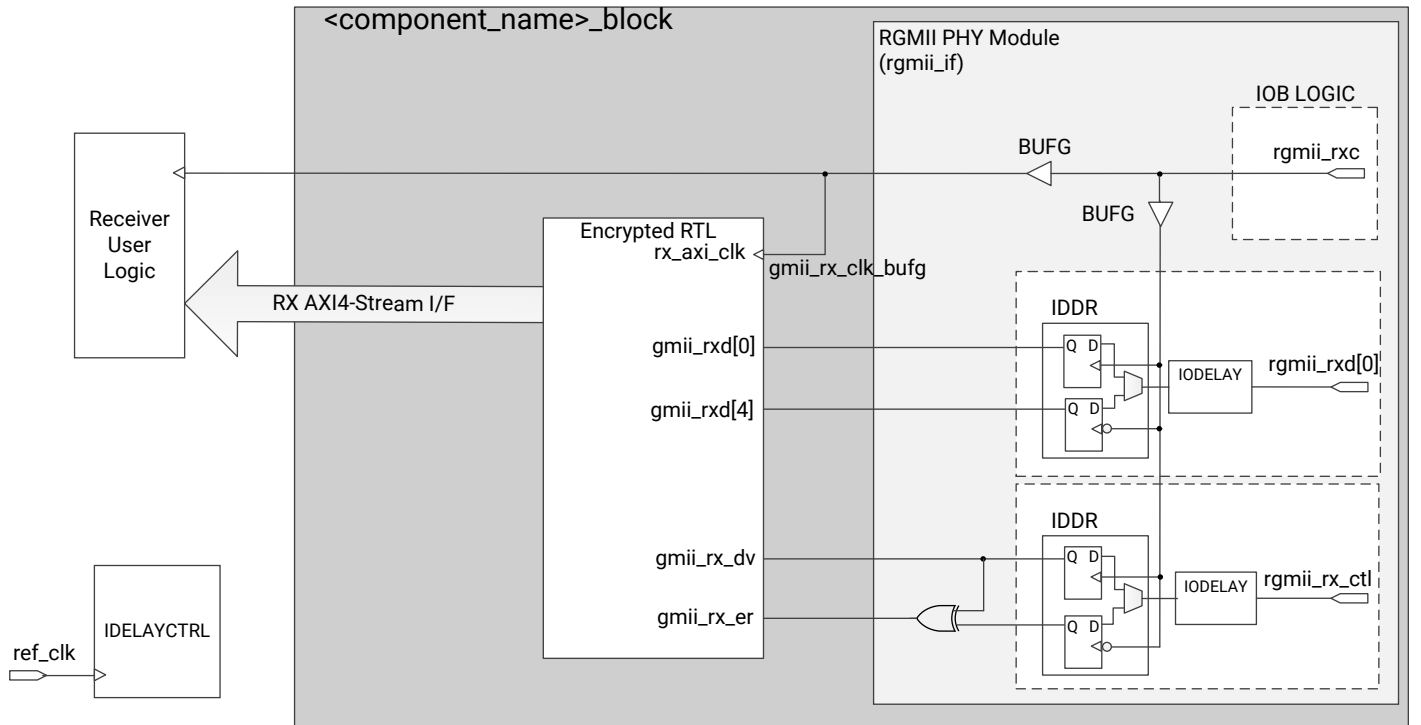
[Figure 91: 1 Gbps RGMII Receiver and Clock Logic](#) shows that the input clock is passed through two parallel global clock routes using BUFG components. One global clock route is used to clock the receiver IOB logic and the other global clock route is for clocking user-side logic which connects to the receiver AXI4-Stream interface of the core. The reason for providing two parallel clock paths is to help Vivado to close timing on the input paths.

It has been observed that, for some devices, Vivado might not succeed in closing timing on the input paths if the input clock net is heavily loaded. Splitting the single clock route into two parallel routes reduces the load on the clock nets and thus helps in closing timing. However, you can modify the clocking structure to use only one global clock path if you can meet the input timing for your selected devices. To perform this, edit the core instance unencrypted HDL file, `<component_name_rgmii_v2_0_if>` present in the core instance `synth/implementation` directory.

IDELAYCTRL

The following figure shows that IODELAY elements are used by the receiver logic. An IDELAYCTRL module must therefore be instantiated in the design. For the RGMII, this is included in the `<component_name>_support` level, present in the core, if the Shared Logic option is selected.

Figure 91: 1 Gbps RGMII Receiver and Clock Logic



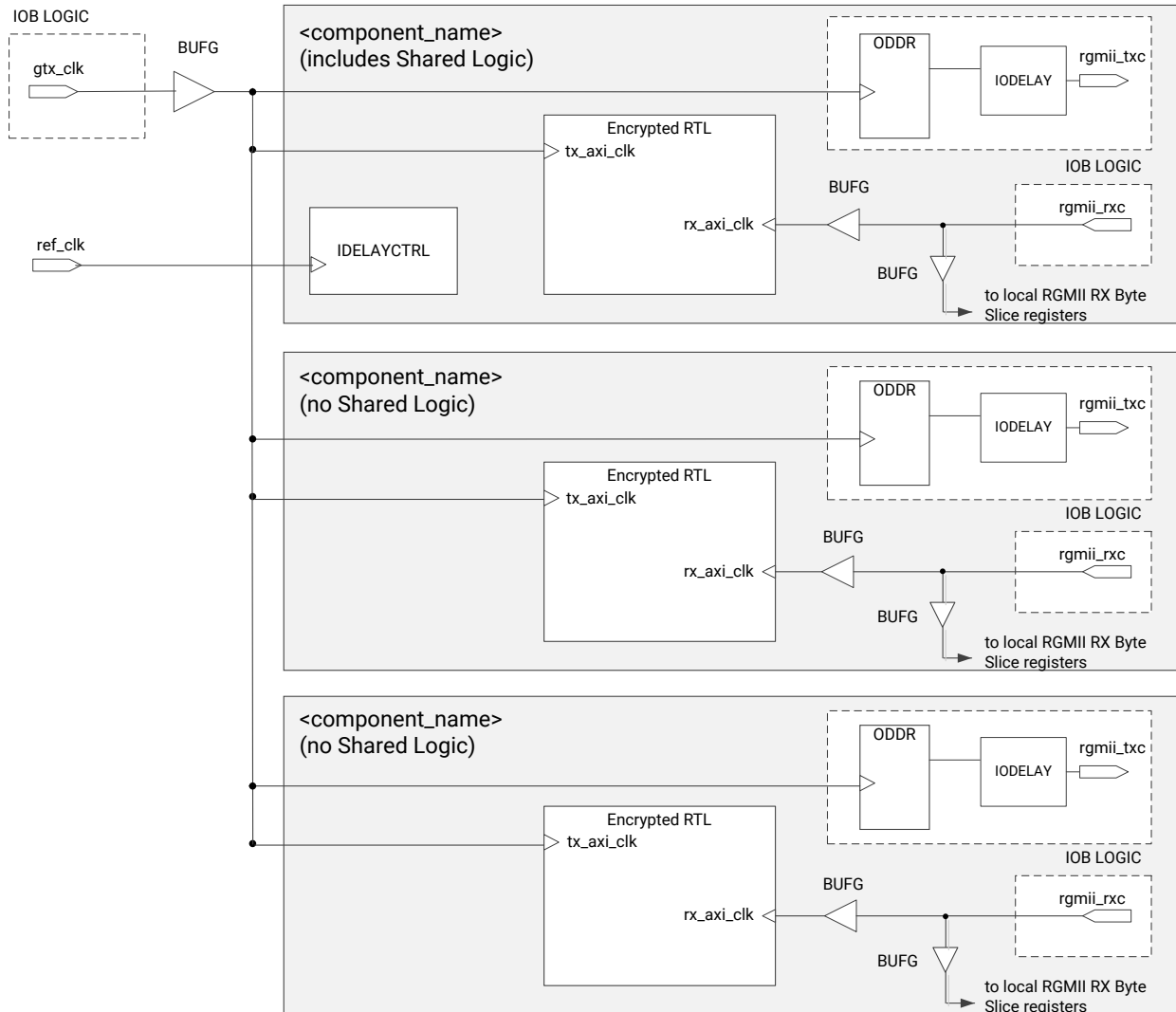
Clock Sharing across Multiple Cores with RGMII for 1 Gbps Operation

The following figure illustrates clock resource sharing across multiple instantiations of the core when using RGMII at 1 Gbps. For all instantiations, `gtx_clk` can be shared between multiple cores, resulting in a common clock domain across the device. The receiver clocks cannot be shared. Each core is provided with its own local version of `rgmii_rxc` from the connected external PHY device as shown in the following figure .

In this figure, the upper core instance has been generated with the Shared Logic option enabled. In both cases this includes an IDELAYCTRL as illustrated. The other core instances have been generated without the Shared Logic option enabled.

The following figure illustrates three cores. However, more can be added using the same principle. This is done by instantiating further cores and sharing `gtx_clk` across all instantiations. The receiver clock, which cannot be shared, is unique for every instance of the core.

Figure 92: Clock Resource Sharing for 1 Gbps RGMII



X25647-080521

Tri-Speed Ethernet MAC Core Interfaces

The HDL example design supplied with the core for tri-speed (10 Mbps, 100 Mbps, and 1 Gbps) operation provides either a GMII or RGMII interface. These are typically used to connect the MAC to an external PHY device.

The MII defined in IEEE 802.3-2008 specification clause 22, is a parallel interface that connects a 10 Mbps and/or 100 Mbps capable MAC to the physical sublayers. The GMII defined in IEEE 802.3-2008 specification, clause 35, is an extension of the MII and is used to connect a 1 Gbps capable MAC to the physical sublayers. MII can be considered a subset of GMII, and as a result, GMII/MII can carry Ethernet traffic at 10 Mbps, 100 Mbps and 1 Gbps.

The voltage standard used depends on the type of I/O used: HR I/O supports GMII at 3.3 V or lower and HP I/O only supports 1.8 V or lower. Therefore, an external voltage converter is required to interoperate with any multi-standard PHY for GMII.

The RGMII is an alternative to the GMII/MII. RGMII can carry Ethernet traffic at 10 Mbps, 100 Mbps, and 1000 Mbps and achieves a 50% reduction in the pin count compared with GMII. This is achieved with the use of double-data-rate (DDR) flip-flops. RGMII is therefore often favored over GMII by PCB designers. A further advantage of the RGMII implementation is that, unlike GMII/MII, clock resources for the transmitter can be shared across multiple core instances. This results in significant clock resource savings when implementing multiple cores in a design.

The voltage standard used depends on the type of I/O used: HR I/O supports RGMII at 2.5 V or lower and HP I/O only supports 1.8 V or lower. Despite this, being the defined RGMII voltage most PHYs require 2.5 V and therefore an external voltage converter is required to interoperate with any multi-standard PHY for RGMII.

GMII Transmitter Interface

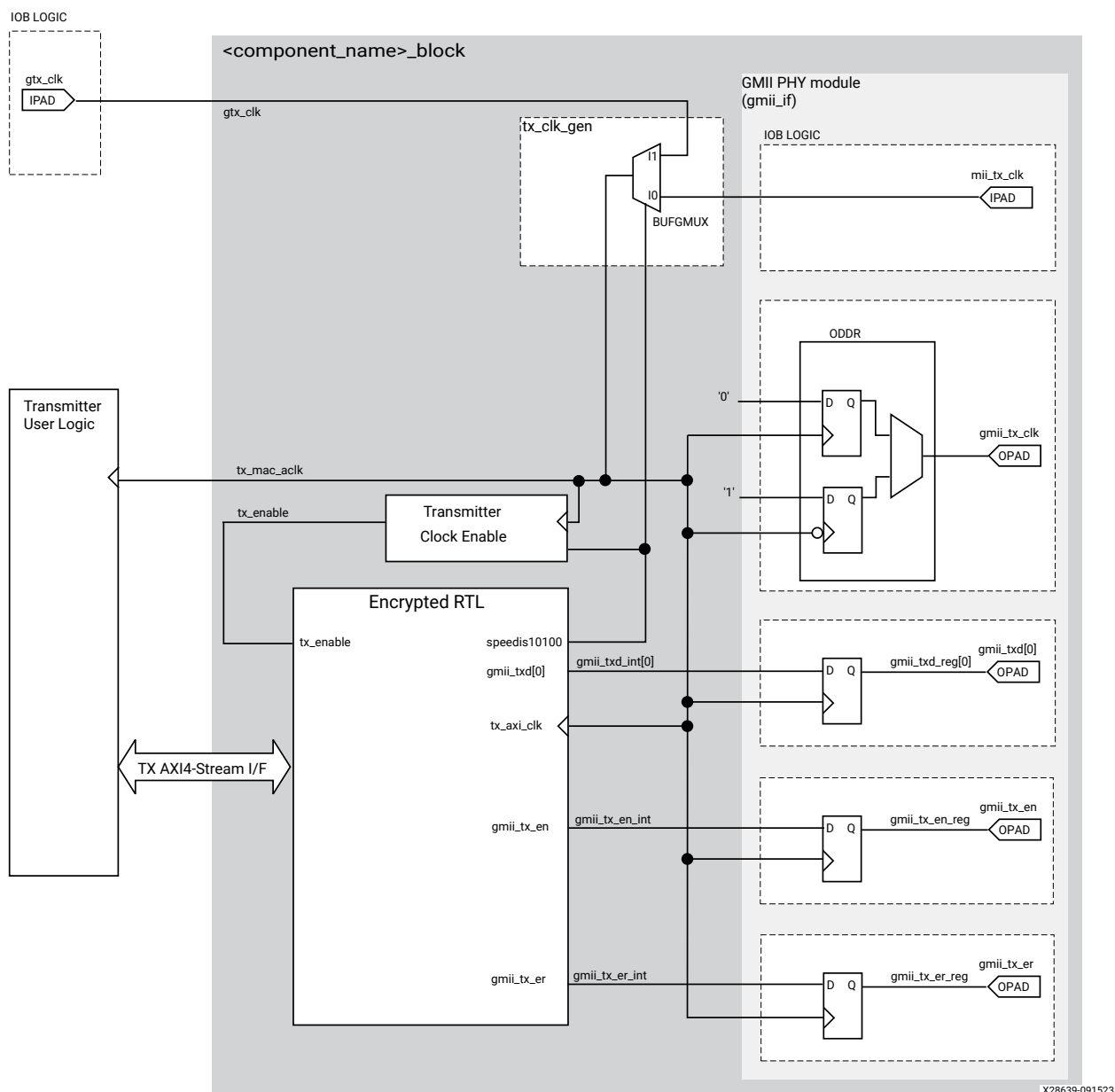
The logic required to implement the GMII transmitter logic is illustrated in the following figure. The `gtx_clk` is a user-supplied 125 MHz reference clock source for use at 1 Gbps. `mii_tx_clk` is sourced by the external PHY device for use at 10 Mbps and 100 Mbps speeds. Consequently, a global clock multiplexer, a BUFGMUX, is used to switch the clock source depending on the operating speed. The output from this BUFGMUX provides the transmitter clock for the core and user logic as illustrated in the following figure.

Closely linked to the clock logic is the use of the `tx_enable` clock enable derivation. This must be provided to the Ethernet MAC core level. All user logic uses the AXI4-Stream interface handshaking to throttle the data to allow for the differing data widths between the 4-bit MII and the cores 8-bit user datapath.

The following figure also illustrates how to use the physical transmitter interface of the core to create an external GMII. The signal names and logic shown in this figure exactly match those delivered with the core for an UltraScale architecture-based device when the GMII is selected.

As shown in the following figure , the output transmitter signals are registered in byte slice before driving them to the device pads. The logic required to forward the transmitter clock for 1 Gbps operation is also shown. This logic uses a byte slice output Double-Data-Rate (DDR) register so that the clock signal produced incurs exactly the same delay as the data and control signals. This clock signal, `gmii_tx_clk` , is inverted with respect to `gtx_clk` so that the rising edge of `gmii_tx_clk` occurs in the center of the data valid window, therefore maximizing setup and hold times across the interface.

Figure 93: Tri-Speed GMII Transmitter and Clock Logic



X28639-091523

GMII Receive Interface

In this implementation, the input clock is used for sampling the GMII RX signal at the device IOBs. This creates placement constraints: a clock capable input pin must be selected, and all other input GMII RX signals must be placed in the respective byte group. See the *UltraScale Architecture Clocking Resources User Guide* ([UG572](#)) for more information.

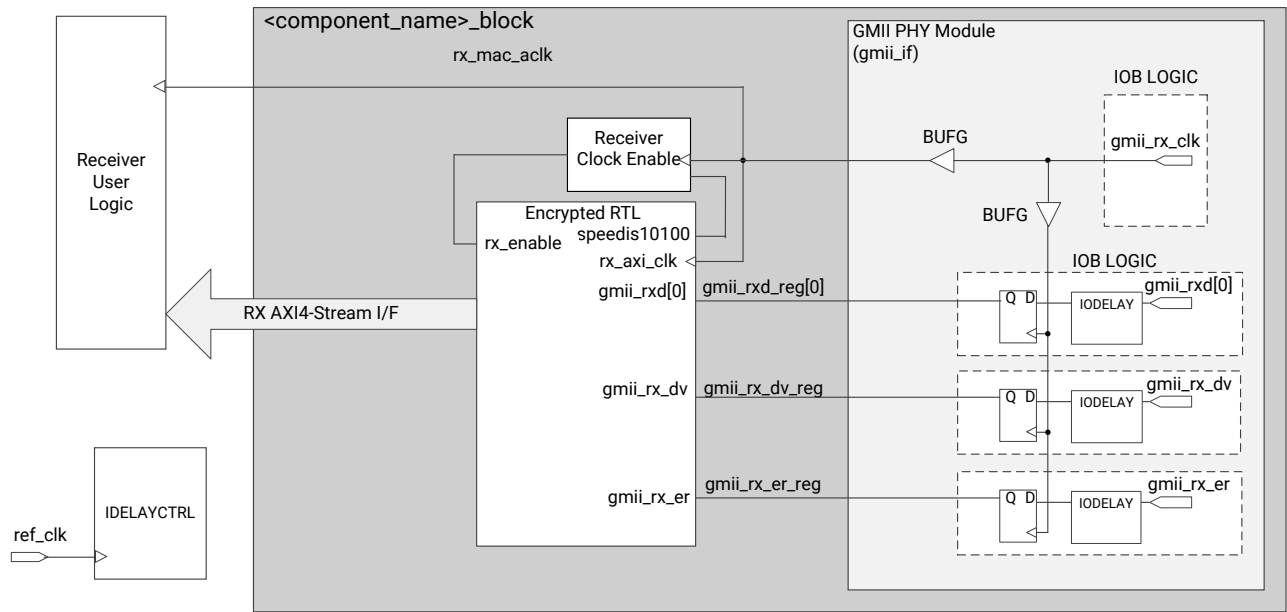
The following figure shows that the input clock is passed through two parallel global clock routes using BUFG components. One global clock route is used to clock the receiver IOB logic and the other global clock route is for clocking user-side logic which connects to the receiver AXI4-Stream interface of the core. The reason for providing two parallel clock paths is to help Vivado to close timing on the input paths.

It has been observed that, for some devices, Vivado might not succeed in closing timing on the input paths if the input clock net is heavily loaded. Splitting the single clock route into two parallel routes reduces the load on the clock nets and thus helps in closing timing. However, you can modify the clocking structure to use only one global clock path if you can meet the input timing for your selected devices. To perform this, edit the core instance unencrypted HDL file, `<component_name_gmii_if>` present in the core instance synth/implementation directory.

The IODELAY elements can be adjusted to fine-tune the setup and hold times at the GMII IOB input flip-flops. The delay is applied to the IODELAY element using constraints in the XDC and these can be edited if desired. See [Constraining the Core](#).

Closely linked to the clock logic is the use of the `rx_enable` clock enable derivation. This must be provided to the Ethernet MAC core level. All user logic uses the AXI4-Stream interface handshaking to throttle the data to allow for the differing data widths between the 4-bit MII and the core 8-bit user datapath.

Figure 94: Tri-Speed GMII Receiver and Clock Logic



X25649-080521

IDELAYCTRL

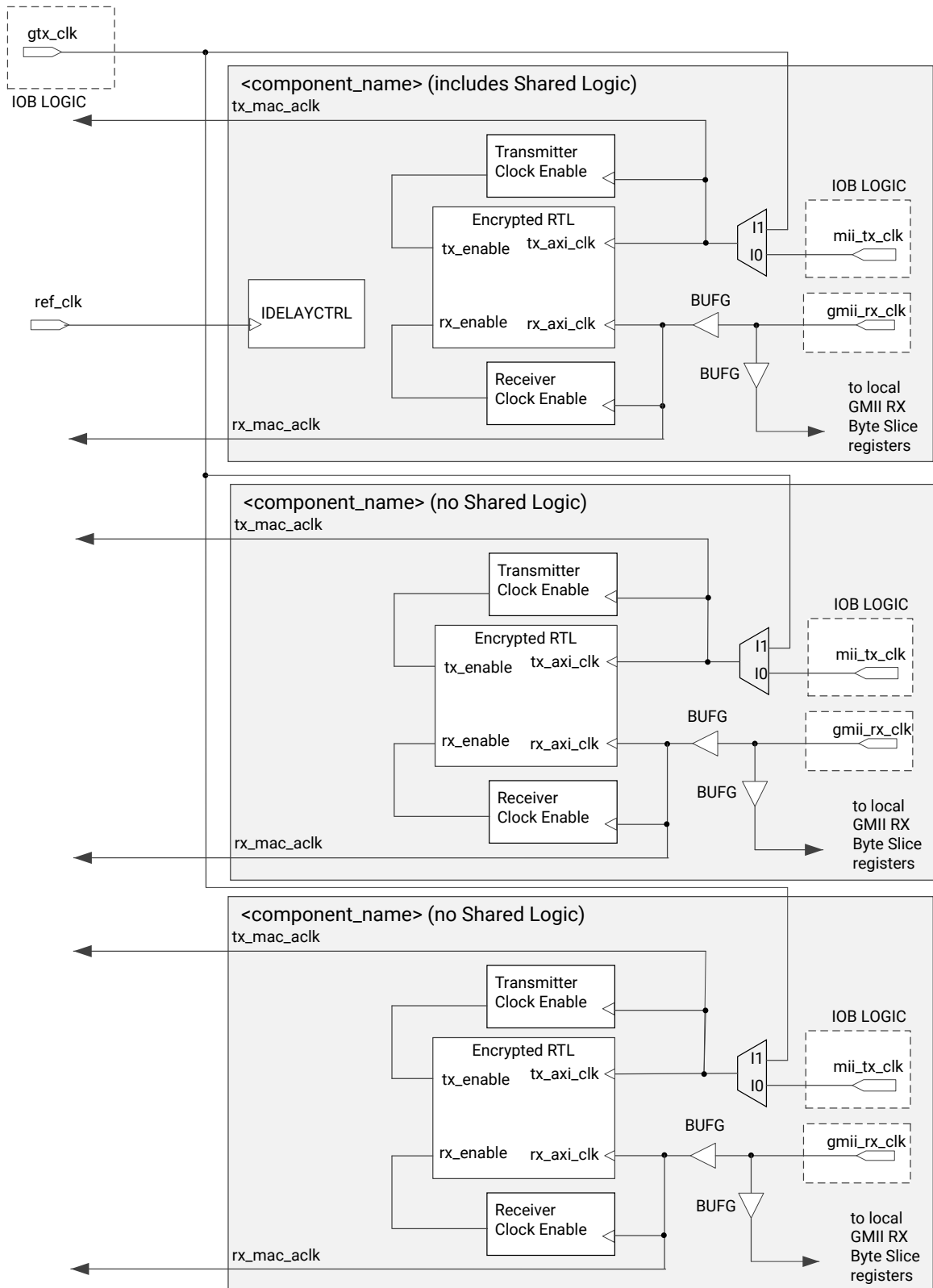
The previous figure shows that the IODELAY elements are used by the receiver logic. An IDELAYCTRL module must therefore be instantiated in the design. For the GMII, this is included in the `<component_name>-support` level, present in the core, if the Shared Logic option is selected.

Clock Sharing across Multiple Cores with GMII for Tri-Speed Operation

Because both `mii_tx_clk` and `gmii_rx_clk` are sourced by the external PHY device connected to the GMII/MII, it is not possible to share global transmitter or receiver clock resources across multiple instantiations of the core. Each instance of the core requires its own endpoint clocking resources. RGMII provides a more optimal solution because it does allow transmitter clock resources to be shared. See [RGMII](#) for more information.

The following figure illustrates that the upper core instance has been generated with the Shared Logic option enabled. For the GMII, this instantiates an IDELAYCTRL as illustrated. The other core instances have been generated without the Shared Logic option enabled. The following figure illustrates three cores. However, more can be added using the same principle. This is done by instantiating further cores without the Shared Logic option.

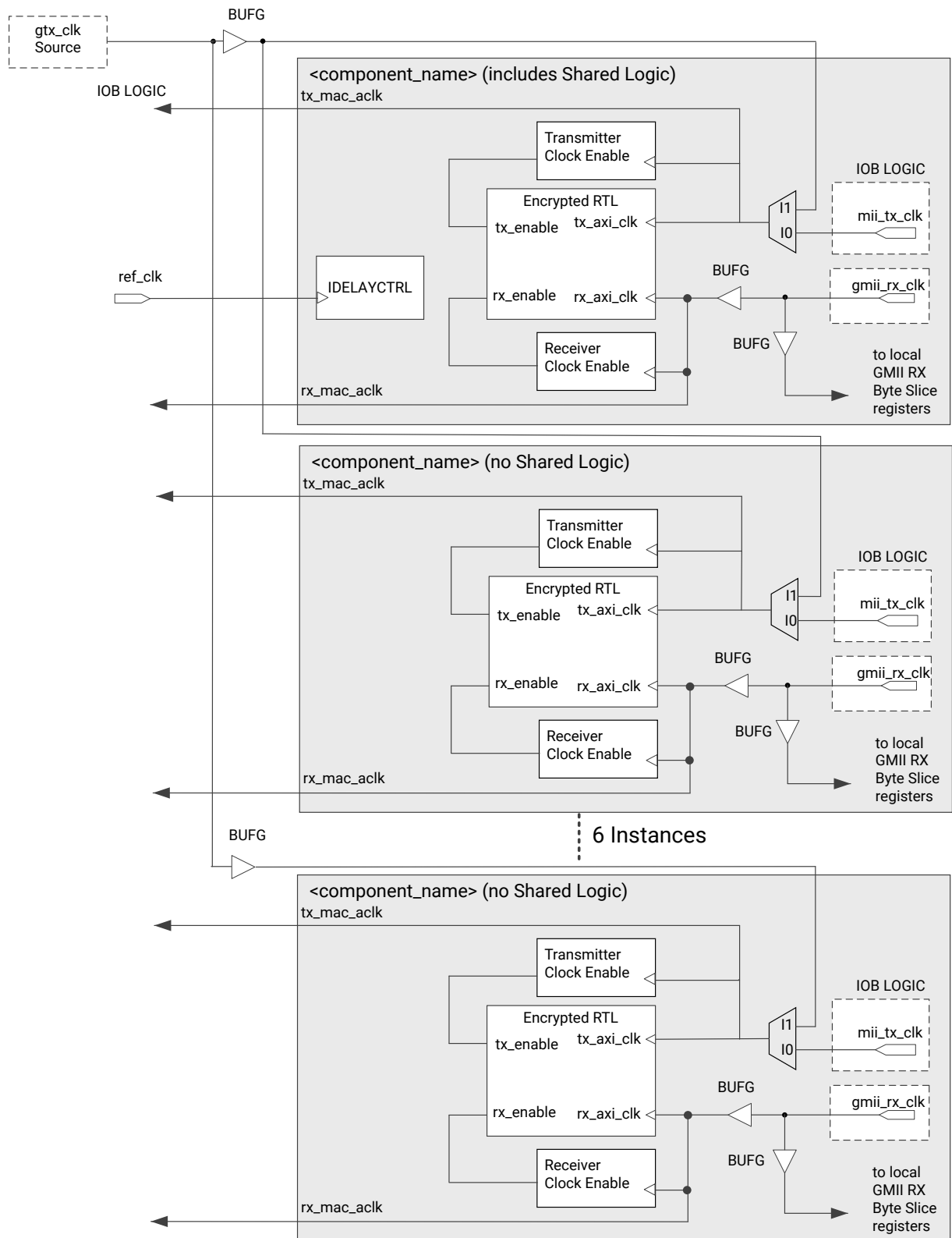
Figure 95: Clock Resource Sharing for Tri-Speed GMII



X25650-080521

If the GTX clock (`gtx_clk`) is sourced from a BUFG, it should be connected to multiple instances of the TEMAC IP, using multiple BUFGs, as shown in the following figure. This is required to limit the number of BUFGs that are cascaded together up to 9.

Figure 96: Clock Resource Sharing for Tri-Speed GMII with GTX Clock BUFG



X25633-080321

RGMII

The logic required to implement the RGMII transmitter logic is illustrated in [Figure 97: Tri-Speed RGMII Transmitter and Clock Logic](#). The `gtx_clk` is a user-supplied 125 MHz reference clock source which is placed onto global clock routing. It provides the clock for all transmitter logic within the core, and for the user-side logic which connects to the transmitter AXI4-Stream interface of the TEMAC.

For RGMII, this global 125 MHz is used to clock transmitter logic at all three Ethernet speeds. The data rate difference between the three speeds is compensated for by the transmitter clock enable logic (the `enable_gen` module from the example design describes the required logic). The derived `tx_enable` signal must be supplied to the Ethernet MAC core level. All user logic uses the AXI4-Stream interfaces built in handshaking to throttle the data appropriately, under control of the Ethernet MAC core level. At all speeds the MAC expects the user logic to supply/accept new data after each validated clock cycle. The generated `tx_enable` signal is always High at 1 Gbps, High for one in ten cycles at 100 Mbps, and High for one in a hundred cycles at 10 Mbps. The advantage of this approach is that it allows common transmitter global clocks to be shared across any number of instantiated cores.

[Figure 97: Tri-Speed RGMII Transmitter and Clock Logic](#) illustrates how to use the physical transmitter interface of the core to create an external RGMII. The signal names and logic shown in this figure exactly match those delivered with the core. [Figure 97: Tri-Speed RGMII Transmitter and Clock Logic](#) shows that the output transmitter signals are registered in byte slice, using DDR registers, before driving them to the device pads.

The logic required to forward the transmitter clock is also shown. This logic uses a byte slice Output Double-Data-Rate (DDR) register so that the clock signal produced incurs exactly the same delay as the data and control signals. However, the clock signal is then routed through an output delay element (ODELAY cascaded with IDELAY) before connecting to the device pad. The result of this is to create a 2 ns delay, which places the `rgmii_txc` forwarded clock in the center of the data valid window for forwarded RGMII data and control signals when operating at 1 Gbps Ethernet speed. At 10 Mbps and 100 Mbps speeds, the `enable_gen` module toggles the DDR input signals at the required frequency so that the forwarded `rgmii_txc` clock is always of the correct frequency for the forwarded data.



CAUTION! *TXC cannot be placed at BITSlice0 of the lower nibble. This is because TXC is routed through a delay chain consisting of ODELAY cascaded with IDELAY. Given that, cascading always happens in a downward direction, until the byte boundary. TXC placed at BITSlice0 of the lower nibble cannot be cascaded with IDELAY.*

RX Data/Control pins or any input pin which needs to be routed through IDELAY cannot be placed at BITSlice below TXC. This is because of the cascaded delay chain on TXC which occupies the IDELAY of the lower BITSlice.

For further information, see *UltraScale Architecture Clocking Resources User Guide* ([UG572](#)).

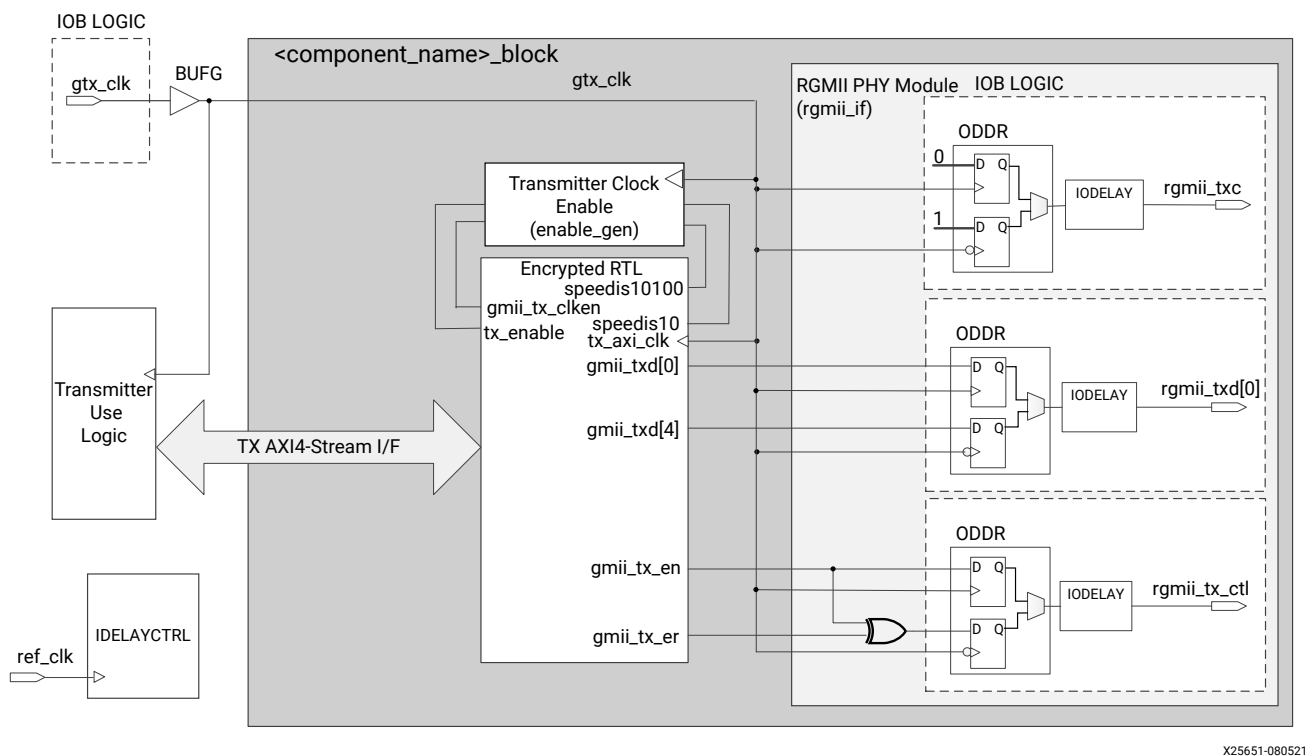


CAUTION! For UltraScale+ devices, RGMII interface logic uses only the IDELAY/ODELAY components to provide skew between clock and data lines. This requirement restricts you from placing the RGMII interface pins in HD I/O banks as these banks do not have any IDELAY/ODELAY elements.

IDELAYCTRL

The following figure shows that the IDELAY elements are used by the transmitter logic. An IDELAYCTRL module must therefore be instantiated in the design. For the RGMII, this is included in the `<component_name>_support` level included in the core, if the Shared Logic option is selected.

Figure 97: Tri-Speed RGMII Transmitter and Clock Logic



X25651-080521

Receiver Logic

In this implementation, the input clock is used for sampling the RGMII RX signal at the device IOBs. This creates placement constraints: a clock capable input pin must be selected, and all other input RGMII RX signals must be placed in the respective byte group. See the *UltraScale Architecture Clocking Resources User Guide* (UG572) for more information.

The following figure shows that the input clock is passed through two parallel global clock routes using BUFG components. One global clock route is used to clock the receiver IOB logic and the other global clock route is for clocking user-side logic which connects to the receiver AXI4-Stream interface of the core. The reason for providing two parallel clock paths is to help Vivado to close timing on the input paths.

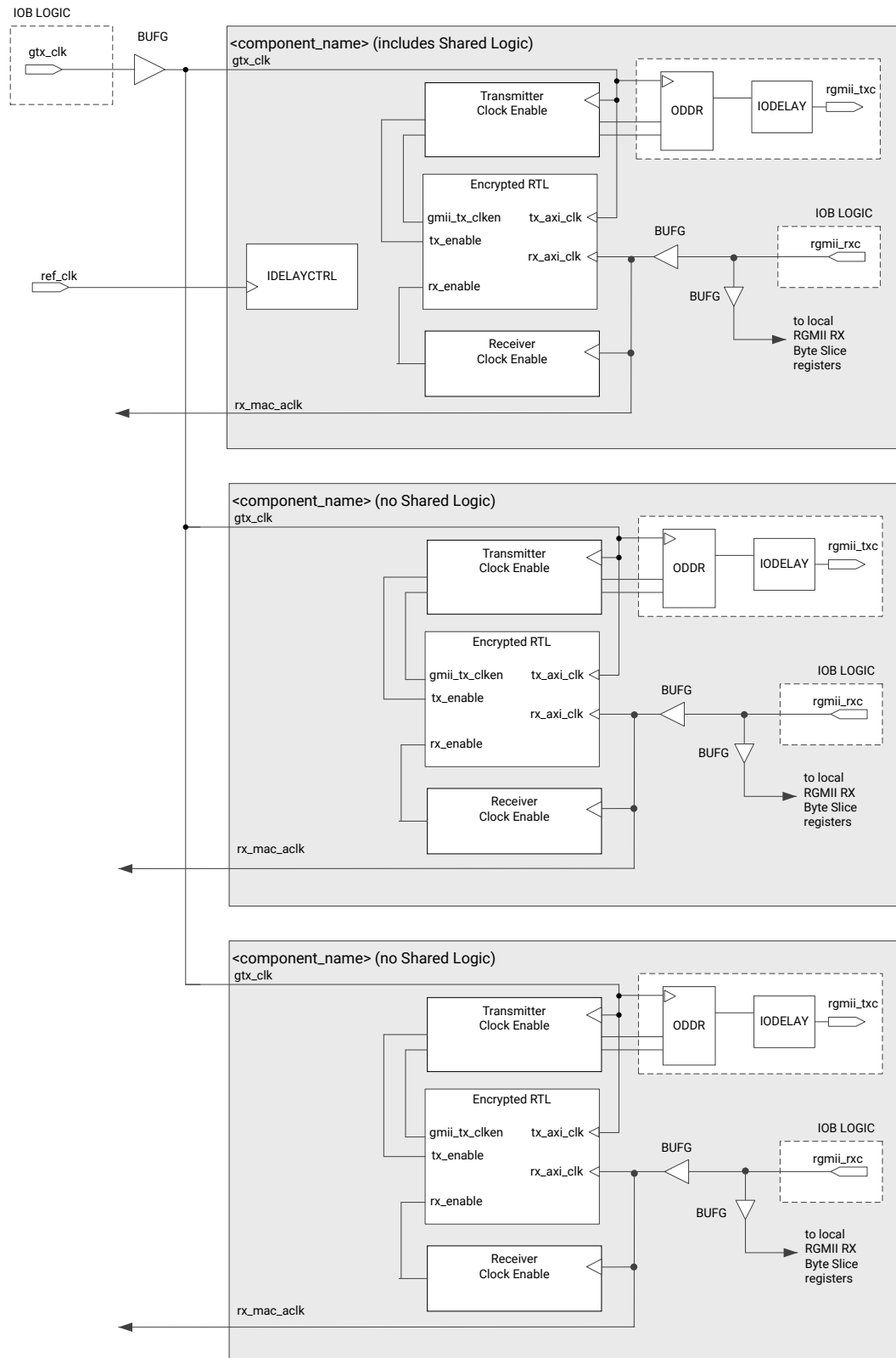
Clock Sharing across Multiple Cores with RGMII for Tri-Speed Operation

The following figure illustrates clock resource sharing across multiple instantiations of the core. For all instantiations, `gtx_clk`, where present, can be shared between multiple cores, resulting in a common clock domain across the device. The receiver clocks cannot be shared. Each core is provided with its own local version of `rgmii_rxc` from the connected external PHY device as shown.

In this figure, the upper core instance has been generated with the Shared Logic option enabled. In both cases, this includes an IDELAYCTRL as illustrated. The other core instances have been generated without the Shared Logic option enabled.

The following figure shows three cores. However, more can be added using the same principle.

Figure 99: Clock Resource Sharing for Tri-Speed RGMII



X25653-080521

Interfacing to Other AMD Ethernet Cores

Ethernet 1G/2.5G PCS/PMA or SGMII Core

The Ethernet MAC core can be integrated into a single device with the Ethernet 1G/2.5G PCS/PMA or SGMII core to provide either:

- A MAC with an SGMII interface to an external PHY device. SGMII can support either tri-speed (10/100/1000 Mbps) designs or 1 Gbps designs.
- A 1 Gbps or 2.5 Gbps Ethernet MAC core with 1G/2.5G PCS/PMA sublayer functionality: this is a 1 Gbps PHY standard which is most commonly used for a fiber optic medium.

For more details on the AMD Ethernet 1G/2.5G PCS/PMA or SGMII core, see the [product web page](#).

The *1G/2.5G Ethernet PCS/PMA or SGMII LogiCORE IP Product Guide* ([PG047](#)) provides the information required to connect the two cores together in any supported configuration.



CAUTION! When using the PCS/PMA-SGMII (transceiver-based) core for UltraScale family devices, ensure that the signal `resetdone`, which is output from the PCS/PMA-SGMII core, is asserted before using the MDIO interface to access the PCS/PMA-SGMII core.

Design Flow Steps

This section describes customizing and generating the core, constraining the core, and the simulation, synthesis, and implementation steps that are specific to this IP core. More detailed information about the standard AMD Vivado™ design flows and the IP integrator can be found in the following Vivado Design Suite user guides:

- *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* ([UG994](#))
- *Vivado Design Suite User Guide: Designing with IP* ([UG896](#))
- *Vivado Design Suite User Guide: Getting Started* ([UG910](#))
- *Vivado Design Suite User Guide: Logic Simulation* ([UG900](#))

Customizing and Generating the Core

This section includes information about using AMD tools to customize and generate the core in the Vivado Design Suite.

The TEMAC solution which comprises the 10/100/1000 Mbps, 1 Gbps, 2.5 Gbps, and 10/100 Mbps cores, generated through the Vivado Design Suite using the Integrated Design Environment (IDE). This chapter describes the Vivado IDE options used to generate and customize the core.

If you are customizing and generating the core in the IP integrator, see the *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* ([UG994](#)) for detailed information.

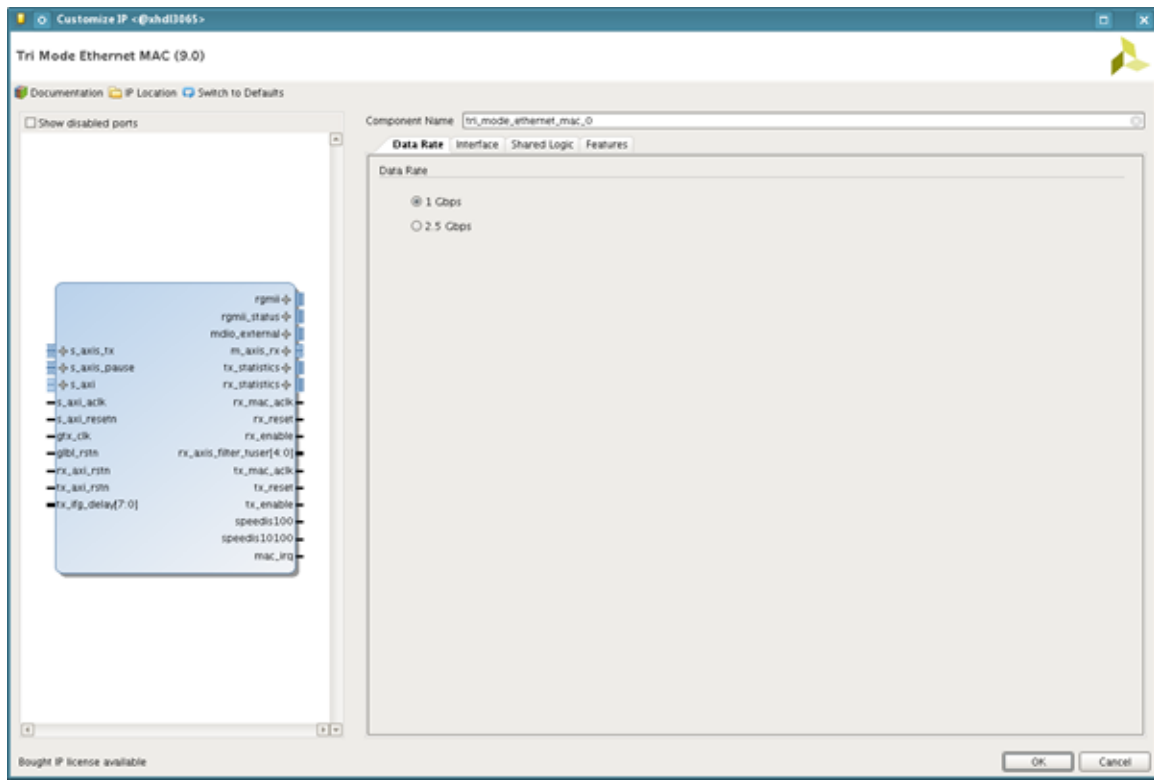
You can customize the IP for use in your design by specifying values for the various parameters associated with the IP core using the following steps:

1. Select the IP from the Vivado IP catalog.
2. Double-click the selected IP or select **Customize IP** command from the toolbar or right-click menu.

For details, see the *Vivado Design Suite User Guide: Designing with IP* ([UG896](#)) and the *Vivado Design Suite User Guide: Getting Started* ([UG910](#)).

- Figures in this chapter are illustrations of the Vivado IDE. This layout might vary from the current version.

Figure 100: Vivado Customize IP IDE – Data Rate Tab



Data Rate

Set the core maximum data rate to the following (see the previous figure):

- **1 Gbps** : Ethernet speeds up to 1 Gbps are supported. All four types of physical interfaces, namely GMII, MII, RGMII, and Internal are available.
- **2.5 Gbps**: Ethernet speed is set to 2.5 Gbps. The physical interface is set to Internal with all other types being disabled.

- **Internal:** The core is generated with no physical interface and is ready for connection to an internal PHY such as the [Ethernet 1G/2.5G PCS/PMA or SGMII LogiCORE](#).

The choice of physical interface determines the content of the example design delivered with the core where the external GMII, MII or RGMII is described in HDL. There is no change in the core HDL between RGMII, GMII, or Internal. If MII is selected, the physical interface datapath is reduced to four bits. The default is to use GMII.

MAC Speed

The TEMAC solution can provide support for 2.5 Gbps and 1 Gbps speed operation, 10 Mbps and 100 Mbps speed operation, and full tri-speed operation (10 Mbps, 100 Mbps, and 1 Gbps speed capability). The choice available for speed support selection is dependent on the chosen physical interface:

- Only Internal mode is available when the MAC data rate is set to 2.5 Gbps.
- If the MAC data rate is set to 1 Gbps and GMII, RGMII, or Internal is selected, then the tri-speed operation and 1 Gbps operation are available for selection.
- If the MAC data rate is set to 1 Gbps and MII is selected, then only 10 Mbps and 100 Mbps operation is available.

Internal Mode Clock Source

Only available when the Physical Interface is set to Internal. This option selects the clock sources for the MAC TX and RX datapaths:

- **User Clk2:** This is the default option and when selected, the MAC TX and RX datapaths are clocked by Userclk2 sourced from the PCS/PMA module.
- **RX User Clk2:** When this option is selected, the MAC TX datapath is clocked by Userclk2 and the RX datapath is clocked by RXOUTCLK. Both are sourced from the PCS/PMA module.

Management Type

In the Interface tab ([Figure 101: Vivado Customize IP IDE – Interface Tab](#)), select the AXI4-Lite option to include the optional Management Interface for the TEMAC configuration (see [Management Interface](#)). If this option is not selected, the core is generated with a replacement configuration vector. If the AXI4-Lite Management Interface is not selected, the AVB option is not available. The default is to have the AXI4-Lite Management Interface selected. These features are included in the Interface tab ([Figure 101: Vivado Customize IP IDE – Interface Tab](#)) and Features tab ([Figure 103: Vivado Customize IP IDE – Features Tab](#)) of the Vivado IDE.

When the Management Interface is enabled, select the **Management Data Input/Output Interface (MDIO)** option to include the optional MDIO interface. If this option is not selected, the generated core does not have the MDIO logic required to manage the objects in the Physical layer.

If MDIO is enabled, select the **Add IO Buffers for MDIO Interface Ports** option to insert I/O buffers for the MDIO interface ports. This creates the bidirectional I/O bus `mdio` and inserts an output buffer for `mdc` . If this option is not selected, the core is generated with `mdio_i` , `mdio_o`, and `mdio_t` ports.

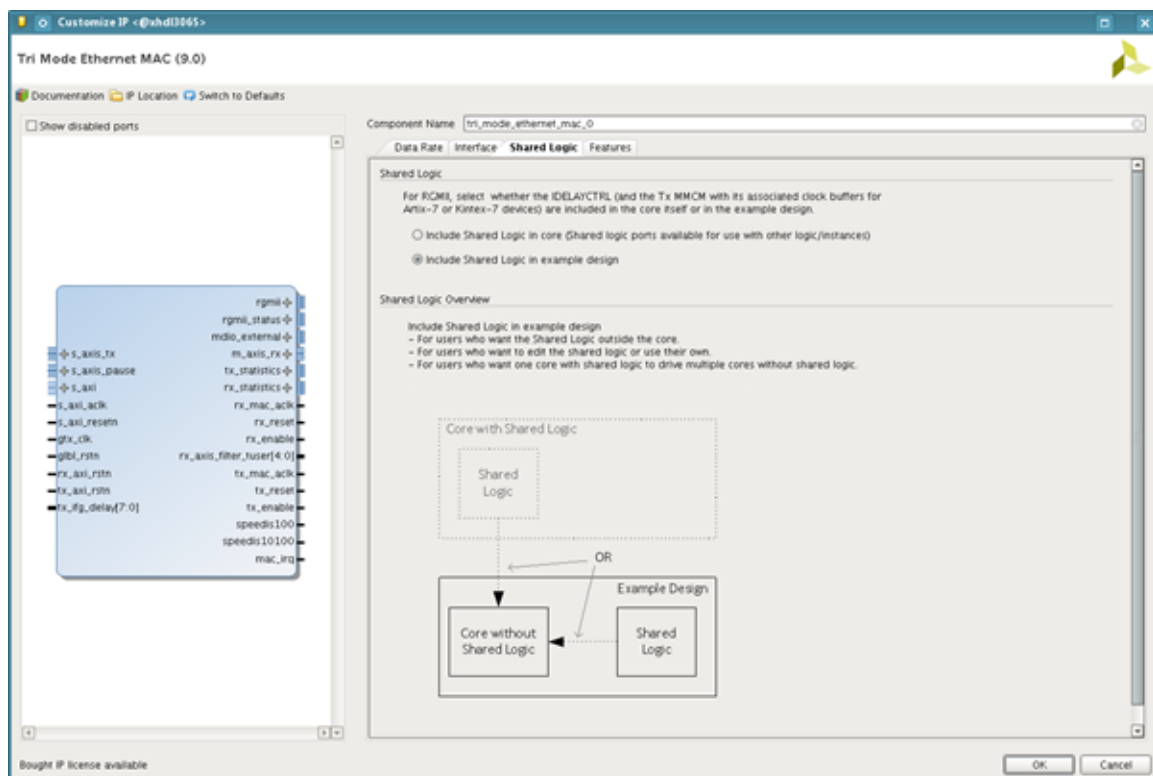
AXI4-Lite Frequency

This specifies the frequency (in MHz) of the AXI4-Lite interface. This is used to provide a default value for synthesis and it is overridden based on connectivity in a system.

Shared Logic

Select whether shared logic (including clocking logic, IDELAYCTRL and resets) is included in the core itself or the example design (see [Shared Logic](#)). The Shared Logic tab is shown in the following figure.

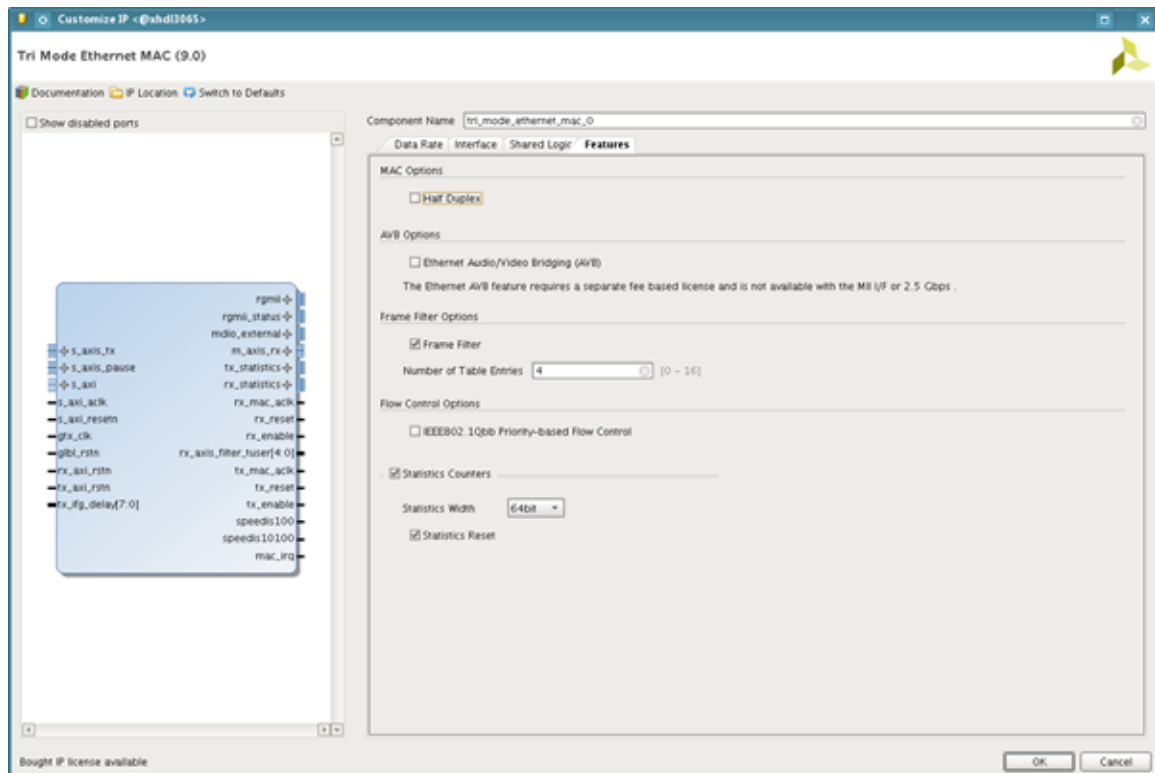
Figure 102: Vivado Customize IP IDE – Shared Logic Tab



MAC Options

The TEMAC solution always provides support for full-duplex Ethernet. However, to provide half-duplex operation, further FPGA logic resources are required. Because many applications require only full-duplex support, the half-duplex logic is therefore optional. When the core is generated with half-duplex logic, full- or half-duplex operation can be selected using TEMAC configuration. The default is not to include half-duplex support. When the data rate is set to 2.5 Gbps or if half-duplex is selected the AVB option is disabled.

Figure 103: Vivado Customize IP IDE – Features Tab



Note: If an MMCM is used on the physical interface receive path to control the clock-to-data relationship, 1 Gbps half-duplex is not supported.

The Features tab is shown in previous figure and includes the following options:

AVB Option

Select the Enable_AVB option if you wish the optional AVB Endpoint front-end logic to be included.

- If the data rate is set to 2.5 Gbps or if half-duplex is selected the AVB option is disabled.
- If the AXI4-Lite management interface is not selected the AVB option is disabled.

If selected, the fee-based Ethernet AVB Endpoint license is required in addition to the Tri-Mode Ethernet MAC license to enable core generation. In the default criteria, the AVB Endpoint is not included.

Frame Filter Options

It is possible to generate the core with a frame filter, which prevents the reception of frames that are not matched by this MAC. This is most commonly used to identify packets which are addressed specifically to this MAC. The default is to use the frame filter.

Number of Table Entries

The frame filter can be generated with a look-up table that holds up to 16 additional valid MAC frame match patterns. You can select an integer between 0 and 16 to define the number of match patterns that are present in the table. The default is to use four table entries.

Enable Priority Flow Control

Select this option to include Priority Flow control support in the core. When included, circuitry to generate PFC frames on transmit and interpret PFC frame on receive is included as well as enhanced XON/XOFF support for IEEE 802.3 pause requests. It is disabled as a default.

Statistics Counters

It is possible to generate the core with built-in statistics counters. The number of counters available is dependent upon the duplex setting of the core with full-duplex requiring 34 counters and half-duplex requiring 41 counters. This option can only be selected when the core is configured with the AXI4-Lite Management Interface. The default is to include the Statistics counters.

Statistics Width

The Statistics counters can be either 32 bits or 64 bits wide. This allows you to control the frequency at which the counters must be polled to avoid information loss due to overflow. The default is to use 64-bit wide counters.

Statistics Reset

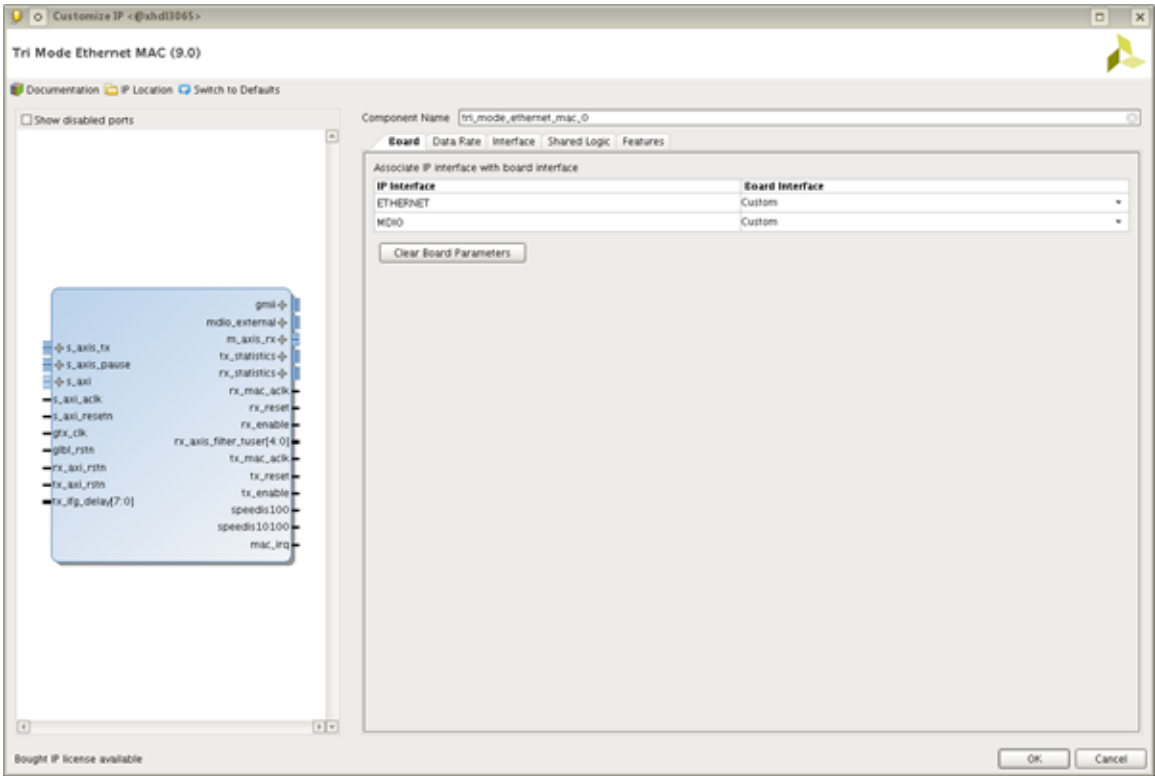
When the Statistics Counters are included, it is possible to include logic to ensure the counters are cleared to zero upon a hardware reset. Without this logic the counter values persist over a reset and are only cleared upon device configuration. The default is to include the counter reset functionality

Board

This tab, shown in the following figure, only appears if a supported board has been pre-selected as the project part.

In the **Associate IP Interface** table, only those interfaces appear that are supported by the selected board. When **Generate Board Based IO Constraints** is selected, the appropriate **Board Interface** field becomes active. **Custom** is the default option, allowing customization of the FPGA pinout. The other options allow selection of the supported board physical interfaces, and if selected, then the correct pinout is assigned automatically. By default, the **Generate Board Based IO Constraints** setting is not enabled.

Figure 104: Vivado Customize IP IDE – Board Tab



IMPORTANT! The 2.5 Gbps data rate option is available only when the Ethernet Board Interface is set to Custom.

User Parameters

The following table shows the relationship between the fields in the Vivado IDE and the User Parameters (which can be viewed in the Tcl Console).

Table 78: Vivado IDE Parameter to User Parameter Relationship

Vivado IDE Parameter/Value ¹	User Parameter/Value ¹	Default Value
component_name	Component_Name	tri_mode_ethernet_mac_v9_0
ETHERNET_BOARD_INTERFACE	ETHERNET_BOARD_INTERFACE	Custom
Custom	Custom	
ETHERNET	ETHERNET	
MDIO_BOARD_INTERFACE	MDIO_BOARD_INTERFACE	Custom
Custom	Custom	
MDIO	MDIO	
Physical_Interface	Physical_Interface	GMII
GMII	GMII	
RGMII	RGMII	
MII	MII	
Internal	Internal	
Data_rate	Data_Rate	1_Gbps
1_Gbps	1_Gbps	
2.5_Gbps	2.5_Gbps	
MAC_Speed	MAC_Speed	Tri_speed
Tri_speed	Tri_speed	
1000_Mbps	1000_Mbps	
10_100_Mbps	10_100_Mbps	
Half_Duplex	Half_Duplex	FALSE
Int_Clk_Src	Int_Clk_Src	User_Clk2
User_Clk2	User_Clk2	
RX_User_Clk2	RX_User_Clk2	
Int_Mode_Type	Int_Mode_Type	BASEX
BASEX	BASEX	
SGMII	SGMII	
Management_Interface	Management_Interface	TRUE
Enable_MDIO	Enable_MDIO	TRUE
Make_MDIO_External	Make_MDIO_External	TRUE
SupportLevel	SupportLevel	0
0	0	
1	1	
Frame_Filter	Frame_Filter	TRUE
Number_of_Table_Entries	Number_of_Table_Entries	4
Enable_AVB	Enable_AVB	FALSE
Enable_1588	Enable_1588	FALSE
Enable_1588_1step	Enable_1588_1step	FALSE
Timer_Format	Timer_Format	Time_of_day
Time_of_day	Time_of_day	

Table 78: Vivado IDE Parameter to User Parameter Relationship (cont'd)

Vivado IDE Parameter/Value ¹	User Parameter/Value ¹	Default Value
Correction_Field_Format	Correction_Field_Format	
Enable_Priority_Flow_Control	Enable_Priority_Flow_Control	FALSE
Statistics_Counters	Statistics_Counters	TRUE
Statistics_Width	Statistics_Width	64bit
64bit	64bit	
32bit	32bit	
Statistics_Reset	Statistics_Reset	TRUE

Notes:

- Parameter values are listed in the table where the Vivado IDE parameter value differs from the user parameter value. Such values are shown in this table as indented below the associated parameter.

Output Generation

For details, see the *Vivado Design Suite User Guide: Designing with IP* (UG896).

The TEMAC solution delivers files into several filegroups. By default, the filegroups necessary for use of the TEMAC or opening the IP Example design are generated when the core is generated. If additional filegroups are required these can be selected using the generate option.

The filegroups generated can be seen in the IP Sources tab of the Sources window where they are listed for each IP in the project.

Constraining the Core

This section contains information about constraining the core in the Vivado Design Suite.

Required Constraints

This chapter defines the constraint requirements of the TEMAC solution. The TEMAC solution is provided with core level XDC files. These provide the constraints for the core which are expected to be applied in all instantiations of the core. These XDC files, named `<compname>.xdc` and `<compname>_clocks.xdc`, can be found in the IP Sources tab of the Sources window in the Synthesis file group.

An example XDC is also provided with the HDL example design to provide the board level constraints. This is specific to the example design and, as such, is only expected to be used as a template for the user design. See [Chapter 6: Example Design](#).

This XDC file, named `<component name>_example_design.xdc`, is found in the IP Sources tab of the Sources window in the Examples file group, in the IP example design project for the core.

The core level XDC file inherits some constraints from the example design XDC file. In any system, it is expected that you would also provide an XDC file to constrain the logic in which the TEMAC solution is instantiated.

Another example design file, `<component name>_user_phytiming.xdc`, is also provided to show how to override default XDC settings provided by the core for setup and hold checks on the selected physical interface.

This is useful in case the interface timing constraints cannot be met. The constraints can be relaxed by adjusting the values in this XDC file which is set to be processed after all other XDC files. This also allows the IODELAY tap delay setting to be adjusted without needing to modify the XDC files provided with the core.

Device, Package, and Speed Grade Selections

The core can be implemented in AMD Kintex™ UltraScale™, AMD Virtex™ 7, AMD Kintex™ 7, and AMD Artix™ 7 devices with these attributes:

- Large enough to accommodate the core
- Contains a sufficient number of IOBs
- Device has a supported speed grade:

Table 79: Supported Speed Grades

Device Family	Speed Grade
Virtex 7	-1 or faster
Kintex 7	-1 or faster
Artix 7	-1 or faster
Kintex UltraScale	-1 or faster
Virtex UltraScale	-1 or faster



IMPORTANT! 2.5 Gbps data rate feature is not available for low-speed Artix 7 devices.

Clock Frequencies

The TEMAC solution has a variable number of clocks with the precise number required being dependent upon the specific parameterization.

As the core targets a specific interface standard (RGMII/GMII or MII) there are associated clock frequency requirements.

Table 80: TEMAC Solution Frequency Requirements

Clock Name	Parameterization	Frequency Requirement (MHz)
gtx_clk	Always present except when core is configured in MII mode and Statistics counters feature is not enabled.	125 (at 1 Gbps) 312.5 (at 2.5 Gbps)
gtx_clk90	RGMI when HRIO used for interface. 90° shifted version of gtx_clk	125
gtx_clk_out	RGMI when HRIO used for interface. Only present when core generated with "Shared logic in core" option.	125
gtx_clk90_out	RGMI when HRIO used for interface. Only present when core generated with "Shared logic in core" option.	125
rx_usr_clk2	Internal mode and Internal Mode Clock Source set to RX User Clk2. Clocks the RX datapath.	125
refclk	RGMI or GMII. Required for the idelayctrl. For UltraScale architecture-based devices the range is 300–1333 MHz.	200-300
mii_tx_clk	GMII or MII. Required for 10/100 Mbps operation	25 (at 100 Mbps) 2.5 (at 10 Mbps)
mii_rx_clk	MII	25 (at 100 Mbps) 2.5 (at 10 Mbps)
gmii_rx_clk	GMII	125 (at 1 Gbps) 25 (at 100 Mbps) 2.5 (at 10 Mbps)
rgmii_rxc	RGMI	125 (at 1 Gbps) 25 (at 100 Mbps) 2.5 (at 10 Mbps)
s_axi_aclk	Management Type set to AXI4-Lite	10-300

I/O Standard and Placement

Among the various interfaces provided, when the TEMAC solution is generated, only the interface to the selected PHY is expected to be propagated to the actual device I/O. As such there are no specific I/O standard/placement requirements on most interfaces.

When the TEMAC is generated with either RGMI/GMII or MII support, the related interfaces and the MDIO interface, if present, are expected to propagate to device I/O and as such there are some limitations which have to be considered.

Depending upon the device family, part and package chosen there are two types of I/O available for use. HP I/O is intended for support of high speed interfaces and as such is limited to 1.8 V support. HP I/O supports both Input and Output Delays components. HR

I/O is intended for interfaces with higher voltage requirements and has a more limited supported frequency range. HR I/O only supports Input Delay components.

Both MII and GMII are 3.3 V standards, with RGMII being a 1.8 V standard. However the majority of PHYs are multi-standard and operate at either 2.5 V or 3.3 V and this is also true for the PHYs selected for AMD development boards. This means that for most applications the physical interfaces are restricted to either using HR I/O, where available, or HP I/O with an external voltage converter to translate between 1.8 V and the minimum level required by the PHY of 2.5 V. For any board design, it is important to identify which type of I/O is available/being used.

For the 1 Gbps interface standards (RGMII and GMII), the received data interface from the PHY can have some placement requirements, depending upon the capture interface used. Across all supported families there are two common capture methods used, these are detailed in [1 Gbps Ethernet MAC Core Interfaces](#) and [Tri-Speed Ethernet MAC Core Interfaces](#) for 7 series devices and [1 Gbps Ethernet MAC Core Interfaces](#) and [Tri-Speed Ethernet MAC Core Interfaces](#) for UltraScale architecture-based devices.

In summary, the receive data sample window is adjusted by either shifting the data using Input delays or by shifting the clock using a PLL/MMCM. When the Data is shifted a BUFIO is used to provide the lowest form of clock routing delay from the input clock to input RX signal sampling at the device IOBs. However, this creates placement constraints. A BUFIO capable clock input pin must be selected, and all other input RGMII/GMII RX signals must be placed in the respective BUFIO region. Refer the relevant family user guide. This requirement does not exist if the PLL/MMCM method is used.

IDELAYCTRL

7 Series Devices

When the core is generated with either an RGMII or GMII interface, the core uses IODELAY elements to align the clock and data of the external ports. An IDELAYCTRL must be always used with IODELAYS.

- If the core is generated using the [Shared Logic](#) option, this IDELAYCTRL is present in the core.
- If the core is generated without the Shared Logic option, an IDELAYCTRL must be instantiated somewhere in the user design. In this case, XDC constraints must be provided to associate the IODELAYS in the core to the instantiated IDELAYCTRL. This is achieved using the “set_property IODELAY_GROUP <name_string>” constraint (see the *Vivado Design Suite User Guide: Using Constraints (UG903)*). The core uses the `<name_string>tri_mode_ethernet_mac_iodelay_grp` for its IODELAY elements for both GMII and RGMII permutations. IDELAYCTRL is always instantiated in the core for UltraScale devices when needed. The XDC constraint associates the IODELAYS with the IDELAYCTRL.

UltraScale Devices

For UltraScale architecture-based devices, IDELAYCTRL is always present at the `<component_name>_block` level.

Simulation

For comprehensive information about Vivado simulation components, as well as information about using supported third-party tools, see the *Vivado Design Suite User Guide: Logic Simulation* ([UG900](#)).

Simulation of the TEMAC solution at the core level is not supported without the addition of a test bench (not supplied). Simulation of the example design is supported.



IMPORTANT! For cores targeting 7 series or Zynq 7000 SoC devices, UNIFAST libraries are not supported. AMD IP is tested and qualified with UNISIM libraries only.

Synthesis and Implementation

For details about synthesis and implementation, see the *Vivado Design Suite User Guide: Designing with IP* ([UG896](#)).

Includes all synthesis sources required by the core. For the TEMAC solution, this is a mix of both encrypted and unencrypted source. Only the unencrypted sources can be made visible in the Sources Hierarchy window by right clicking on the core and selecting the “IP Hierarchy” menu item to select the “Show All IP Hierarchy” option.

Optionally, unencrypted HDL files can be edited by using the **Managed IP** property option. For detailed information, see *Vivado Design Suite User Guide: Designing with IP* ([UG896](#)).

Example Design

This section provides detailed information about the example design, including a description of the file groups, contents of the example HDL wrappers, and the operation of the demonstration test bench.

The example design, under certain core configurations, is intended to directly target key AMD Demonstration Families. The currently supported boards are the AMD Kintex™ 7 ([KC705 Board](#)) and AMD Artix™ 7 ([AC701 Board](#)) FPGAs.

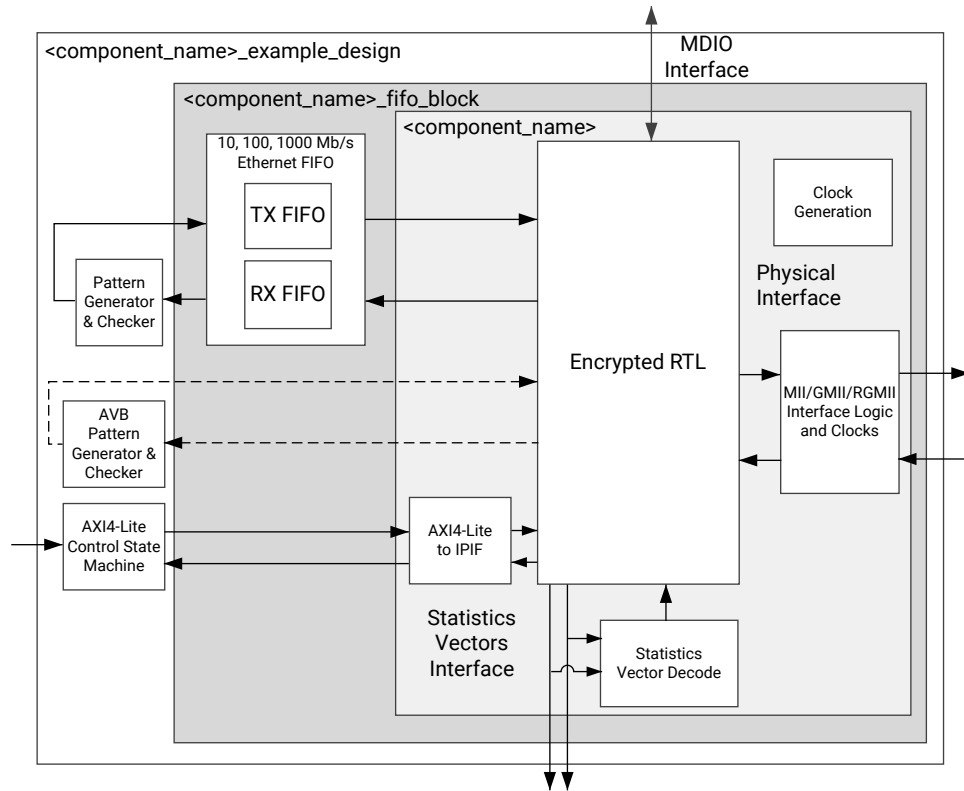
The example design includes a basic state machine which brings up the external PHY and MAC to allow basic frame transfer through the AXI4-Lite interface.

A Simple Frame Generator and Frame Checker are also included which can be used to turn a particular board into a packet generator with any received data optionally being checked. If the TEMAC is generated with the Optional AVB Endpoint another frame generator and frame checker are included to exercise the additional AV datapath.

Loopback functionality is provided as either MAC RX to TX loopback, where the loopback logic becomes the packet source in place of the packet generator, or PHY TX to RX loopback, with the loopback replacing the demonstration test bench stimulus and checker. At present, the loopback logic is not available for the 2.5 Gbps rates. Basic control of the state machine, allowing MAC speed change is achieved using push buttons and DIP switches on the board. See the board specific sections in [Targeting the Example Design to a Board](#).

The following figure illustrates the top-level design for the TEMAC solution example design.

Figure 105: HDL Example Design



X25621-080321

The HDL example design contains the following:

- An instance of the TEMAC solution.
- Clock management logic including MMCM and Global Clock Buffer instances, where required.
- MII, GMII, or RGMII interface logic including IOB and DDR registers instances, where required.
- Statistics vector decode logic.
- AXI4-Lite to IPIF interface logic.
- User Transmit and Receive FIFOs with AXI4-Stream interfaces.
- User basic pattern generator module that contains a frame generator and frame checker plus loopback logic. At present, the loopback logic is not available for 2.5G rates.
- User AVB pattern generator module providing a second frame generator and frame checker for designs including the AVB Endpoint.
- A simple state machine to bring up the PHY (if any) and MAC ready for frame transfer

The HDL example design provides basic loopback functionality on the user side of the TEMAC solution and connects the GMII/RGMII interface to external IOBs. It can also operate as a pattern generator with data being optionally looped back externally, on the PHY side, and automatically checked.

This allows the functionality of the core to be demonstrated either using a simulation package, as discussed in this guide, or in hardware if placed on a suitable board. The simple state machine assumes a standard PHY address and registers content as per standard AMD demonstration boards.

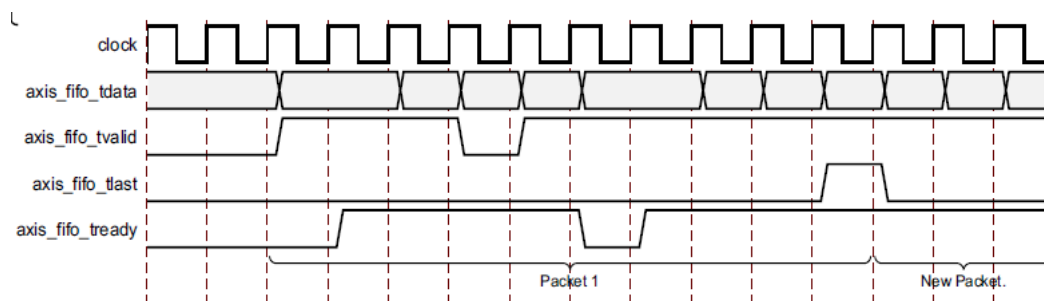
10, 100, 1000 Mbps Ethernet FIFO

The 10, 100, and 1000 Mbps Ethernet FIFO is available for MAC data rates of up to 1 Gbps. It is described in the following files:

```
<component_name>_ten_100_1g_eth_fifo.v[hd]
<component_name>_tx_client_fifo.v[hd]
<component_name>_rx_client_fifo.v[hd]
```

The 10, 100, and 1000 Mbps Ethernet FIFO contains an instance of tx_client_fifo to connect to the MAC TX AXI4-Stream interface, and an instance of the rx_client_fifo to connect to the MAC RX AXI4-Stream interface. Both transmit and receive FIFO components implement an AXI4-Stream user interface, through which the frame data can be read/written. The AXI4-Stream user interface and the MAC TX/RX AXI4-Stream interfaces are eight bits wide. The following figure illustrates a straightforward frame transfer across the user-side AXI4-Stream interface.

Figure 106: Frame Transfer across AXI4-Stream Interface



rx_client_fifo

The `rx_client_fifo` is built around a Dual Port Inferred RAM, giving a total memory capacity of 4,096 bytes. The receive FIFO writes in data received through the TEMAC core. If the frame is not errored, that frame is presented on the AXI4-Stream FIFO interface for reading by you, (in this case the `basic_pat_gen` module). If the frame is errored, that frame is dropped by the receive FIFO.

If the receive FIFO memory overflows, the frame currently being received is dropped, regardless of whether it is a good or bad frame, and the signal `rx_overflow` is asserted. Situations in which the memory can overflow are:

- The FIFO can overflow if the receiver clock is running at a faster rate than the transmitter clock or if the inter-packet gap between the received frames is smaller than the interpacket gap between the transmitted frames. If this is the case, the TX FIFO is not able to read data from the RX FIFO as fast as it is being received.
- The FIFO size of 4,096 bytes limits the size of the frames that it can store without error. If a frame is larger than 4,000 bytes, the FIFO can overflow and data is then lost. Therefore, it is recommended that the example design is not used with the TEMAC solution in jumbo frame mode for frames larger than 4,000 bytes.

tx_client_fifo

The `tx_client_fifo` is built around a Dual Port Inferred RAM, giving a total memory capacity of 4,096 bytes.

When a full frame has been written into the transmit FIFO, the FIFO presents data to the MAC transmitter. The MAC uses `tx_axis_mac_tready` to throttle the data until it has control of the medium.

If the FIFO memory fills up, the `tx_axis_fifo_tready` signal is used to halt the AXI4-Stream interface writing in data, until space becomes available in the FIFO. If the FIFO memory fills up but no full frames are available for transmission. For example, if a frame larger than 4,000 bytes is written into the FIFO, the FIFO asserts the `tx_overflow` signal and continues to accept the rest of the frame from you. The overflow frame is dropped by the FIFO. This ensures that the AXI4-Stream FIFO interface does not lock up.

2.5 Gbps Ethernet FIFO

The 2.5 Gbps Ethernet FIFO is available for MAC data rates of 2.5 Gbps only. It is described in the following files:

```
<component_name>_two_g5_eth_fifo.v[hd]  
<component_name>_tx_client_fifo_2g5.v[hd]  
<component_name>_rx_client_fifo_2g5.v[hd]
```

The 2.5 Gbps Ethernet FIFO contains an instance of `tx_client_fifo_2g5` to connect to the MAC TX AXI4-Stream interface, and an instance of the `rx_client_fifo_2g5` to connect to the MAC RX AXI4-Stream interface. Both transmit and receive FIFO components implement an AXI4-Stream user interface, through which the frame data can be read/written. The AXI4-Stream user interface is 32 bits wide and the MAC TX/RX AXI4-Stream interfaces are eight bits wide.

rx_client_fifo_2g5

The `rx_client_fifo_2g5` is built around a Dual Port instantiated block RAM, giving a total memory capacity of 4,096 bytes. The receive FIFO writes in data received through the TEMAC core. If the frame is not errored, that frame is presented on the AXI4-Stream FIFO interface for reading by you, (in this case the `basic_pat_gen` module). If the frame is errored, that frame is dropped by the receive FIFO.

If the receive FIFO memory overflows, the frame currently being received is dropped, regardless of whether it is a good or bad frame, and the signal `rx_overflow` is asserted.

Situations in which the memory can overflow are:

- The FIFO can overflow if the receiver clock is running at a faster rate than the transmitter clock or if the inter-packet gap between the received frames is smaller than the interpacket gap between the transmitted frames. If this is the case, the TX FIFO is not able to read data from the RX FIFO as fast as it is being received.
- The FIFO size of 4,096 bytes limits the size of the frames that it can store without error. If a frame is larger than 4,000 bytes, the FIFO can overflow and data is then lost. It is therefore recommended that the example design is not used with the TEMAC solution in jumbo frame mode for frames larger than 4,000 bytes.

tx_client_fifo_2g5

The `tx_client_fifo_2g5` is built around a Dual Port instantiated block RAM, giving a total memory capacity of 4,096 bytes.

When a full frame has been written into the transmit FIFO, the FIFO presents data to the MAC transmitter. The MAC uses `tx_axis_mac_tready` to throttle the data until it has control of the medium.

If the FIFO memory fills up, the `tx_axis_fifo_tready` signal is used to halt the AXI4-Stream interface writing in data, until space becomes available in the FIFO. If the FIFO memory fills up, no full frames are available for transmission. For example, if a frame larger than 4,000 bytes is written into the FIFO, the FIFO asserts the `tx_overflow` signal and continues to accept the rest of the frame from you. The overflow frame is dropped by the FIFO. This ensures that the AXI4-Stream FIFO interface does not lock up.

Basic Pattern Generator Module for 10, 100, 1000 Mbps Data Rates

The basic pattern generator for 10, 100, 1000 Mbps data rates are described in the following files:

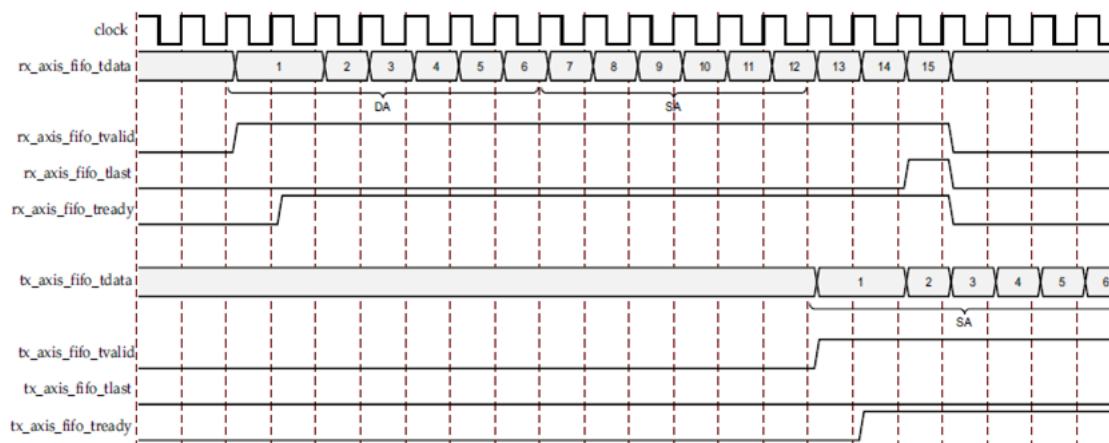
```
<component_name>_basic_pat_gen.v[hd]  
<component_name>_axi_pat_gen.v[hd]  
<component_name>_axi_pat_check.v[hd]  
<component_name>_axi_mux.v[hd]  
<component_name>_axi_pipe.v[hd]  
<component_name>_address_swap.v[hd]
```

The basic pattern generator has two main functional modes: loopback and generator. In loopback, the data from the RX FIFO is passed to the address swap module and passed from there to the TX FIFO. In generator mode, the TX data is provided by the pattern generator, with RX Data being optionally checked by the pattern checker.

Address Swap

The address swap module can be enabled for use on the loopback path. This would allow the example design, targeted to a suitable board, to be connected to an Ethernet protocol tester. The address swap module waits until both the DA and SA have been received before starting to send data to the TX FIFO.

Figure 107: Modification of Frame Data by Address Swap Module



If enabled, the module swaps the destination and source addresses of each frame as shown in the previous figure. To ensure that the outgoing frame destination address matches the source address of the link partner, otherwise the DA and SA are left untouched. The module transmits the frame control signals with equal latency to the frame data.

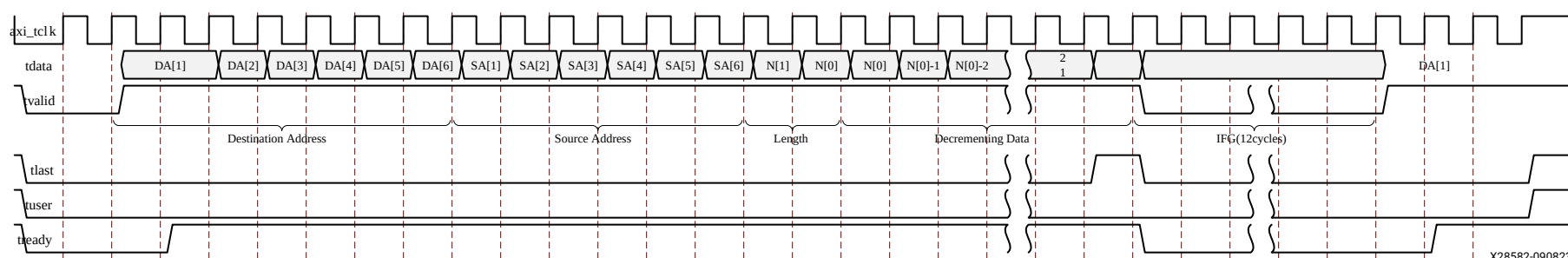
Pattern Generator

This pattern generator can be enabled/disabled using a DIP switch. When enabled, the data from the RX FIFO is flushed and the `axi_pat_gen` module drives the `address_swap` modules inputs.

The pattern generator allows user modification of the Destination Address, Source Address, minimum frame size, and maximum frame size using parameters. When enabled, using a dedicated input mapped to a DIP switch on a board, it starts with the minimum frame size and after each frame is sent, increments the frame size until the maximum value is reached, it then starts again at the minimum frame size.

In all cases, the Destination and Source address are as provided by HDL parameters, with the Type/Length field being dependent upon the frame size and the frame data being a decrementing count starting from the value in the type/length field. This should mean that the final data byte in all frames is 0x1. This is shown in the following figure.

Figure 108: Pattern Generator Frame Structure



In a loopback scenario (using a second board as the loopback), the ppm difference between the oscillators on the two boards can cause overflows in the slower board—resulting in errors. This is normally observed when the slower board is operating as the loopback board. To avoid this issue the data rate provided by the pattern generator is throttled to below the selected line rate.

Pattern Checker

The `axi_pat_check` module provides a simple sanity check that the data is being received correctly. It uses the same parameters as the `axi_pat_gen` module and therefore expects the same frame contents and frame size increments. Because the Frame data can or cannot have the DA and SA swapped the pattern checker allows either value to be in either location.

When enabled, using a dedicated input which uses a board DIP switch, the output from the RX_FIFO is monitored. The first step is to identify where in the frame sequence the data is, this is done by capturing the value in the type/length field. After this is done, the following frame is expected to be incrementally bigger (unless you happen to be at the wrap point). If an error is detected an error is raised on the byte or bytes which mismatch and the error condition is latched and output to a dedicated output, this is displayed using a board LED. The pattern checker state machine then re-synchronizes to the data. A dedicated input, connected to one of the push buttons, is used to clear this latched error state, enabling a feel for the frequency of errors (if any).

The pattern checker also provides a simple activity monitor. This toggles a dedicated output, which flashes a board LED, to indicate that RX Data is being received correctly. This ensures that the lack of a detected error is not due to all frames being dropped.

Basic Pattern Generator Module for 2.5 Gbps Data Rate

The basic pattern generator for 2.5 Gbps data rate is described in the following files:

```
<component_name>_basic_pat_gen_2g5.v[hd]  
<component_name>_axi_pat_gen_2g5.v[hd]  
<component_name>_axi_pat_check_2g5.v[hd]
```

At present, the basic pattern generator has only one functional mode: generator. In the generator mode, the TX data is provided by the pattern generator with the RX Data being optionally checked by the pattern checker. Both the pattern generator and checker have a 32-bit datapath as the user side AXI4-Stream interface is 32 bits for 2.5 Gbps data rate.

Pattern Generator

This pattern generator can be enabled/disabled using a DIP switch. When enabled, the data from the RX FIFO is flushed and the `axi_pat_gen_2g5` module drives the `tx_client_fifo_2g5` module inputs.

The pattern generator allows user modification of the Destination Address, Source Address, minimum frame size, and maximum frame size using parameters. When enabled, using a dedicated input mapped to a DIP switch on a board, it starts with the minimum frame size and after each frame is sent, increments the frame size until the maximum value is reached, it then starts again at the minimum frame size.

In all cases, the Destination and Source address are as provided by HDL parameters, with the Type/Length field being dependent upon the frame size and the frame data being an incrementing count starting from 0x0. The payload value increment across successive frames and rolls over when it is 0xFF.

Pattern Checker

The `axi_pat_check_2g5` module provides a simple sanity check that data is being received correctly. It uses the same parameters as the `axi_pat_gen_2g5` module and therefore expects the same frame contents and frame size increments. Because the Frame data can or cannot have the DA and SA swapped the pattern checker allows either value to be in either location.

When enabled, using a dedicated input which uses a board DIP switch, the output from the RX_FIFO is monitored. The first step is to identify where in the frame sequence the data is, this is done by capturing the first received frame first payload byte. After this is completed, the following payload bytes are expected to be incrementally bigger (unless you happen to be at the wrap point).

If an error is detected, an error is raised on the byte or bytes which mismatch and the error condition is latched and output to a dedicated output, this is displayed using a board LED.

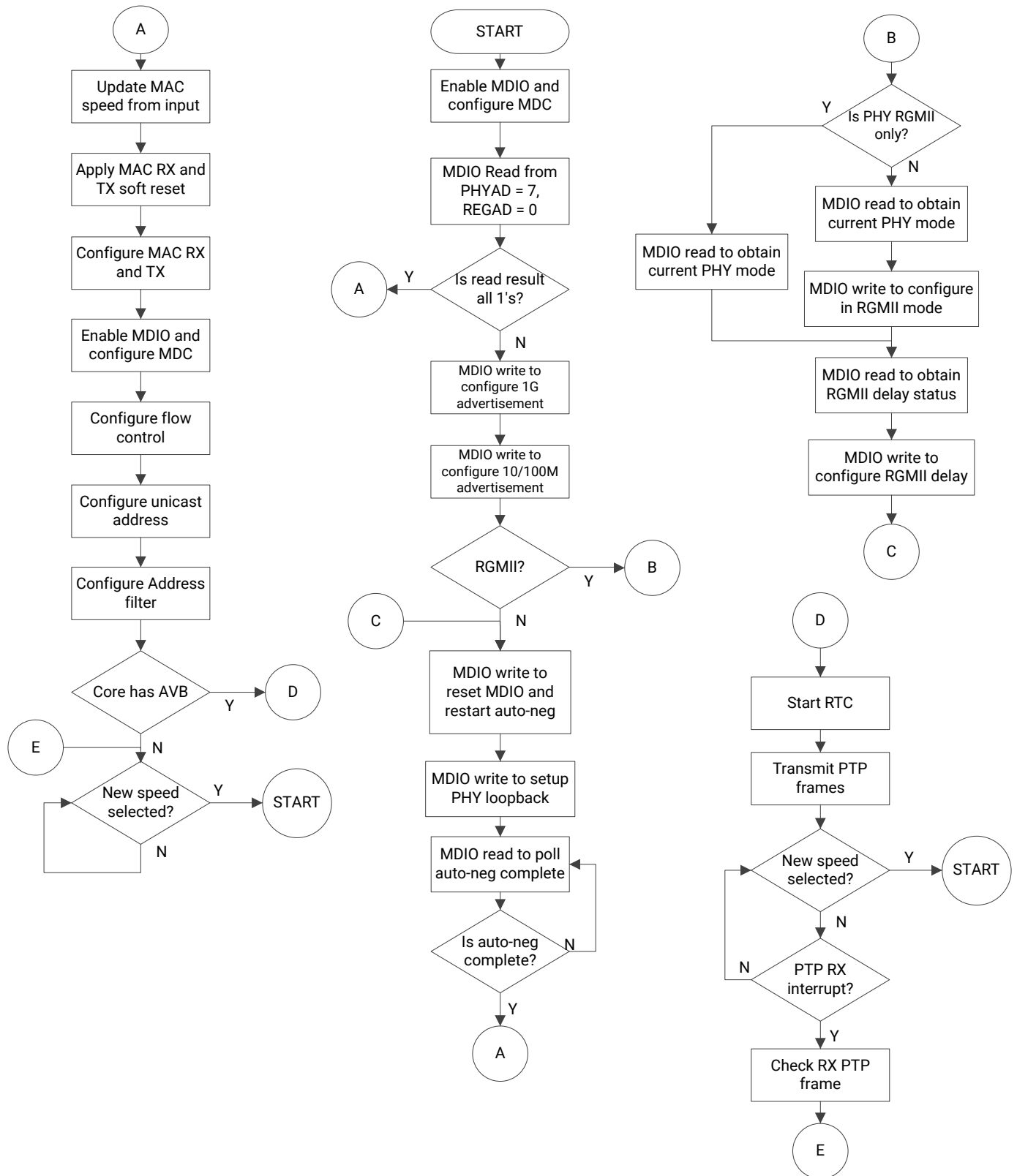
The pattern checker state machine then re-synchronizes to the data. A dedicated input, connected to one of the push buttons, is used to clear this latched error state, enabling a feel for the frequency of errors (if any).

The pattern checker also provides a simple activity monitor. This toggles a dedicated output, which flashes a board LED, to indicate that RX Data is being received correctly. This ensures that the lack of a detected error is not due to all frames being dropped.

AXI4-Lite Control State Machine

The AXI4-Lite state machine, which is present when the core is generated with AXI4-Lite support enabled, provides basic accesses to initialize the PHY and MAC to allow basic frame transfer.

Figure 109: State Machine Flow Diagram



X25616-080221

The previous figure shows the accesses performed by the state machine. After a reset, and allowing settling time for internal resets to complete, the state machine first writes to the MAC to enable the MDIO and configure the MDIO clock (this assumes an `s_axi_aclk` running at 100 MHz). An MDIO read is then performed from PHYAD 7, which is the standard address used on AMD demonstration boards. If this returns all 1s, it implies that no PHY exists at this location and further MDIO accesses are skipped.

This MDIO read enables the demonstration test bench to limit the number of MDIO accesses performed and reduce the runtime of the simulations while still allowing the correct MDIO accesses to take place on a board. If the PHY is present, the MDIO read data has a value other than all 1s, the state machine performs the necessary MDIO writes to configure the PHY speed advertisement as per the `mac_speed` inputs. If RGMII is selected, a read-modify-write is performed to select RGMII, unless the PHY is only RGMII, avoiding the need to change jumper settings on the board. Finally the PHY is reset and auto-negotiation restarted.

After auto-negotiation completes the MAC speed is updated, as per the `mac_speed` inputs. The MAC is then configured to disable flow control. If Frame Filter is enabled, initialize Frame Filter value and mask value bytes [11:0] and set the address filter to promiscuous mode if the PHY is present or if the MAC data rate is set to 2.5 Gbps, or to non-promiscuous mode if the PHY is not present.

If Frame Filter is disabled, initialize the unicast address and set the Frame Filter to promiscuous mode. If the AVB Endpoint logic is present, then the RTC is started and PTP frames both transmitted and received (this assumes an external loopback is in use). Finally the state machine sits and waits, if the `update_speed` input asserts it returns to the initial MDIO read state and the new `mac_speed` input is captured and applied.

With the state machine only applying a fixed core configuration, logic can be stripped during logic optimization. To avoid this, the state machine has a serial interface, `serial_command` and `serial_response`, which can be used to access any location and either perform a read or a write. This uncertainty prevents functions unused by the state machine from being stripped.

Targeting the Example Design to a Board

For each supported device, there are certain TEMAC solution configurations which can be targeted directly to the AMD connectivity board for that device. The XDC included with the example design provides the required pin placements for the specific board. In each case, the board DIP switches, push buttons, and LEDs are used to provide basic control over the MAC functionality. This is described in more detail in the board specific sections.

TEMAC Solution Configurations Supported

There are some basic requirements for the example design to function correctly when targeted at a board. The TEMAC must:

- Include an AXI4-Lite Management interface
- Target the relevant part for the specific board – See the board specific section

Due to I/O limitations some connectivity boards use a RGMII only PHY. This is the case on the Artix 7 AC701. This means that the TEMAC must be configured for RGMII to be targeted to this board.

Bring-Up Sequence

When the example design is first targeted at a board, the following sequence is suggested to check if the various features are working. This is common for all boards.

1. Attach an Ethernet cable between the board and a PC installed with wireshark or similar.
2. Select the desired speed using the DIP switches.
3. Push the update speed pushbutton.
4. Ensure the link status LEDs show the expected speed.
5. Enable the `pattern_generator` using the DIP switch.
6. Capture and check the received frames at the PC and ensure they have the expected data pattern.

To use the pattern checker, check both datapaths. Two boards are required.

Board A: Operates as a frame source and optionally checker.

Board B: This board operates as a simple loopback board.

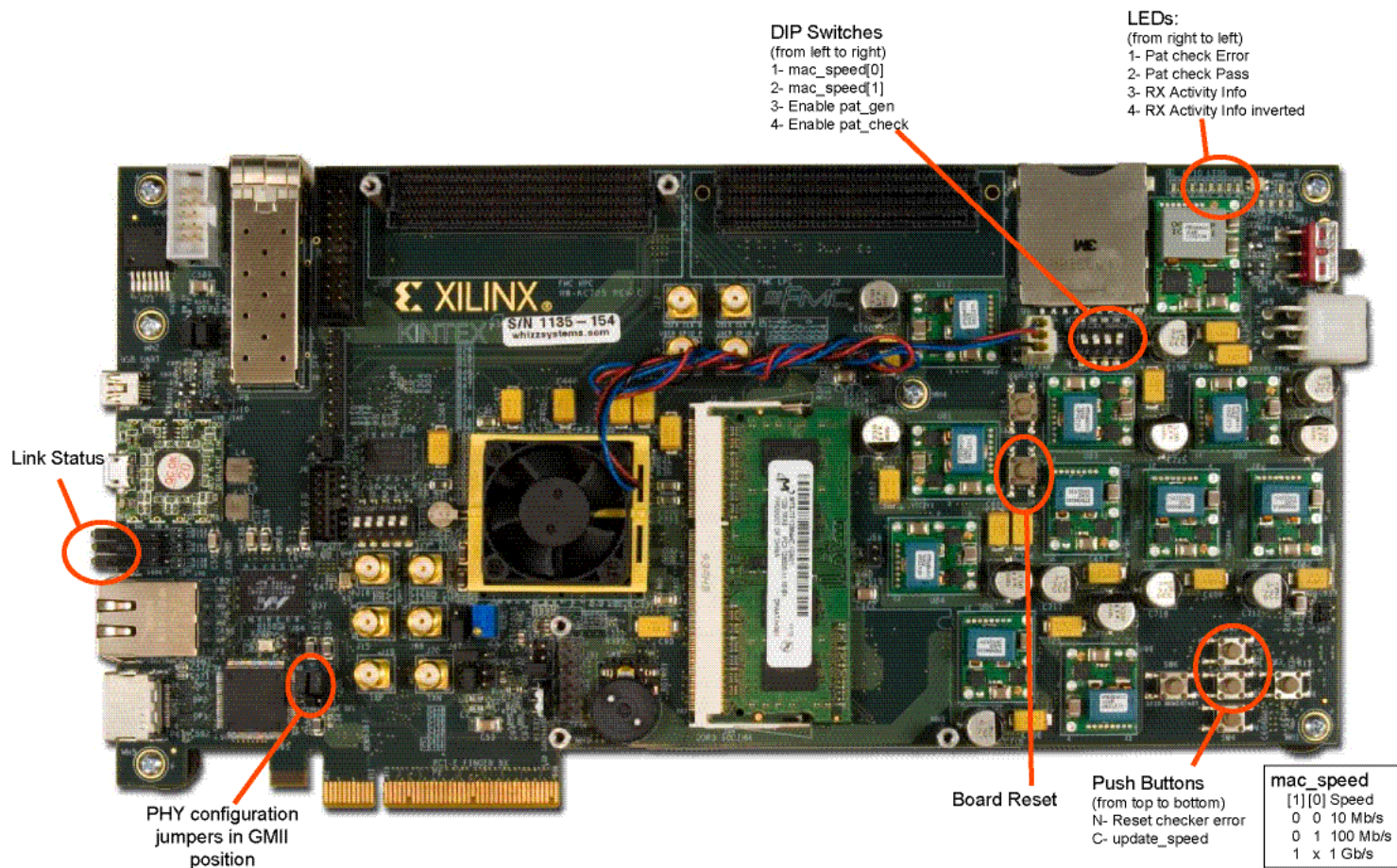
Bring-up process:

1. Attach an Ethernet cable between the two boards.
2. Select the desired speed on both boards. This must be the same setting.
3. Push the update speed button on both boards.
4. Check the link status LEDs show the correct speed.
5. Enable the pattern generator on Board A, ensure it is disabled on Board B.
6. Check the Link Status RX/TX LEDs all light up.
7. If desired the Pattern checker can be enabled on both boards or Board A.
8. Ensure the RX activity LED is flashing.

KC705 Board

The XDC targets the KC705 when either the correct Kintex 7 FPGA part (XC7K325TFFG900) is selected, or when the KC705 board is targeted directly in the Vivado project options.

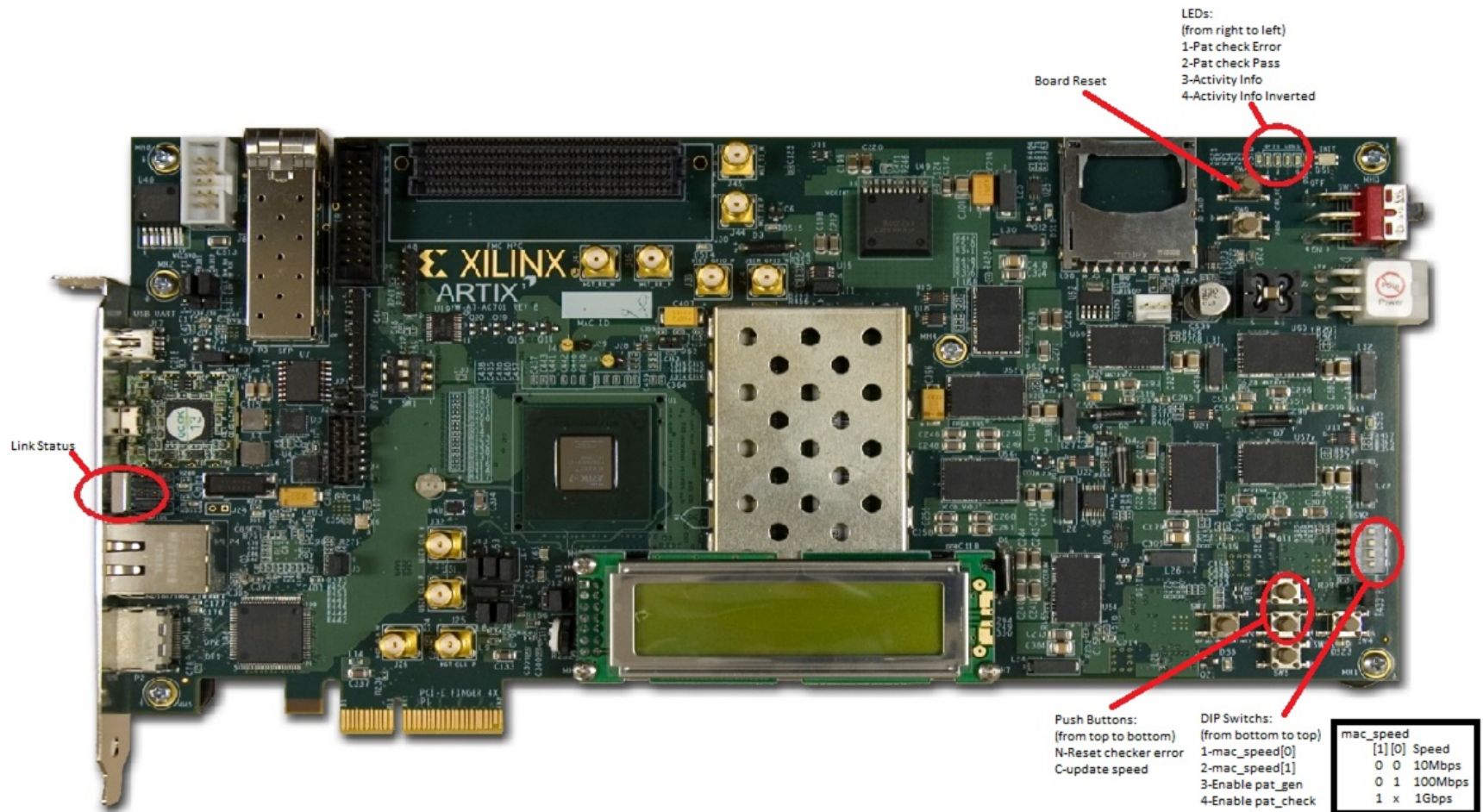
Figure 110: KC705 Board Connectivity



AC701 Board

The XDC targets the AC701 when either the correct Artix 7 FPGA part (XC7A200TFBG676) is selected, or when the AC701 board is targeted directly in the Vivado project options.

Figure 111: AC701 Board Connectivity



When the MAC data rate is set to 2.5 Gbps, this additional module is present:

```
loopback_module.v[hd]
```

The demonstration test bench is a simple VHDL or Verilog program to exercise the example design and the core itself. It has two modes of operation, DEMO and Built-in Self Test (BIST), with BIST being the default mode only when the AVB Endpoint is included and frame filter is disabled. When the core is configured for 2.5 Gbps data rate, only the BIST mode of operation is available. In all other configurations, the test bench defaults to DEMO mode.

The test bench consists of the following:

- Clock generators
- DEMO – A stimulus block that connects to the GMII/MII or RGMII receiver interface of the example design.
- DEMO – A monitor block to check data returned through the GMII/MII or RGMII transmitter interface.
- DEMO (Frame Filter enabled) – Basic frame filter that looks at the DA/SA fields of frame inserted into the GMII/MII or RGMII receiver interface.
- BIST – A simple loopback from the GMII/MII or RGMII transmit interface to the receiver.
- BIST (AVB only) – A basic AV data bandwidth monitor.
- BIST (2.5 Gbps only) - A simple pattern checker module for TX and RX. Also, this module loops back the TX data to RX inputs.
- A management block to control the speed selection.
- An MDIO monitor/stimulus to check and respond to MDIO accesses, if a management interface is selected and MDIO is enabled.

Core with Management Interface

DEMO Mode

The demonstration test bench performs the following tasks:

- Input clock signals are generated.
- A reset is applied to the example design.
- The required speed is selected using `mac_speed` and `update_speed`
- The MDIO stimulus/response block responds to a read with all 1s – to indicate no PHY is present.
- Four frames are pushed into the GMII/MII or RGMII receiver interface at the fastest MAC speed supported:

- The first frame is a minimum length frame.
- The second frame is a type frame.
- The third frame is an errored frame.
- The fourth frame is a padded frame.
- The frames received at the GMII/MII or RGMII transmitter interface are checked against the stimulus frames to ensure data is the same. The monitor process takes into account the source/destination address field and FCS modifications resulting from the address swap module.
- If either the Tri-speed or MII configurations have been selected, `mac_speed` is updated to run at the next fastest available speed. This is 100 Mbps or 10 Mbps respectively. `update_speed` is then pulsed.
- The MDIO stimulus/response block responds to a read with all 1s – to indicate no PHY is present.
- The same four frames are then sent to the MII/GMII or RGMII interface and checked against the stimulus frames.
- If the Tri-speed configuration has been selected, `mac_speed` is updated to run at 10 Mbps. `update_speed` is then pulsed.
- The MDIO stimulus/response block responds to a read with all 1s – to indicate no PHY is present.
- The same four frames are then sent to the MII/GMII or RGMII interface and checked against the stimulus frames.
- For the Tri-speed configuration, the speed is then changed back to 1 Gbps and the same four frames are sent and checked for a final time. This tests the speed switching between 1 Gbps and 10/100 Mbps in both directions.

DEMO Mode with Frame Filter

The test bench performs the additional task of checking the destination address and source address of the frame in the following way:

- A fifth frame is pushed into the GMII/MII or RGMII receiver interface.
- The address filter looks at the DA/SA fields of each frame.
- If there is a mismatch between the DA/SA field of a frame and the 12-byte value the address filter has been pre-programmed with (in the AXI4-Lite state machine), then the frame is dropped.
- By default only the fifth frame is dropped because of this mismatch.

BIST Mode

The demonstration test bench performs the following tasks:

- Input clock signals are generated.
- A reset is applied to the example design.
- The required speed is selected using `mac_speed` and `update_speed`.
- The MDIO stimulus/response block responds to a read with all 1s – to indicate no PHY is present.
- If the AVB Endpoint is included the AXI4-Lite state machine requests two TX PTP frames and checks they are received.
- The pattern generator(s) and checker(s) are enabled.
- The simulation runs for a fixed duration, allowing a large number of frames to pass.
- Any detected errors or lack of RX activity are reported as errors.
- If the AVB Endpoint is included the bandwidth of the two data streams is reported.

Core with No Management Interface

DEMO Mode

The demonstration test bench performs the following tasks:

- Input clock signals are generated.
- A reset is applied to the example design.
- The required speed is selected using `mac_speed` and `update_speed`.
- The stimulus block pushes four frames into the GMII/MII or RGMII receiver interface at the fastest speed supported by the selected configuration:
- The first frame is a minimum-length frame.
- The second frame is a type frame.
- The third frame is an errored frame.
- The fourth frame is a padded frame.
- The frames received at the GMII/MII or RGMII transmitter interface are checked against the stimulus frames to ensure data is the same. The monitor process takes into account the source/destination address field and FCS modifications resulting from the address swap module.
- If either the Tri-speed or MII configurations have been selected, `mac_speed` is updated to run at the next fastest available speed. This is 100 Mbps or 10 Mbps, respectively. `update_speed` is then pulsed.
- The same four frames are then sent to the MII/GMII or RGMII interface and checked against the stimulus frames.

- If the Tri-speed configuration has been selected, `mac_speed` is updated to run at 10 Mbps. `update_speed` is then pulsed. The same four frames are then sent to the MII or RGMII interface and checked against the stimulus frames.
- For the Tri-speed configuration, the speed is then changed back to 1 Gbps and the same four frames are sent and checked for a final time. This tests the speed switching between 1 Gbps and 10/100 Mbps in both directions.

BIST Mode

The demonstration test bench performs the following tasks:

- Input clock signals are generated.
- A reset is applied to the example design.
- The required speed is selected using `mac_speed` and `update_speed`.
- The pattern generator and checker are enabled.
- The simulation runs for a fixed duration, allowing a large number of frames to pass.
- Any detected errors or lack of RX activity are reported as errors.

Changing the Test Bench

The Demonstration test bench defaults to DEMO mode for all implementations except when AVB endpoint is enabled and Frame Filter is disabled. In that case, the test bench defaults to BIST mode.

The mode is set using the `TB_MODE` parameter in the `demo_tb.v[hd]`. To change the mode, set the parameter value to DEMO or BIST.

DEMO Mode

Changing Frame Data

The contents of the frame data passed into the TEMAC receiver can be changed by editing the DATA fields for each frame defined in the test bench. The test bench automatically calculates the new FCS field to pass into the TEMAC, as well as calculating the new expected FCS value. Further frames can be added by defining a new frame of data.

If frame filter is enabled, the first 4 frames have the same DA and SA data matching the value of what the frame filter is set to. The fifth frame is dropped because by default it has different DA/SA data than the first 4.

In the example design the address filter is set to:

DA = 48'h0605040302DA

SA = 48'h06050403025A

If none of the frames has this DA/SA setup, all frames are dropped by the address filter.



IMPORTANT! In the DEMO test bench, the `address_filter_value` parameter has no control over the value the address filter is set to.

Changing Frame Error Status

Errors can be inserted into any of the pre-defined frames by changing the error field to 1 in any column of that frame. When an error is introduced into a frame, the `bad_frame` field for that frame must be set to disable the monitor checking for that frame. The error currently written into the third frame can be removed by setting all error fields for the frame to 0 and unsetting the `bad_frame` field.

BIST Mode

In BIST mode the data is provided by the [Basic Pattern Generator Module for 10, 100, 1000 Mbps Data Rates](#). This allows a degree of control over the frames generated using the module parameters:

```
DEST_ADDR
SRC_ADDR
MAX_SIZE
MIN_SIZE
ENABLE_VLAN
VLAN_ID
VLAN_PRIORITY
```

The pattern generator does not have an error injection capability.

Upgrading

This appendix contains information about migrating a design from the AMD Design Suite to the AMD Vivado™ Design Suite and upgrading to a more recent version of the IP core. For customers upgrading in the Vivado Design Suite, important details (where applicable) about any port changes and other impacts on user logic are included.

Migrating to the Vivado Design Suite

For information on migrating from the AMD Design Suite tools to the Vivado Design Suite, see the *ISE to Vivado Design Suite Migration Guide* ([UG911](#))

Upgrading in the Vivado Design Suite

This section provides information about any changes to the user logic or port designations that take place when you upgrade to a more current version of this IP core in the Vivado Design Suite.

Shared Logic

As part of the hierarchical changes to the core that were implemented in v8.0, it is now possible for an instance of the core to include logic that can be shared between multiple cores, this shared logic was previously exposed in the example design for the core.

If you are updating a version of the core previous to v8.0 to a new one with Shared Logic there might not be a simple upgrade path and it is recommended to see [Shared Logic](#) for guidance.

Port Additions in v9.0 Rev 4

The following table shows the optional ports added when the Physical Interface is set to Internal and Internal Mode Clock Source is set to RX User Clk2.

Table 81: Added Optional Ports

Port Name and Width	Direction	Description	What to do
rx_usr_clk2	Input	Clock for the RX datapath	Connect this port to the PCS/PMA core's rxoutclk port
clk_enable_rx	Input	Clock enable for the RX datapath	Connect this port to the PCS/PMA core's sgmmi_rx_clk_en port

Port Changes from v8.3 to v9.0

Ports Removed

The following table shows the removed port from the core when it is generated for the AMD UltraScale™ architecture devices using GMII or RGMII and only when the Shared Logic option “Include shared logic in example design” option is selected.

Table 82: Removed Ports

Port Name and Width	Direction	Description	What to do
Refclk	Input	Required for IDELAYCTRL, 300 to 1,333 MHz	Remove this port and its driver from the port map on your instance(s) of this core.

The following table shows the removed port from the core only when it is generated using MII and when Statistics Counters is not enabled.

Table 83: Removed Ports

Port Name and Width	Direction	Description	What to do
gxt_clk	Input	Global 125 MHz clock	Remove this port and its driver from the port map on your instance(s) of this core.

Other Changes from v8.3 to v9.0

Shared Logic for UltraScale Architecture Devices

In previous versions of the core, there were no sharable resources available for UltraScale architecture devices. In this version, IDELAYCTRL has been made a shareable resource. This makes the core more flexible when its I/Os need to share the XiPHY Byte with other modules I/Os.

Port Changes from v8.2 to v8.3

No new ports have been added or removed. The MDIO interface ports are now made available only when the MDIO interface is enabled.

Other Changes from v8.2 to v8.3

MDIO Logic

MDIO logic is now optional. When generating the core you can choose to either include or exclude the MDIO logic from the core. MDIO logic is enabled by default.

MDIO Interface

MDIO interface ports are available only when the MDIO logic is enabled. An option is now provided to make the interface pins external or internal. In previous versions, the physical interface type of the core decided whether the MDIO interface pins would be internal or external. In v8.3 of the core, you can make this choice, irrespective of the physical interface selected.

Port Changes from v8.1 to v8.2

Ports Added

Ports added for Priority Flow (PFC) interface. These ports are available only when the IP is generated with PFC support. See [Table 11: Priority Flow Control Ports](#) for the complete port list.

Ports Removed

None.

Other Changes from v8.1 to v8.2

IDLEAYCTRL

For UltraScale architecture-based devices, IDELAYCTRL is always present at the `<component_name>_block` level.

Number of Filter Table Entries

The number of Filter Table Entries has been increased to 16.

Port Changes from v8.0 to v8.1

None.

Port Changes from v7.0 to v8.0

Ports Added

The ports in the following table are only available when using RGMII in AMD Artix™ 7 or AMD Kintex™ 7 devices (see [RGMII for 7 Series Devices](#)), and only when the Shared Logic option is selected with the “Include shared logic in core” option.

Table 84: Additional Ports

Port Name and Width	Direction	Description	What to do
gtx_clk_out	Output	This clock has a 0o phase shift with respect to the gtx_clk input and is used for RGMII data transmission.	This output clock can be used by other TEMAC core instances when sharing clocking resources. See Figure 75: Clock Resource Sharing for 1 Gbps RGMII in 7 Series Using HP I/O or Figure 85: Clock Resource Sharing for Tri-Speed RGMII in 7 Series Devices Using HR I/O for a connection illustration.
gtx_clk90_out	Output	This clock has a 90o phase shift with respect to the gtx_clk input and is used for RGMII transmitter clock forwarding.	This output clock can be used by other TEMAC core instances when sharing clocking resources. See Figure 75: Clock Resource Sharing for 1 Gbps RGMII in 7 Series Using HP I/O or Figure 85: Clock Resource Sharing for Tri-Speed RGMII in 7 Series Devices Using HR I/O for a connection illustration.

Ports Removed

The ports in the following table were removed from the core when using GMII or RGMII only when the Shared Logic option “Include shared logic in example design” option is selected.

Table 85: Removed Ports

Port Name and Width	Direction	Description	What to do
refclk	Input	Required for idelayctrl, 200-300 MHz	Remove this port and its driver from the port map on your instance(s) of this core.

Other Changes from v7.0 to v8.0

IDELAYCTRL

For GMII and RGMII physical interfaces IODELAY elements are used by the receiver logic. An IDELAYCTRL module must therefore be instantiated in the design. For the GMII and RGMII, this is included in the <component_name>_support level, present in the core, if the Shared Logic option is selected.

IP Upgrade

In v9.0 revision 6, a new parameter, C_INT_MODE_TYPE was added. In internal mode, this indicates if the PCS/PMA core connected to TEMAC is configured for 1000BASE-X mode or SGMII mode.

In v9.0 revision 4, a new parameter, C_SGMII_CLK_SRC was added. In internal mode, this selects the clock for RX datapath.

In v9.0 of the core a new parameter, C_HAS_2G5 was added. This indicates if the core is 2.5 Gbps capable or not.

In v8.3 of the core a new parameter, C_HAS_MDIO was added. This controls the inclusion of MDIO logic.

In v8.2 of the core, a new parameter, C_PFC was added. This controls the inclusion of the PFC feature.

There are no changes in the parameters between v8.0 and v8.1 of the core.

For any version prior to v8.0, when an IP upgrade has been run in your design the Shared Logic option setting and port changes of the upgraded core are determined from the configuration of the older core as shown in the following table.

Table 86: Upgrade Scenarios

Older Core Configuration		After IP Upgrade		
Interface	Device Family	Shared Logic	Port Changes	Action
Internal	All	Not applicable	None	None
MII	All	Not applicable	None	None
GMII	All	Included in the core	None	None
RGMII	Virtex 7	Included in the core	None	None

Table 86: Upgrade Scenarios (cont'd)

Older Core Configuration		After IP Upgrade		
RGMI	Zynq, Artix 7 and Kintex 7	Not part of the core	refclk removed	Remove this port and its driver from the port map on your instance(s) of this core. Also, make sure an IDELAYCTRL is instantiated in the design. Use the 'set_property IODELAY_GROUP' XDC constraint to link this IDELAYCTRL with the core (see IDELAYCTRL).

Calculating the MMCM Phase Shift or IODelay Tap Setting

Two differing methods can be used by the core to meet input bus (GMII/MII or RGMII) setup and hold timing specifications. The following topics are included in this appendix:

- **MMCM Usage:** An MMCM can be used in the receiver clock path to meet the input setup and hold requirements when implementing GMII/MII and RGMII. See the Physical Interface sections in this Guide in [Chapter 4: Designing with the Core](#).
- **IODelay Usage:** IODelays can be used in the receiver clock path to meet the input setup and hold requirements when implementing GMII/MII and RGMII. See the Physical Interface sections in this guide in [Chapter 4: Designing with the Core](#).

MMCM Usage

MMCM Phase Shifting Requirements

When using an MMCM, a fixed-phase shift offset is applied to the receiver clock MMCM to skew the clock. This performs static alignment by using the receiver clock MMCM to shift the internal version of the receiver clock such that the input data is sampled at the optimum time. The ability to shift the internal clock in small increments is critical for sampling high-speed source synchronous signals. For statically aligned systems, the MMCM output clock phase offset (as set by the phase shift value) is a critical part of the system, as is the requirement that the PCB is designed with precise delay and impedance-matching for all the GMII/MII or RGMII receiver data bus and control signals.

You must determine the best MMCM setting (phase shift) to ensure that the target system has the maximum system margin to perform across voltage, temperature, and process (multiple chips) variations. Testing the system to determine the best MMCM phase shift setting has the added advantage of providing a benchmark of the system margin based on the UI (unit interval or bit time). The System margin is defined as the following:

$$\text{System Margin (ps)} = \text{UI(ps)} \times (\text{working phase shift range}/128)$$

Finding the Ideal Phase Shift Value

AMD cannot recommend a singular phase shift value that is effective across all hardware families. AMD does not recommend attempting to determine the phase shift setting empirically. In addition to the clock-to-data phase relationship, other factors such as package flight time (package skew) and clock routing delays (internal to the device) affect the clock to data relationship at the sample point (in the IOB) and are difficult to characterize.



RECOMMENDED: AMD recommends extensive investigation of the phase shift setting during hardware integration and debugging. The phase shift settings provided in the example design constraint file are placeholders, and work successfully in a back-annotated simulation of the example design.

Perform a complete sweep of phase-shift settings during your initial system test. Use a test range which covers at least half of the clock period or 128 taps. This does not imply that 128 phase-shift values must be tested. Increments of 4 (52, 56, 60, and so forth) correspond to roughly one MMCM tap at 125 MHz, and consequently provide an appropriate step size. Additionally, it is not necessary to characterize areas outside the working phase-shift range.

At the edge of the operating phase shift range, system behavior changes dramatically. In eight phase shift settings or less, the system can transition from no errors to exhibiting errors. Checking the operational edge at a step size of two (on more than one board) refines the typical operational phase shift range. After the range is determined, choose the average of the high and low working phase shift values as the default.



RECOMMENDED: During the production test, AMD recommends that you re-examine the working range at corner case operating conditions to determine whether any final adjustments to the final phase shift setting are needed.

IODelay Usage

IODelay Tap Setting Requirements

With this method, an IODelay is used on either the clock or data (or both) to adjust the clock/data relationship such that the input data is sampled at the optimum time. The ability to adjust this relationship in small increments is critical for sampling high-speed source synchronous signals. For statically aligned systems, the IODelay Tap setting is a critical part of the system, as is the requirement that the PCB is designed with precise delay and impedance-matching for all the GMII/MII or RGMII receiver data bus and control signals.

Note: You must determine the best IODelay Tap setting to ensure that the target system has the maximum system margin to perform across voltage, temperature, and process (multiple chips) variations.

Finding the Ideal Tap Setting Value

AMD cannot recommend a singular tap value that is effective across all hardware families. AMD does not recommend attempting to determine the tap setting empirically. In addition to the clock-to-data phase relationship, other factors such as package flight time (package skew) and clock routing delays (internal to the device) affect the clock to data relationship at the sample point (in the IOB) and are difficult to characterize.



RECOMMENDED: *AMD recommends extensive investigation of the tap setting during hardware integration and debugging. The tap settings provided in the example design constraint file are placeholders, and work successfully in a back-annotated simulation of the example design.*

Perform a complete sweep of tap settings during your initial system test. If possible use a test range which covers at least half of the clock period. This does not imply that all values must be tested as it might be simpler to use a large step size initially to identify a tighter range for a subsequent run. Additionally, it is not necessary to characterize areas outside the working range. If an IODelay is used on both Clock and Data then ensure this test range covers both clock only and data only adjustments. At the edge of the operating range, system behavior changes dramatically. In four tap settings or less, the system can transition from no errors to exhibiting errors.



RECOMMENDED: *During the production test, AMD recommends that you re-examine the working range at corner case operating conditions to determine whether any final adjustments to the final setting are needed. Where IODelays are used on the data it might be necessary or beneficial to use slightly different values for each bit.*

Verification, Compliance, and Interoperability

The TEMAC solution has been verified with extensive simulation and hardware verification.

Simulation

A highly parameterizable transaction based test bench was used to test the core. Testing included the following:

- Reset and Initialization
 - Register Access including MDIO
 - Frame transmission
 - Frame reception, frame filtering and error handling
 - All supported PHY interfaces
-

Hardware Testing

The example design provided with this core can be targeted to [Reference Boards](#).

These designs have been used to perform hardware validation of the TEMAC solution. The KC705 and AC701 designs have been hardware tested using the AMD Vivado™ Design Suite.

Debugging

This appendix includes details about resources available on the AMD Support website and debugging tools.



IMPORTANT! *IP license level is ignored at checkpoints. The test confirms a valid license exists. It does not check IP license level.*

Finding Help with AMD Adaptive Computing Solutions

To help in the design and debug process when using the core, the [Support web page](#) contains key resources such as product documentation, release notes, answer records, information about known issues, and links for obtaining further product support. The [Community Forums](#) are also available where members can learn, participate, share, and ask questions about AMD Adaptive Computing solutions.

Documentation

This product guide is the main document associated with the core. This guide, along with documentation related to all products that aid in the design process, can be found on the [AMD Adaptive Support web page](#) or by using the AMD Adaptive Computing Documentation Navigator. Download the Documentation Navigator from the [Downloads page](#). For more information about this tool and the features available, open the online help after installation.

Answer Records

Answer Records include information about commonly encountered problems, helpful information on how to resolve these problems, and any known issues with an AMD Adaptive Computing product. Answer Records are created and maintained daily to ensure that users have access to the most accurate information available.

Answer Records for this core can be located by using the Search Support box on the main [AMD Adaptive Support web page](#). To maximize your search results, use keywords such as:

- Product name
- Tool message(s)
- Summary of the issue encountered

A filter search is available after results are returned to further target the results.

Master Answer Record for the Core

AR [54251](#).

Technical Support

AMD Adaptive Computing provides technical support on the [Community Forums](#) for this AMD LogiCORE™ IP product when used as described in the product documentation. AMD Adaptive Computing cannot guarantee timing, functionality, or support if you do any of the following:

- Implement the solution in devices that are not defined in the documentation.
- Customize the solution beyond that allowed in the product documentation.
- Change any section of the design labeled DO NOT MODIFY.

To ask questions, navigate to the [Community Forums](#).

Debug Tools

There are many tools available to address <Tri-Mode Ethernet MAC> design issues. It is important to know which tools are useful for debugging various situations.

Vivado Design Suite Debug Feature

The AMD Vivado™ Design Suite debug feature inserts logic analyzer and virtual I/O cores directly into your design. The debug feature also allows you to set trigger conditions to capture application and integrated block port signals in hardware. Captured signals can then be analyzed. This feature in the Vivado IDE is used for logic debugging and validation of a design running in AMD devices.

The Vivado logic analyzer is used to interact with the logic debug LogiCORE IP cores, including:

- ILA 2.0 (and later versions)
- VIO 2.0 (and later versions)

See the *Vivado Design Suite User Guide: Programming and Debugging* ([UG908](#)).

Reference Boards

Various AMD development boards support the <Tri-Mode Ethernet MAC> core. These boards can be used to prototype designs and establish that the core can communicate with the system.

The provided example design can, if generated with the correct part and core options, be targeted directly to the following list of boards. For more information, see [Targeting the Example Design to a Board](#).

7 series evaluation boards:

- KC705
- AC701

Link Analyzers

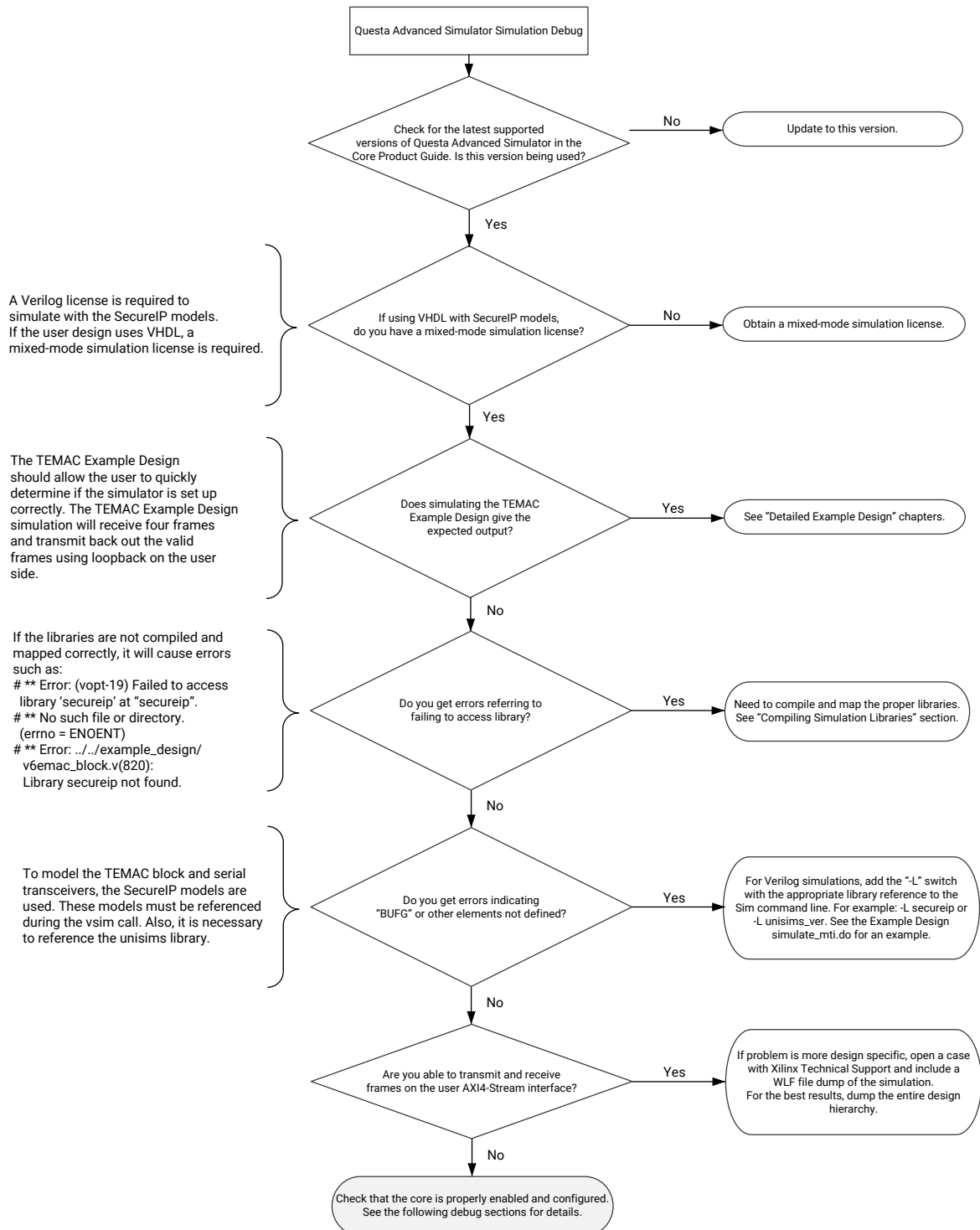
Link analyzers can be used to generate and analyze traffic for hardware debug and testing. Common link analyzers include:

- Spirent SmartBits
- IXIA brand 10/100/1000 Ethernet test chassis
- Wireshark (a free packet sniffer software application)

Simulation Debug

The simulation debug flow for Mentor Graphics Questa Advanced Simulator is illustrated in the following figure. A similar approach can be used with other simulators.

Figure 113: Simulator Flow Diagram



X25623-080321

Compiling Simulation Libraries

Compile the AMD simulation libraries, either by using the AMD Simulation Library Compilation Wizard, or by using the `compxlib` command line tool.

AMD Simulation Library Compilation Wizard

A GUI wizard provided as part of the AMD software can be launched to assist in compiling the simulation libraries by typing `compxlib` in the command prompt.

For more information see the Software Manuals and specifically the *Command Line Tools Reference Guide* under the section titled `compxlib`.

Assuming the AMD and Questa Advanced Simulator environments are set up correctly, this is an example of compiling the SecureIP and UNISIMs libraries for Verilog into the current directory.

```
compxlib -s mti_se -arch virtex7 -l verilog -lib secureip -lib unisims -  
dir ./
```

There are many other options available for `compxlib` described in the Command Line Tools Reference Guide (UG628).

`Compxlib` produces a `questasim.ini` file containing the library mappings. In Questa Advanced Simulator, to see the current library mappings, type `vmap` at the prompt. The mappings can be updated in the `.ini` file or to map a library at the Questa Advanced Simulator prompt type:

```
vmap [<logical_name>] [<path>]
```

For example:

```
vmap unisims_ver C:\my_unisim_lib
```

Implementation and Timing Errors

Regional Clocking Errors

When implementing the Ethernet MAC with either a GMII or RGMII physical interface, regional clocking methodologies are used. This means that there are the following requirements:

1. The receive-side physical interface clock (GMII_RX_CLK for GMII, or RGMII_RXC for RGMII) must be placed at a clock-capable I/O (CCIO) pin. If this requirement is not met, an error similar to the following one might be seen during implementation:

```
ERROR: [Vivado 12-1411] Cannot set LOC property of ports, Could not legally place instance trimac_fifo_block/trimac_sup_block/tri_mode_ethernet_mac_i/inst/gmii_interface/gmii_rx_clk_ibuf_i at Y14 (IOB_X1Y49) since it belongs to a shape containing instance trimac_fifo_block/trimac_sup_block/tri_mode_ethernet_mac_i/inst/gmii_interface/bufio_gmii_rx_clk. The shape requires relative placement between trimac_fifo_block/trimac_sup_block/tri_mode_ethernet_mac_i/inst/gmii_interface/gmii_rx_clk_ibuf_i and trimac_fifo_block/trimac_sup_block/tri_mode_ethernet_mac_i/inst/gmii_interface/bufio_gmii_rx_clk that cannot be honored because it would result in an invalid location for trimac_fifo_block/trimac_sup_block/tri_mode_ethernet_mac_i/inst/gmii_interface/bufio_gmii_rx_clk.
```

2. All receive-side physical interface signals must be placed at package pins that correspond to the same clock region as the receive-side physical interface. If this requirement is not met, a warning similar to the following might be seen during implementation:

```
CRITICAL WARNING: [Vivado 12-1411] Cannot set LOC property of ports, Illegal to place instance trimac_fifo_block/trimac_sup_block/tri_mode_ethernet_mac_i/inst/gmii_interface/bufio_gmii_rx_clk on site T1E0FF_X101Y96. The location site type does not match the instance type. Instance trimac_fifo_block/trimac_sup_block/tri_mode_ethernet_mac_i/inst/gmii_interface/bufio_gmii_rx_clk belongs to a shape with reference instance trimac_fifo_block/trimac_sup_block/tri_mode_ethernet_mac_i/inst/gmii_interface/bufio_gmii_rx_clk. Shape elements have relative placement respect to each other. The invalid location might results from a constraint on any of the instance in the shape. [/home/username/dev_proj/test/warning_recreate/tri_mode_ethernet_mac_0_example/tri_mode_ethernet_mac_0_example.srcs/constrs_1/imports/example_design/tri_mode_ethernet_mac_0_example_design.xdc:125]
```

For more information on these requirements, see the Vivado Design Suite, See [I/O Standard and Placement](#).

Timing Failed for GMII/RGMII/MII OFFSET IN Constraint

To satisfy setup and hold requirements for these standards, either:

- Fixed-mode IODELAYs are placed on the receive data and control signals when using the GMII, RGMII, or MII wrapper files.

In the example design XDC, the fixed value delays are set based on the pinout used in the example design. With a different pinout, it might be required to adjust the fixed DELAY value to still meet the setup and hold requirements.

- An MMCM is used on the input clock source for the GMII, RGMII, or MII.

In the example design XDC, a fixed phase shift value is set based on the pinout used in the example design. With a different pinout, it might be required to adjust the phase shift value to still meet the setup and hold requirements.

For more details on adjusting this delay to meet setup and hold requirements, see the Vivado Design Suite, [I/O Standard and Placement](#).

Timing Violations on I/O Paths in GMII/RGMII – UltraScale Architecture Devices

These techniques can help close timing on I/O paths covered by `set_input_delay` / `set_output_delay` constraints:

- **TX Path:** Lock the clocking elements used to generate the transmit clock, that is the MMCM and BUFG, to be in the same clock region. Locking the TX I/O pins to this clock region results in better timing margins.
- **RX Path:** Lock the BUFG on the RX clock input and the RX I/O pins to be in the same clock region.

To illustrate how these techniques can be used, the following constraints are developed for the TEMAC example design generated for an XCKU040-FFVA1156 device in RGMII:

```
# 200 MHz input clock
set_property PACKAGE_PIN AF10 [get_ports clk_in_p]
set_property PACKAGE_PIN AG10 [get_ports clk_in_n]
# RGMII TX pins
set_property PACKAGE_PIN AH9 [get_ports rgmii_txc]
set_property PACKAGE_PIN AH8 [get_ports rgmii_tx_ctl]
set_property PACKAGE_PIN AK8 [get_ports rgmii_txd[0]]
set_property PACKAGE_PIN AL8 [get_ports rgmii_txd[1]]
set_property PACKAGE_PIN AJ9 [get_ports rgmii_txd[2]]
set_property PACKAGE_PIN AJ8 [get_ports rgmii_txd[3]]
# RGMII RX pins
set_property PACKAGE_PIN AG11 [get_ports rgmii_rxc]
set_property PACKAGE_PIN AM11 [get_ports rgmii_rx_ctl]
set_property PACKAGE_PIN AE13 [get_ports rgmii_rxd[0]]
set_property PACKAGE_PIN AF13 [get_ports rgmii_rxd[1]]
set_property PACKAGE_PIN AK13 [get_ports rgmii_rxd[2]]
set_property PACKAGE_PIN AL13 [get_ports rgmii_rxd[3]]
# BUFG on 200 MHz input clock
set_property CLOCK_REGION X2Y0 [get_cells {example_clocks/bufg_clk_in1}]
# BUFG on GTX Clock
set_property CLOCK_REGION X2Y0 [get_cells {example_clocks/clock_generator/
clkout1_buf}]
# BUFG on RX Clock input
set_property CLOCK_REGION X2Y0 [get_cells -hier -filter {NAME =~ */
rgmii_interface/bufg_rgmii_rx_clk}]
set_property CLOCK_REGION X2Y0 [get_cells -hier -filter {NAME =~ */
rgmii_interface/bufg_rgmii_rx_clk_iddr}]
# Create Pblock to place the design in the vicinity of IO pins
startgroup
create_pblock pblock_1
resize_pblock pblock_1 -add CLOCKREGION_X2Y0:CLOCKREGION_X2Y0
endgroup
add_cells_to_pblock pblock_1 -top
```

Hardware Debug

Hardware issues can range from link bring-up to problems seen after hours of testing. This section provides debug steps for common issues. The AMD Vivado™ debug feature is a valuable resource to use in hardware debug. The signal names mentioned in the following individual sections can be probed using the debug feature for debugging the specific problems.

General Checks

Ensure that all the timing constraints for the core were properly incorporated from the example design and that all constraints were met during implementation.

- Does it work in post-place and route timing simulation? If problems are seen in hardware but not in timing simulation, this could indicate a PCB issue. Ensure that all clock sources are active and clean.
- If using MMCMs in the design, ensure that all MMCMs have obtained lock by monitoring the `locked` port.
- If your outputs go to 0, check your licensing.

Problems with Transmitting and Receiving Frames

Problems with data reception or transmission can be caused by a wide range of factors. The following list contains common causes to check for:

- Verify that the whole TEMAC block is not being held in reset. The whole block is held in reset if the main reset input or if a locked signal from an MMCM is Low.
- Verify that both the receiver and transmitter are enabled and not being held in reset. For more information, see the receiver and transmitter configuration words in [Table 28: Receiver Configuration Word 0 \(0x400\)](#) and [Table 30: Transmitter Configuration Word \(0x408\)](#) respectively.
- Verify that the TEMAC is configured correctly and that the latest core version is being used. Try running a simulation to check if the failure is hardware-specific.
- If using GMII or RGMII, check if setup and hold requirements are met. For more information, see the section on debugging [Implementation and Timing Errors](#).
- Verify that the link is up between the PHY and its link partner. If using the Ethernet 1G/2.5G PCS/PMA or SGMII core, see the Debugging Guide section of the *1G/2.5G Ethernet PCS/PMA or SGMII LogiCORE IP Product Guide (PG047)*.
- If using an external PHY, is data received correctly if the PHY is put in loopback? If so, the issue might be on the link between the PHY and its link partner.

- Check if the address filter is enabled. If frames are not being received correctly, try disabling the address filter to ensure that the frame is not being dropped by the address filter. For more information, see [Frame Filter](#).
- Verify that the TEMAC has been configured to operate at the correct speed negotiated with the PHY.
- Are received frames being dropped by user logic because `rx_axis_mac_tuser` is asserted? See [Frame Reception with Errors](#) for details on why frames are marked bad by the Ethernet MAC. The Vivado analyzer tool can be inserted to get more details on the bad frames.
- Add the Vivado analyzer tool to the design to look at the RX and TX AXI4-Stream and physical interface data signals, control signals, and statistics vectors.

Issues with the MDIO

See [Accessing PHY Configuration Registers, through MDIO Using the Management Interface](#) for detailed information about performing MDIO transactions.

Things to check for:

- Check that the MDC clock is running and that the frequency is 2.5 MHz or less. If using the MDIO control registers to perform MDIO accesses, the MDIO interface does not work until the clock frequency is set with `CLOCK_DIVIDE`. The MDIO clock with a maximum frequency of 2.5 MHz is derived from the `s_axi_aclk` clock.
- Ensure that the TEMAC and PHY are not held in reset. Be sure to check the polarity of the reset to your external PHY. Many PHYs have an active-Low reset.
- Read from a configuration register that does not have all 0s as a default. If all 0s are read back, the read was unsuccessful.
- If using the management interface to access the MDIO, check if the issue is with the MDIO control registers or if there are also issues reading and writing MAC registers with the management interface.
- If accessing MDIO registers of the Ethernet 1G/2.5G PCS/PMA or SGMII core, check that the PHYAD field placed into the MDIO frame matches the value placed on the `phyad[4:0]` port of the Ethernet 1G/2.5G PCS/PMA or SGMII core.
- Has a simulation been run? Verify in simulation and/or a Vivado analyzer tool capture that the waveform is correct for accessing the management interface for an MDIO read/write. The demonstration test bench delivered with the core provides an example of MDIO accesses.

Configuring the Ethernet MAC to the Correct Speed

When operating in tri-mode, the PHY negotiates the highest speed available with its link partner. The user application can set the speed of the Ethernet MAC after auto-negotiation completes by doing the following:

1. The user application can either monitor auto-negotiation interrupt from the external PHY or internal Ethernet 1G/2.5G PCS/PMA or SGMII core or poll for auto-negotiation (see the relevant PHY documentation).
2. When auto-negotiation completes, the user application can read the MDIO auto-negotiation registers to obtain the negotiated speed.
3. The user application then needs to set this speed in the Ethernet MAC configuration registers using the host interface.

If auto-negotiation is disabled, the Ethernet MAC, PHY, and the PHY link partner must all be set to the same speed.

Additional Resources and Legal Notices

Finding Additional Documentation

Technical Information Portal

The AMD Technical Information Portal is an online tool that provides robust search and navigation for documentation using your web browser. To access the Technical Information Portal, go to <https://docs.amd.com>.

Documentation Navigator

Documentation Navigator (DocNav) is an installed tool that provides access to AMD Adaptive Computing documents, videos, and support resources, which you can filter and search to find information. To open DocNav:

- From the AMD Vivado™ IDE, select **Help** → **Documentation and Tutorials**.
- On Windows, click the **Start** button and select **Xilinx Design Tools** → **DocNav**.
- At the Linux command prompt, enter `docnav`.

Note: For more information on DocNav, refer to the *Documentation Navigator User Guide* ([UG968](#)).

Design Hubs

AMD Design Hubs provide links to documentation organized by design tasks and other topics, which you can use to learn key concepts and address frequently asked questions. To access the Design Hubs:

- In DocNav, click the **Design Hubs View** tab.
- Go to the [Design Hubs](#) web page.

Support Resources

For support resources such as Answers, Documentation, Downloads, and Forums, see [Support](#).

References

These documents provide supplemental material useful with this guide:

1. IEEE Standard 802.1Qbb, "IEEE Standard for Local and Metropolitan Area Networks—Virtual Bridged Local Area Networks—Amendment: Priority-based Flow Control"
2. *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* ([UG994](#))
3. *AXI 1G/2.5G Ethernet Subsystem Product Guide* ([PG138](#))
4. *AMBA AXI4-Stream Protocol Specification* ([ARM IHI 0051A](#))
5. IEEE 802.3-2008 specification
6. *1G/2.5G Ethernet PCS/PMA or SGMII LogiCORE IP Product Guide* ([PG047](#))
7. *7 Series FPGAs Clocking Resources User Guide* ([UG472](#))
8. *Vivado Design Suite User Guide: Designing with IP* ([UG896](#))
9. *Vivado Design Suite User Guide: Getting Started* ([UG910](#))
10. *Vivado Design Suite User Guide: Using Constraints* ([UG903](#))
11. *Vivado Design Suite User Guide: Logic Simulation* ([UG900](#))
12. *ISE to Vivado Design Suite Migration Guide* ([UG911](#))
13. *Vivado Design Suite User Guide: Programming and Debugging* ([UG908](#))
14. *Command Line Tools User Guide* ([UG628](#))
15. Reduced Gigabit Media Independent Interface (RGMII), version 2.0
16. IEEE 802.1AS, Timing and Synchronization for Time-Sensitive Applications in Bridged Local Area Networks
17. IEEE 802.1BA-2011, Audio Video Bridging (AVB) Systems
18. IEEE 802.1Q-2011, Media Access Control (MAC) Bridges and Virtual Bridged Local Area Networks
19. IEEE 1588, A Precision Clock Synchronization Protocol for Networked Measurement and Control Systems
20. *7 Series FPGAs Configuration User Guide* ([UG470](#))
21. *7 Series FPGAs Configurable Logic Block User Guide* ([UG474](#))

22. UltraScale Architecture Clocking Resources User Guide ([UG572](#))

23. UltraScale Architecture SelectIO Resources User Guide ([UG571](#))

24. Vivado Design Suite User Guide: Release Notes, Installation, and Licensing ([UG973](#))

Revision History

The following table shows the revision history for this document.

Section	Revision Summary
12/11/2024 Version 9.0	
Entire document	General Updates
11/07/2023 Version 9.0	
Entire document	General Updates
05/17/2023 Version 9.0	
Clock, Speed Indication, and Reset Signal Definition	Added a note for Table.
12/02/2022 Version 9.0	
Flow Control Interface (IEEE 802.3)	Added a table note
05/09/2022 Version 9.0	
I/O Delay Calibration Ports for TEMAC RGMII Interface	Added a new table.
08/05/2021 Version 9.0	
User Error Indication	Added tuser assertion information and updated for Vivado 2021.1 release.
07/08/2020 Version 9.0	
Table 13: Alternative to Optional Management Interface – Configuration Vector Signal Pinout	Updated rx/tx_configuration_vector
User Error Indication	Added paragraph at the end
Frame Collisions – Half-Duplex Operation Only	Added paragraph at the end
RGMII and RGMII	Added UltraScale+ caution note
04/04/2018 Version 9.0	
Chapter 3: Product Specification	- Updated mac_irq description in Interrupt Signals table - Updated rtc_nanosec_field and rtc_sec_field descriptions in AVB Specific Signals table - Added header file description in Register Space section - Added bit description to Interrupt Controller section
Chapter 4: Designing with the Core	Added MAC description in User Error Indication section
10/04/2017 Version 9.0	
Chapter 4: Designing with the Core	- Updated rx_pfc_p[0-7]_tready description in Pause Frame Reception Enabled section - Updated description in Finding the Ideal Phase Shift Value section. - Updated code in Timing Violations on I/O Paths in GMII/RGMII – UltraScale Architecture Devices section.

Section	Revision Summary
10/05/2016 Version 9.0	
IP Facts	Added Spartan-7
XON/XOFF Extended Functionality	Added note
RGMII	Added a Caution note
User Parameters	Updated Vivado IDE Parameter to User Parameter Relationship table.
Appendix A: Upgrading	Added description in IP Upgrade.
04/06/2016 Version 9.0	
Chapter 3: Product Specification	- Updated Important description in MAC description. - Updated Table 2-13: Clock and Speed Indication Signals. - Updated Bits[79:32] in Table 2-21: tx_configuration_vector Bit Definitions. - Updated Bits[79:32] in Table 2-22: rx_configuration_vector Bit Definitions. - Updated RX Alignment Errors and RX Bad Opcode descriptions in Table 2-24: Statistics Counter Definitions.
Chapter 5: Design Flow Steps	- Added description in Maximum Permitted Frame Length section. - Updated Fig. 4-2. - Added Internal mode Clock Source in Design Flow Steps chapter. - Added Int_Clk_Src in Table 4-1: Vivado IDE Parameter to User Parameter Relationship. - Added rx_usr_clk2 in Table 4-3: TEMAC Solution Frequency Requirements.
11/18/2015 Version 9.0	
UltraScale Devices	Added support for UltraScale+ families.
09/30/2015 Version 9.0	
Chapter 4: Designing with the Core	- Updated the Resource Utilization section. - Updated speedis100 description in Clock and Speed Indication Signals. - Updated Fig. 3-56: Management Register Write Timing. - Updated RX Oversize Frames description in Statistics Counter Definitions. - Added loopback logic availability in Example Design section. - Updated Basic Pattern Generator Module for 10, 100, 1000 Mbps Data Rates. - Added Basic Pattern Generator Module for 2.5 Gbps Data Rate section. - Added description in 10, 100, 1000 Mbps Ethernet FIFO section. - Added 2.5 Gbps Ethernet FIFO section. - Added 2.5 Gbps description in Test Bench Functionality section.

Section	Revision Summary
04/01/2015 Version 9.0	
Chapter 3: Product Specification	- Added 2.5 Gbps Ethernet feature throughout product guide. - Updated description in Ethernet Overview. - Updated Table 2-22: Clock and Speed Indication Signals. - Updated Table 2-45: Ability Register (0x4FC). - Added GMII/RGMII for UltraScale Devices and MII or Internal Interface for UltraScale Devices sections. - Updated description in Clocking section.
Chapter 4: Designing with the Core	- Updated description in 10 or 100 Mbps Ethernet MAC Core Interfaces MII Transmitter and Receiver Interface sections. - Updated description in 1 Gbps Ethernet MAC Core Interfaces GMII Receive Interface section. - Updated description in RGMII section. - Updated description in Tri-Speed Ethernet MAC Core Interfaces (UltraScale) section. - Updated Caution note in Ethernet 1000BASE-X PCS/PMA or SGMII Core section.
Chapter 5: Design Flow Steps	- Added Data Rate section and updated Figs. 4-1 to 4-5 - Updated Table 4-1: Vivado IDE Parameter to User Parameter Relationship. - Added Virtex UltraScale in Table 4-2: Supported Speed Grades. - Added UNISIM important note in Simulation section. - Updated Synthesis and Implementation section.
Appendix A: Upgrading	Updated the appendix.
Appendix D: Debugging	Added Timing Violations on I/O Paths in GMII/RGMII – UltraScale Architecture Devices section
10/01/2014 Version 8.3	
MDIO Interface	Added MDIO interface and I/O buffer selection
04/02/2014 Version 8.2	
Chapter 3: Product Specification	- Added Priority Flow Control Section - Added Physical interface for UltraScale Family devices section - Updated Table 2-11, Table 2-13, Table 2-15, Table 2-17, Table 2-34, Table 2-38, Table 2-42, Table 2-43, Table 2-49, Table 2-80 and Table 2-81
12/18/2013 Version 8.1	
Chapter 3: Product Specification	- Added UltraScale architecture support - Updated Table 2-22, Table 2-24, and Table 2-76. - Updated Shared Logic - Updated Clocking

Section	Revision Summary
Chapter 4: Designing with the Core	- Updated Constraining the Core with description of a new example design file - Updated IDELAYCTRL with UltraScale related information - Updated Regional Clocking Errors with Vivado error messages
10/02/2013 Version 8.0	
Chapter 4: Designing with the Core	- Added Shared Logic section - Added IDELAYCTRL information - Updated Clock Resource Sharing information
06/19/2013 Version 7.0	
Chapter 3: Product Specification	- Revision number advanced to 7.0 to align with core version number. - Updated s_axi_awaddr[31:0] and s_axi_araddr[31:0] descriptions in Table 2-12 Optional AXI4-Lite Signal Pinout. - Updated Frame Filter Enable (0x70C) to Bits[31:1].
Chapter 4: Designing with the Core	Updated Figs. 3-5 to 3-6.
Chapter 5: Design Flow Steps	- Updated GUIs in Figs. 4-1 and 4-2. - Added descriptions in AXI4-Lite Control State Machine section. - Added descriptions in Test Bench Functionality section. - Added DEMO Mode with Frame Filter descriptions. - Added Changing Frame Data descriptions. - TX/RX clock port changes. - AXI4-Lite address bus width shrinkage. - demo_tb enhancement to illustrate frame filtering.
03/20/2013 Version 2.1	
Chapter 3: Product Specification	- Updated to v6.0 for Vivado Design Suite only. Removed . - Updated Device Utilization tables (Table 2-1 to 2-4). - Updated descriptions in Port Descriptions. - tx_enable, rx_enable, speedis10100 and speedis100 outputs now present for all parameterizations - tx_axis_mac_tuser is now defined as std_logic_vector for all VHDL parameterizations
Chapter 5: Design Flow Steps	- Updated Figs. 4-1 and 4-2 GUIs - Updated AXI4-Lite Control State Machine in Example Design chapter. - Added Artix 7 AC701 board support in Example Design chapter. - Updated to Questa Advanced Simulator.

Section	Revision Summary
12/18/2012 Version 2.0	
Chapter 4: Designing with the Core	- Updated to v5.5, Design Suite 14.4/Vivado Design Suite 2012.4. - Removed Windows and Linux references in System Requirements. - Added note on IEEE 1588 timestamping - Updated to 6 cycles in Frame Collisions – Half Duplex Operation section - Updated “bytes of transmit data” to Interframe Gap Adjustment Full-Duplex section - Updated GMII and RGMII subsections with 7 series in 1 Gbps Ethernet MAC IP Core and Tri-Speed (10 Mbps, 100 Mbps, and 1 Gbps) Ethernet MAC IP Core sections
Chapter 5: Design Flow Steps	Updated GUI in Fig. 4-1
Chapter 7: Test Bench	Updated State Machine Flow Diagram in Fig. 6-5
Appendix A: Upgrading	Updated GUI in Fig. 7-1
Appendix D: Debugging	Updated Debug section and minor document updates.
07/25/2012 Version 1.0	
Initial Xilinx release.	This Product Guide is derived from DS818 and UG777.

Please Read: Important Legal Notices

The information presented in this document is for informational purposes only and may contain technical inaccuracies, omissions, and typographical errors. The information contained herein is subject to change and may be rendered inaccurate for many reasons, including but not limited to product and roadmap changes, component and motherboard version changes, new model and/or product releases, product differences between differing manufacturers, software changes, BIOS flashes, firmware upgrades, or the like. Any computer system has risks of security vulnerabilities that cannot be completely prevented or mitigated. AMD assumes no obligation to update or otherwise correct or revise this information. However, AMD reserves the right to revise this information and to make changes from time to time to the content hereof without obligation of AMD to notify any person of such revisions or changes. THIS INFORMATION IS PROVIDED "AS IS." AMD MAKES NO REPRESENTATIONS OR WARRANTIES WITH RESPECT TO THE CONTENTS HEREOF AND ASSUMES NO RESPONSIBILITY FOR ANY INACCURACIES, ERRORS, OR OMISSIONS THAT MAY APPEAR IN THIS INFORMATION. AMD SPECIFICALLY DISCLAIMS ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, OR FITNESS FOR ANY PARTICULAR PURPOSE. IN NO EVENT WILL AMD BE LIABLE TO ANY PERSON FOR ANY RELIANCE, DIRECT, INDIRECT, SPECIAL, OR OTHER CONSEQUENTIAL DAMAGES ARISING FROM THE USE OF ANY INFORMATION CONTAINED HEREIN, EVEN IF AMD IS EXPRESSLY ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

AUTOMOTIVE APPLICATIONS DISCLAIMER

AUTOMOTIVE PRODUCTS (IDENTIFIED AS "XA" IN THE PART NUMBER) ARE NOT WARRANTED FOR USE IN THE DEPLOYMENT OF AIRBAGS OR FOR USE IN APPLICATIONS THAT AFFECT CONTROL OF A VEHICLE ("SAFETY APPLICATION") UNLESS THERE IS A SAFETY CONCEPT OR REDUNDANCY FEATURE CONSISTENT WITH THE ISO 26262 AUTOMOTIVE SAFETY STANDARD ("SAFETY DESIGN"). CUSTOMER SHALL, PRIOR TO USING OR DISTRIBUTING ANY SYSTEMS THAT INCORPORATE PRODUCTS, THOROUGHLY TEST SUCH SYSTEMS FOR SAFETY PURPOSES. USE OF PRODUCTS IN A SAFETY APPLICATION WITHOUT A SAFETY DESIGN IS FULLY AT THE RISK OF CUSTOMER, SUBJECT ONLY TO APPLICABLE LAWS AND REGULATIONS GOVERNING LIMITATIONS ON PRODUCT LIABILITY.

Copyright

© Copyright 2012-2024 Advanced Micro Devices, Inc. AMD, the AMD Arrow logo, Artix, Kintex, Spartan, UltraScale, UltraScale+, Versal, Virtex, Vivado, Zynq, and combinations thereof are trademarks of Advanced Micro Devices, Inc. AMBA, AMBA Designer, Arm, ARM1176JZ-S, CoreSight, Cortex, PrimeCell, Mali, and MPCore are trademarks of Arm Limited in the US and/or elsewhere. Other product names used in this publication are for identification purposes only and may be trademarks of their respective companies.